

년월일 2007-08-01	문서번호	변경 코드	파일/참고	페이지 1 (308)
작성자	승인자	문서관리자	검토	문서종류
제목		TITLE		

ZigBee Protocol Stack

컴퓨터네트워크 연구실

인제대학교

(초록) 본 문서에서는 ... 에 대해 기술한다.

목 차

1	사용자 요구사항 정의 (Definition of user's requirements)	5
1.1	General	5
1.1.1	Purpose of the Project	5
1.1.2	Specific Objectives	5
1.1.3	Abbreviations	5
1.1.4	References	5
1.2	System Requirements	6
1.3	ZigBee Overview	7
1.4	Full Function Device	11
1.5	Reduced Function Device	11
2	시스템 설계서 (System Design) – ZigBee Protocol Stack	12
2.1	개요 (Introduction)	12
2.2	시스템 설계 개념 (Concepts for System Design)	12
2.2.1	용어의 정의 (Definition of Terminologies)	12
2.2.2	설계 개념 (Design Concept)	13
2.3	시스템 구조 설명	13
2.3.1	시스템 구성 (System Architecture)	13
2.3.2	전체적 기능 흐름 (Overall functional flow)	13
2.3.3	Zigbee component	14
2.3.4	802.15.4 Component	14
2.3.5	OS-AL	14
3	시스템 설계서 (System Design) – Zigbee Protocol Stack Layers	14
3.1	개요 (Introduction)	14
3.2	시스템 설계 개념 (Concepts for System Design)	15
3.2.1	용어의 정의 (Definition of Terminologies)	15
3.3	시스템 구성 (System Architecture)	15
3.4	구성 요소 설명 (Description of internal components)	16
3.4.1	HAL	16
3.4.2	PHY	16
3.4.3	Packet manager	16
3.4.4	Scheduling	16
3.4.5	TRANS	17
3.4.6	RX	17
3.4.7	MAC_STATE	17

3.4.8	DISPATCH_CMD.....	17
3.4.9	BEACON.....	17
4	PHY Layer Design and Primitives	18
5	MAC Layer Design and Primitives	19
5.1	MAC Data Service.....	20
5.2	MAC Data Service.....	20
5.2.1	MCPS - Data.....	21
5.2.2	MCPS - Purge	25
5.3	MAC Management Service	25
5.3.1	MLME-Reset	26
5.3.2	MLME-Beacon-Notify	27
5.3.3	MLME-GET	28
5.3.4	MLME-SET	28
5.3.5	MLME-COMM-STATUS.....	29
5.3.6	MLME-SYNC / MLME-SYNC-LOSS.....	30
5.3.7	MLME-POLL.....	32
5.3.8	MLME-START.....	35
5.3.9	MLME-RX-ENABLE	37
5.3.10	MLME-SCAN.....	39
5.3.11	MLME-ORPHAN.....	42
5.3.12	MLME-ASSOCIATE	44
5.3.13	MLME-DISASSOCIATE	44
6	NWK Layer Design and Primitives.....	45
6.1	Network Layer Data Entity (NLDE).....	46
6.2	Network Layer Management Entity (NLME).....	46
7	Upper Layer.....	51
7.1	Zigbee Device Object.....	51
7.2	Zigbee Profile.....	52
7.3	Application Support Sub-Layer	52
7.4	Application Framework	53
8	Test and Evaluation	53
8.1	Test of Primitives	53
8.1.1	MAC layer	53
8.1.2	Network layer	67
8.1.2	Application Framework (AF)	75
8.1.1	Application Support Layer (APS).....	78



8.1.2	Zigbee Device Object (ZDO)	80
8.2	Test Scenario	84
8.2.1	Hardware.....	84
8.2.2	Software	84
9	Source Code	89

1 사용자 요구사항 정의 (Definition of user's requirements)

1.1 General

1.1.1 Purpose of the Project

The purpose of this project is to design and develop a Protocol Stack for ZigBee, IEEE 802.15.4. This includes implementation of Physical, Media Access Control, Network and Application Layers for Full Function Device (FFD) and Reduced Function Device (RFD).

1.1.2 Specific Objectives

The specific objectives of the project are listed below:

- To develop a stack that can function as one of the following
 - Coordinator
 - Router
 - Endpoint
- To limit the stack size to not so large value.

1.1.3 Abbreviations

The following abbreviations are used in this documentation:

Acronym	Meaning
MAC	Media Access Control
NWK	Network
MLME	MAC Layer Management Entity
NLDE	NWK Layer Data Entity
NLME	NWK Layer Management Entity
NIB	Network Information Base

Table 1. List of Abbreviations

1.1.4 References

[1] IEEE Standard 802.15.4-2003

Part 15.4: Wireless Medium Access Control (MAC) and Physical Layer (PHY)
Specifications for Low-Rate Wireless Personal Area Networks (LR-WPANS)

[2] ZigBee Specification

ZigBee™ Alliance, Document 053474r06 version 1.0.

December 14, 2004

1.2 System Requirements

The development platform is being done in Windows XP with installed Programmer's Notepad with WinAVR compiler and AVRStudio.

1.2.1 Programmer's Notepad

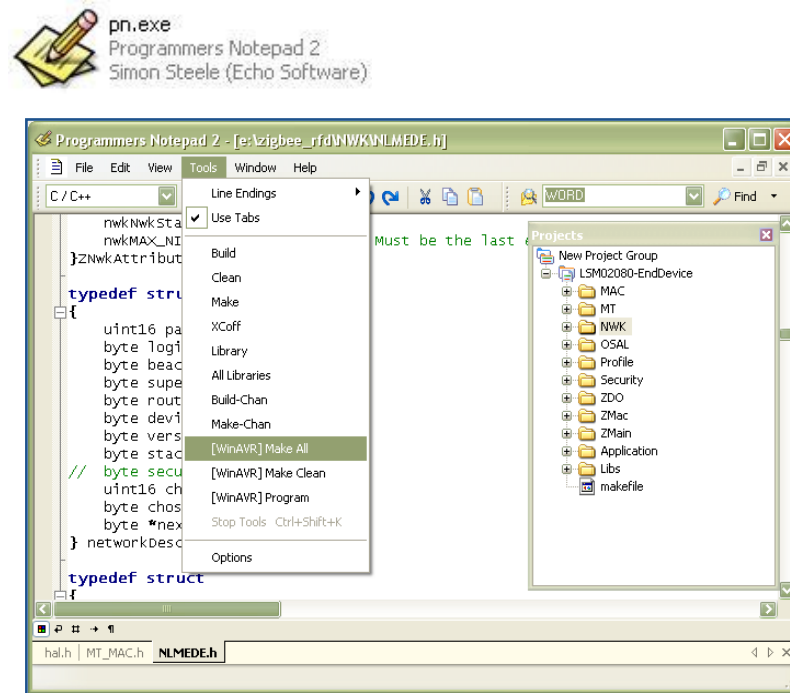


Figure 1. Programmer's Notepad

Programmer's Notepad is a source code editor program that can be integrated with WinAVR for seamless compiling and linking. The full features and downloadable versions are available in <http://www.pnotepad.org/>.

1.2.2 WinAVR

WinAVRTM (pronounced "whenever") is a suite of executable, open source software development tools for the Atmel AVR series of RISC microprocessors hosted on the Windows platform. It includes the GNU GCC compiler for C and C++ . This is available at <http://winavr.sourceforge.net/>

1.2.3 AVRStudio

AVR Studio® 4 is the new professional Integrated Development Environment (IDE) for writing and debugging AVR® applications in Windows® 9x/NT/2000/XP

environments.

AVR Studio 4 includes an assembler and a simulator. The following AVR development tools are also supported:

ICE50, ICE40, JTAGICE mkII, JTAGICE, ICE200, STK500/501/502/503/504/505/520, AVRISP mkII, and AVRISP. More details about AVR Studio can be found here:
http://www.atmel.com/dyn/products/tools_card.asp?tool_id=2725

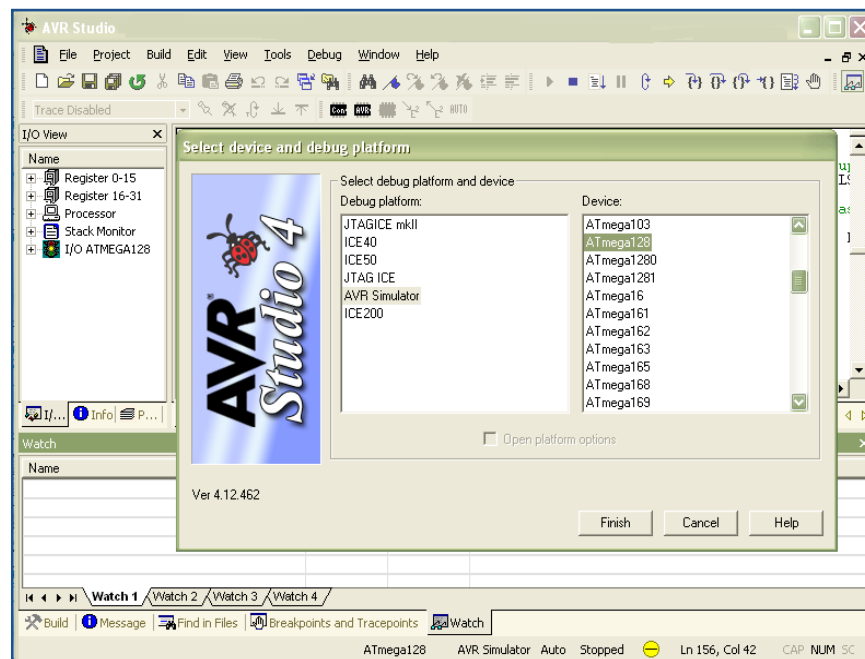


Figure 2. AVR Studio

1.3 ZigBee Overview

The ZigBee is a low-rate wireless networking standard. It is built based on IEEE standard 802.15.4 which defines the PHY and MAC layers. An organization called ZigBee Alliance organized this standard.

The ZigBee standards set of solutions called “layers” which in time would evolve to the ZigBee Protocol stack. These layers are:

a. Application Layer

The Application layer consists of the Application Support (APS) layer, ZigBee Device Object (ZDO) and manufacturer defined applications. The APS is responsible for binding or matching two devices together based

on their needs and services, and acting as an interface between them, and discovery or detecting other devices that are operating in a particular devices operating space. The ZDO is responsible for assigning roles for each device in the network. Manufacturer defined application objects implement the application based on the application descriptions defined y the ZigBee network.

b. Network Layer

The network layer allows for spatial network growth without the need for high powered transmitters. The design of the network layer also allows for relatively low latency handling of large numbers of nodes. Network layer responsibilities include; network establishment and access, device configuration and addressing, Device synchronization within a network, Frames security and routing.

c. Media Access Control Layer

The Media Access Control (MAC) Layer allows for multiple topologies without complexity. It can handle a large number of devices without having them "parked", due to the power management operation in the MAC layer. This power management operation does not require multiple operating modes, and has a reduced functionality device (RFD) that doesn't require Flash memory or large amounts of RAM or ROM to utilize.

d. Physical (PHY) Layer

The physical layer allows for a high degree of integration. The analog circuitry uses direct sequence. The simplicity of this form of analog circuitry affords itself a low cost, and is very tolerant of inexpensive implementations.

Figure 3 shows these three ZigBee standard layers [3] as described above.

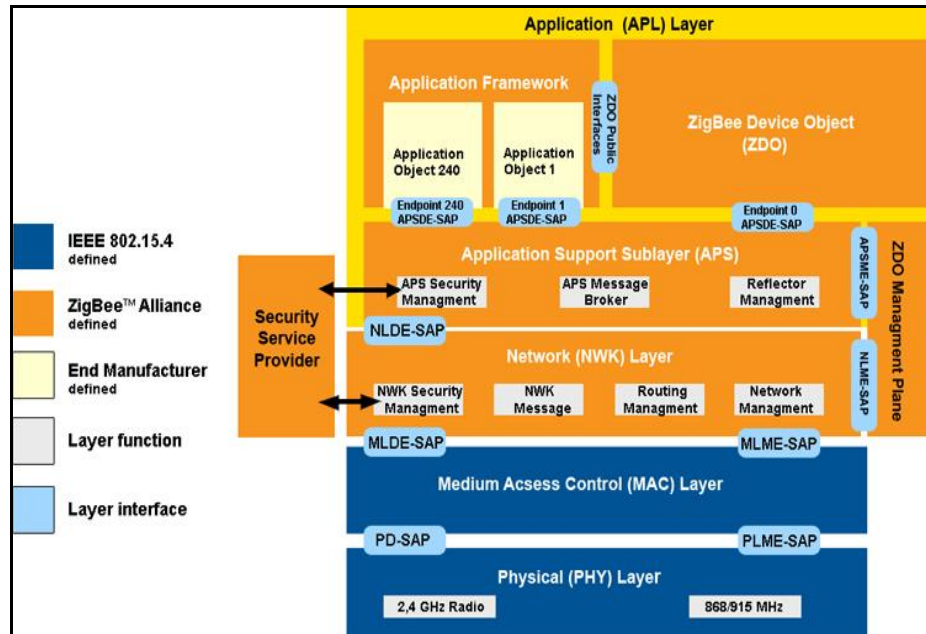
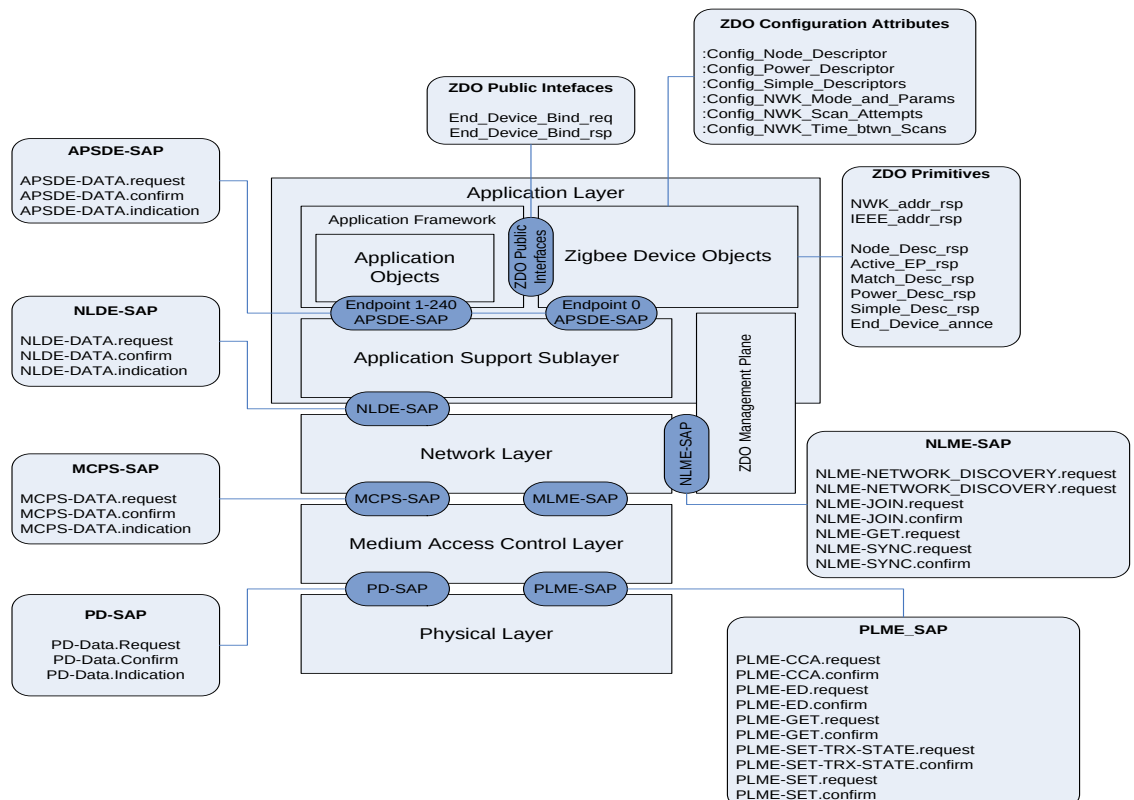
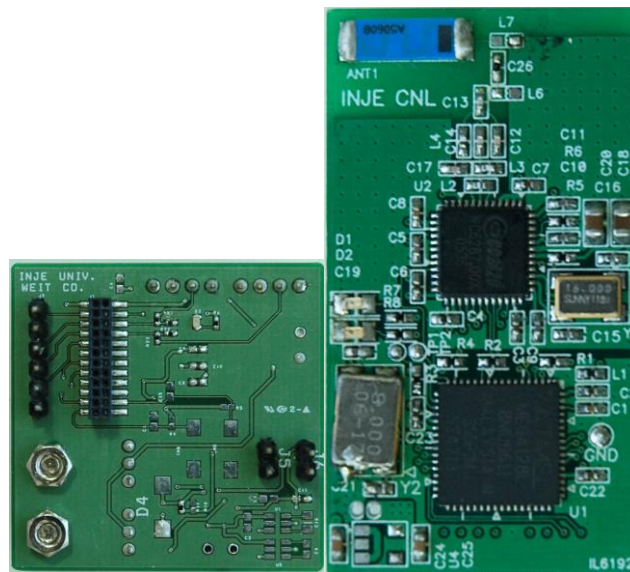


Figure 3. ZigBee Protocol Stack





There are three types of ZigBee devices categorized according to its functionality and its capabilities. They are:

- a. ZigBee Coordinator – the most capable ZigBee device in the network. Serves as the central node and might bridge all the other nodes in the network. There is exactly one ZigBee coordinator in a network.
- b. Full Function Device (FFD) – This is also known as the ZigBee Router. The FFD is an intermediary router transmitting data from other devices. It needs lesser memory than the ZigBee coordinator node and performs lesser functions. It can operate in all topologies (star, cluster tree, mesh) and can act as a coordinator.
- c. Reduced Function Device (RFD) – Also known as the ZigBee End Device. This device is just capable of communicating to its parent node; it cannot relay data from other devices. Requiring even less memory since it has lesser functions.

Below is a list of the characteristics of a ZigBee node which distinguished it from any network type node.

- a. Low duty cycle
- b. Low data latency
- c. Support for multiple network topologies
- d. Direct Sequence Spread Spectrum
- e. 64-bit IEEE addressing (up to 65000 networks)

- f. 128-bit data encryption
- g. Collision Avoidance
- h. Maximum data rates allowed for each of these frequency bands are fixed as 250 kbps @2.4 GHz, 40 kbps @ 915 MHz, and 20 kbps @868 MHz
- i. Link quality indication

For ZigBee is designed to have low power, its applications can range from consumer to industrial applications. Some applications are patient monitoring, fitness control, home and industrial light control, building automation, wireless peripheral connections, and environmental monitoring.

1.4 Full Function Device

Full function device supports all of the features specified by the standard and takes on complex tasks, such as network routing functions.

1.4.1 Coordinator

The coordinator is used to initiate, terminate and route communication in the network and is therefore the primary controller of the personal area network.

1.4.2 Router

Similarly, the router is implemented as FFD but has less functions compared to the coordinator. Generally, it transmits data in the network. A router may act as coordinator within its personal operating space, that is capable of routing messages between devices and supporting associations

1.4.3 End Device

An End device can be FFD or RFD participating in a zigbee network which is neither a zigbee coordinator nor a router.

1.5 Reduced Function Device

A reduced function device is generally used at the network edge, and can be as simple as a transceiver, an 8-bit microprocessor or a device supported by pair of AAA batteries.

The three tables below described the primitives required for a Reduced Function Device (RFD).

Table 2. PHY Layer RFD Primitives

Name	Request	Response	Indication	Confirm
PLME-Get	√			√
PLME-ED	√			
PLME-CCA	√			
PLDE-Data	√		√	√
PLME-SetTrxState	√			√
PLME-Set	√			√

Table 3. MAC Layer RFD Primitives

Name	Request	Response	Indication	Confirm
MCPS-Data	√		√	√
MLME-Associate	√			√
MLME-Beacon Notify			√	
MLME-Get	√			
MLME-Reset	√			√
MLME-Scan	√			√
MLME-Set	√			√

Table 4. Network Layer RFD Primitives

Name	Request	Response	Indication	Confirm
NLDE-Data	√		√	√
NLME-NetworkDiscovery	√			√
NLME-Join	√		√	√
NLME-Reset	√			√
NLME-Get	√			√
NLME-Set	√			√

2 스템 설계서 (System Design) – ZigBee Protocol Stack

2.1 개요 (Introduction)

This document defines software system architecture of ZigBee protocol stack.
 Specification of the on board system is to be defined.

2.2 시스템 설계 개념 (Concepts for System Design)

2.2.1 용어의 정의 (Definition of Terminologies)

ZPS	Zigbee protocol stack
INT	Interrupt

2.2.2 설계 개념 (Design Concept)

The software is designed by dividing it into functional blocks. All methods to describe software functionality are described in this document.

2.3 시스템 구조 설명

2.3.1 시스템 구성 (System Architecture)

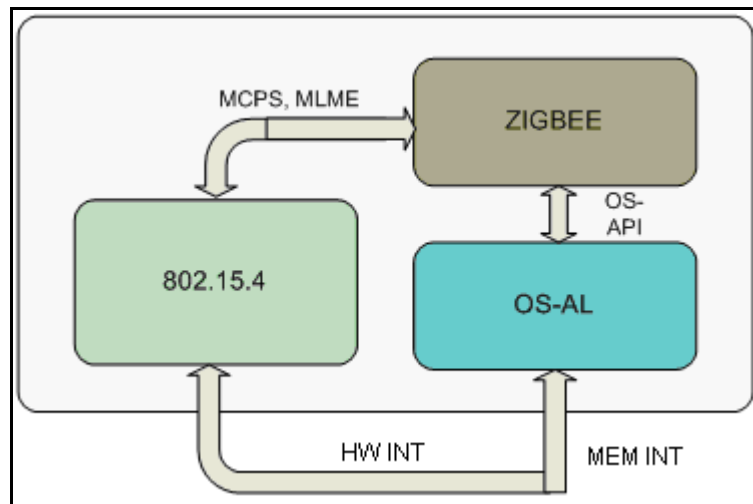


Figure 4. Zigbee system architectures

The system is divided into three components:

- ZigBee: Software component that works on behalf of the ZigBee protocol stack defined in the ZigBee standard.
- 802.15.4: Mac layer following the standard 802.15.4. This is an independent software component that will manage the INT from hardware to provide the functions of MAC.
- OS-AL: Operating system Abstract Layer that will provide functionality of a reduced (or full) operating system.

2.3.2 전체적 기능 흐름 (Overall functional flow)

ZigBee component uses MCPS and MLME services from 802.15.4 to transmit and receive data frames. Scheduling and resource management of ZigBee stack



and ZigBee application component will be supported by OS-AL.

INTs will be managed by 802.15.4 and OS-AL:

- Hardware related INT: will be managed by 802.15.4. They are INT related to hardware and timer.
- Memory INT: will be managed by OS-AL. They are all INT except hardware INT. This includes power INT, timer INT and others external INT.

2.3.3 Zigbee component

Implementation of Zigbee stack defined in the Zigbee standard.

2.3.4 802.15.4 Component

This component will provide the following services:

- MCPS and MLME services as defined in the 802.15.4 standard
- Independent scheduling to schedule for MAC services
- Independent memory management services for MAC services
- INT routine to handle hardware related INT

2.3.5 OS-AL

This component acts as a simple OS (or full OS) that will manage task and resource in ZigBee stack and ZigBee applications. OS-AL is needed to simplify the development phase for ZigBee and ZigBee applications. Necessary services must be provided by this component:

- Task scheduling
- Inter-process communication
- Memory management

To reduce the load for the micro processor, OS-AL must also have following characteristics:

- API is simple to used
- Without any unused functionality
- Size should be as small as possible

3 시스템 설계서 (System Design) – Zigbee Protocol Stack Layers

3.1 개요 (Introduction)

This document defines the software architecture of the layers as conformed to standard 802.15.4.

3.2 시스템 설계 개념 (Concepts for System Design)

3.2.1 용어의 정의 (Definition of Terminologies)

ZPS Zigbee protocol stack

PHY Physical layer

3.3 시스템 구성 (System Architecture)

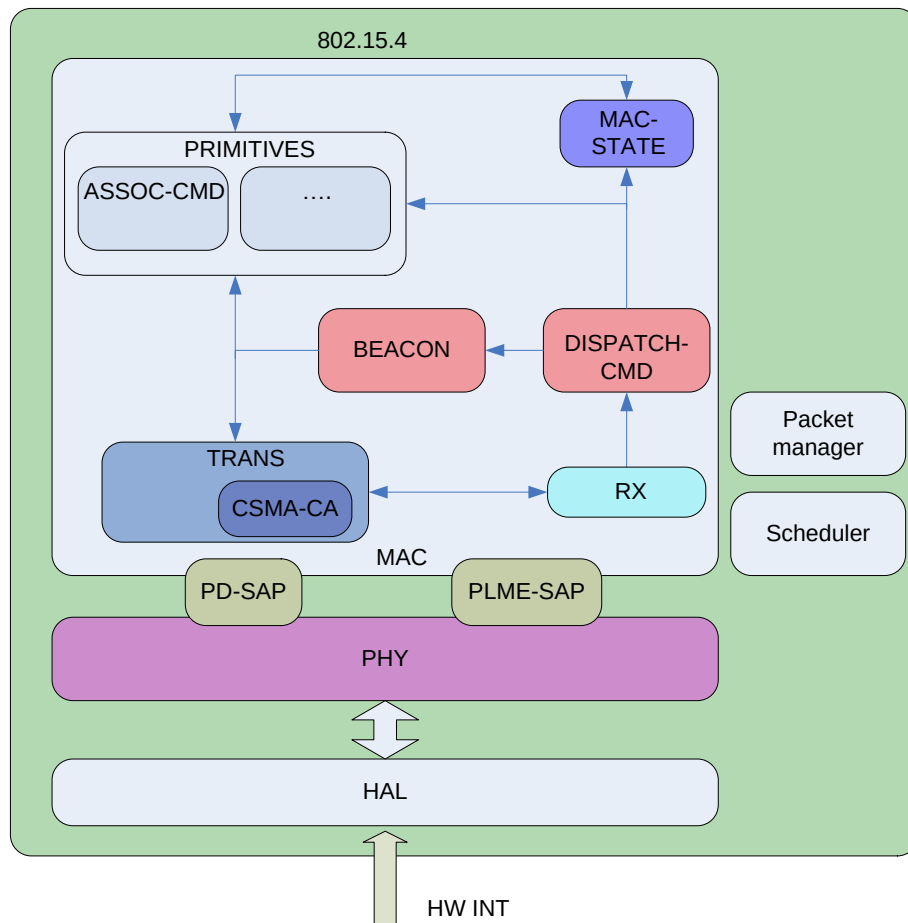


Figure 5. Software architecture of MAC and PHY

HAL	Provide functions to interact with the hardware	
PHY	Provide services as defined in the 802.15.4	
Packet manager	Provides services to manage packet used in the MAC and PHY	
Scheduler	Schedule task of MAC and PHY	
MAC	TRANS	Provides services to transmit packet following procedure of 802.15.4

	CSMA_CA	Provides services to transmit packet using csma-ca defined in 802.15.4
	RX	Received a MPDU from PHY and analyze it, send it to TRANS, if it it ACK or DISPATCH-CMD if others
	MAC_STATE	Manage states (transaction) of the MAC layers.
	PRIMITIVES	Provides primitives as defines in the 802.15.4

Table 5. Component in MAC and PHY

3.4 구성 요소 설명 (Description of internal components)

3.4.1 HAL

This component defines function to simplify the interaction with the hardware. This component provides:

- Bit mask to access hardware pin/register/memory
- Macro to access hardware pin/register/memory

3.4.2 PHY

This component provides service PD-SAP and PLME-SAP as defined in the 802.15.4. It also provides services to:

- Handle the interrupt to receive packet from the hardware
- Control On/Off the transceiver

3.4.3 Packet manager

This component will manage packet used in the MAC and PHY. The services provided by this component will be:

- Packet allocation/de-allocation
- Indirect packet queue management: This function will manage the status of indirect packet queue in the MAC. That includes pushing, pop, and changing status of packet in the indirect packet queue.

3.4.4 Scheduling

This component will provide service to manage the tasks is the MAC. Task here is defined as a routine of source code (function) need to be activated. Task scheduling in MAC must have following properties:

- Task must have priority. Higher priority task is run first
- For all tasks have the same priority, first in first run.
- Tasks will be activated periodically until it is removed
- Tasks are re-schedulable. Re-scheduled task will be add to the tail of the task queue

3.4.5 TRANS

This component provides services to transmit packet using CSMA-CA or not using CSMA-CA. This component will:

- Manage to transmit packet following the procedure in 802.15.4
- Report the status of transmission procedures
- Manage transmission timeout/re-transmission of packet
- Acknowledgment of the sent packet

3.4.6 RX

This component will get the MPDU from the PHY and analyze it. Then

- If the packet it ACK, packet is will be send to TRANS
- If others, packet will be sent to DISPATCH-CMD

3.4.7 MAC_STATE

This component will manage MAC state. That includes

- MLME-GET/SET primitives
- MAC transaction management services

3.4.8 DISPATCH_CMD

This component will dispatch the received command to specific component that process that command.

- Beacon will be dispatched to BEACON
- ACK will be dispatched to TRANS
- Others command will be dispatched to the PRIMITIVES

3.4.9 BEACON

This component will handle function related to beacon:

- Periodically generation of beacon frame
- Process the beacon frame in the beacon network to synchronize the

network devices.

3.4.10 NWK and Application

The application is divided into three steps, namely:

- Network Formation
 - The network formation is started by coordinator by sending beacons. These beacons can be heard by the routers. The routers will issue an associate request to the coordinator or other router. If the association is successful, the nodes issuing the request will receive a network address allocated by the coordinator.
- Address resolution
 - In tree routing, a child node may send a packet to another node provided it knows the network address of the recipient. To do this, a child node who wishes to send a data would need to broadcast a request from its parent to inquire the network address of the recipient provided that the external address is known.
- Data sending
 - After address resolution, a node can send data using the network address of the recipient. The data route still follows tree routing.

For this application, we could show the differences among endpoint, router and coordinator.

4 PHY Layer Design and Primitives

The PHY layer provides interface between the MAC sublayer and the physical radio channel, via the RF firmware and hardware. The PHY conceptually includes a management entity called the PLME. This entity provides the layer management service interfaces through which layer management functions may be invoked. The PLME is also responsible for maintaining a database of managed objects pertaining to the PHY. This database is referred to as the PHY PAN information base (PIB).

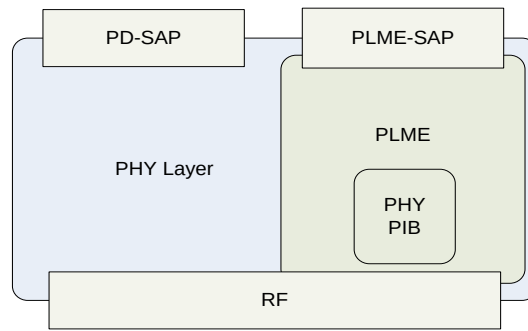


Figure 6. PHY reference model

The physical layer is implemented in the `/Dev/MAC/phy_primitives`. Table 6 shows the files in the protocol stack that implements PHY.

File Name	Description
<code>phy_primitives.c</code>	Implements PHY functional primitives
<code>phy_primitives.h</code>	Implements type definitions of PHY primitives
<code>phy_support.c</code>	Supporting functions for PHY
<code>phy_support.h</code>	Type definitions for the supporting functions
<code>trx_interrupt.c</code>	Implements from packet transmission

Table 6. Source files for PHY

4.1.1 PHY Data Service

The PD-SAP supports the transport of MPDUs between peer MAC sublayer entities.

4.1.2 PHY Management Service

The PLME-SAP allows the transport of management commands between the MLME and PLME.

5 MAC Layer Design and Primitives

The MAC sublayer defines all accesses to the physical radio channel and is responsible for the following tasks:

- Generating network beacons if the device is a coordinator
- Synchronizing to the beacons
- Supporting PAN association and disassociation
- Employing the CSMA-CA mechanism for channel access
- Handling and maintaining the GTS mechanism

- Providing a reliable link between two peer MAC entities [1]

MAC primitives define fundamental operations of services such as request, confirm, response and/or indication. This also includes describing how the services can be accessed with an abstraction of the actual operation.

The MAC implementation is in /Dev/MAC/mac_primitive

File Name	Description
mac_state.c	Provides functions regarding mac states
mac_state.h	Defines type definitions for mac_state.c
mac_mlme.c	Defines management entity primitives
mac_mlme.h	Type definitions for mac_mlme.h
mac_mcps.c	Defines datab

Table 7. Source files for MAC

5.1 MAC Data Service

The MCPS-SAP supports the transport of SSCS protocol data units (SPDUs) between peer SSCS entities. All devices shall provide interface for the MCPS-SAP primitives.

The MAC sublayer defines all accesses to the physical radio channel and is responsible for the following tasks:

- Generating network beacons if the device is a coordinator
- Synchronizing to the beacons
- Supporting PAN association and disassociation
- Employing the CSMA-CA mechanism for channel access
- Handling and maintaining the GTS mechanism
- Providing a reliable link between two peer MAC entities [1]

MAC primitives define fundamental operations of services such as request, confirm, response and/or indication. This also includes describing how the services can be accessed with an abstraction of the actual operation.

5.2 MAC Data Service

The MCPS-SAP supports the transport of SSCS protocol data units (SPDUs) between peer SSCS entities. All devices shall provide interface for the MCPS-SAP



primitives.

5.2.1 MCPS - Data

The MCPS- Data primitive handles the transfer of a data SPDU (Service Specific Convergence Sublayer protocol data units)

- a. The MCPS - DATA.request primitive requests the transfer of a data SPDU from local SSCS (Service Specific Convergence Sublayer) entity to a single SSCS entity.
- b. The MCPS - DATA.confirm primitive reports the results of a request to transfer a data SPDU from a local SSCS entity to a single peer SSCS entity.
- c. The MCPS - DATA.indication primitive indicates the transfer of a data SPDU from the MAC sublayer to the local SSCS entity.

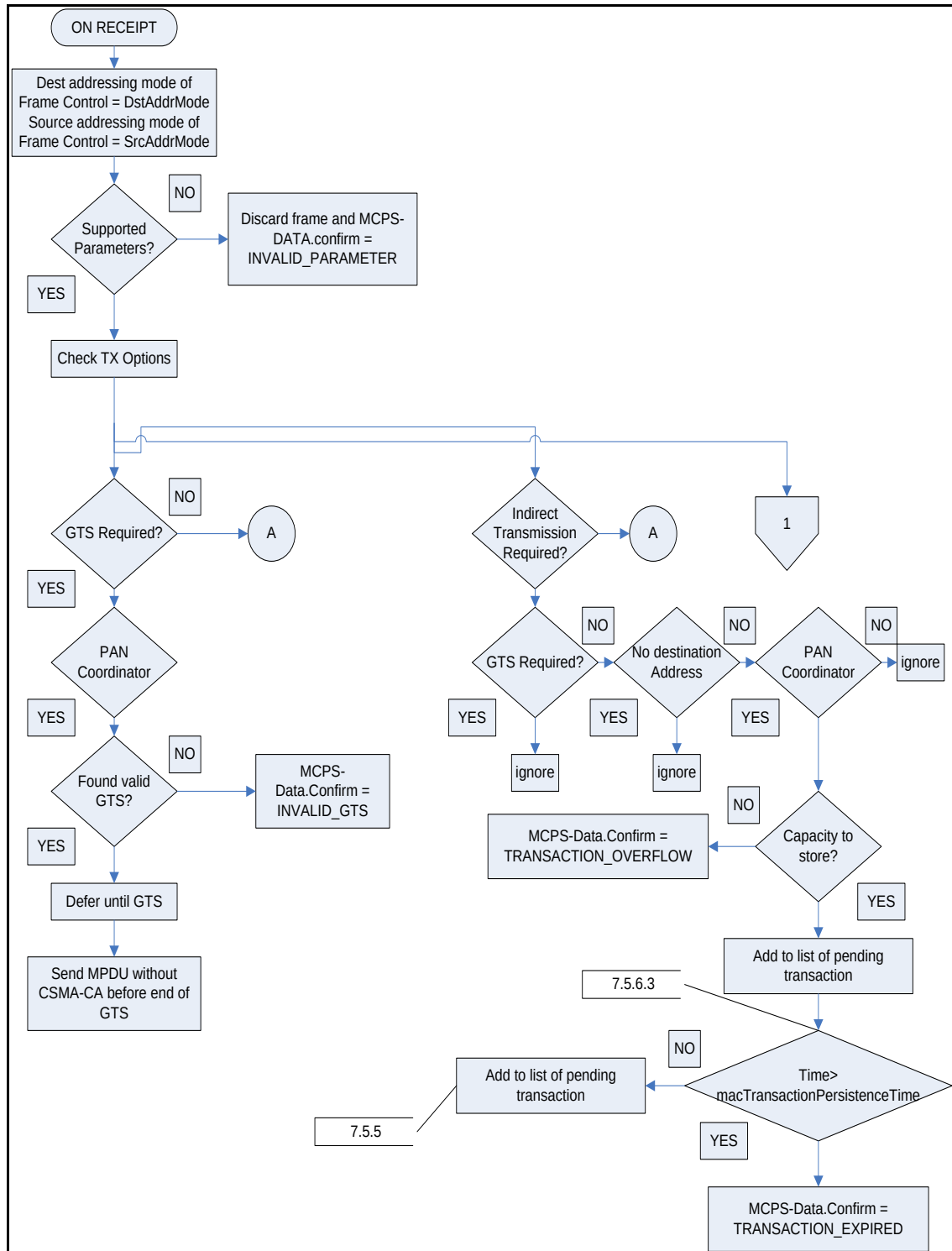


Figure 7. Flowchart of MCPS-Data service operation.

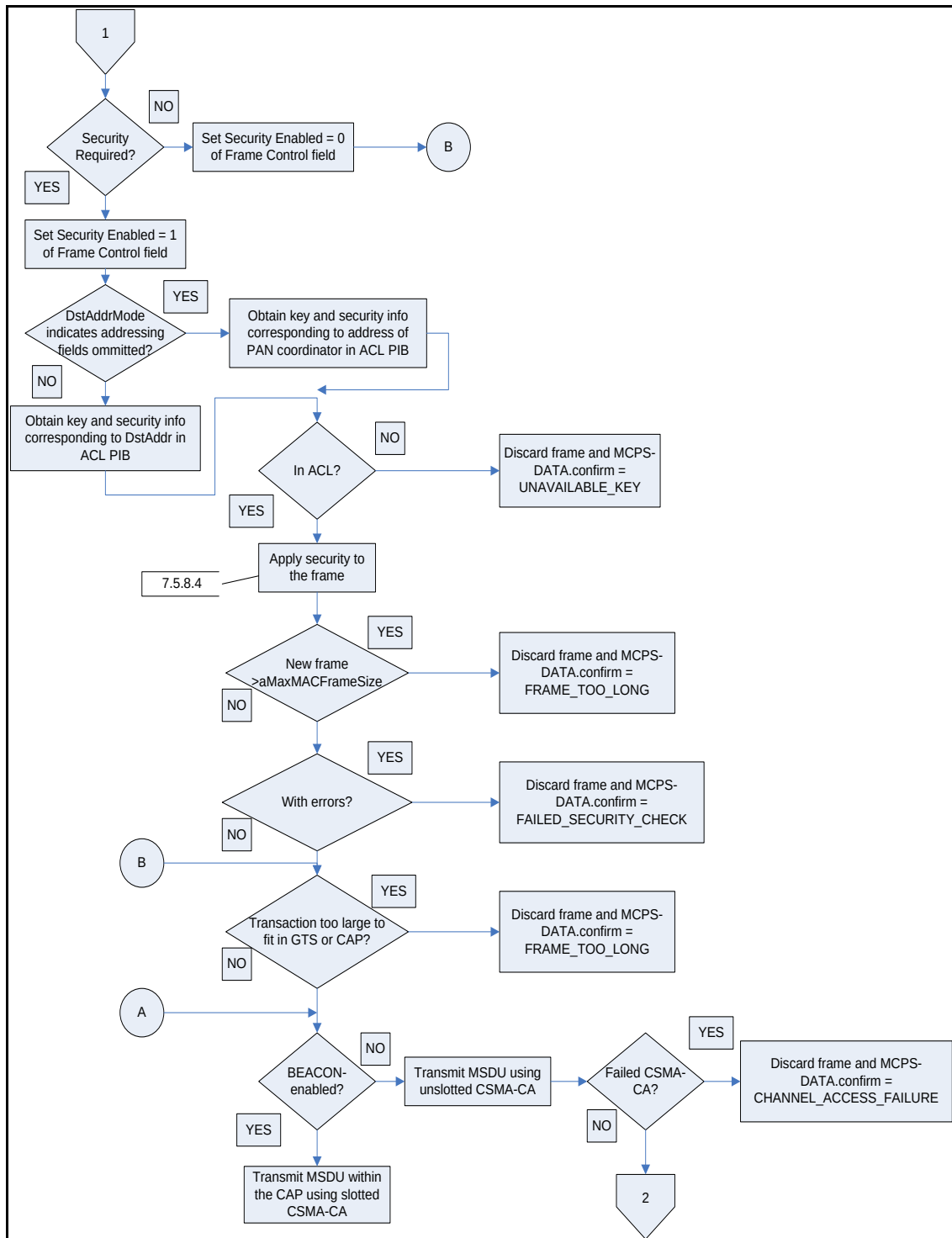


Figure 8. Flowchart of MCPS-Data service operation.

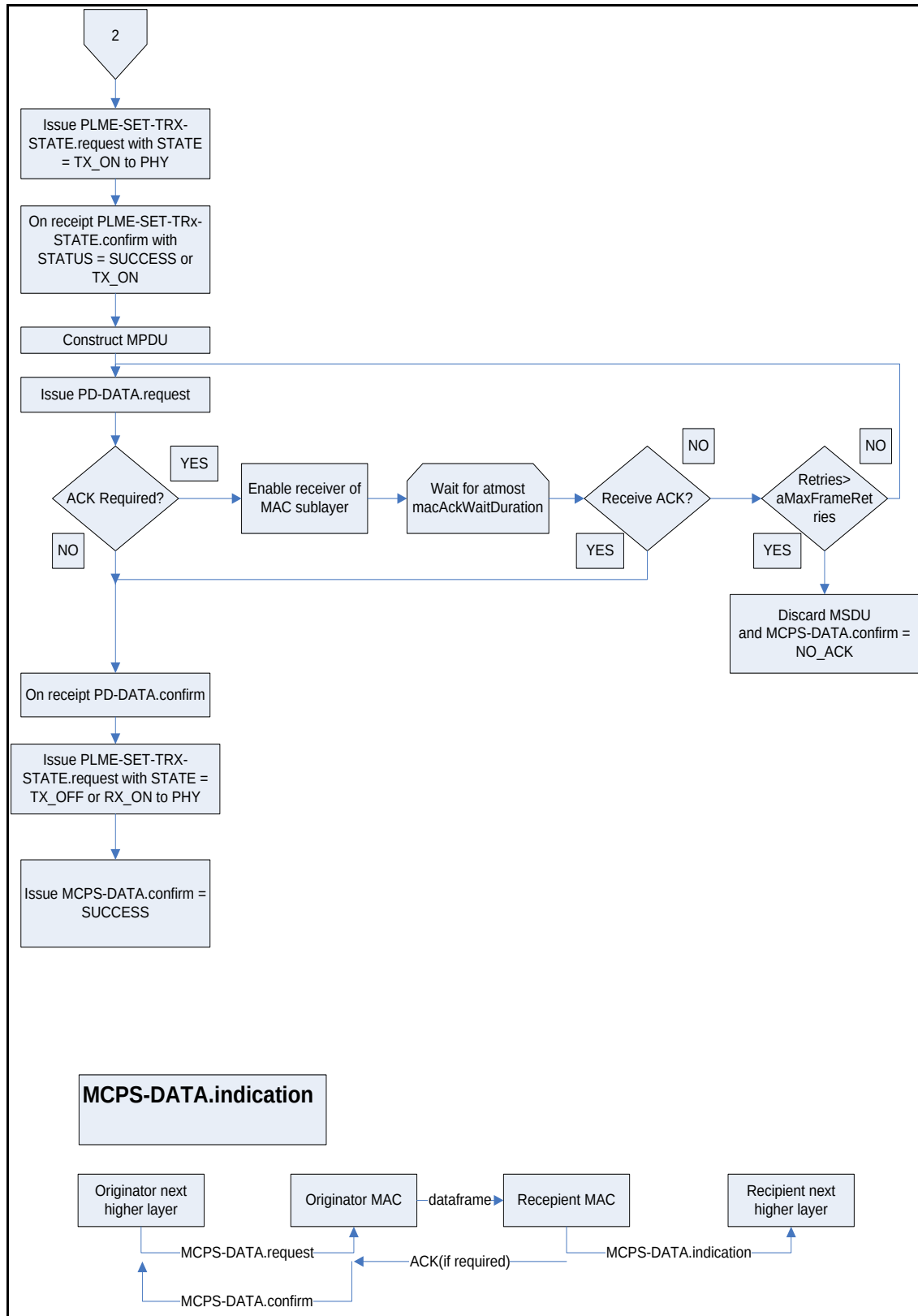


Figure 9. Flowchart of MCPS-Data service operation.

5.2.2 MCPS – Purge

- MCPS-PURGE.request primitive allows the next higher layer to purge an MSDU from the transaction queue.
- MCPS-PURGE.confirm primitive allows the MAC sublayer to notify the next higher layer of the success of its request to purge an MSDU from the transaction queue

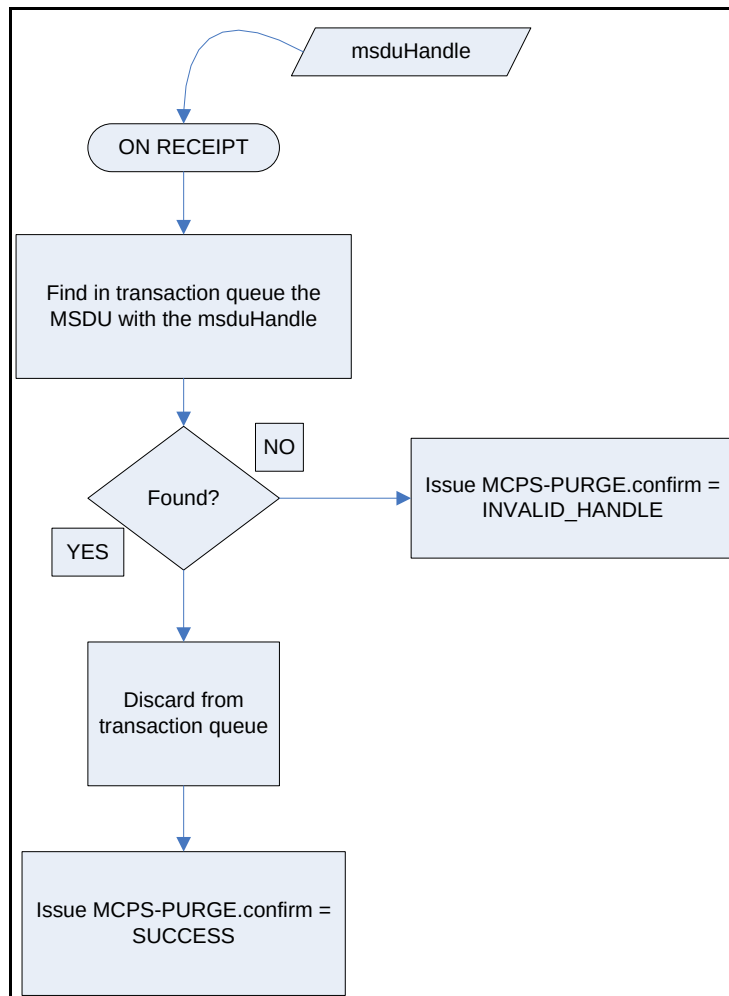


Figure 10. Flowchart of MCPS-Purge service operation.

5.3 MAC Management Service

The MLME –SAP allows the transport of management commands between the next higher layer and the MLME.

5.3.1 MLME-Reset

- MLME-RESET.request primitive allows the next higher layer to request that the MLME performs a reset operation
- MLME-RESET.confirm primitive reports the results of the reset operation

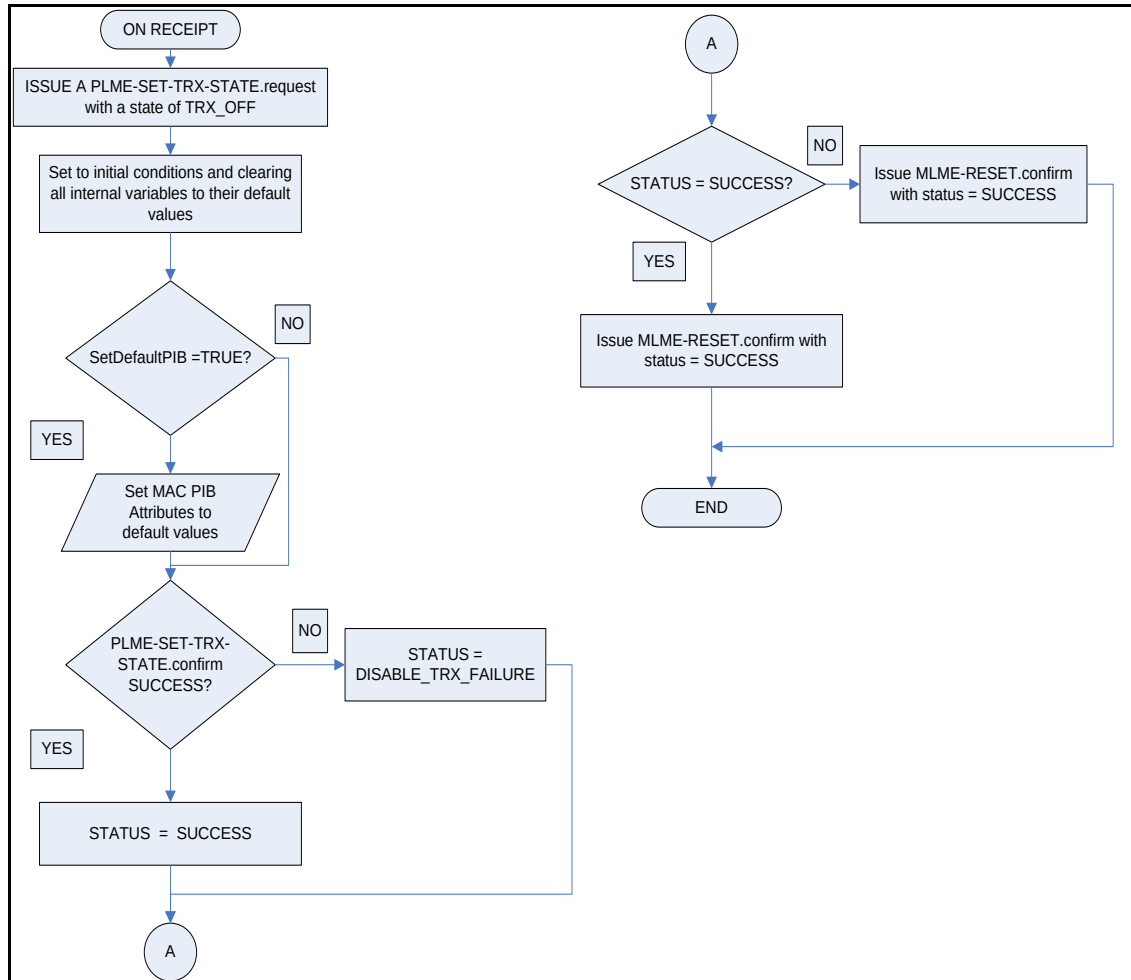


Figure 11. Flowchart of MLME-Reset service operation.

5.3.2 MLME-Beacon-Notify

The MLME-SAP beacon notification primitive defines how a device may be notified when a beacon is received during normal operating conditions.

- a. MLME-BEACON-NOTIFY.indication primitive is used to send parameters contained within a beacon frame received by the MAC sublayer to the next higher layer. The primitive also sends a measure of the LQ and the time the beacon frame was received.

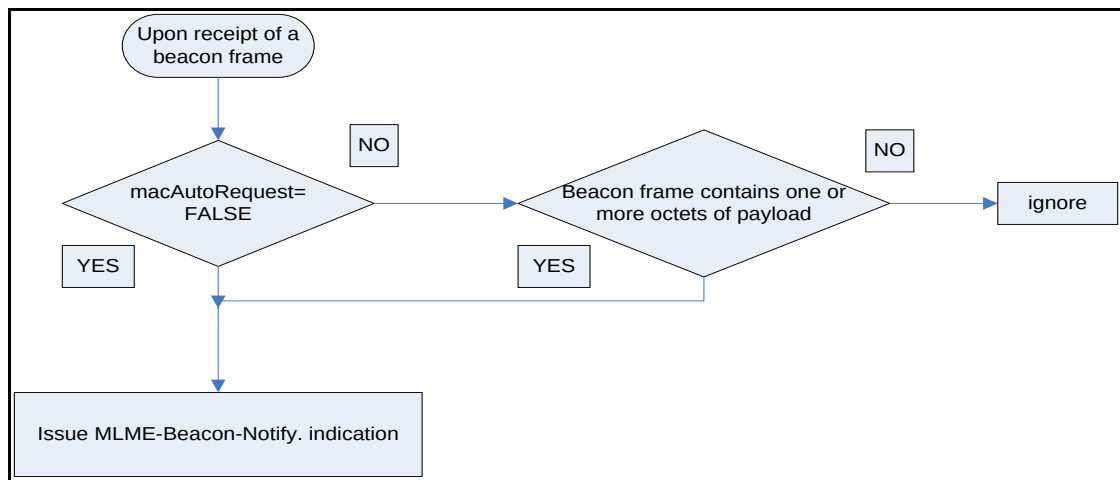


Figure 12. Flowchart of MLME-Beacon-Notify service operation.

5.3.3 MLME-GET

- The MLME-GET.request primitive requests information about a given PIB attribute.
- The MLME-GET.confirm primitive reports the results of an information request from the MAC PIB.

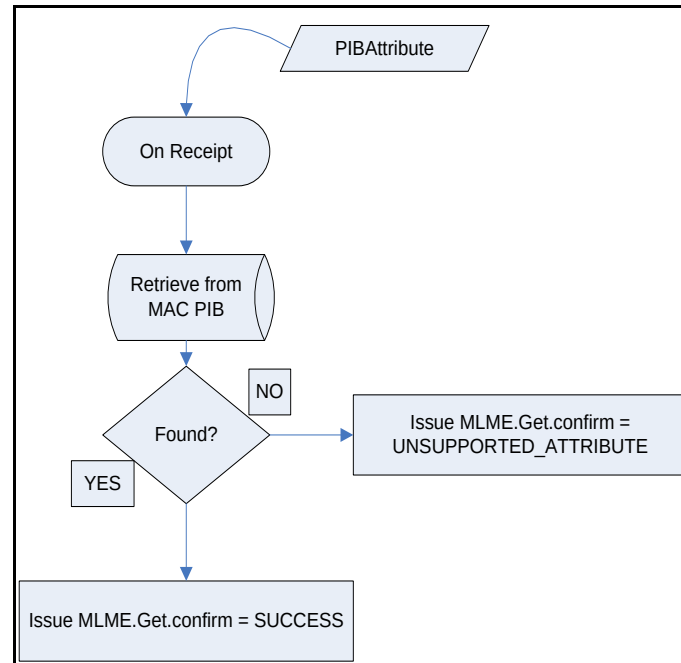


Figure 13.Flowchart of MLME-Get service operation.

5.3.4 MLME-SET

- The MLME-SET.request attempts to write the given value to the indicated MAC PIB attribute.
- MLME-SET.confirm primitive reports the results of an attempt to write a value to a MAC PIB attribute.

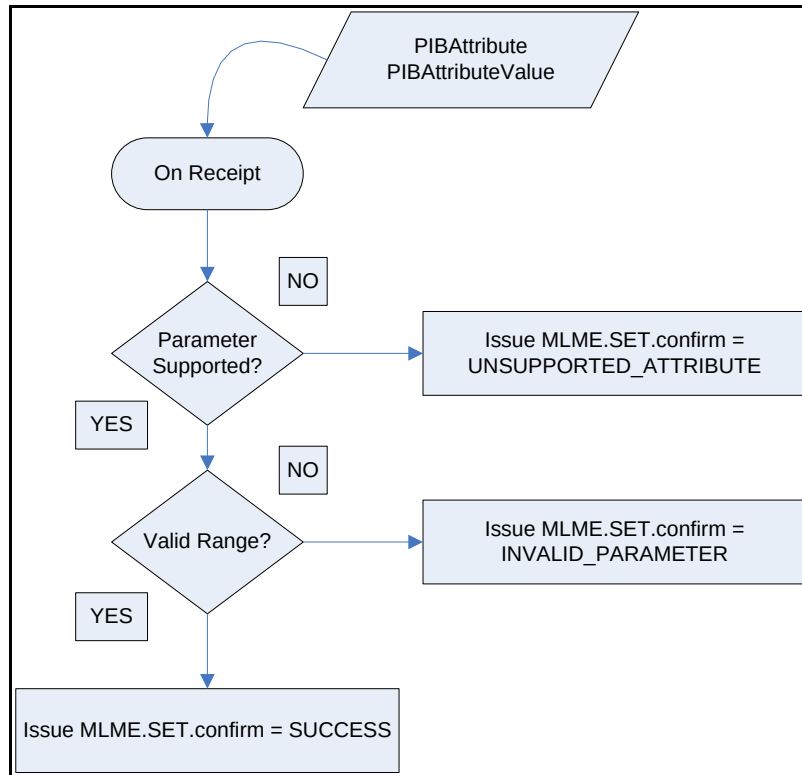


Figure 14. Flowchart of MLME-Set service operation

5.3.5 MLME-COMM-STATUS

The MLME-SAP communication status primitive defines how the MLME communicates to the next higher layer about transmission status, when the transmission was not instigated by a .request primitive and security errors on incoming packets.

- a. The MLME-COMM-STATUS.indication primitive allows the MLME to indicate a communication status.

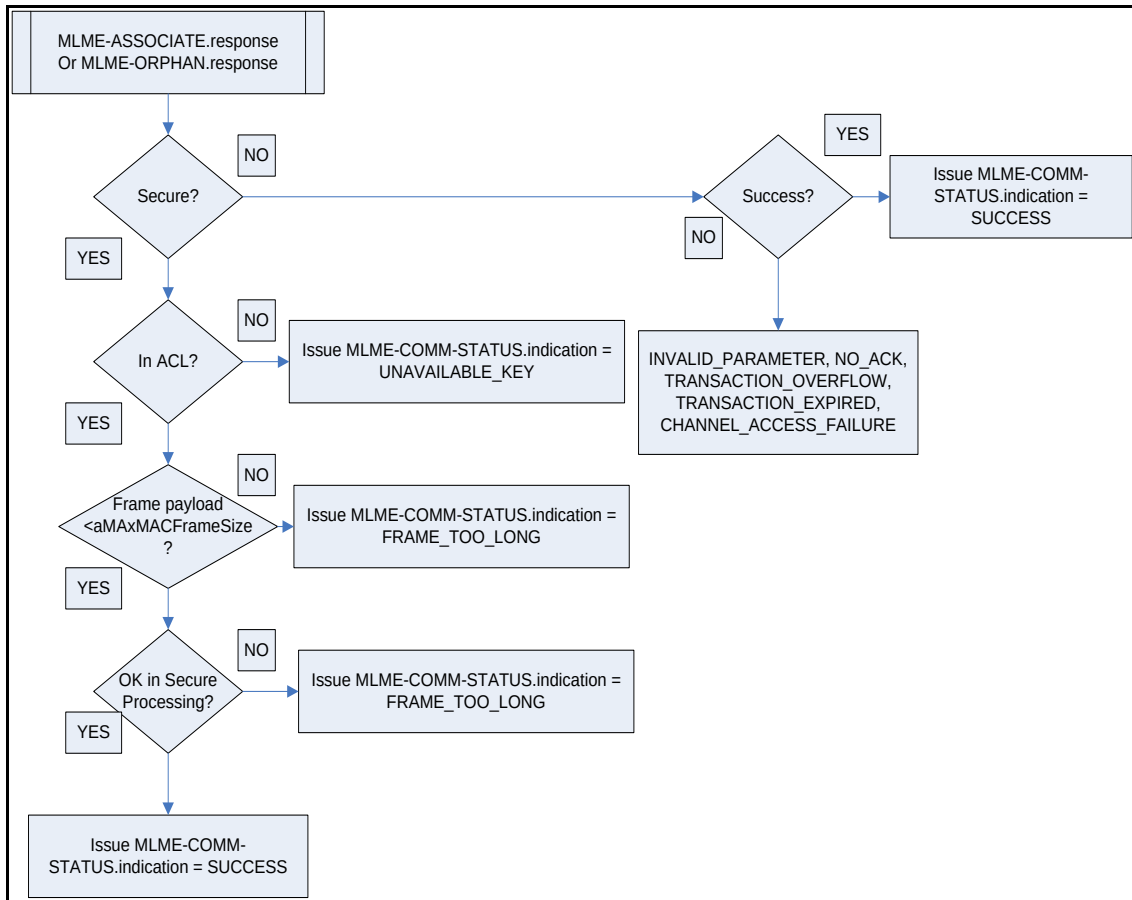


Figure 15. Flowchart of MLME-Comm-Status service operation

5.3.6 MLME-SYNC / MLME-SYNC-LOSS

MLME-SAP synchronization primitives define how synchronization with a coordinator may be achieved and how a loss of synchronization is communicated to the next higher layer.

- The MLME-SYNC.request primitive requests to synchronize with the coordinator by acquiring and if specified, tracking its beacons.
- The MLME-SYNC-LOSS.indication primitive indicates the loss of synchronization with a coordinator.

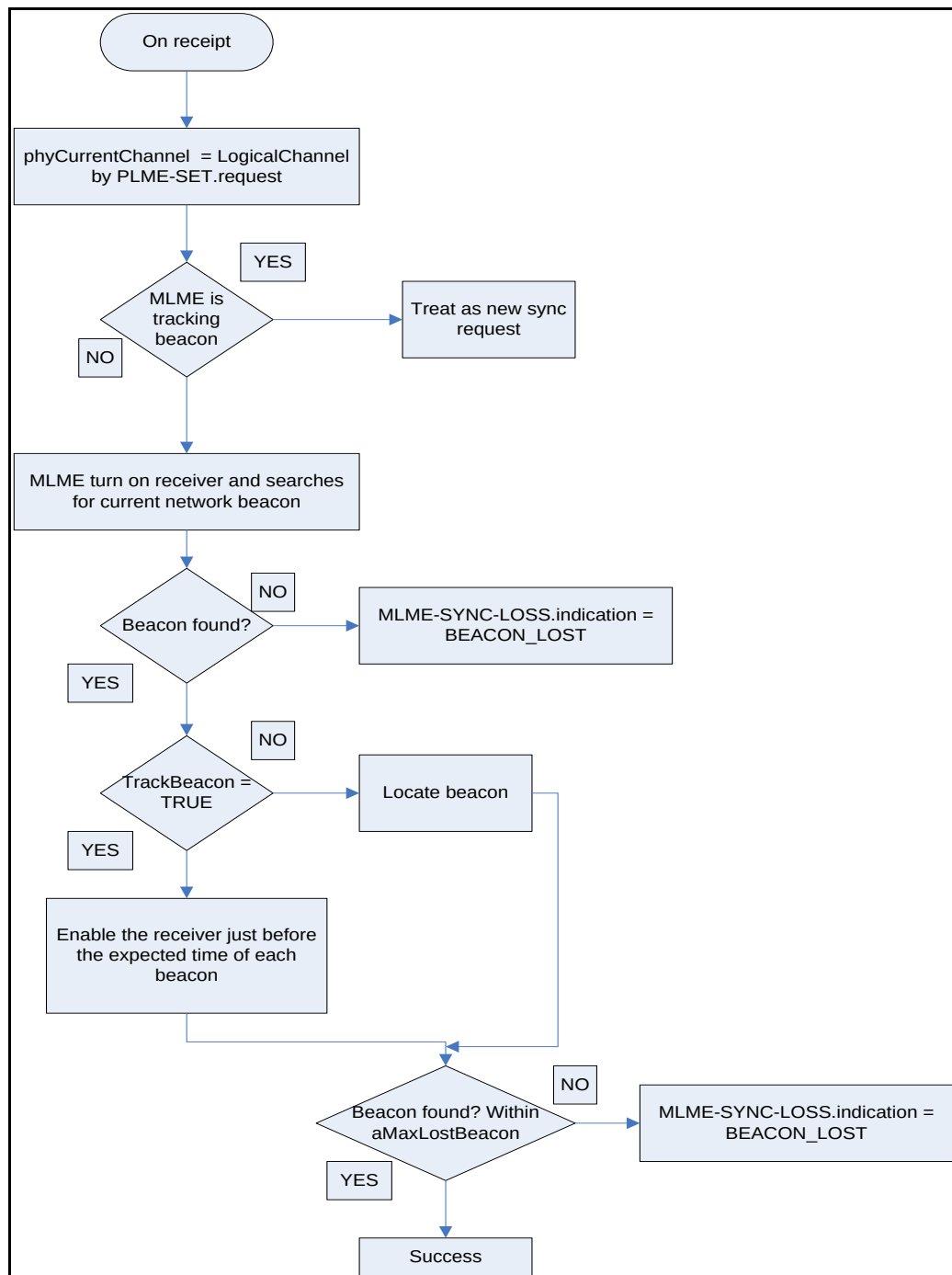


Figure 16. Flowchart of MLME-SYNC / MLME-SYNC-LOSS service operation

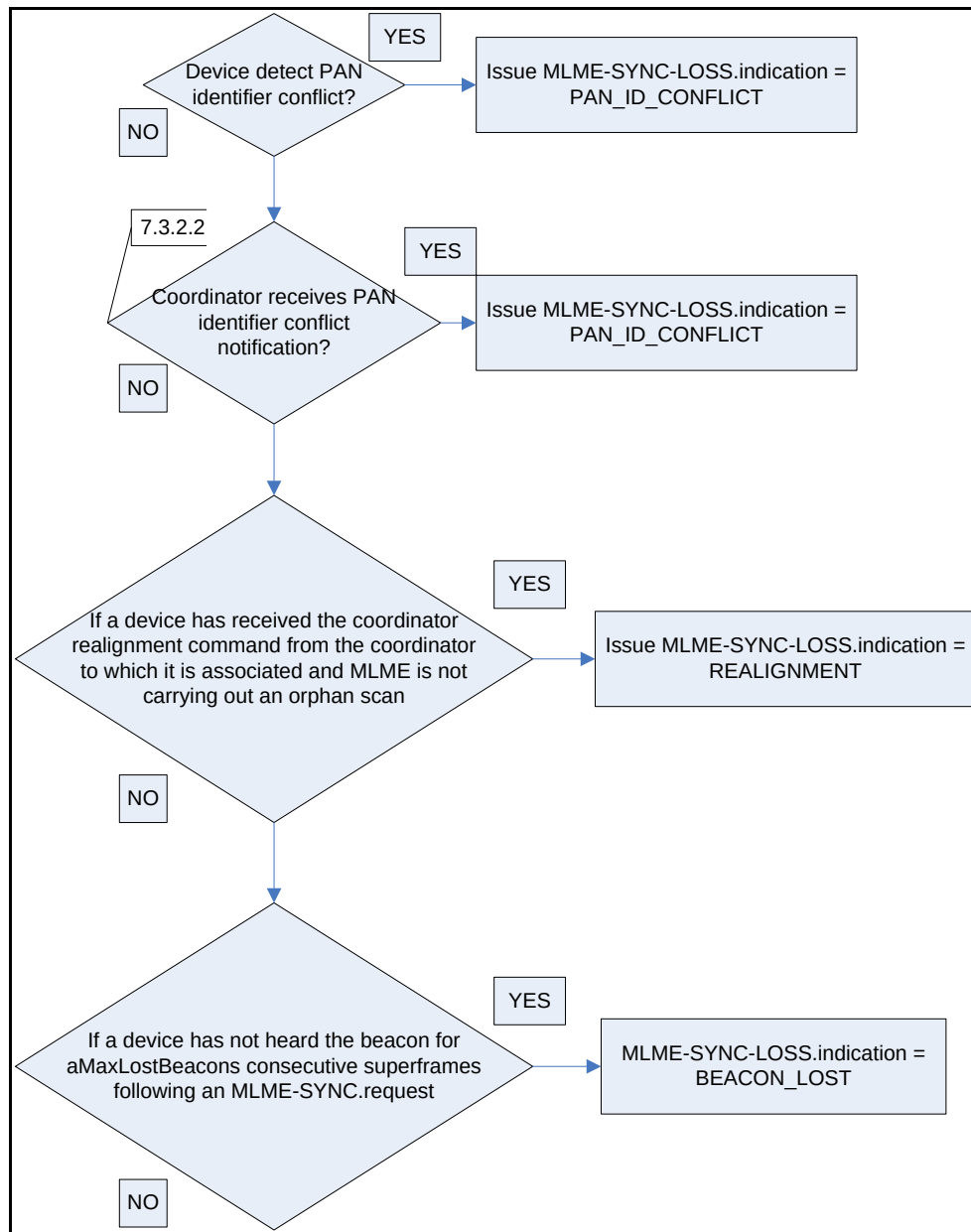


Figure 17. Flowchart of MLME-SYNC-LOSS service operation

5.3.7 MLME-POLL

MLME-SAP polling primitives define how to request data from a coordinator

- MLME-POLL.request primitive prompts the device to request data from the coordinator.
- MLME-POLL.confirm primitive reports the results of a request to poll the coordinator for data.

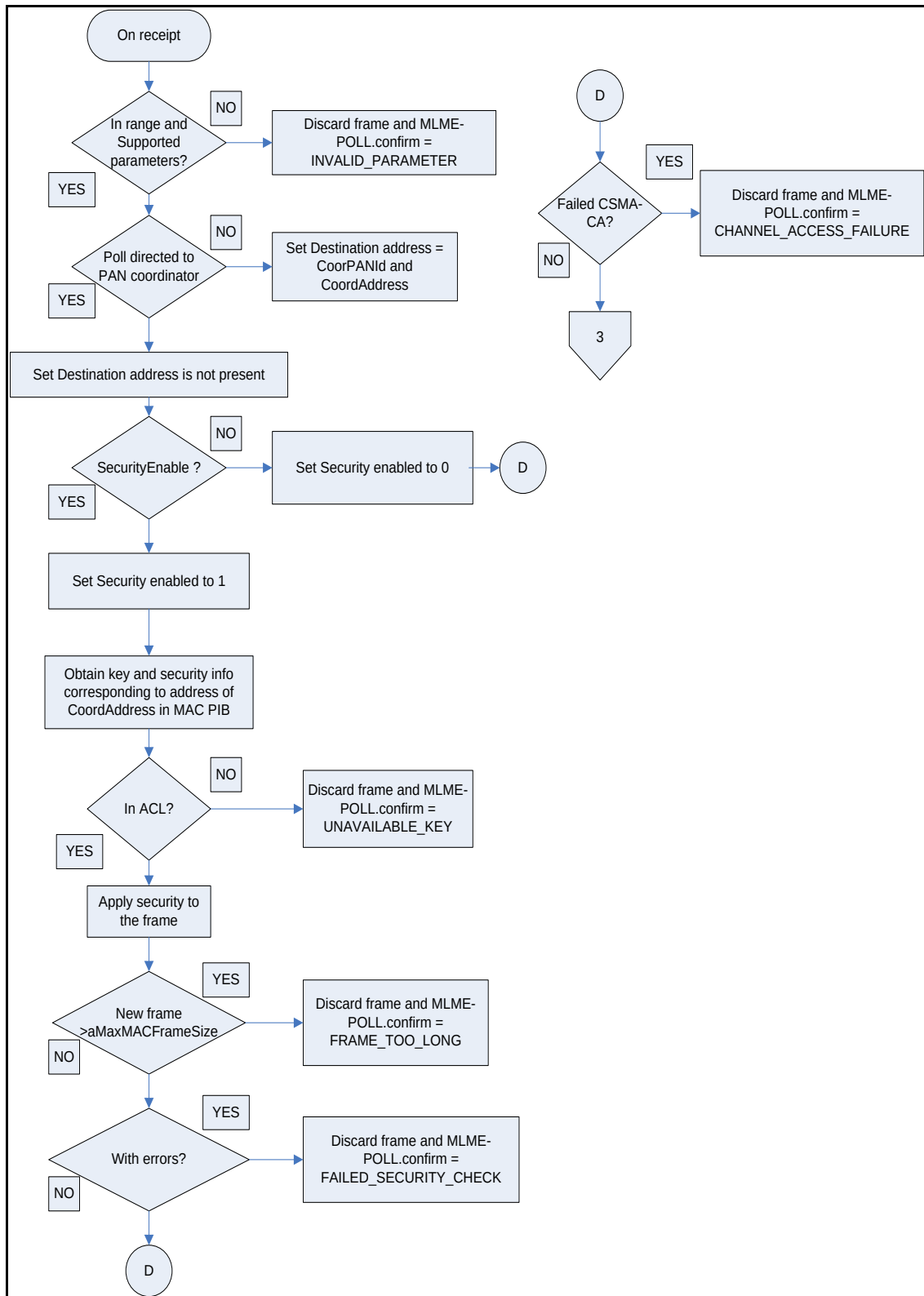


Figure 18. Flowchart of MLME-POLL service operation

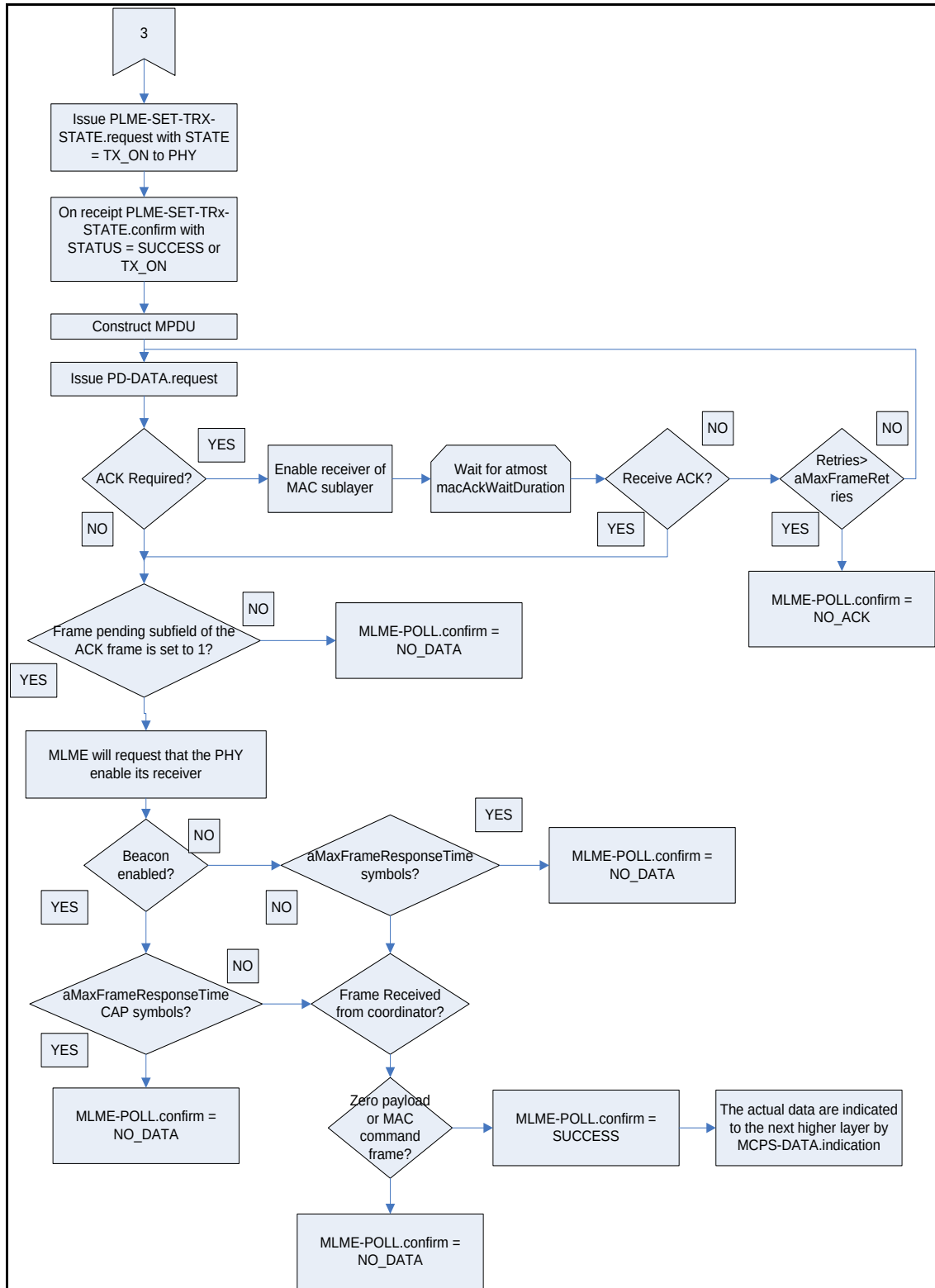


Figure 19. Flowchart of MLME-POLL service operation



5.3.8 MLME-START

MLME-SAP start primitives define how an FFD can request to start using a new superframe configuration in order to initiate a PAN, begin transmitting beacons on already existing PAN, facilitating device discovery or to stop transmitting beacons.

- a. The MLME-START.request primitive makes a request for the device to start using a new superframe configuration.
- b. The MLME-START.confirm primitive reports the results of the attempt to start using a new superframe configuration.

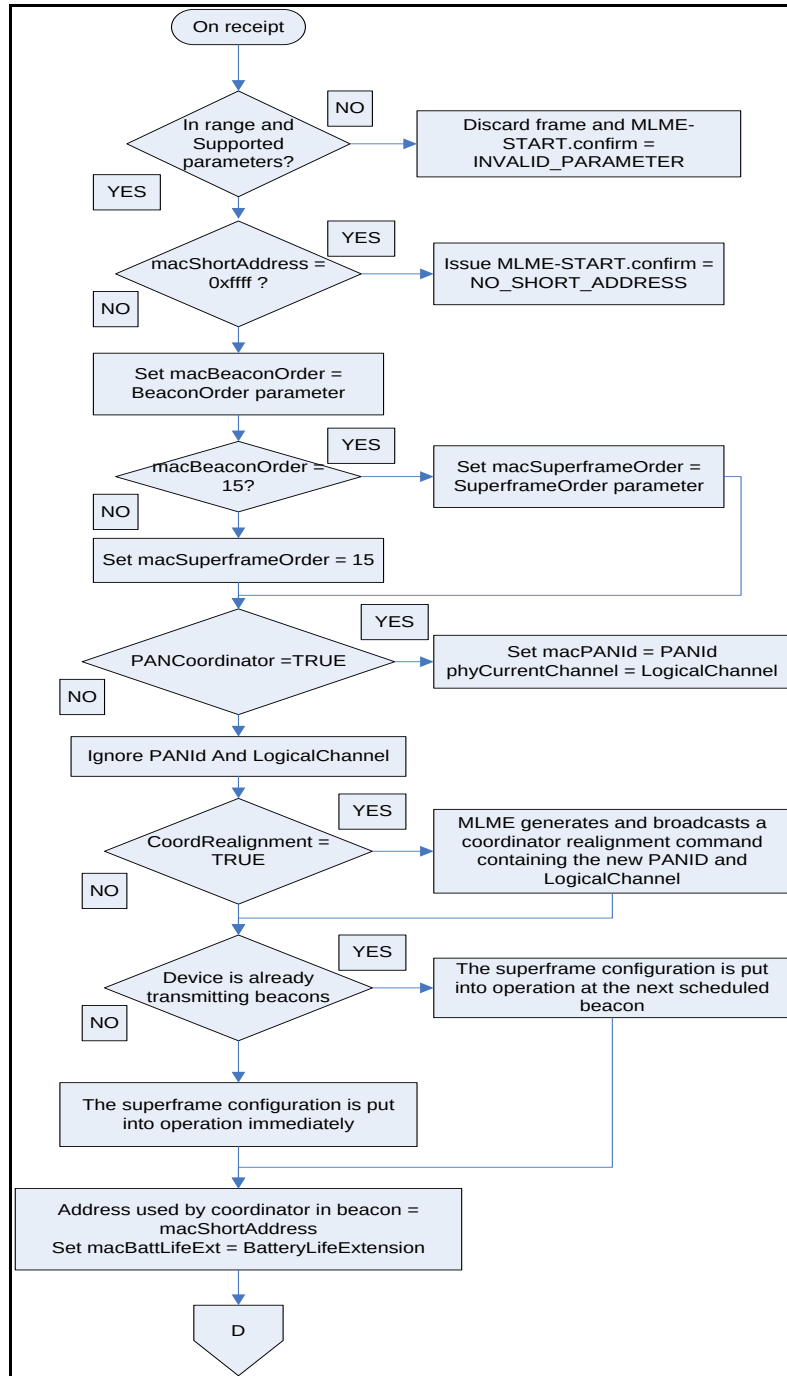


Figure 20. Flowchart of MLME-START service operation.

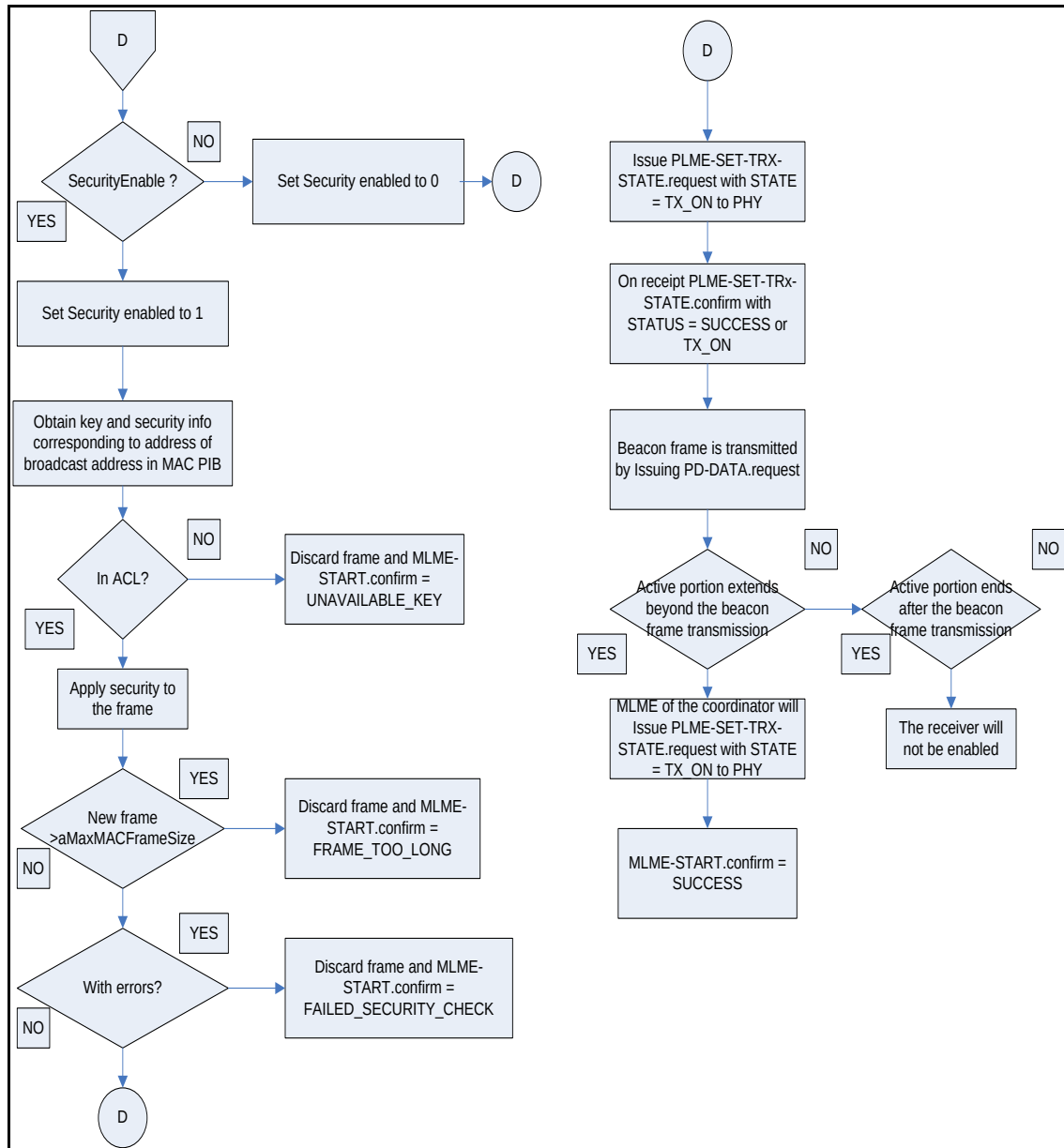


Fig. 11b Flowchart of MLME-START service operation.

5.3.9 MLME-RX-ENABLE

MLME-SAP receiver state primitives define how a device can enable or disable the receiver at a given time.

- The MLME-RX-ENABLE.request primitive allows the next higher layer to request that the receiver is enable for a finite period of time.
- The MLME-RX-RNABLE.confirm primitive reports the results of the attempt to enable the receiver.

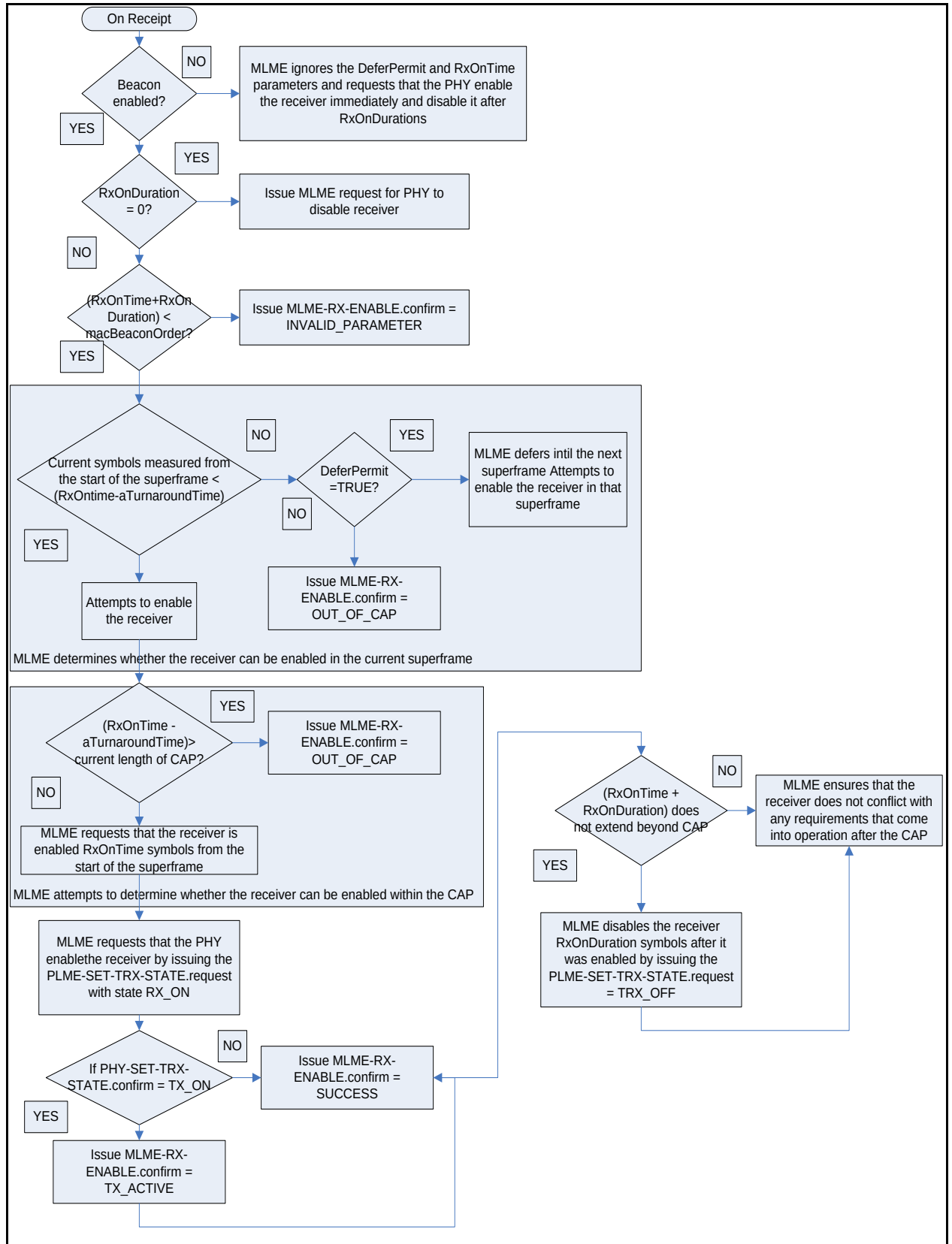


Figure 21. Flowchart of MLME-RX-ENABLE service operation.



5.3.10 MLME-SCAN

MLME-SAP scan primitives define how a device can determine the energy usage or the presence or absence of PANs in a communication channel.

- a. The MLME-SCAN.request primitive is used to initiate a channel scan over a given list of channels. A device can use a channel scan to measure the energy on the channel, search for the coordinator with which it associated or search for all coordinators transmitting beacon frames within the POS of the scanning device.
- b. The MLME-SCAN.confirm primitive reports the results of the channel scan request.

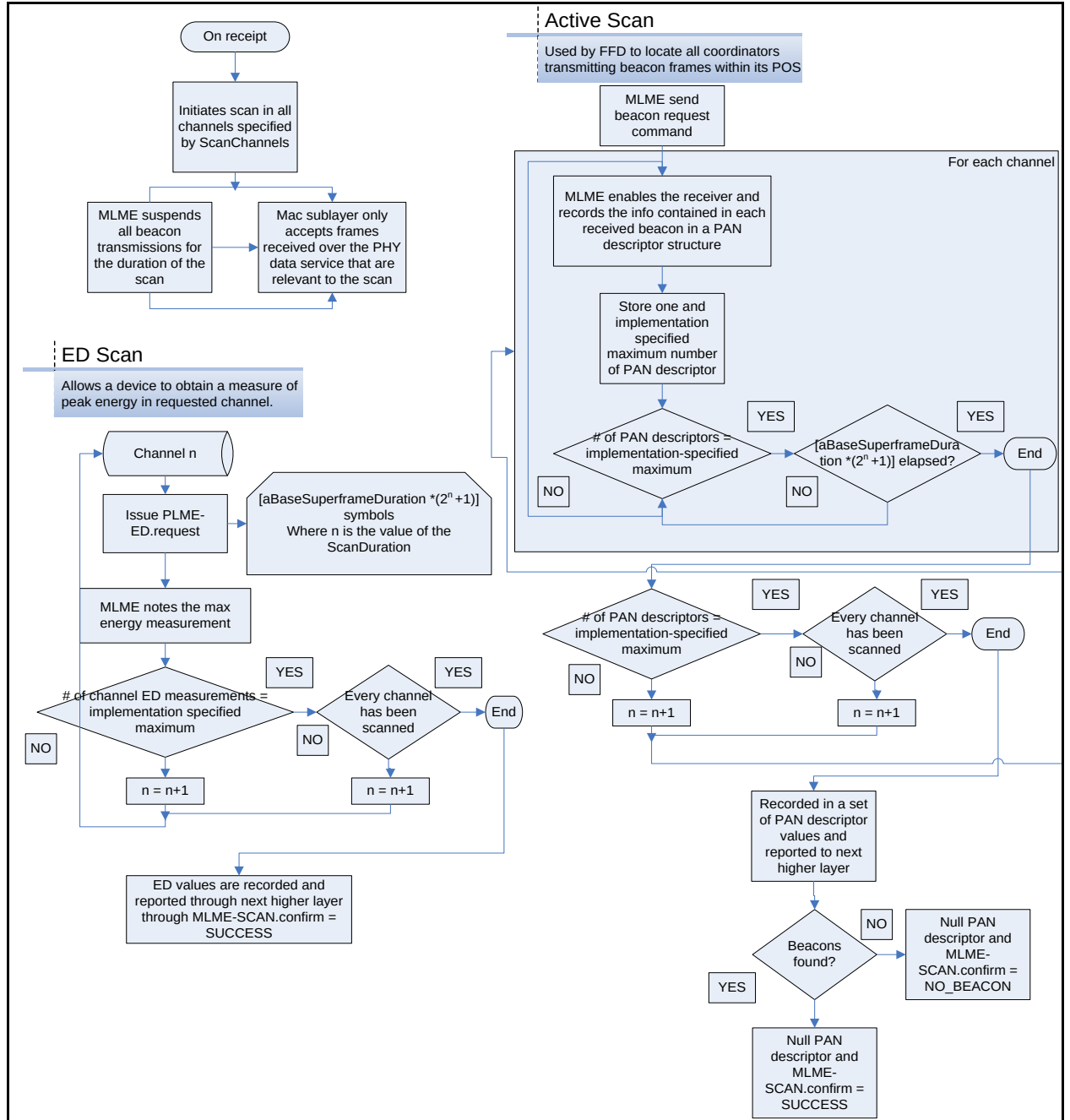


Figure 22. Flowchart of MLME-SCAN service operation.

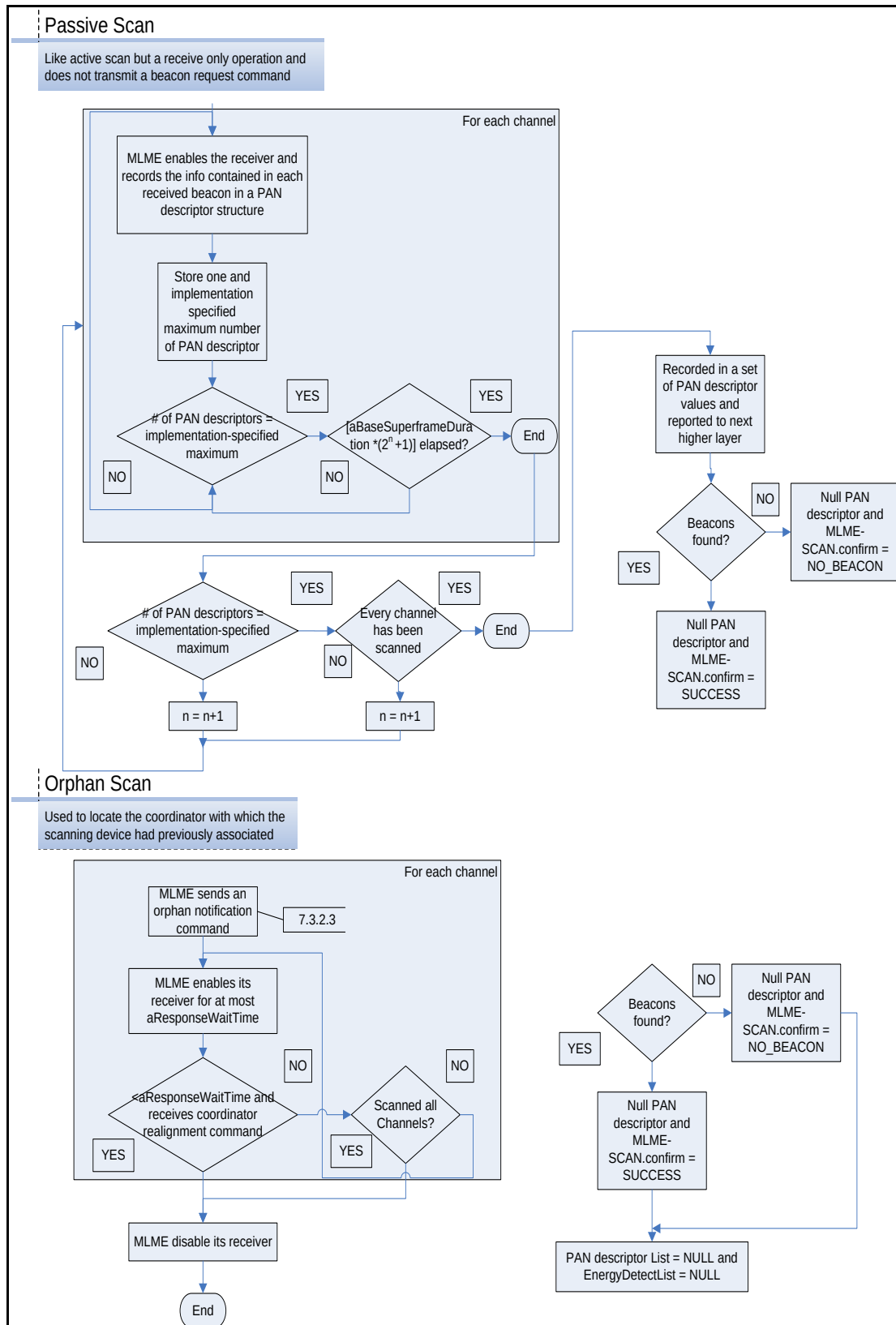


Figure 23. Flowchart of MLME-SCAN service operation.

5.3.11 MLME-ORPHAN

MLME-SAP orphan notification primitives define how a coordinator can issue a notification of an orphaned device.

- The MLME-ORPHAN.indication primitive allows the MLME of a coordinator to notify the next higher layer of the presence of an orphaned device.
- The MLME-ORPHAN.response primitive allows the next higher layer of a coordinator to respond to the MLME-ORPHAN.indication.primitive.

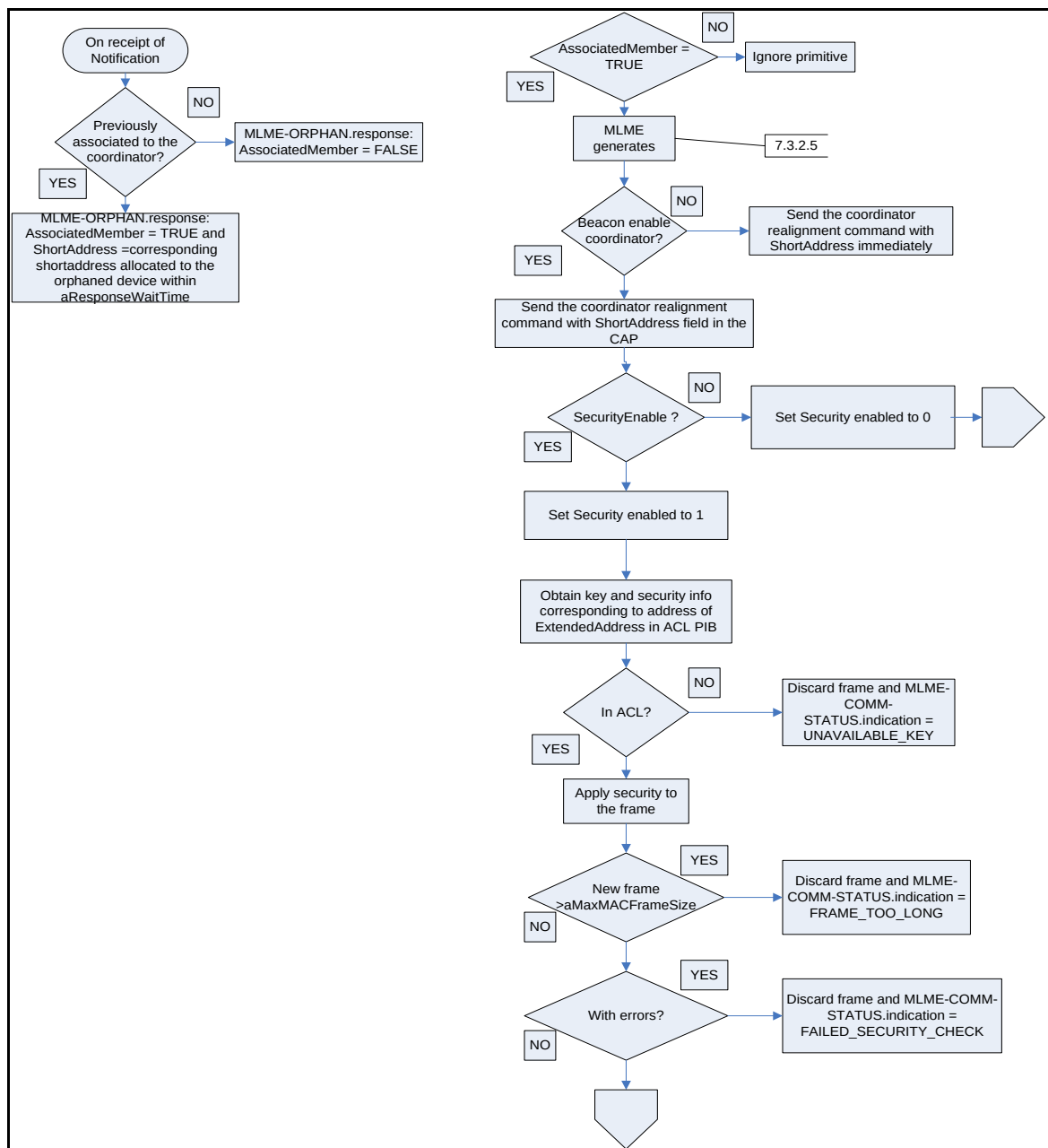


Figure 24. Flowchart of MLME-ORPHAN service operation

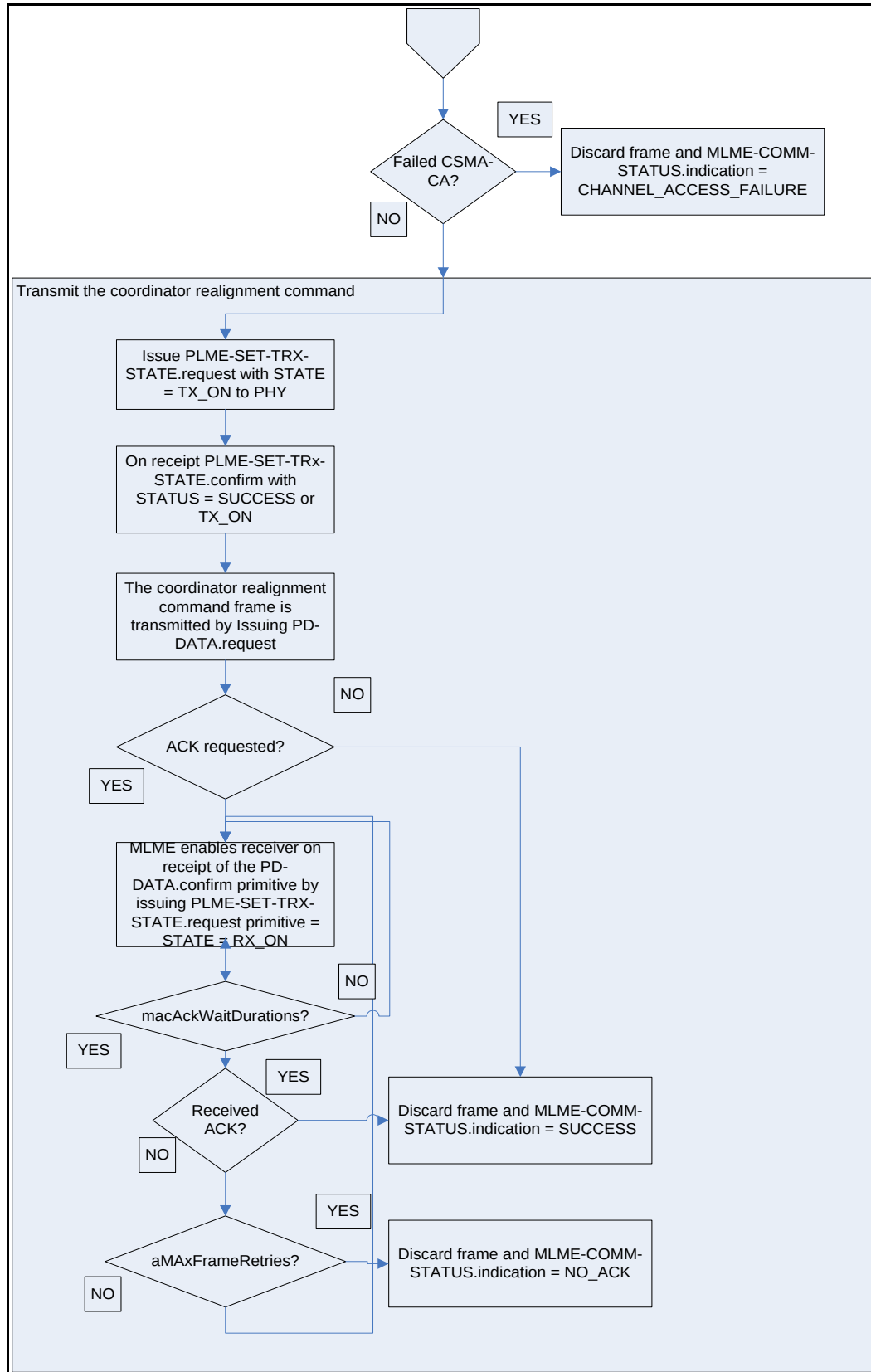


Figure 25. Flowchart of MLME-ORPHAN service operation

5.3.12 MLME-ASSOCIATE

MLME-SAP Association primitives define how a device becomes associated with a PAN.

- MLME-ASSOCIATE.request primitive allows a device to request an association with a coordinator.
- MLME-ASSOCIATE.indication primitive is used to indicate the reception of an association request command.
- MLME-ASSOCIATE.response primitive is used to initiate a response to MLME-ASSOCIATE.indication primitive
- MLME-ASSOCIATE.confirm primitive is used to inform the next higher layer of the initiating device whether its request to associate was successful or unsuccessful.

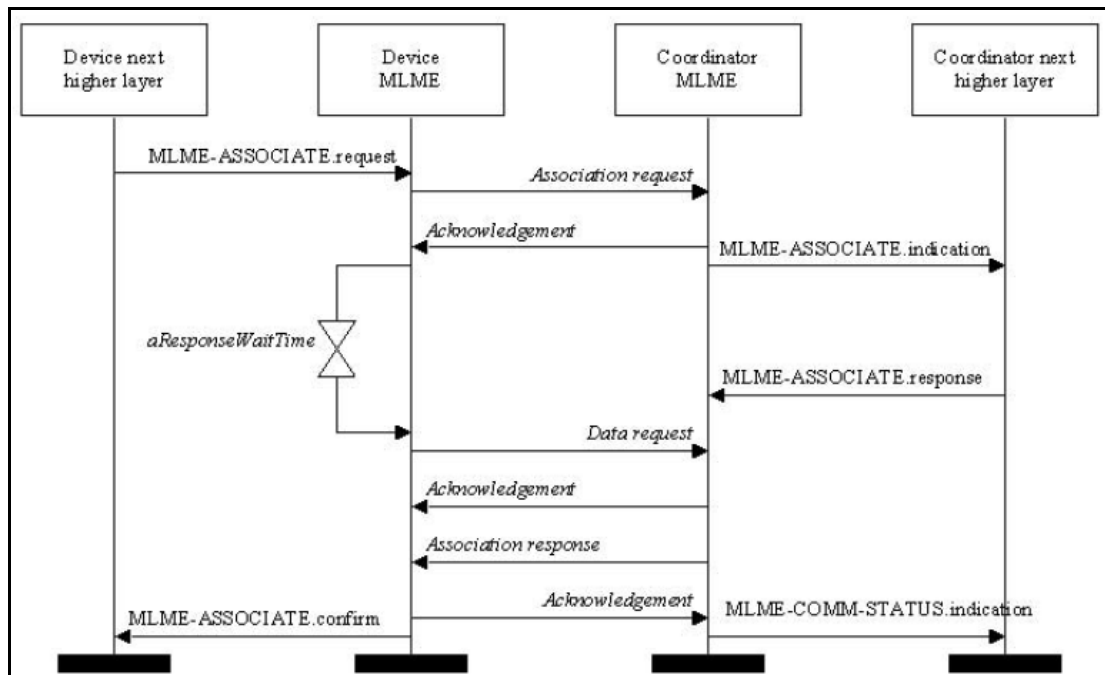


Figure 26. Message sequence chart of MLME-Associate service[1].

5.3.13 MLME-DISASSOCIATE

MLME-SAP Disassociation primitives define how a device can disassociate from a PAN.

- MLME-DISASSOCIATE.request primitive is used by an associated device to notify the coordinator of its intent to leave the PAN. It is also used by the coordinator to instruct an associated device to leave the PAN.
- MLME-DISASSOCIATE.indication primitive is used to indicate the reception of a disassociation notification command.
- MLME-DISASSOCIATE.confirm primitive reports the results of an MLME-DISASSOCIATE.request primitive

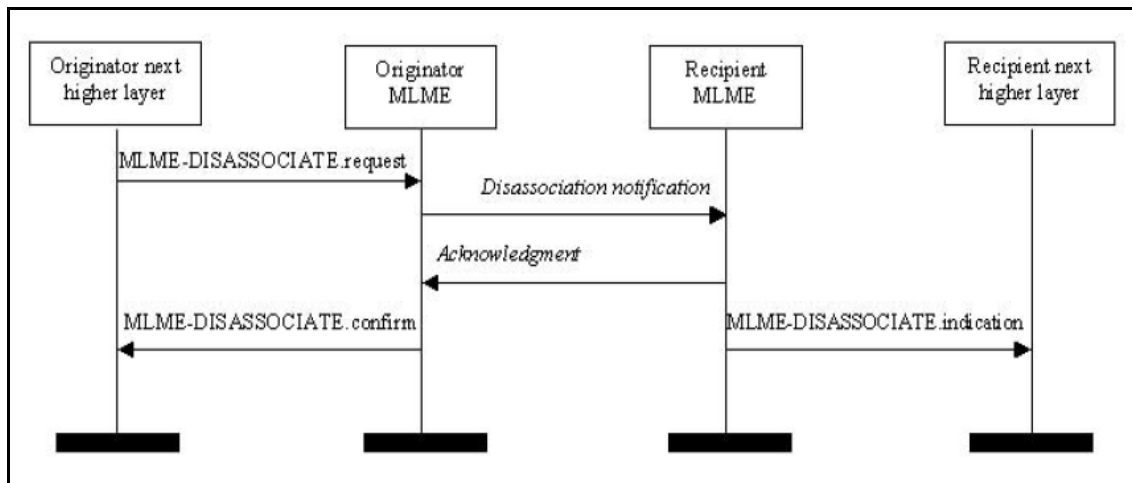


Figure 27. Message sequence chart of MLME-Disassociate service.

6 NWK Layer Design and Primitives

The NWK layer defines a functionality to ensure correct operation of IEEE 802.15.4-2003 MAC sub-layer and provide a suitable service interface to the application layer. The NWK layer is implemented /Dev/NWK. The

File Name	Description
nwk_routing.c	Implements routing in the network layer
nwk_routing.h	Type definitions for routing functions
data_queue.c	Handles the data queue to be used in NWK and APS
data_queue.h	Type definitions for the above functions
nwk.c	Implements intertask communication mechanism that conformed with the scheduler mechanism detailed in AS-OS
nwk.h	Type definitions for the above functions
nwk_common.c	Functions shared among NWK functions

nwk_common.h	Type definitions for the above functions
nwk_data.c	Data handler file (tx, reayl and broadcast)
nwk_data.h	Type definitions for the above functions
nwk_msg.h	Define the msg_type and msg_format for the event in the NWK layer
nwk_nlme.c	Implements the nwk management entity of the networ

Table 8. Source files for NWK

6.1 Network Layer Data Entity (NLDE)

The NLDE shall provide a data service to allow an application to transport application protocol data units between two or more devices. The devices themselves must be located on the same network. The NLDE will provide the following services;

- *Generation of the Network Level (NPDU).* The NLDE shall be capable of generating an NPDU from an application support sub-layer PDU through the addition of an appropriate protocol header.
- *Topology specific routing.* The NLDE shall be able to transmit an NPDU to an appropriate device that is either a final destination of the communication or the next step towards the final destination of the communication chain.

6.1.1 NWK Data Service

The NWK layer data entity SAP supports the transport of application protocol data units between peer application entities.

6.2 Network Layer Management Entity (NLME)

The NLME shall provide a management service to allow an application to interact with the stack. The NLME shall provide the following services:

- *Configuring a new device.* The ability to sufficiently configure the stack for operation as required. Configuration options include beginning operation as a ZigBee coordinator or joining an existent network
- *Starting a network.* The ability to establish a new network.
- *Joining and leaving a network.* The ability to join or leave a network
- *Addressing:* Zigbee coordinators and routers has ability to assign addresses to deviced joining the network.

- *Neighbor discovery.* The ability to discover, record, and report information pertaining to the one-hop neighbors of the device
- *Route Discovery.* The ability to discover and record paths through the network whereby messages may be efficiently routed.
- *Reception control.* The ability of a device to control when the receiver is activated and for how long, enabling MAC sub-layer synchronization or direct reception.

6.2.1 NLME Network Discovery

The NWK layer management entity SAP (NLME-SAP) supports the discovery of operating networks.

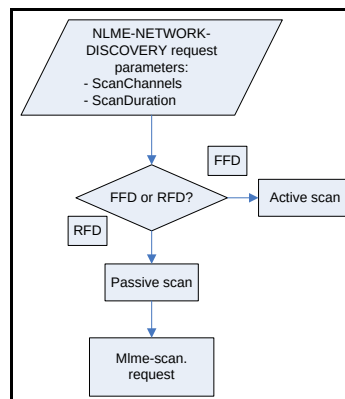


Figure 28. NLME Network Discovery Request.

6.2.2 NLME Join

This set of primitives defines how the next higher layer of a device can:

- o request to join a network through association
- o request to join a network directly
- o request to re-join a network if orphaned

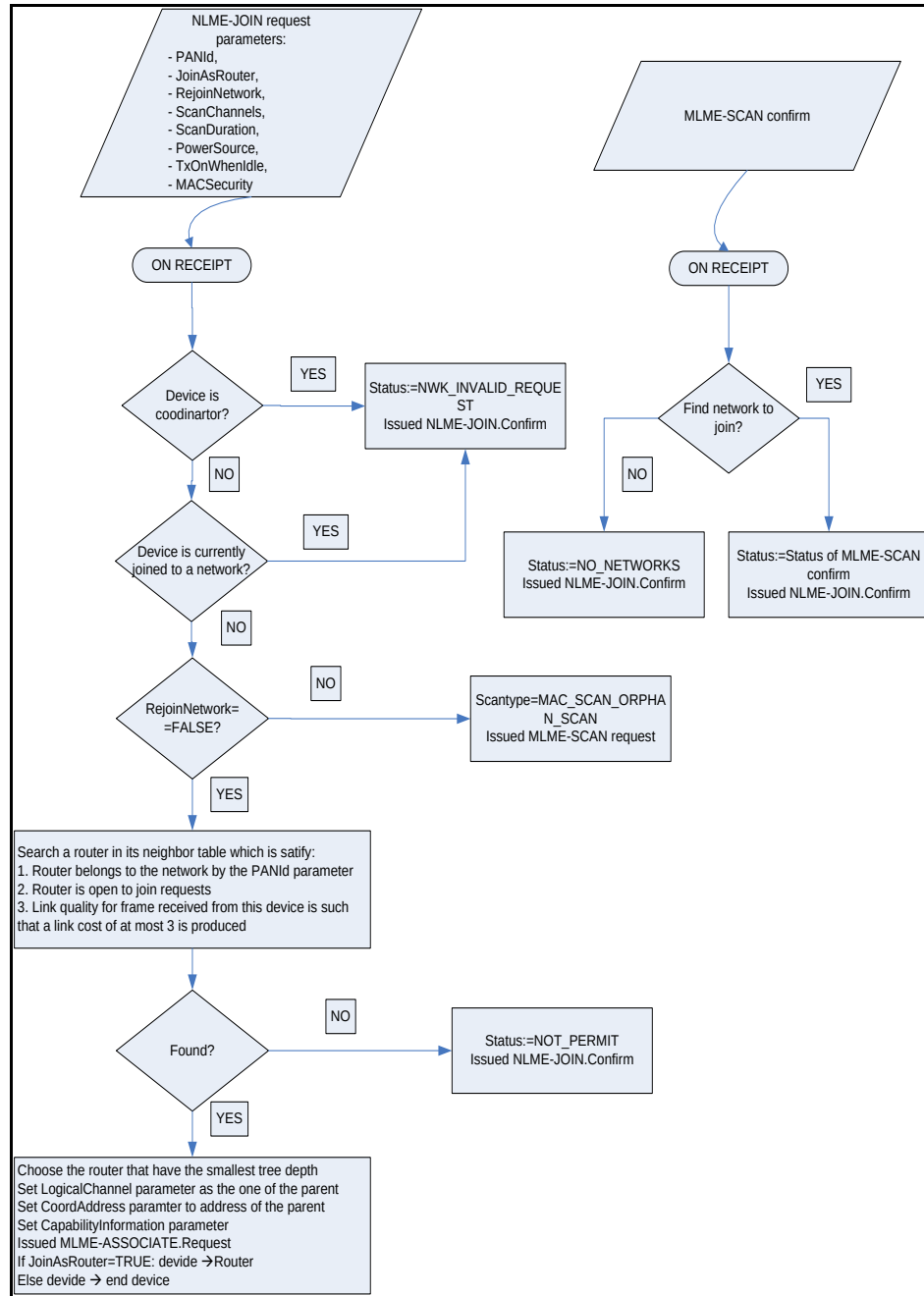


Figure 29. NLME Join Request.

6.2.3 NLME Reset

This primitive defines how the next higher layer of a device can request that the NWK layer is reset.

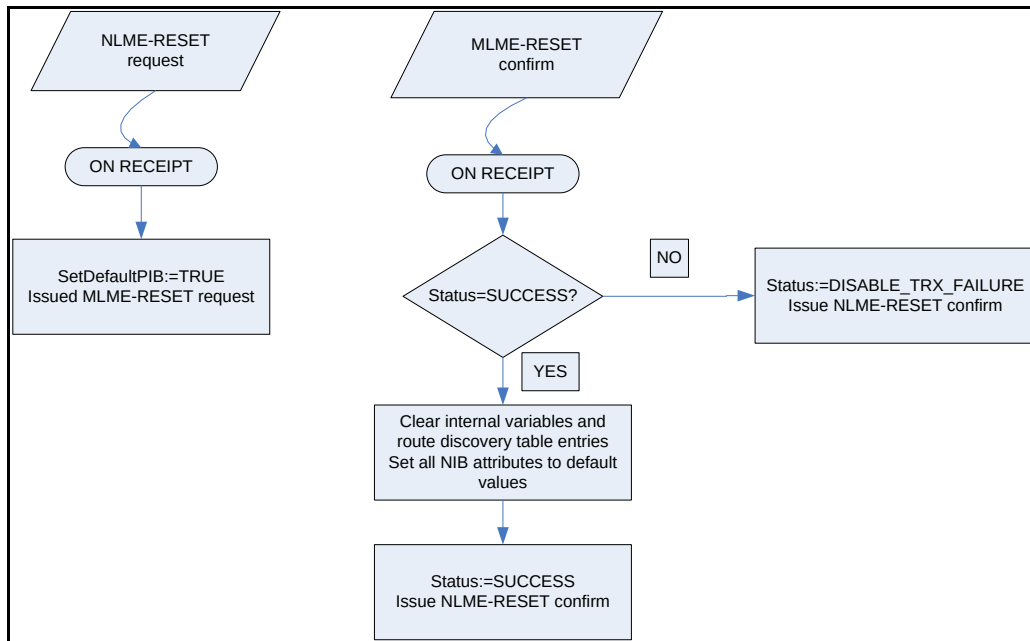


Figure 30. NLME Reset Request.

6.2.5 NLME Get

This primitive defines how the next higher layer of a device can read attributes in the NIB.

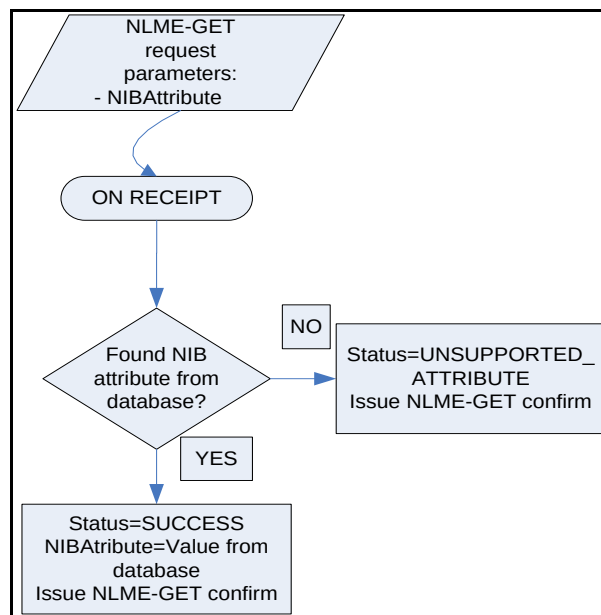


Figure 31. NLME Get Request.

6.2.6 NLME Set

This primitive defines how the next higher layer of a device can write attributes in the Network Information Base.

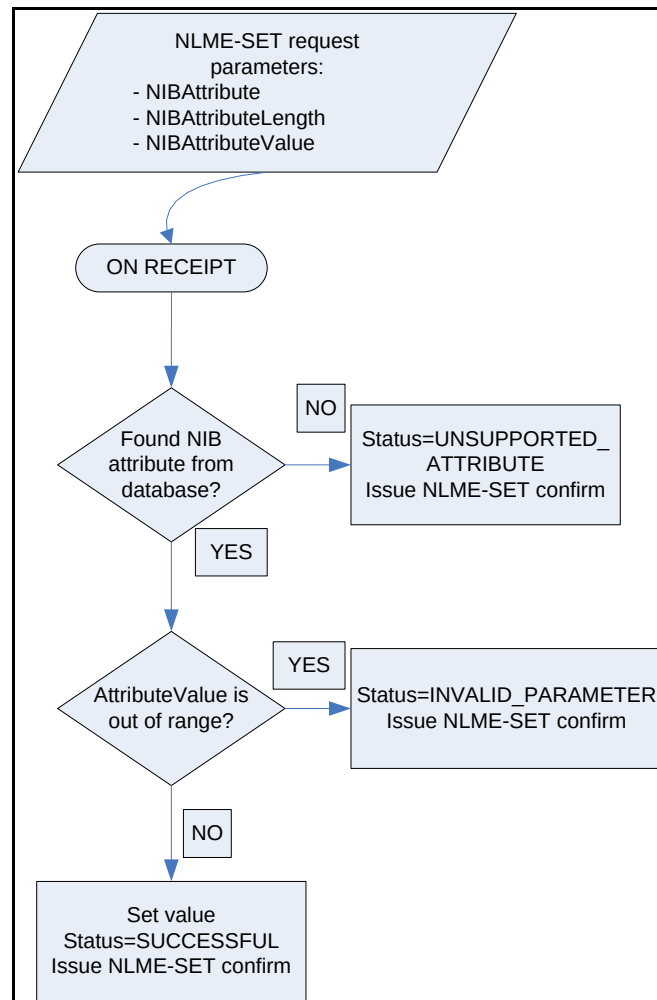


Figure 32. NLME Set Request.

6.2.7 Addressing – Address Allocation

Addressing functionality are specific for coordinator and router. This function is fully describe in the Zigbee Standard. Zigbee Addressing allocation in the distributed addresss scheme are calculated basing on the network tree and the depth of that tree. Given values for the maximum number of children a parent may have, $nkwMaxChildrent (C_m)$, the maximum demth in the network, $nwkMaxDepth (L_m)$, and the maximnum number of routers a parent may have (R_m). $Cskip(d)$ are the size of the address sub-block being distributed by each parent at its depth d .

$$Cskip(d) = \begin{cases} 1 + Cm.(L, -d - 1), & \text{if } Rm = 1 \\ \frac{1 + Cm - Rm - Cm.Rm^{Lm-d-1}}{1 - Rm} & \text{otherwise} \end{cases}$$

Network address shall be assigned to endevices in a sequential manner with the n^{th} address, $A_n = A_{\text{parent}} + Cskip(d).Rm + n$ where $1 \leq n \leq (Cm - Rm)$ and A_{parent} represent the address of the parent.

6.2.8 Route Discovery

Route Discovery function in Zigbee network use tree routing procedure. Tree routing procedure base on the address tree of network then it can only on distriuted addressing scheme.

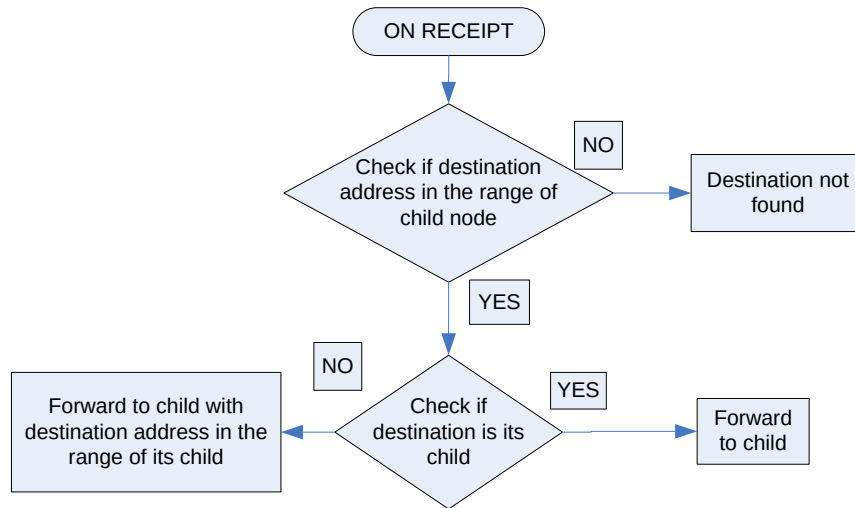


Figure 33. Algorithm of rout discovery

7 Upper Layer

The upper layers are composed of 4 components: Zigbee profile, ZDO, and application support sublayer, application framework.

7.1 Zigbee Device Object

Zigbee Device Object is a portion of application layer responsible for defining the role of the device within the network, initiating and/or responding to binding and discovery requests and establishing a secure relationship between network devices.

File Name	Description
zdo_app_interface.c	Implements the interface of event from ZDO and application
zdo_app_interface.h	Type definition for above
zdo.c	Implements the behaviors of the ZDO of Coordinator, Router or End Device
zdo.h	Type definition for above

Table 9. Zigbee device Object

7.2 Zigbee Profile

A profile is a description which together forms a cooperative application.

File Name	Description
zdp.c	ZDP functionality of Zigbee stack
zdp.h	Type definition for above

Table 9. Zigbee Profile

7.3 Application Support Sub-Layer

This layer provides an interface between the network layer and the application layer through a general set of services that are used by both the ZDO and manufacturer defined application objects.

File Name	Description
aps_ib.c	implement the internal information block of APS
aps_ib.h	Type definition for above
aps.c	Implements the Application Data Entity Primitive
aps.h	Type definition for above

Table 10. Application Support Sub-Layer

7.4 Application Framework

The application framework is an environment in which application objects are hosted in zigbee devices.

File Name	Description
apf_format.c	Implements the format of kvp/msg
apf_format.h	Type definition for above
app_framework.c	Implements the application framework for zigbee app
app_framework.h	Type definition for above

Table 11. Application Framework

These 4 are located in /Dev/APL in the protocol stack.

8 Test and Evaluation

8.1 Test of Primitives

8.1.1 MAC layer

a. Figure 33 shows the snapshot of mlmeResetRequest while debugging. Generally, mlmeResetRequest primitive resets the values of Attributes to default values as defined by IEEE 802.15.4-2003 Table 17 (page 135). We added watches on mpib.macAckWaitDuration, mpib.macAssociationPermit, mpib.macAutoRequest, mpib.macBattLifeExt and mpib.macBattLifeExtPeriods attribute variables. As we step over the lines of the code, (notice the debugger pointer position), we could see on the watch dialog box in the bottom of Fig. 18, that mpib.macAckWaitDuration is now set to 54, mpib.macAssociationPermit is set to FALSE, mpib.macAutoRequest is set to TRUE, mpib.macBattLifeExt is set to FALSE and finally, mpib.macBattLifeExtPeriods is set to 6.

The screenshot displays an IDE with the following components:

- Left Panel (Project Explorer):** Shows a tree view of the project structure. The 'I/O ATMEGA128' folder is expanded, listing various hardware modules like AD_CONVERTER, ANALOG_COMPARATC, BOOT_LOAD, CPU, EEPROM, EXTERNAL_INTERRUPT, JTAG, MISC, PORTA, PORTB, PORTC, PORTD, PORTE, PORTF, and PORTG.
- Center Panel (Code Editor):** Displays the source code for the `RESET_MF()` function. The code initializes MAC command state, sets other state variables, and configures MAC/PHY PIB attributes to their default values. A yellow arrow points to the line `mpib.macBattLifeExtPeriods = 6;`.
- Right Panel (Watch Window):** A table showing the current values of variables being monitored.

Name	Value	Type	Location
<code>mpib.macAckWaitDuration</code>	54		
<code>mpib.macAssociationPermit</code>	FALSE		
<code>mpib.macAutoRequest</code>	TRUE		
<code>mpib.macBattLifeExt</code>	FALSE		
<code>mpib.macBattLifeExtPeriods</code>	6		

Figure 34. Snapshot of mlmeResetRequest



b. Figure 34 shows the snapshot of mlmeSetRequest while debugging. Generally, mlmeSetRequest primitive set the value of an attribute based from the function caller's parameters: Attribute to be set and its value. For this example, the attribute is MAC_ASSOCIATION_PERMIT and the value to set is TRUE, these values were assigned when mlmeSetRequest function is called to variables pibAttribute and *pPibAttributeValue, respectively. The function finds the case defined by pibAttribute and assigned the *pPibAttributeValue to that attribute. So that, in this case,

```
case MAC_ASSOCIATION_PERMIT
mpib.macAssocationPermit = (BOOL) pPibAttributeValue;
break;
```

is executed. Based from our watch, the actual MAC PIB attribute: mpib.macAssocationPermit gets the value of boolean value of pPibAttributeValue.

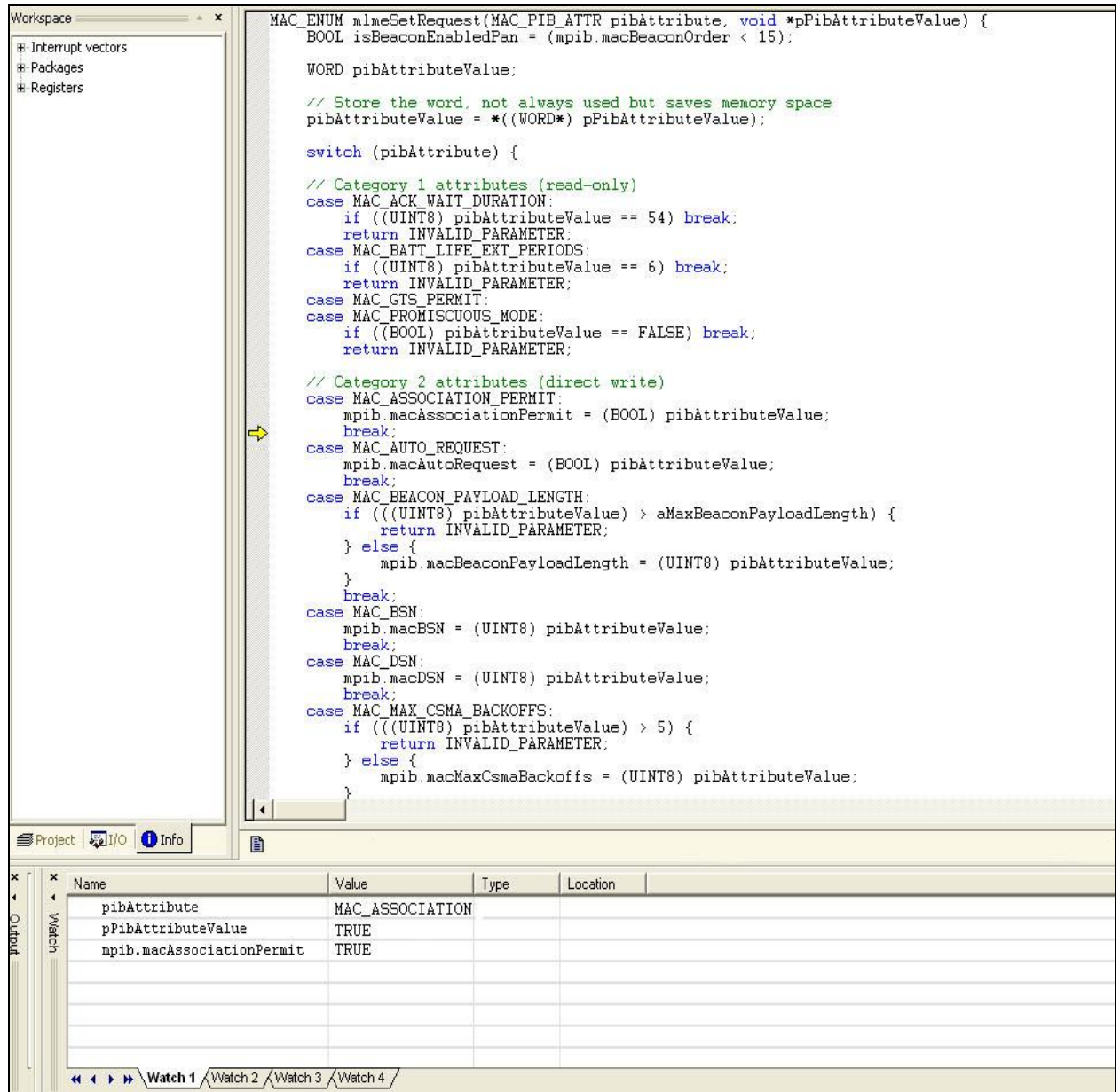


Figure 35. Snapshot of mlmeSetRequest

c. Figure 35 shows the snapshot of `mcpsPurgeRequest` while debugging. A purge request is invoked whenever a dataframe is to be purged. For this example, no packet is available to be purged in the transaction queue defined by `miqInfo.firstIndirectPacket = NO_PACKET` thus, an `invalid_handle` message was returned (see also Fig.3 for the flowchart). From the flowchart, we expect that the whole function wouldn't do anything and skip the entire process. This is why the debugger skipped the whole while function and went directly to the line which indicates: return an `INVALID_HANDLE` as shown in the picture with the

yellow debugger arrow.

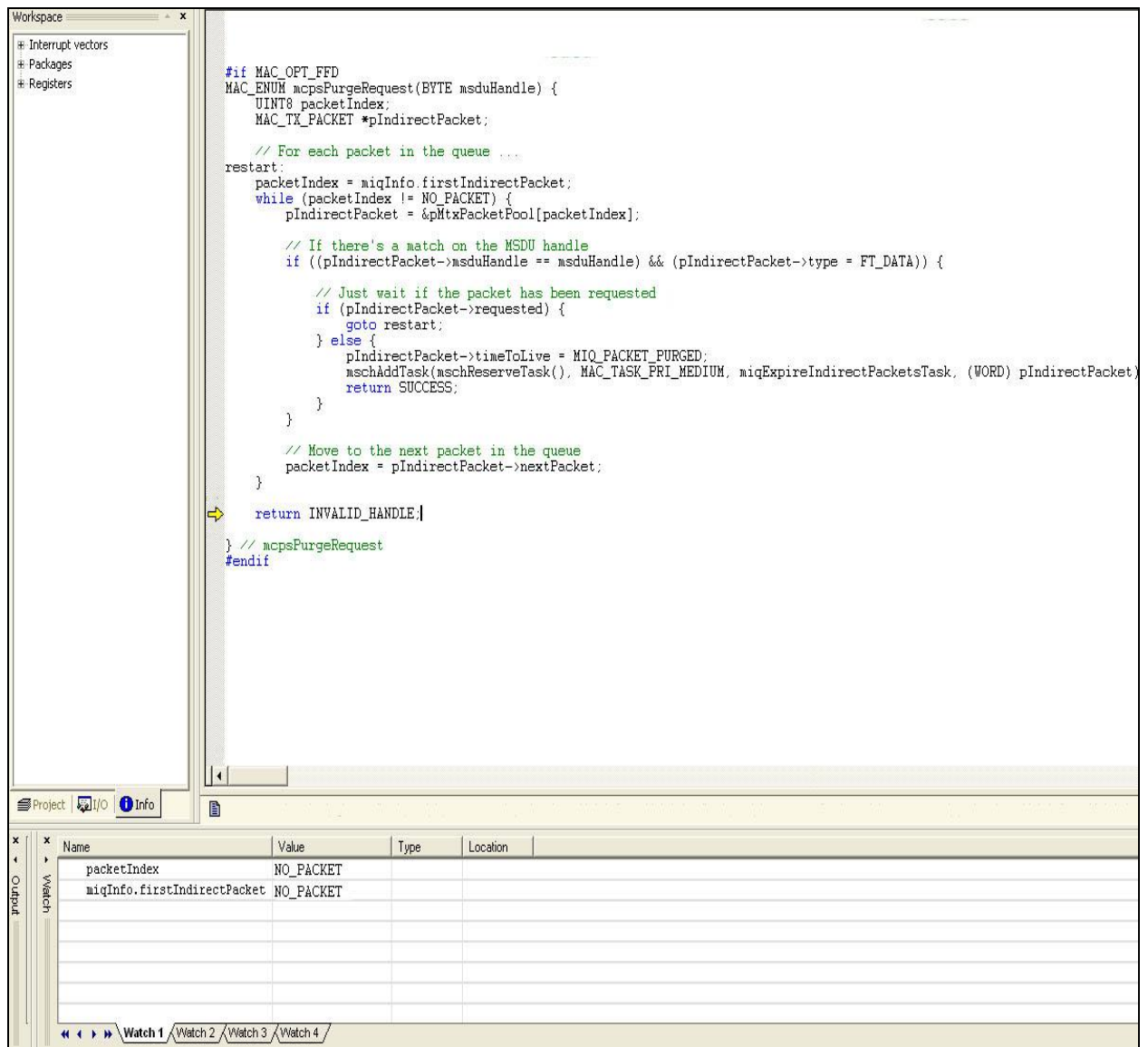


Figure 36. Snapshot of mcpsPurgeRequest.

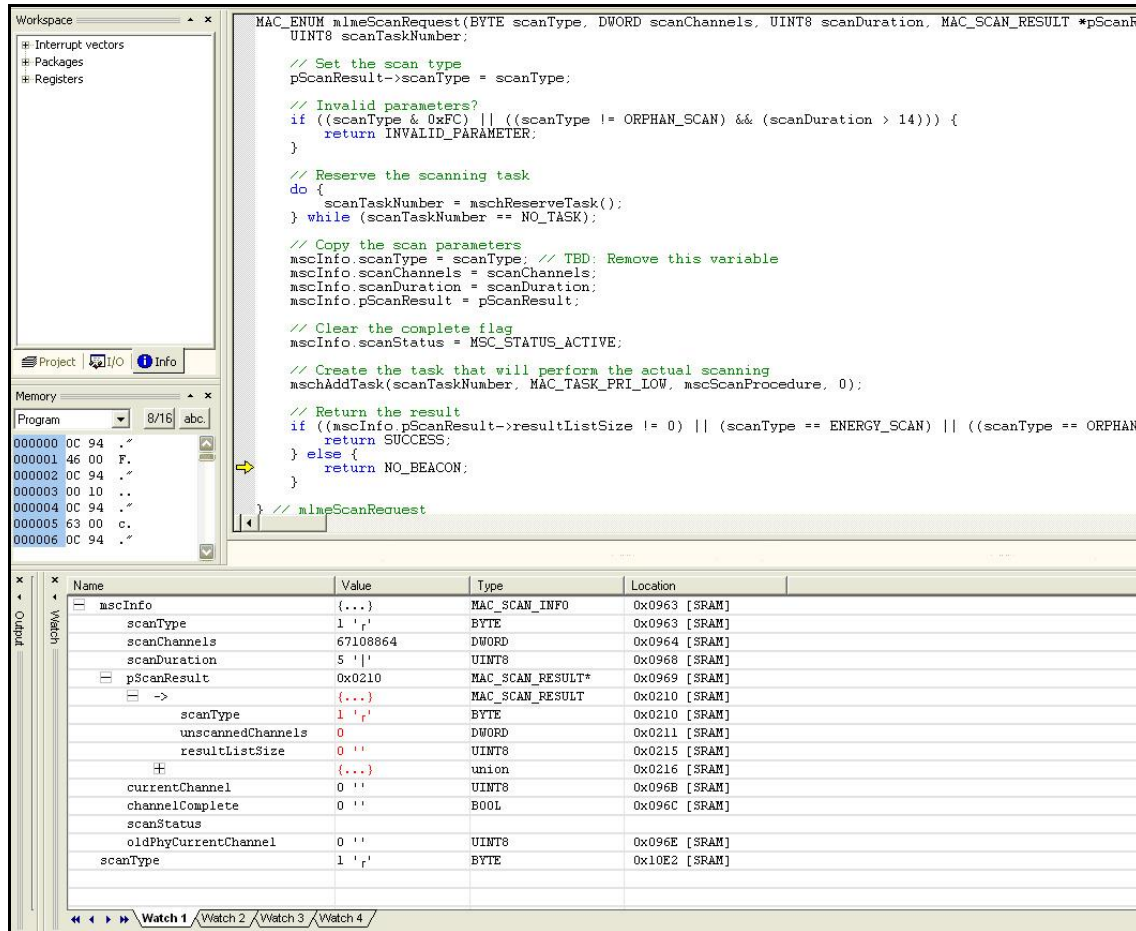


Figure 37. Snapshot of mlmeScanRequest.

e. Fig. 36 shows that `mlmeScanRequest` returned a value of `NO_BEACON` after its execution. On function call, `scanType` had a value of 1 and `scanDuration` had a value of 5. Therefore, the test for Invalid Parameter was skipped. `ENERGY_SCAN`, `ACTIVE_SCAN`, `PASSIVE_SCAN` and `ORPHAN_SCAN` have 0, 1, 2, 3 as their values, respectively. Since `scanType = 1` and `mscInfo.pScanResult->resultListSize = 0`, the “if” statement was false and therefore `NO_BEACON` was returned.

f. Fig. 37 shows the snapshot of watch and a portion of `mlmeStartRequest`. The test for `NO_SHORT_ADDRESS` was skipped in the beginning since `macShortAddress` is not equal to `0xFFFF`. Also, the test for `INVALID_PARAMETER` was skipped in the statement:

```
if ((!msupChannelValid(logicalChannel)) || (beaconOrder > 15) || ((superframeOrder > beaconOrder) && (superframeOrder != 15)))
{...}
```


because, as per watch, beaconOrder is not greater than 15 but 15 and superframeOrder is equal to beaconOrder which is 15 and the channel is valid. Therefore, the process continued and returned SUCCESS in the end.

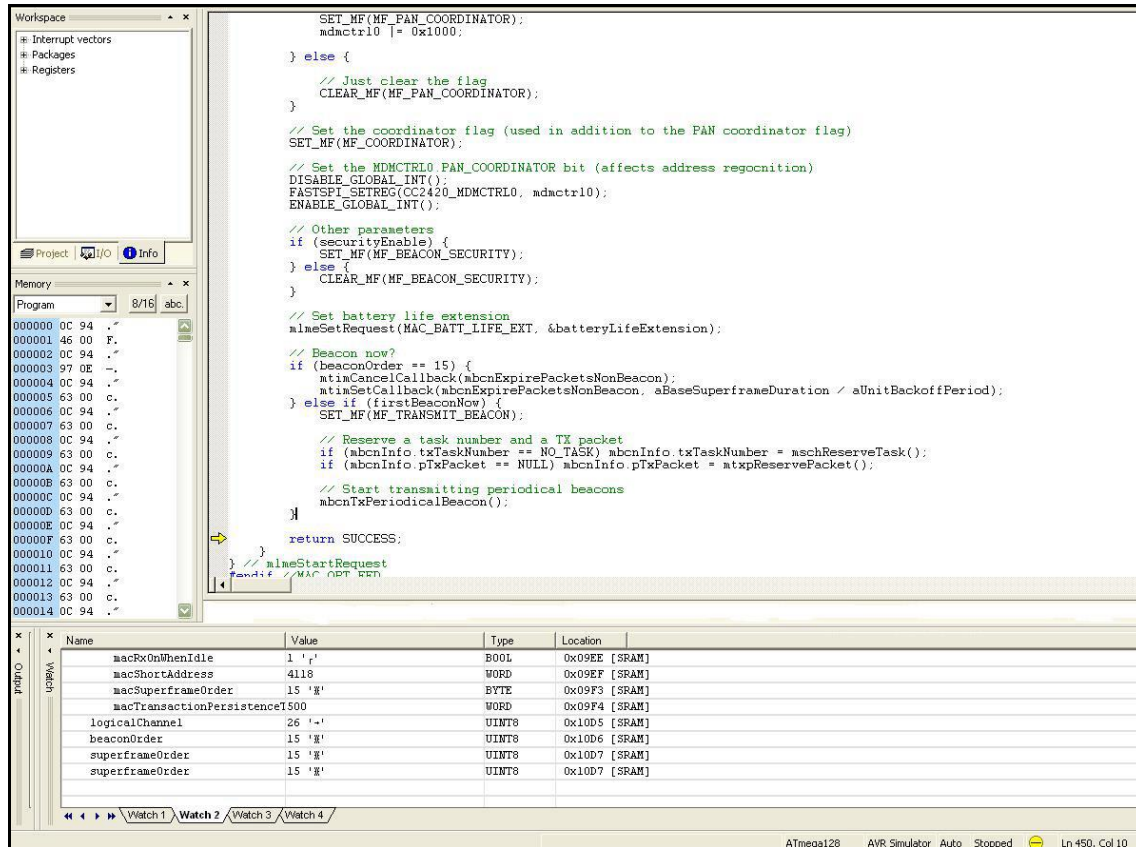


Figure 38. Snapshot of mlmeStartRequest.

- f. Figure 38 & 39 show snapshots of mlmeSyncRequest. Here we found a task to sync with. The logicalChannel is shifted 8 bits to the left and cast to word and ORed to trackbeacon. This is used to sync and to queue the task.

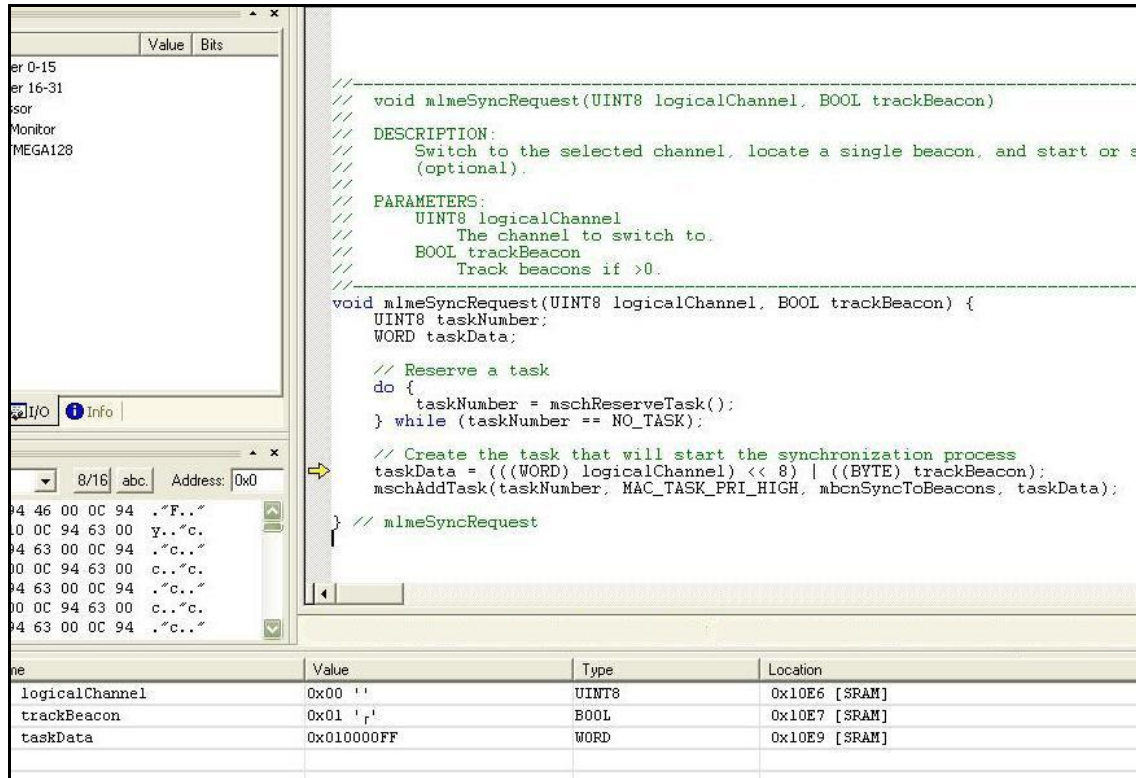


Figure 39. Snapshot of mlmeSyncRequest.

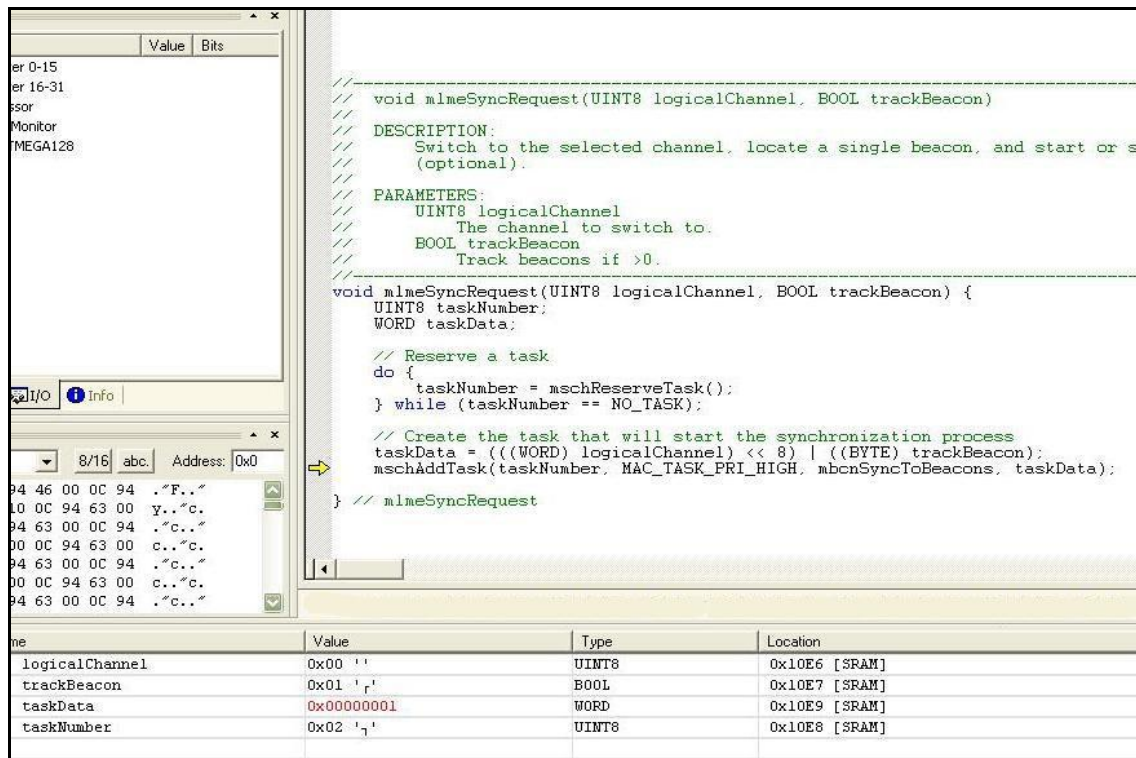


Figure 40. Snapshot of mlmeSyncLossIndication.

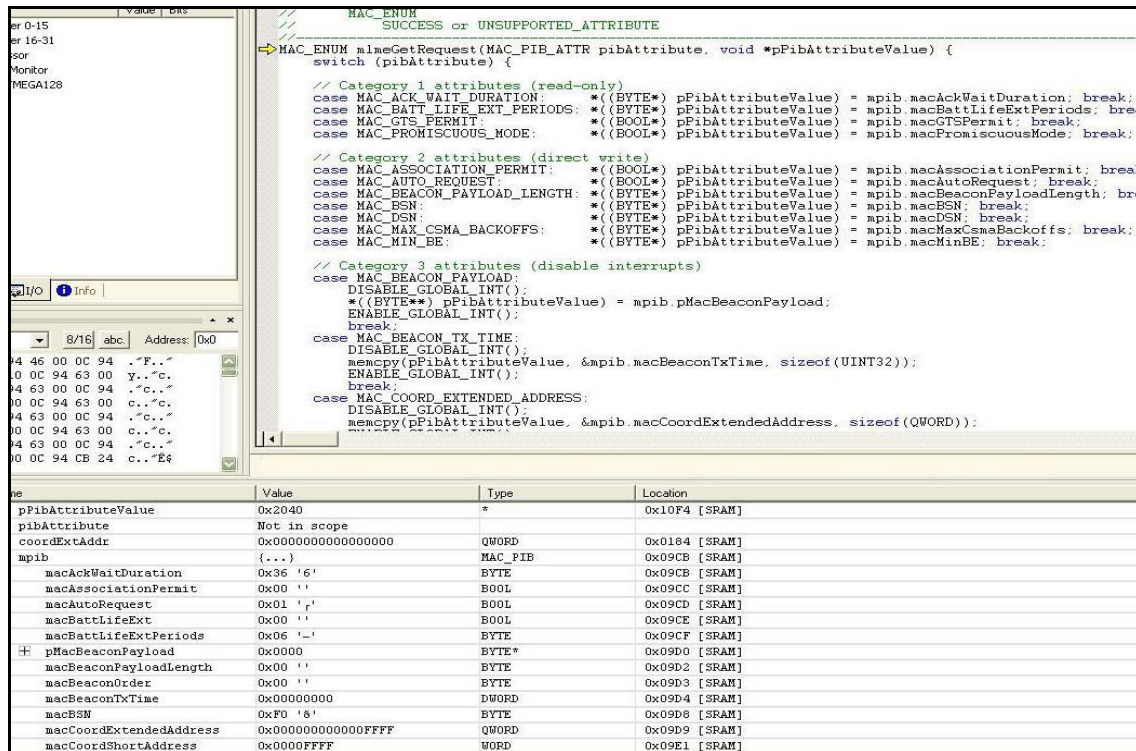


Figure 41. Snapshot of mlmeGetRequest.

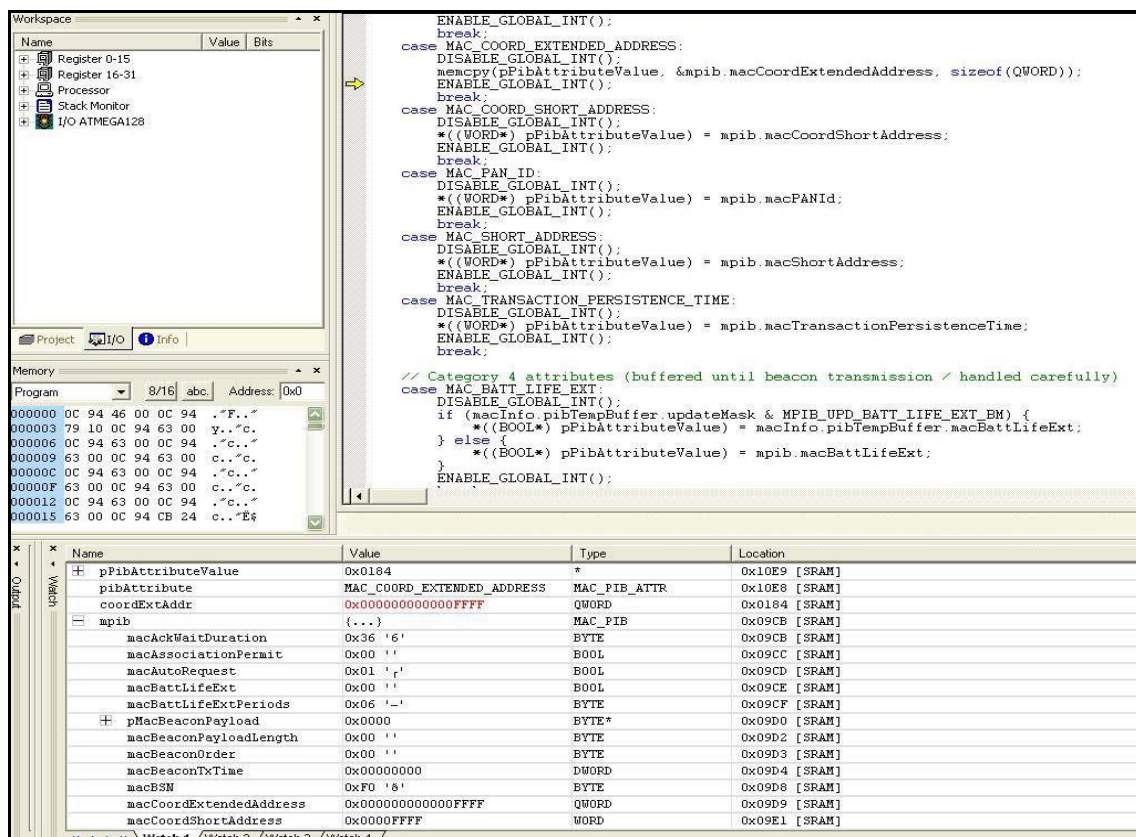


Figure 42. Snapshot of mlmeGetRequest.

- g. Figure 40 & 41 are snapshots mlmeGetRequest. Getrequest retrieves information from PIB. PibAttribute to fetch is MAC_Coord_Extended_Address. After the line, `memcpy(pPibAttributeValue, &mpib.macCoordExtendedAddress, sizeof(QWORD));`, memory area from `&mpib.macCoordExtendedAddress` is copied to memory `pPibAttributeValue`

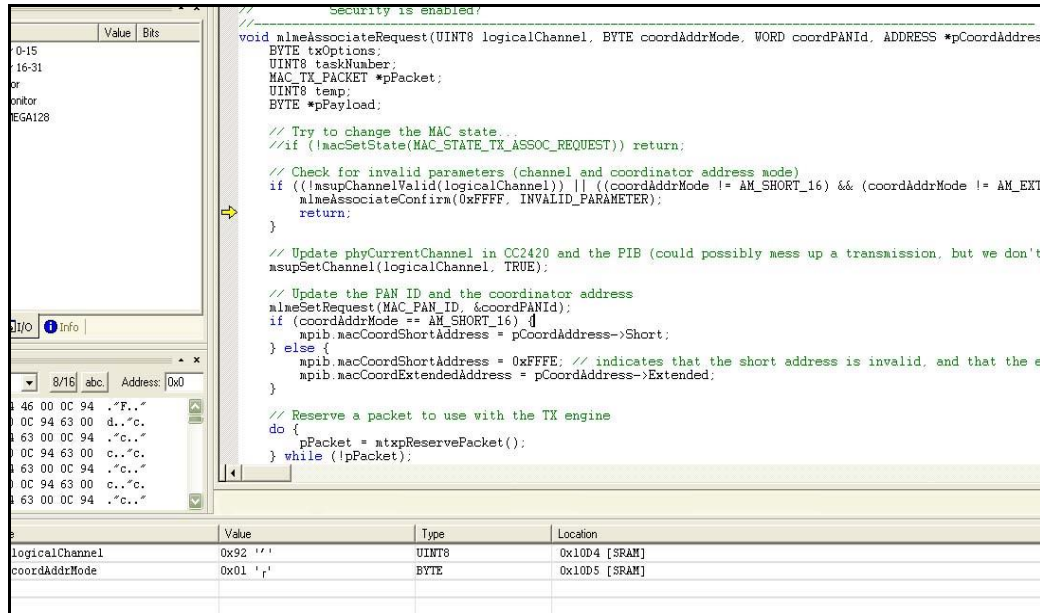


Figure 43. Snapshot of mlmeAssociateRequest

- h. Figure 42 shows the snapshot of mlmeAssociateRequest. Here the logical channel is 0x92 and coordAddrMode is 0x01. The IF statement (where the debug arrow is) checks if coordAddrMode is not equal to AM_SHORT_16 and AM_EXTENDED which are 2 and 3, respectively. Therefore, this will issue an INVALID_PARAMETER on mlmeAssociateConfirm.
- i. Figure 43 shows a snapshot of mlmeDisassociateRequest. `&mpib.macCoordExtendedAddress` did not match with `pDeviceAddress` and since the device is not a coordinator it skipped disassociate scheme and issued mlmeDisassociateConfirm of Invalid_Parameter.
- j. Figure 44 shows a simplified mlmePollRequest wherein result is equal to SUCCESS which is 0x00. If an error occurs such as no data, no ack, etc they were discarded and just issued a reset.

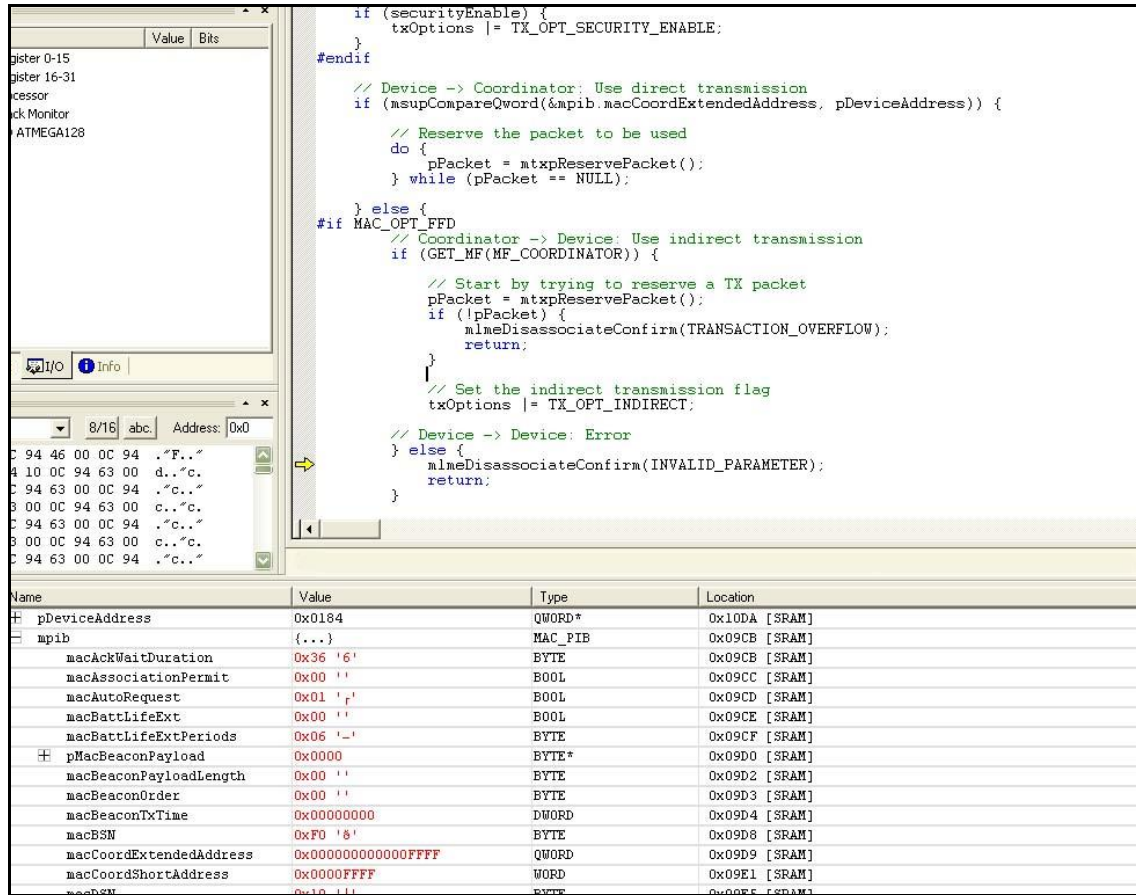


Figure 44. Snapshot of mlmeDisassociateRequest

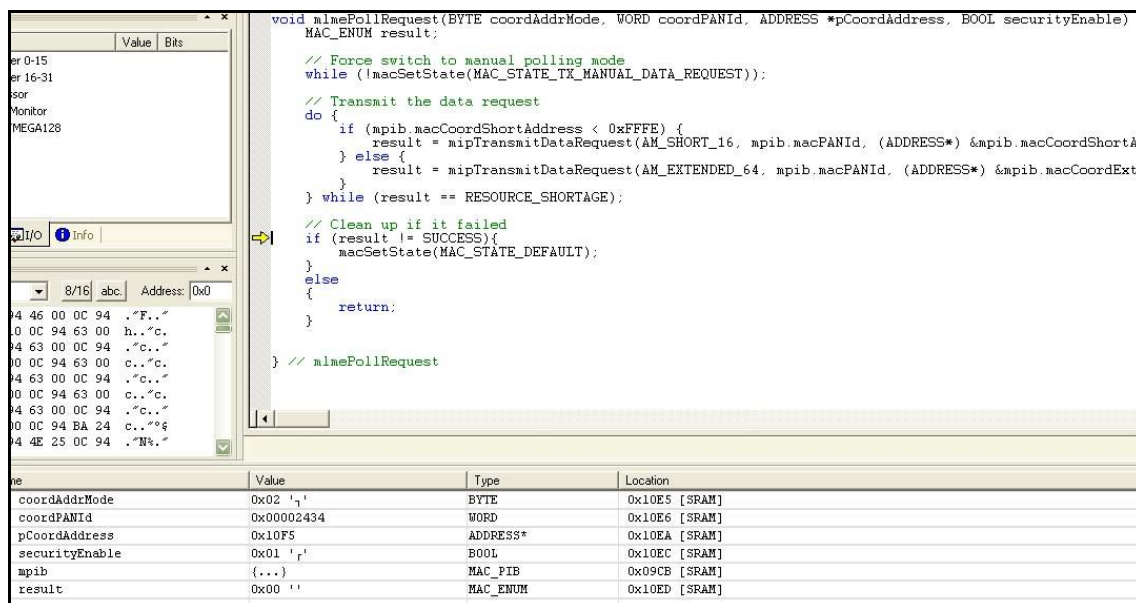


Figure 45. Snapshot of mlmePollRequest

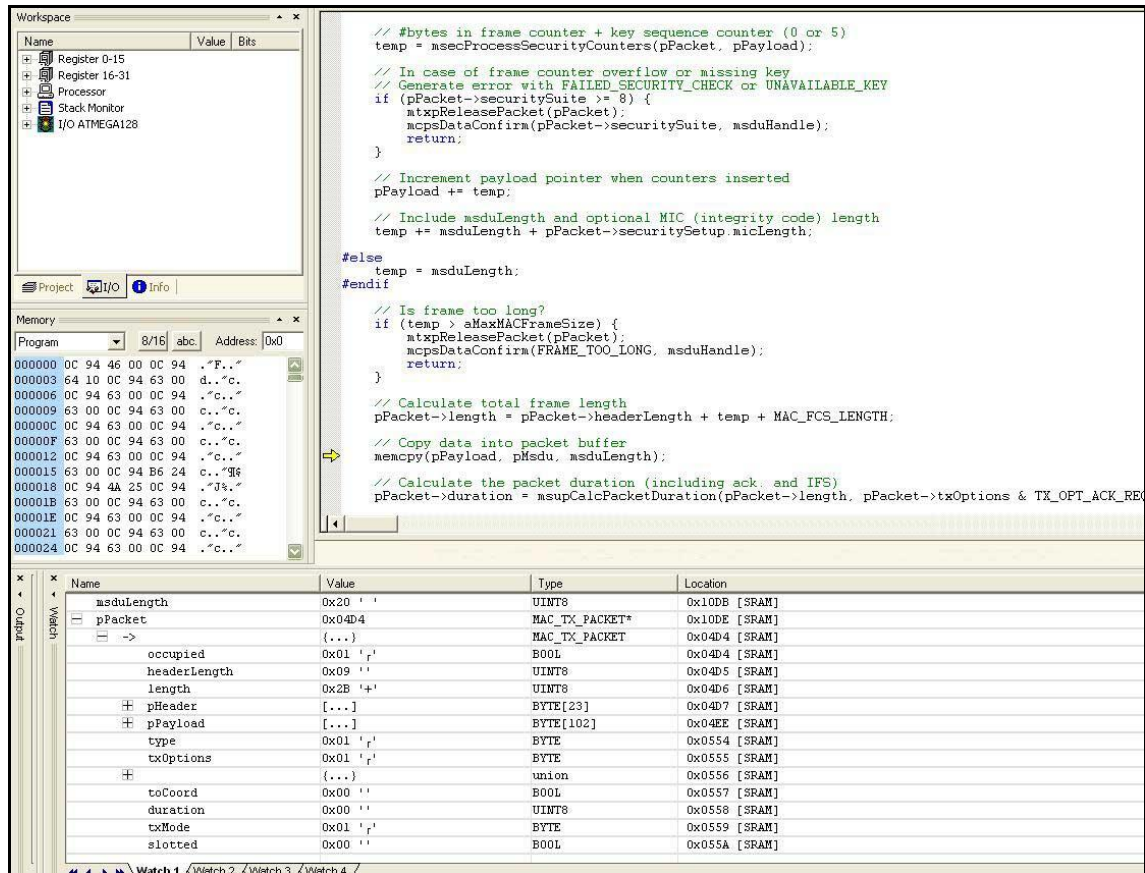


Figure 46. Snapshot of mcpsDataRequest

- k. Figure 46 shows the snapshot of `mcpsDataRequest`. Here the MPDU (mac protocol data unit) `msduLength` was created and was checked if exceeded the maximum frame size and it was not. A `txOption` that requires an acknowledged was implemented as defined by `pPacket.txOptions`. Furthermore, the packet will be transmitted using unslotted CSMA-CA as define by `pPacket.slotted`.
- l. Figure 47 shows the snapshot of `mlmeRxEnableRequest`. This request simply add a task into the periodical scheduler.
- m. Figure 48 shows the snapshot of `mlmeBeaconNotifyIndication`. This function add a event of network layer into the event queue. The beacon will be processed by function `nwk_processNwkEvent`.
- n. Figure 49 shows the snapshot of `mlmeComStatusIndication`.
- o. Figure 50 shows an example of testing of primitive `mlmeOrphanIndication`. This function will call to `processNwkEvent` and correctness of this primitive will be tested at that function.

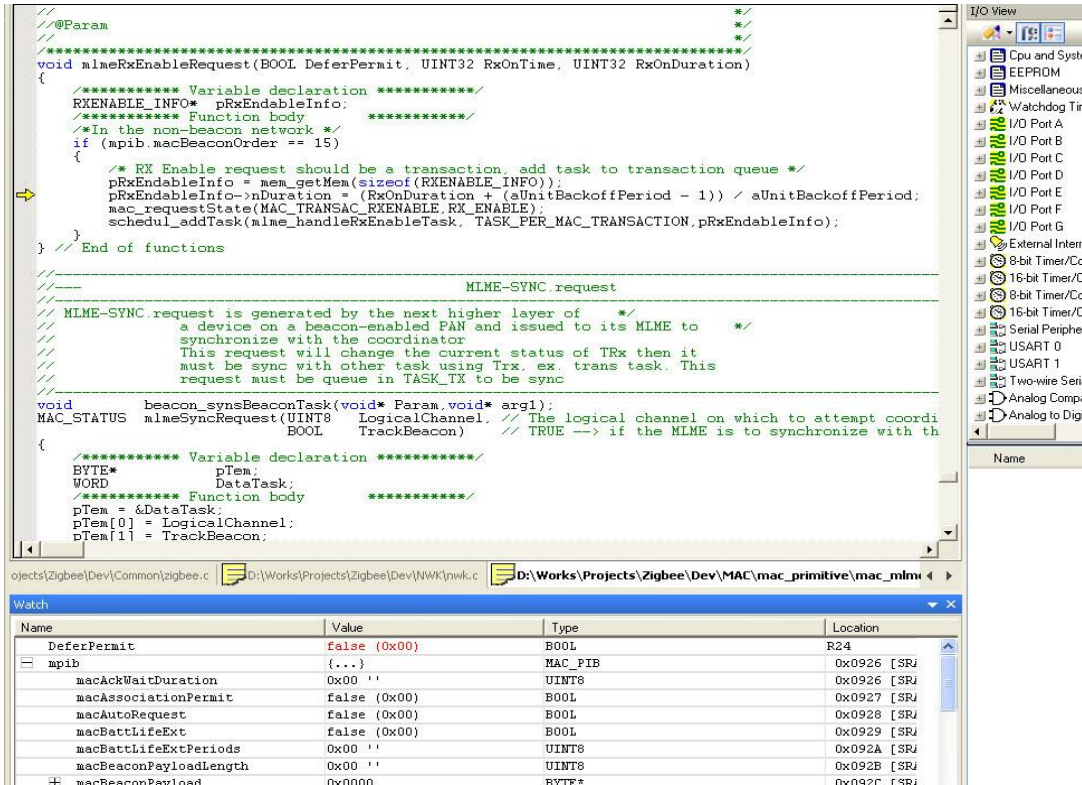


Figure 47. Snapshot of mlmeRxEnableRequest

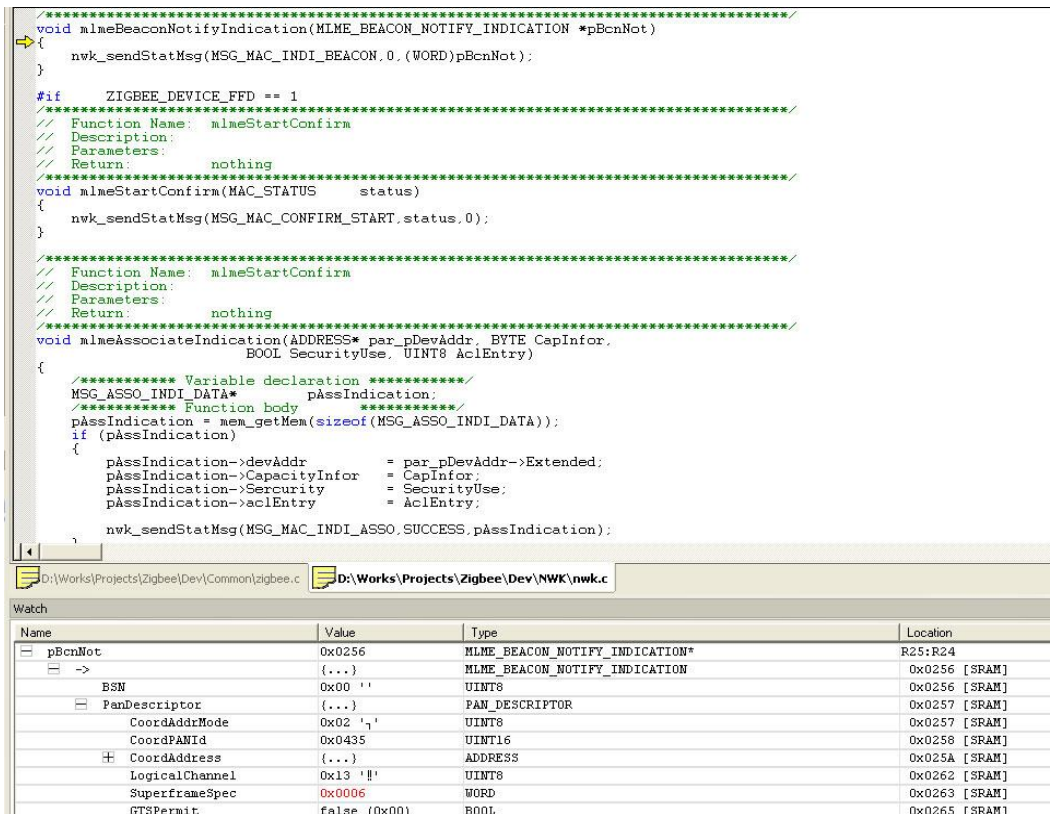


Figure 48. Snapshot of mlmeBeaconNotifyIndication

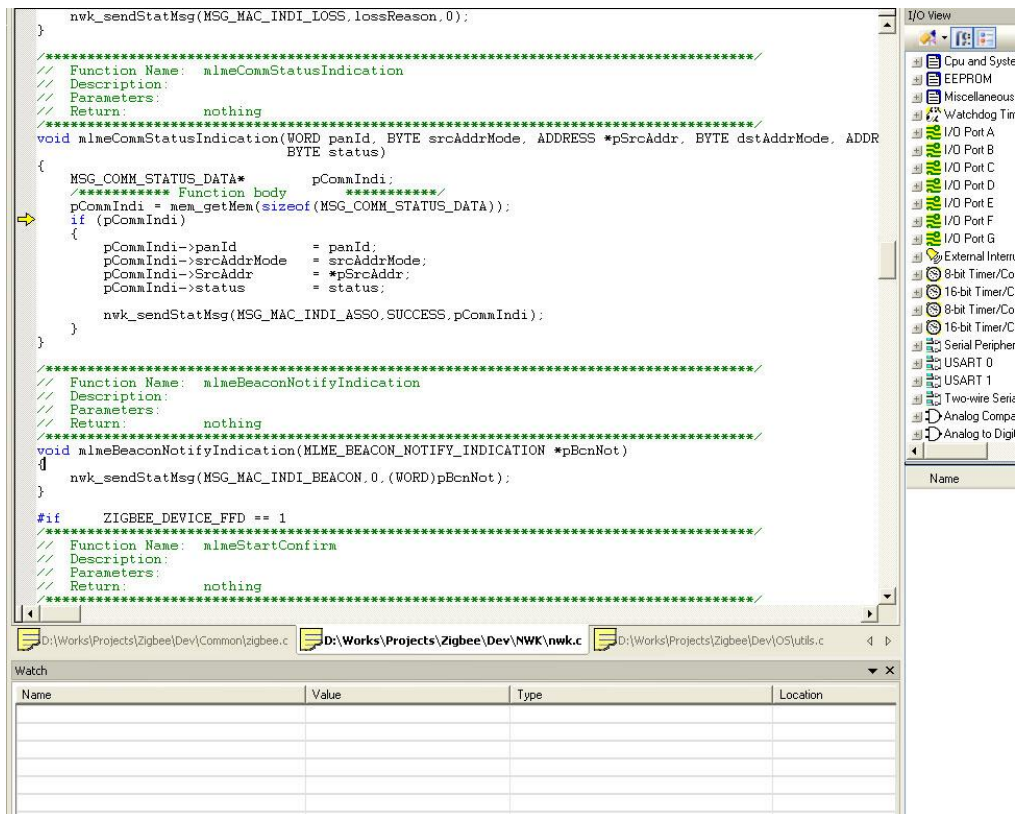


Figure 49. Snapshot of mlmeComStatusIndication

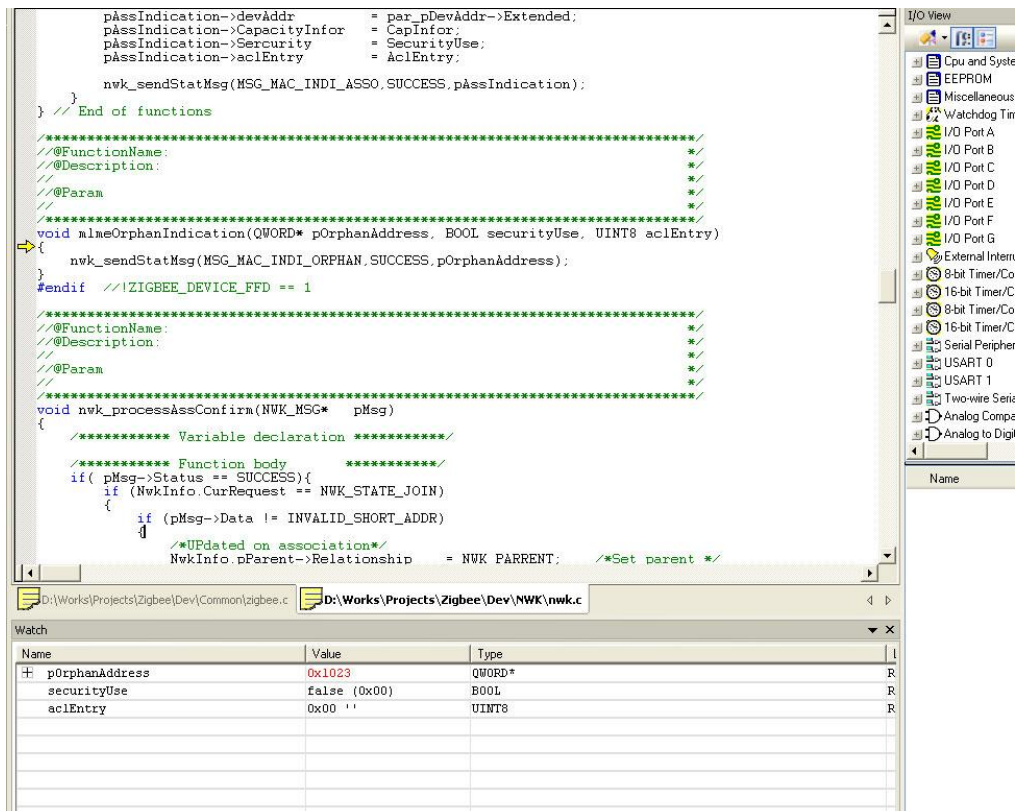


Figure 50. Snapshot of mlmeOrphanIndication

8.1.2 Network layer

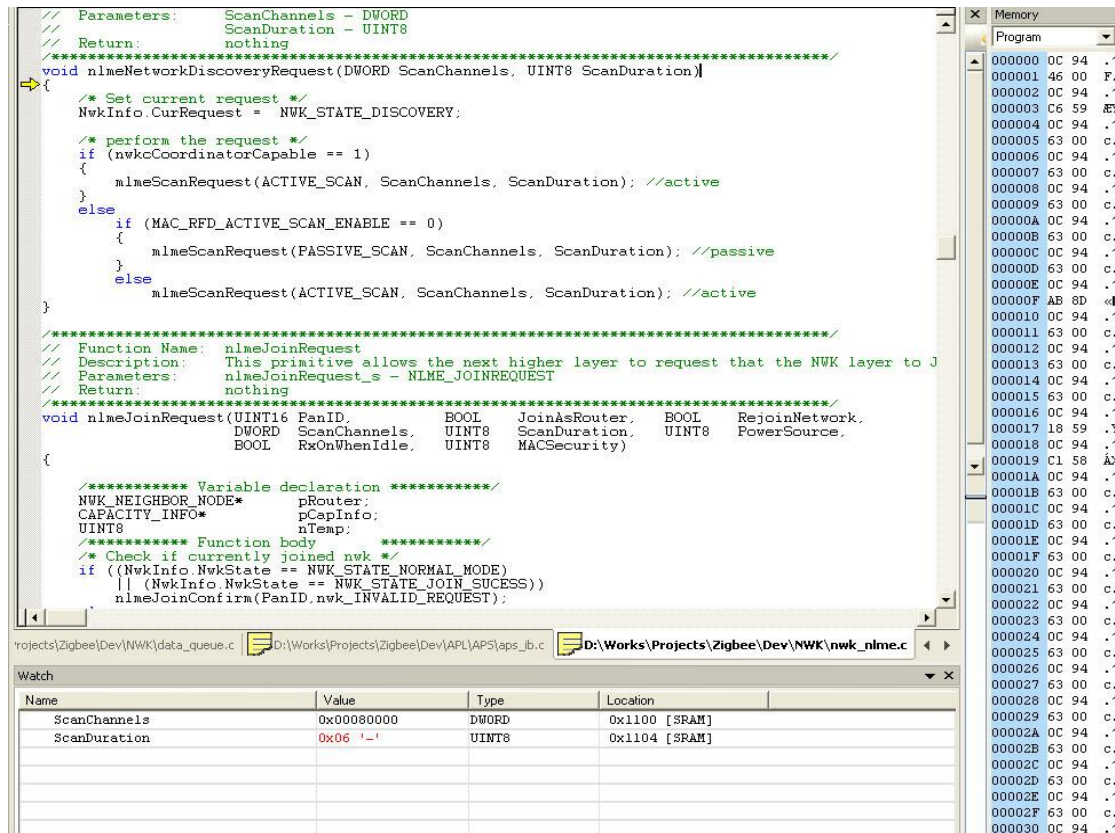
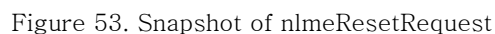
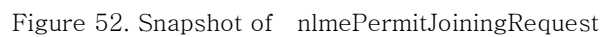


Figure 51. Snapshot of nlmeNetworkDiscoveryRequest

- Figure 51 shows the snapshot of nlmeNetworkDiscoveryRequest. This request uses the mlmeScanRequest to perform the functionality. Depending on current capacity of device defined in the variable nwkcCoordinatorCapable, different type of scan request will be sent to the MAC layer.
- Figure 52 show the testing snapshot of primitive nlmePermitJoiningRequest. This primitive firstly check the condition to perform this function on the current device by checking device capacity (nwkcCoordinatorCapable). Then it call primitive mlmeSetRequest to set MAC_ASSOCIATION_PERMIT attribute to MAC layer.



- c. Figure 53 show the snapshot of the nlmeReset primitive. Firstly, this primitive simple call the reset primitive mlmeResetRequest on the MAC layer. If this primitive returns success, network layer will continue to call to nlme_resetNwk to reset the internal parameters.
- d. Figure 54 shows the snapshot of the nlmeSetRequest primitive. This primitive use pointer NIBAttributeValue contain the value which is going to be set to the network parameter. According to the attribute type (NIBAttribute), that value will be set to correct parameter.

```

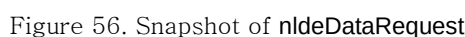
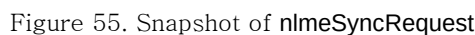
// Description: This primitive allows the next higher layer to write the value of
//              an attribute into the NIB*/
// Parameters: nlmeSetRequest_s - NLME_SETREQUEST
// Return:      nothing
//*****
➔ NWK_STATUS nlmeSetRequest(BYTE NIBAttribute, UINT8 NIBAttributeLength, void *NIBAttributeValue){
    //***** Function body *****
    NWK_STATUS SetStatus;
    //***** Function body *****
    SetStatus = nwk_SUCCESS;
    if(NIBAttribute == NWKSEQUENCENUMBER){
        nwk_ib.nwkSequenceNumber = *(WORD*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKPASSIVEACKTIMEOUT){
        nwk_ib.nwkPassiveAckTimeout = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXBROADCASTRETRIES){
        nwk_ib.nwkMaxBroadcastRetries = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXCHILDREN){
        nwk_ib.nwkMaxChildren = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXDEPTH){
        nwk_ib.nwkMaxDepth = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXROUTERS){
        nwk_ib.nwkMaxRouters = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKNEIGHBORTABLE){
        nwk_ib.nwkNeighborTable = (void*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKNETWORKBROADCASTDELIVERYTIME){
        nwk_ib.nwkBroadcastDeliveryTime = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKREPORTCONSTANTCOST){
        nwk_ib.nwkReportConstantCost = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKROUTEDISCOVERYRETRIESPERMITTED){
        nwk_ib.nwkRouteDiscoveryRetriesPermitted = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKSYMLINK){
        nwk_ib.nwkRouteTable = (void*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKROUTEDISCOVERYRETRIESPERMITTED){
        nwk_ib.nwkSymLink = *(UINT8*)NIBAttributeValue;
    }
}

```

Name	Value	Type	Location
nwkUseTreeRouting	false (0x00)	BOOL	0x052E [SRAM]
nwkNextAddress	0x0000	UINT16	0x052F [SRAM]
nwkAvailableAddresses	0x0000	UINT16	0x0531 [SRAM]
nwkAddressIncrement	0x0000	UINT16	0x0533 [SRAM]
nwkTransactionPersistenceTime	0x0000	UINT16	0x0535 [SRAM]

Figure 54. Snapshot of nlmeSetRequest

- e. Figure 55 shows the snapshot of the nlmeSyncRequest primitive. This request is performed only when network layer is in appropriate state. The state is controlled by function nkw_setNwkState. If not, the primitive will return nwk_NOT_PERMIT. According to the parameter *Track*, which define if the behavior of polling, the primitive mlmeSyncRequest will be sent to MAC layer.



f. Figure 56 shows the snapshot of the `nldeDataRequest` primitive. This primitive uses `nwk_makeNpdu` to format the network data packet and MAC primitive `mlmeDataRequest` to send data to the MAC layer.

```
// Return: nothing
//*****

NWK_STATUS nlmeGetRequest(BYTE NIBAttribute, void *pNIBAttribValue, UINT8* NIBAttribLength){
    switch (NIBAttribute){
        case NWKSEQUENCENUMBER:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkSequenceNumber;
            *NIBAttribLength = sizeof(nwk_ib.nwkSequenceNumber);
            break;
        case NWKPASSIVEACKTIMEOUT:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkPassiveAckTimeout;
            *NIBAttribLength = sizeof(nwk_ib.nwkPassiveAckTimeout);
            break;
        case NWKMAXBROADCASTRETRIES:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxBroadcastRetries;
            *NIBAttribLength = sizeof(nwk_ib.nwkMaxBroadcastRetries);
            break;
        case NWKMAXCHILDREN:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxChildren;
            *NIBAttribLength = sizeof(nwk_ib.nwkMaxChildren);
            break;
        case NWKMAXDEPTH:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxDepth;
            *NIBAttribLength = sizeof(nwk_ib.nwkMaxDepth);
            break;
        case NWKMAXROUTERS:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxRouters;
            *NIBAttribLength = sizeof(nwk_ib.nwkMaxRouters);
            break;
        case NWKNEIGHBORTABLE:
            *((QWORD**)pNIBAttribValue) = nwk_ib.nwkNeighborTable;
            *NIBAttribLength = sizeof(nwk_ib.nwkNeighborTable);
            break;
        case NWKNETWORKBROADCASTDELIVERYTIME:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkBroadcastDeliveryTime;
            *NIBAttribLength = sizeof(nwk_ib.nwkBroadcastDeliveryTime);
            break;
        case NWKREPORTCONSTANTCOST:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkReportConstantCost;
            *NIBAttribLength = sizeof(nwk_ib.nwkReportConstantCost);
            break;
        case NWKROUTEDISCOVERYRETRIESPERMITTED:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkRouteDiscoveryRetriesPermitted;
            *NIBAttribLength = sizeof(nwk_ib.nwkRouteDiscoveryRetriesPermitted);
            break;
        case NWKROUTETABLE:
            break;
    }
}
```

Name	Value	Type	Location
nwkMaxRouters	0x00	UINT8	0x0523 [SRAM]
nwkNeighborTable	0x0000	void*	0x0524 [SRAM]
nwkBroadcastDeliveryTime	0x00	UINT8	0x0526 [SRAM]
nwkReportConstantCost	0x00	UINT8	0x0527 [SRAM]
nwkRouteDiscoveryRetriesPermi	0x00	UINT8	0x0528 [SRAM]
nwkRouteTable	0x0000	void*	0x0529 [SRAM]

Figure 57. Snapshot of `nlmeGetRequest`

g. Figure 57 shows the snapshot of the `nlmeGetRequest` primitive. This primitive uses `NIBAttribute` to indicate the parameter need to be retrieved and the parameter `pNIBAttributeValue` as pointer to contain the value of that attribute.

h. Figure 58 shows the snapshot of the `nlmeJoinRequest` primitive. This primitive perform joining of a network node. This function can only be activate when network node are after scanning. As in this example, function can process further only when `NwkInfo.NwkStata` are not `NWK_STATE_NORMAL_MODE` or not `NWK_STATE_JOIN_SUCESS`.

```
// Return: nothing
//***** Variable declaration *****/
void nlmeJoinRequest(UINT16 PanID, BOOL JoinAsRouter, BOOL RejoinNetwork,
                    DWORD ScanChannels, UINT8 ScanDuration, UINT8 PowerSource,
                    BOOL RxOnWhenIdle, UINT8 MACSecurity)
{
    /****** Variable declaration *****/
    NWK_NEIGHBOR_NODE* pRouter;
    CAPACITY_INFO* pCapInfo;
    UINT8 nTemp;
    /****** Function body *****/
    /* Check if currently joined nwk */
    if ((NwkInfo.NwkState == NWK_STATE_NORMAL_MODE)
        || (NwkInfo.NwkState == NWK_STATE_JOIN_SUCESS))
        nlmeJoinConfirm(PanID, nwk_INVALID_REQUEST);
    else
    {
        /* Rejoin the network or not */
        if (!RejoinNetwork) /* Join new network */
        {
            pCapInfo = (CAPACITY_INFO*)&nwk_ib.nwkCapabilityInformation;
            if (JoinAsRouter)
            {
                NwkInfo.NwkJoinedDeviveType = ZIGBEE_ROUTER;
                pCapInfo->DeviceType = TRUE;
                pCapInfo->AllocAddr = TRUE;
            }
            else
            {
                NwkInfo.NwkJoinedDeviveType = ZIGBEE_ENDPOINT;
                pCapInfo->DeviceType = FALSE;
                pCapInfo->AllocAddr = FALSE;
            }
            pRouter = nwk_findRouterInNT(PanID, ScanChannels);
            if (pRouter)
            {
                NwkInfo.pParent = pRouter;
                NwkInfo.NwkPANId = PanID;
                NwkInfo.NwkChannel = pRouter->LogicalChannel;
            }
        }
    }
}

D:\Works\Projects\Zigbee\Dev\Common\zigbee.c D:\Works\Projects\Zigbee\Dev\NWK\nwk_nlme.c
```

Name	Value	Type	Location
pCapInfo	0x0000	CAPACITY_INFO*	0x10E7 [SRAM]
AlterCoord	0x00 ''	UINT8	0x0000 [SRAM]
DeviceType	0x00 ''	UINT8	0x0000 [SRAM]
PwrSrc	0x00 ''	UINT8	0x0000 [SRAM]
RxOnWhenIdle	0x00 ''	UINT8	0x0000 [SRAM]
Reserved	0x00 ''	UINT8	0x0000 [SRAM]

Figure 58. Snapshot of nlmeJoinRequest

```
// Parameters:
// Return: nothing
//***** Variable declaration *****/
void nlmeLeaveRequest(ADDRESS* pDeviceAddress, BOOL RemoveChildren, BOOL MACSecurityEnable)
{
    /****** Variable declaration *****/
    NWK_NEIGHBOR_NODE* pNode;
    NWK_FRM_TYPE FrameType = NWK_FRM_CMD;
    NWK_CMD_TYPE CmdType = NWK_CMD_LEAVE;
    P_NPDU npdu;
    BYTE Nsdu;
    /****** Function body *****/
    if (NwkInfo.NwkState != NWK_STATE_NORMAL_MODE){
        nlmeLeaveConfirm(pDeviceAddress, nwk_INVALID_REQUEST);
    }
    else
    {
        /* Remove devices from the network */
        if (pDeviceAddress->Extended != 0)
        {
            /* Remove this child from the network 2.7.1.7.2*/
            pNode = nwk_findNodeInNT(pDeviceAddress);
            if (!pNode) /* Node does not exist in the NT*/
            {
                nlmeLeaveConfirm(pDeviceAddress, nwk_UNKNOWN_DEVICE);
                return;
            }
            else /* Device found, remove it */
            {
                /* Check if the device initiating the Leave is a coord or a router
                if ((ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COORD) || (ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER)){
                    /* Begin the transaction */
                    Nsdu = 0;
                    NsduLength = sizeof(Nsdu);
                    npdu = nwk_makeNpdu(NWK_FRM_DATA, NsduLength, Nsdu);
                    /* Send th LEAVE request */
                    mcpsDataRequest(AM_SHORT_16, NwkInfo.NwkPANId, NwkInfo.NwkAddr, pNode->NetworkAddress, AM_SHORT_16,
                                   pDeviceAddress->Extended, NsduLength, npdu, 0, 0);
                }
            }
        }
    }
}

D:\Works\Projects\Zigbee\Dev\NWK\nwk_nlme.c
```

Name	Value	Type	Location
NwkInfo	{...}	NWK_INFO	0x04E6 [SRAM]
NwkState	0x00 ''	UINT8	0x04E6 [SRAM]
CurRequest	0x00 ''	UINT8	0x04E7 [SRAM]
pParent	0x0000	NWK_NEIGHBOR_NODE*	0x04E8 [SRAM]
NwkPANId	0x0000	UINT16	0x04EA [SRAM]
NwkChannel	0x00 ''	UINT8	0x04EC [SRAM]

Figure 59. Snapshot of nlmeLeaveRequest

- i. Figure 59 shows the snapshot of the nlmeLeaveRequest primitive. Leave request have to check the current state of the network layer. If the current state is not NWK_STATE_NORMAL_MODE, this request can not be performed due to the consistency of the state. This in case, NwkInfo.State = 0, it's not normal state, then this request can not be performed, nlmeLeaveConfirm with INVALID_REQUEST will must be called.

```

//@Param
//
//
//
void nlmeJoinDirectRequest(ADDRESS* pDeviceAddr, CAPACITY_INFO CapInfor)
{
    /******* Variable declaration *****/
    NWK_STATUS status;
    /******* Function body *****/
    status = nwk_addNewChild(pDeviceAddr, 0x00, CapInfor, NwkInfo.SecurityEnable, 0x00);
    nlmeJoinDirectConfirm(pDeviceAddr, status);
} // End of functions

#endif

#endif

/******* Function Name: nlmeLeaveRequest *****/
/******* Description: *****/
/******* Parameters: *****/
/******* Return: nothing *****/
void nlmeLeaveRequest(ADDRESS* pDeviceAddress, BOOL RemoveChildren, BOOL MACSecurityEnable)
{
    /******* Variable declaration *****/
    NWK_NEIGHBOR_NODE* pNode;
    NWK_FRM_TYPE FrameType = NWK_FRM_CMD;
    NWK_CMD_TYPE CmdType = NWK_CMD_LEAVE;
    P_NPDU npdu;
    BYTE Nsdu;
    BYTE NsduLength;
    /******* Function body *****/
    if (NwkInfo.NwkState != NWK_STATE_NORMAL_MODE){
        nlmeLeaveConfirm(pDeviceAddress, nwk_INVALID_REQUEST);
    }
    else
    {
        /* Remove devices from the network */
        if (pDeviceAddress->Extended != 0 )
        {

```

Figure 60. Snapshot of nlmeJoinDirectRequest

- j. Figure 60 shows the snapshot of the nlmeJoinDirectRequest primitive. This request directly add a child in to the neighbor table. Correctness of this function is correctness of nwk_addNewChild function. nwk_addNewChild is tested independently and of scope of this primitive.

```

void nldeDataIndication(UINT8 SrcAddrMode, ADDRESS* pSrcAddress, BYTE NsduLength, BYTE *Nsdu, BYTE LinkQuality)
{
    /****** Variable declaration *****/
    P_APS_FRM pFrm;
    P_APS_FRM pACK;
    ACK_RECORD* pRec;
    UINT8 index;
    UINT8 Status;
    /****** Function body *****/
    pFrm = (P_APS_FRM)Nsdu;
    switch(pFrm->FrmType)
    {
        case APS_FRM_ACK:
            if (pFrm->DeliveryMode == APS_DELIVERY_NORMAL)
            {
                pACK = pFrm;
                index = dataqueue_findRec(&ApsDataHandler, pACK->ApsCounter);
                if (index != APS_INVALID_HANDLE)
                {
                    pRec = ApsDataHandler.RecList[index].pData;
                    /*Check the ack*/
                    if (pACK->SrcEndPoint == pRec->DestEp)
                    {
                        aps_endTrans(pACK, pRec, index);
                    }
                }
            }
            break;
        case APS_FRM_CMD:
            break;
        case APS_FRM_DATA:
            if ((pFrm->DeliveryMode == APS_DELIVERY_NORMAL)
                || ((pFrm->DeliveryMode == APS_DELIVERY_INDIRECT)
                    && (pFrm->IndirectAddrMode == 0)))
            {
                if (app_finEp(pFrm->DestEndpoint))
                {
                    apsDataIndication( pFrm, NsduLength,
                                      (SrcAddrMode==AM_SHORT_16)?APS_ADDR_SHORT:APS_ADDR_EXT,
                                      pSrcAddress, LinkQuality, FALSE);
                    Status = SUCCESS;
                }
            }
    }
}

```

D:\Works\Projects\Zigbee\Dev\Common\zigbee.c D:\Works\Projects\Zigbee\Dev\APL\APS\aps.c

Name	Value	Type	Location
SrcAddrMode	0x02 '1'	UINT8	0x1108 [SRAM]
pSrcAddress	0x7D1B	ADDRESS*	0x1109 [SRAM]
Short	0x4543	WORD	0x7D1B [SRAM]
Extended	0xFFFFFFFFFFFF4543	QWORD	0x7D1B [SRAM]
NsduLength	0x12 '1'	BYTE	0x110B [SRAM]
Nsdu	0x0046	BYTE*	0x110C [SRAM]
LinkQuality	0x00 ''	BYTE	0x110E [SRAM]

Figure 61. Snapshot of nldeDataIndication

k. Figure 61 shows the snapshot of the nldeDataIndication primitive. In this example, we set input value for the function and check if data is transferred correctly to upper layer. By changing the parameter, we can get into every branch of this function.

```

// @Param
//
//
void nlmeStartRouterRequest(UINT8 BeaconOrder, UINT8 SuperframeOrder, BOOL BatLifeExt)
{
    /****** Variable declaration *****/
    UINT8 nTemp;
    CAPACITY_INFO* pCapInfor;
    /****** Function body *****/
    /* Check if it joined as router */
    if (NwkInfo.NwkJoinedDevicetype != ZIGBEE_ROUTER)
    {
        nlmeStartRouterConfirm(nwk_INVALID_REQUEST);
    }
    else
    {
        /*If already created and now change the config or initializing new router */
        if (((NwkInfo.NwkState == NWK_STATE_NORMAL_MODE) && ((NwkInfo.NwkBeaconOrder != BeaconOrder) ||
            (NwkInfo.NwkSuperFrmOrder != SuperframeOrder) || (NwkInfo.BatLifeExt != BatLifeExt))) ||
            (NwkInfo.NwkState == NWK_STATE_JOIN_SUCCESS))
        {
            NwkInfo.NwkBeaconOrder = BeaconOrder;
            NwkInfo.NwkSuperFrmOrder = SuperframeOrder;
            NwkInfo.BatLifeExt = BatLifeExt;
            NwkInfo.CurRequest = NWK_STATE_START_ROUTER;
            NwkInfo.BcnPayload.EndDeviceCapacity = ENDDEVICE_CAP;
            NwkInfo.BcnPayload.nwkExtendedPANID = 0x00; /*TED*/
            NwkInfo.BcnPayload.RouterCapacity = ROUTER_CAP;

            #if ZIGBEE_DEVICE_FFD == 1
            NwkInfo.BcnPayload.DeviceDepth = NwkInfo.pParent->Depth+1;
            /*Init routing*/
            routing_init();
            #endif

            /* Modify necessary PIB attributes */
            nTemp = TRUE;
            nlmeSetRequest(MAC_ASSOCIATION_PERMIT, &nTemp);

            nTemp = sizeof(BEACON_PAYLOAD);
        }
    }
}

```

D:\Works\Projects\Zigbee\Dev\Common\zigbee.c D:\Works\Projects\Zigbee\Dev\NWK\nwk_nlme.c

Name	Value	Type	Location
pParent	0x0000	NWK_NEIGHBOR_NODE*	0x04E9 [SRAM]
NwkPANId	0x0000	UINT16	0x04EB [SRAM]
NwkChannel	0x00 ''	UINT8	0x04ED [SRAM]
NwkJoinedDevicetype	0x00 ''	UINT8	0x04EE [SRAM]
NwkAddr	0x0000	WORD	0x04EF [SRAM]
SecurityEnable	false (0x00)	BOOL	0x04F1 [SRAM]
NwkSuperFrmOrder	0x00 ''	UINT8	0x04F2 [SRAM]

Figure 62. Snapshot of nlmeStartRouterRequest

- Figure 62 shows the snapshot of the nlmeStartRouterRequest primitive. This primitive is applied for only device as Router then only the device has configuration value *NwkInfo.NwkJoimDevicetype* as Router can perform this function. If it is router, *NwkInfo.NwkState* is check to have the consistency in functionality of this primitive.

8.1.2 Application Framework (AF)

```

*****
//@FunctionName:
//@Description: This function will send a single KVP using services in APS
//@Param
*****
Z_STATUS apf_sendSingleKVP(UINT8 TransSeq, UINT8 CmdId, UINT8 AttrDataType, WORD AttrId,
                          UINT8 ErrCode, BYTE* pData, UINT8 nDataLen,
                          AF_ADDR* pDstAddr, BYTE ClusterId,
                          BYTE SrcEndpoint, BYTE TxOptions)
{
    ***** Variable declaration *****
    BYTE* pBuffer;
    BYTE *pTemp;
    BYTE nBufLen;
    Z_STATUS Status;
    ENDPOINT_DESCRIPTOR* pEp;
    ***** Function body *****
    /*Calculate length of single KVP*/
    nBufLen = SIZE_OF_KV_HR_RSP + nDataLen;
    pBuffer = mem_getMem(nBufLen);
    if (pBuffer)
    {
        /*Header */
        pTemp = pBuffer;
        *pTemp++ = BUILD_UINT8(AF_FRM_KVP,1);
        *pTemp++ = TransSeq;
        *pTemp++ = BUILD_UINT8(CmdId,AttrDataType);
        *pTemp++ = IO_UINT16( AttrId);
        *pTemp++ = HI_UINT16( AttrId);
        /*Error code*/
        if (CmdId > RESERVED_BEFORE_RES)
        {
            *pTemp++ = ErrCode;
            mem_cpy(pTemp,pData,nDataLen);
            pEp = app_finEp(SrcEndpoint);
            if (pEp)
            {
                Status = apsdeDataRequest(pDstAddr->AddrMode,&pDstAddr->Addr, pDstAddr->EP, pEp->SimpleDescriptor.AppProf:
                    ClusterId, SrcEndpoint,nBufLen,pBuffer,TxOptions, FALSE, DEFAULT_RADIUS);
            }
            else
            {
                Status = ZDP_INVALID_EP;
            }
        }
        mem_freeMem(pBuffer);
    }
}

```

Figure 63. Snapshot of Apf_sendSingleKVP

- In the figure 63, function Apf_sendSingleKV performs a main functionality of application framework which is format data in to Key Value Pair and send it to a specified destination defined by pDstAddr. In this example, we specified the input value of the message, and monitor what is sent to the network layer. By comparing the input parameter with the frame sent to network layer, we can confirm the correctness of the Key Value Pair functionality of AF.
- In the figure 64, function Apf_sendSingleMsg perform a main functionality of AF. To test this function, we will also put a set of input data into parameter of function and monitor the message which is sent to the network layer.


```

//***** Variable declaration *****//
Z_STATUS apf_sendSingleMsg(BYTE* pData, UINT8 nDataLen, UINT8 TransSeq,
                           AF_ADDR* pDstAddr, BYTE DstEndpoint, BYTE ClusterId, BYTE SrcEndpoint,
                           BYTE TxOptions)
{
    /****** Function body *****//
    /*Calculate length of single msg*/
    nBufLen = 1 + 1 + 1 + nDataLen;
    pBuffer = mem_getMem(nBufLen);
    if (pBuffer)
    {
        /*Header */
        pTemp = pBuffer;
        *pTemp++ = BUILD_UINT8(AF_FRM_MSG, 1);
        *pTemp++ = TransSeq;
        *pTemp++ = nDataLen;
        mem_cpy(pTemp, pData, nDataLen);
        pEp = app_finEp(SrcEndpoint);
        if (pEp)
        {
            Status = apsdeDataRequest(pDstAddr->AddrMode, &pDstAddr->Addr, DstEndpoint, pEp->SimpleDescriptor.AppProfileId,
                                      ClusterId, SrcEndpoint, nBufLen, pBuffer, TxOptions, FALSE, DEFAULT_RADIUS);
        }
        else
        {
            Status = ZDP_INVALID_EP;
            mem_freeMem(pBuffer);
            return Status;
        }
    }
    else
    {
        return TRANSACTION_OVERFLOW;
    }
} // End of functions

//***** @FunctionName: *****//
//***** @Description: This function will send a single KVP using services in APS *****//
//***** @Param *****//

```

Figure 64. Snapshot of Apf_sendSingleMsg

```

//***** @Param par_pMsg - ZAPP_MSG* *****//
//***** @FunctionName: *****//
//***** @Description: This function will send a apf frame *****//
//***** @Param *****//
void apf_dispatchEvent(ZAPP_MSG* par_pMsg)
{
    /****** Variable declaration *****//
    /****** Function body *****//
    /* Incoming data is broadcasting to endpoint */
    if (par_pMsg->EndPointId == EP_ID_BROADCAST)
    {
        /* Send message to application msg queue */
        for (int i=1; i<ZIGBEE_NUM_EP; i++)
        {
            if (EndpointDescriptors[i])
            {
                par_pMsg = (ZAPP_MSG*)par_pMsg;
                par_pMsg->EndPointId = EndpointDescriptors[i]->SimpleDescriptor.EndPoint;
                schedul_sendMsg(TASK_APER_APP, par_pMsg);
            }
        }
    }
    /* Search for endpoint */
    else if (par_pMsg->EndPointId == EP_ID_ZDO)
    {
        zdo_processEvent(par_pMsg);
    }
    else
    {
        /* Send message to application msg queue */
        schedul_sendMsg(TASK_APER_APP, par_pMsg);
    }
} // End of functions

//***** @FunctionName: *****//
//***** @Description: This function will send a apf frame *****//
//***** @Param *****//
Z_STATUS apf_sendFrm(AF_FRM* pFrm, UINT8 DestAddrMode, ADDRESS* pDestAddr, BYTE DstEndpoint,
                    BYTE ClusterId, BYTE SrcEndpoint, BYTE TxOptions)
{
    /****** Variable declaration *****//

```

Figure 65. Snapshot of Apf_dispatchEvent

- c. Figure 65 shows the function `Apf_dispatchEvent` which performs message dispatching function of application layer. Message sent from APS will be sent to correct application endpoint by this function. In this example, we initialize a `par_pMsg` with a predefined endpoint and monitor if this is dispatched to correct endpoint or not.

8.1.1 Application Support Layer (APS)

```

APSME-BIND/UNBIND Service
*****
// @FunctionName: apsmeBindRequest
// @Description: This function will add a binding in the binding table
//               - IEEE addr is mapped to
// @Param
//
Z_STATUS apsmeBindRequest( WORD SrcAddr, UINT8 SrcEndPoint, UINT8 ClusterId,
                           WORD DesAddr, UINT8 DesEndPoint)
{
    ***** Variable declaration *****
    BINDING_ENTRY* pEntry;
    UINT8 index;
    ***** Function body *****
    if (ZIGBEE_DEVICE_TYPE != ZIGBEE_PAN_COOR)
        return APS_ILLEGAL_DEVICE;

    /*Search if existed an such binding*/
    pEntry = ib_searchBindingEntry(SrcAddr, SrcEndPoint, DesAddr, DesEndPoint);
    if (pEntry)
    {
        /*Existed*/
        if (!ib_addCluster(pEntry, ClusterId)) /*Full*/
            return ZDP_TABLE_FULL;
        return ZDP_SUCCESS;
    }
    else
    {
        if (ib_addBindingEntry(SrcAddr, SrcEndPoint, DesAddr, DesEndPoint, 1, &ClusterId))
            return ZDP_SUCCESS;
        else
            return ZDP_TABLE_FULL;
    }
} // End of functions

*****
// @FunctionName: apsmeUnbindRequest
// @Description: This function will remove a binding in the binding table
//

```

Dr:\Works\Projects\Zigbee\Dev\APL\APS\aps.c

Name	Value	Type	Location
pEntry	0xFFFF	BINDING_ENTRY*	0x1101 [SRAM]

Figure 66. Snapshot of `apsmeBindRequest`

- a. Figure 66 shows the `apsmeBindRequest` of APS. To test this primitive, in this example, we put a example binding including a source address (`SrcAddr`), destination address (`DesAddr`) to add to the Information Base of APS. After that, we will check if there exist that binding in the Information Base or not to confirm the correctness of `apsmeBindRequest`.

```

//@Param
//
//*****
Z_STATUS apsmeUnbindRequest(WORD SrcAddr,UINT8 SrcEndPoint, UINT8 ClusterId,
WORD DesAddr,UINT8 DesEndPoint)
{
    /****** Variable declaration *****/
    BINDING_ENTRY* pEntry;
    UINT8 index;
    /****** Function body *****/
    if (ZIGBEE_DEVICE_TYPE != ZIGBEE_PAN_COOR)
        return APS_ILLEGAL_DEVICE;

    /*Search if existed an such binding*/
    pEntry = ib_searchBindingEntry(SrcAddr,SrcEndPoint,DesAddr,DesEndPoint);
    if (pEntry)
    {
        /*Existed*/
        if (ib_removeCluster(pEntry,ClusterId)) /*Full*/
        {
            if(pEntry->numClusterIds == 0) /*remove entry*/
                ib_removeBindingEntry(pEntry);

            return ZDP_SUCCESS;
        }
    }
    return APS_INVALID_BINDING;
} // End of functions

```

Dr:\Works\Projects\Zigbee\Dev\APL\APS\aps.c

Name	Value	Type	Location
pEntry	0x0000	BINDING_ENTRY*	0x10F2 [SRAM]

Figure 67. Snapshot of apsmeUnbindRequest

- b. Figure 67 shows the apsmeUnbindRequest of APS. To test this primitive, in this example, we put a example binding including a source address (SrcAddr), destination address (DesAddr) to add to the Information Base of APS. After that, we will check if that record is removed from the the Information Base or not to confirm the correctness of apsmeBindRequest.
- c. Figure 68 shows the apsdeDataRequest of APS. In this test, we put a set of input data to make a APS data message. That message will be sent to the network layer using nlmeDataRequest. By checking data frame at nlmeDataRequest, we can confirm the correctness of this primitive.

```

pFrm->AckRequest      = (TxOptions&APS_TX_ACK_REQ); \
pFrm->Security         = Ser; \
pFrm->ProfileId       = ProfileId; \
pFrm->ClusterId       = ClusterId; \
pFrm->ApsCounter      = ApsCounter++;

Z_STATUS apsdeDataRequest( UINT8 DestAddrMode, ADDRESS* pDestAddr, BYTE DstEndpoint,
                           UINT16 ProfileId, BYTE ClusterId, BYTE SrcEndpoint,
                           BYTE asduLength, BYTE *asdu, BYTE TxOptions,
                           BYTE DiscoverRoute, BYTE RadiusCounter )
{
    /****** Variable declaration *****/
    UINT8 tHandle;
    P_APS_FRM par_pFrm;
    P_APS_FRM_TOCOORD par_pToCoord;
    UINT8 nLen;
    ADDR_MAP_ENTRY* pAddrMap;
    Z_STATUS Status;
    ACK_RECORD* pAckRec;
    UINT8 index;
    /****** Function body *****/
    /* Keep the transmission to evaluate ACK or resend*/
    tHandle = dataqueue_getFreeSlot(&ApsDataHandler);
    if (tHandle == APSDataHandler.Size)
        return TRANSACTION_OVERFLOW;

    Status = SUCCESS;
    if (DestAddrMode == APS_ADDR_INDIRECT)
    {
        Status = APS_NO_BOUND_DEVICE;
        #if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COORD /*TBD*/
        /*Check there is space for the current node*/
        for (index = 0; index < MAX_APS_INDIRECT_QUEUE_SIZE; index++)
            if (CurIndirectRec[index].Status == APS_INDIRECT_NONE)
                break;

        if (index == MAX_APS_INDIRECT_QUEUE_SIZE)
            Status = TRANSACTION_OVERFLOW; /*Support only number of indirect request at a time*/
        else
        {
            CurIndirectRec[index].pCurBinding =
                ib_searchIterativeBindingEntry(NwkInfo.NwkAddr, SrcEndpoint, NULL);
        }
    }
}

```

Dr:\Works\Projects\Zigbee\Dev\APL\APS\aps.c | Dr:\Works\Projects\Zigbee\Dev\Common\zigbee.c | Dr:\Works\Projects\Zigbee\Dev\NWK\nwk_data.c

Name	Value	Type	Location
ApsDataHandler	{...}	DATA_QUEUE	0x0A9A [SRAM]
RecList	0x0000	DATA_REC*	0x0A9A [SRAM]
FirstIndex	0x00 ''	UINT8	0x0A9C [SRAM]
LastIndex	0x00 ''	UINT8	0x0A9C [SRAM]
Size	0x00 ''	UINT8	0x0A9D [SRAM]

Figure 68. Snapshot of apsdeDataRequest

8.1.2 Zigbee Device Object (ZDO)

Zigbee Device Object is set of function which support for manipulation of data on the Device Object Description. Those function related to data processing and message formatting. For that reason, we can test those function using debugger. A set of data is created as input parameter of every function, then debugger is used to monitor data changing and result of data formatting and updating in Device Object Description. Following snapshot shows examples of using debugger to test those functions.

```

}
}

//*****
//@FunctionName: zdp_EndDeviceBindReq
//@Description: This function will generate a end_device_bind request
//
//@Param
//*****
Z_STATUS zdp_EndDeviceBindReq( AF_ADDR* pDstAddr,
                               UINT16 LocalCoordinator,
                               BYTE endPoint,
                               UINT16 ProfileID,
                               BYTE NumInClusters, BYTE *InClusterList,
                               BYTE NumOutClusters, BYTE *OutClusterList,
                               BYTE SecurityEnable )
{
    BYTE *buf;
    BYTE *pBuf;
    BYTE buflen;
    Z_STATUS stat;

    // Calculate length needed
    buflen = 2 + 1 + 2 + 1 + 1;
    // LocalCoordinator + epIntf + ProfileID + NumInClusters + NumOutClusters
    buflen += (NumInClusters + NumOutClusters);

    // Allocate the buffer
    buf = mem_getMem( buflen );

    if ( buf )
    {
        // Fill in the buffer
        pBuf = buf;

        *pBuf++ = LO_UINT16( LocalCoordinator );
        *pBuf++ = HI_UINT16( LocalCoordinator );

        *pBuf++ = endPoint;

        *pBuf++ = LO_UINT16( ProfileID );
        *pBuf++ = HI_UINT16( ProfileID );
    }
}

```

Watch

Name	Value	Type	Location

Figure 69. Snapshot of Zdp_EndDeviceBindReq

```

//@Param
//*****
//@FunctionName: zdp_NwkAddrReq
//@Description: This function will send a response to a address request
//*****
Z_STATUS zdp_NwkAddrReq( QWORD *pIEEEAddress, BYTE ReqType,
                        UINT8 StartIndex, BOOL SecurityUse )
{
    //***** Variable declaration *****
    AF_MSG AfMsg;
    AF_FRM* pAfFrm;
    WORD BroadAddr;

    //***** Function body *****
    //Init af msg*
    AfMsg.pData = mem_getMem(16);
    if (AfMsg.pData == NULL)
        return TRANSACTION_OVERFLOW;

    AfMsg.TransSeqNumber = zdp_trans++;
    mem_copy(AfMsg.pData, pIEEEAddress, ADDR_LEN_EXTENDED_64);
    AfMsg.pData[ADDR_LEN_EXTENDED_64] = ReqType;
    AfMsg.pData[ADDR_LEN_EXTENDED_64 + 1] = StartIndex;
    AfMsg.DataLen = ADDR_LEN_EXTENDED_64 + 1+1;

    pAfFrm = apf_initFrm(AF_FRM_MSG);
    if (pAfFrm)
    {
        if (apf_addMsg(pAfFrm, &AfMsg))
        {
            /* Send message to the destination */
            BroadAddr = BROADCAST_ADDR;
            apf_sendFrm(pAfFrm, APS_ADDR_SHORT, &BroadAddr, 0,
                       0, EP_ID_ZDO, 0x00);
        }
        apf_releaseFrm(pAfFrm);
        mem_freeMem(AfMsg.pData);
        return SUCCESS;
    }
    return TRANSACTION_OVERFLOW;
} // End of functions

//*****
//@FunctionName:
//@Description: This function will send a response to a address request
//*****

```

Watch

Name	Value	Type	Location

Figure 70. Snapshot of Zdp_NwkAddrReq

```

//@Description: This function will send a response to a address request
//
//@Param
//
//*****
void zdp_NwkAddrRes(    BOOL    bIEEE_AF_ADDR* pDstAddr, BYTE Status,
                      BYTE    *IEEEAddrRemoteDev, UINT16 nwkAddr, BYTE ReqType,
                      BYTE    NumAssocDev, BYTE    StartIndex, UINT16 *NWKAddrAssocDevList,
                      BOOL    SecurityEnable )
{
    BYTE    nTemp;
    BYTE    *buf;
    BYTE    *pBuf;
    BYTE    bufLen;
    Z_STATUS stat;

    // Calculate buffer length needed
    bufLen = 1 + ADDR_LEN_EXTENDED_64 + 1 + 1 + 2;
    /* Status + IEEEAddrRemoteDev + nwkAddr + NumAssocDev + StartIndex */

    //Check kind of request*/
    if ( ReqType == ZDP_ADDR_REQ_EXTENDED )
        bufLen += ((NumAssocDev - StartIndex) * sizeof( UINT16 ));
    else
        NumAssocDev = 0;

    // Allocate the buffer
    buf = mem_getMem( bufLen );

    if ( buf )
    {
        // Fill in the buffer
        pBuf = buf;

        *pBuf++ = Status;

        mem_cpy( pBuf, IEEEAddrRemoteDev, ADDR_LEN_EXTENDED_64 );
        pBuf += ADDR_LEN_EXTENDED_64;

        *pBuf++ = LO_UINT16( nwkAddr );
        *pBuf++ = HI_UINT16( nwkAddr );

        *pBuf++ = NumAssocDev;
    }
}

```

Watch

Name	Value	Type	Location

Figure 71. Snapshot of Zdp_NwkAddrRes

```

//@Description: This function will generate a bin/unbind request and send
//              to the coord.
//
//@Param
//
//*****
Z_STATUS zdp_BindUnbindReq( BOOL BindOrUnbind, AF_ADDR* pDstAddr,
                          BYTE *pSourceAddr, BYTE SrcEndPoint,
                          BYTE ClusterID,
                          BYTE *pDestinationAddr, BYTE DstEndPoint,
                          BYTE SecurityEnable )
{
    BYTE    *pBuf;
    BYTE    buf;
    BYTE    bufLen;
    Z_STATUS stat;

    // Calculate length needed
    bufLen = ADDR_LEN_EXTENDED_64 + 1 + 1 + ADDR_LEN_EXTENDED_64 + 1;
    // SourceAddr + SrcEndPoint + ClusterID + DestinationAddr + DstEndPoint

    // Allocate the buffer
    buf = mem_getMem( bufLen );

    if ( buf )
    {
        pBuf = buf;
        mem_cpy( pBuf, pSourceAddr );
        *pBuf++ = SrcEndPoint;

        *pBuf++ = ClusterID;

        mem_cpy( pBuf, pDestinationAddr, ADDR_LEN_EXTENDED_64 );
        *pBuf++ = DstEndPoint;

        // Send the message
        stat = apf_sendSingleMsg(buf, bufLen, zdp_trans, pDstAddr,
                                EP_ID_ZDO, (BindOrUnbind==TRUE)?Bind_req:Unbind_req,
                                EP_ID_ZDO, 0x00 );

        // Free the memory
        mem_freeMem( buf );
    }
    else
        return Z_FAILURE;
}

```

Watch

Name	Value	Type	Location

Figure 72. Snapshot of Zdp_BindUnbindReq

```

stat = apf_sendSingleMsg(buf, buflen, zdp_trans, pDstAddr,
// Free the memory
mem_freeMem( buf );
}
else
stat = TRANSACTION_OVERFLOW;
return stat;
}
#endif // ZDO_ENDDDEVICE_BIND_REQ
#if defined ( ZDO_ENDDDEVICE_BIND_RSP )
//*****
//FunctionName: zdp_EDBindRes
//Description: This function will generate ED_bin_ to the source
//*****
//Param
//*****
Z_STATUS zdp_EDBindRes(AF_ADDR* pDstAddr, UINT8 Trans, UINT8 Status)
{
//***** Variable declaration *****
//***** Function body *****
return apf_sendSingleMsg(&Status, sizeof(Status), Trans, pDstAddr,
EP_ID_ZDO, End_Device_Bind_rsp,
EP_ID_ZDO, 0x00 );
} // End of functions
#endif // !defined ( ZDO_ENDDDEVICE_BIND_RSP )

#if defined ( ZDO_ENDDDEVICE_BIND_UNBIND_REQ )
//*****
//FunctionName: zdp_BindUnbindReq
//Description: This function will generate a bin/unbind request and send
//to the coord.
//*****
//Param
//*****
Z_STATUS zdp_BindUnbindReq( BOOL BindOrUnbind, AF_ADDR* pDstAddr,
BYTE *pSourceAddr, BYTE SrcEndPoint,

```

Figure 73. Snapshot of Zdp_EDBindRes

```

//*****
//Param
//*****
Z_STATUS zdp_EndDeviceAnnce( UINT16 NwkAddr, BYTE* pIEEEAddr, BYTE SecurityEnable )
{
BYTE Msg[2 + ADDR_LEN_EXTENDED_64];
AF_ADDR BtAddr;

Msg[0] = LO_UINT16( NwkAddr );
Msg[1] = HI_UINT16( NwkAddr );
mem_cpy( &Msg[2], pIEEEAddr, ADDR_LEN_EXTENDED_64 );

BtAddr.Addr.Short = BROADCAST_ADDR;
BtAddr.Addr.Mode = APS_ADDR_SHORT;
return apf_sendSingleMsg(Msg, sizeof(Msg), zdp_trans, &BtAddr, EP_ID_ZDO,
End_Device_annce, EP_ID_ZDO, 0x00 );
}
#endif // !defined ( ZDO_END_DEVICE_ANN )

//*****
// Binding
//*****
#if defined ( ZDO_ENDDDEVICE_BIND_REQ )
//*****
//FunctionName:
//Description:
//*****
//Param
//*****
void zdp_sendEDBindReq( BYTE endPoint )
{
SIMPLE_DESCRIPTOR *sDesc;
ENDPOINT_DESCRIPTOR *epDesc;
AF_ADDR Addr;
if ( endPoint == EP_ID_ZDO )
return; // Can't do for ZDO

epDesc = app_finEp( endPoint );
if ( epDesc )
{
sDesc = &epDesc->SimpleDescriptor;

```



Figure 74. Snapshot of Zdp_EndDeviceAnnce

8.2 Test Scenario

8.2.1 Hardware

The scenario will involve 1 coordinator, 2 routers and 1 endpoint.

8.2.2 Software

The software has 3 steps:

- Network Formation
- Address Resolution
- Sending Data

These three steps are described in 3.4.10. The procedures can be further described using the standard.

9.1.2.1 Network Formation

Step 1: Network Formation

As per standard:

- A node will send beacons periodically and assigned itself as the PAN coordinator
- A node who could hear beacons from the coordinator will try to associate via association request.
- The assigned address will be sent as a response to the request.

Actual Implementation:

- 1 Turn on coordinator
- 2 Coordinator broadcasts beacons
- 3 Turn on router 1
- 4 Associate router 1
- 5 Turn on router 2
- 6 Associate router 2
- 7 Turn on end point
- 8 Associate endpoint to router

Frame Display - FTS4ZB - Air

File Edit View Format Live Filter Options Window Help

Summary: MAC MAC with Auto-traverse

B...	Frame#	Delta	Type	MAC Cnd	Security	Seq#	Ack	Prd	InPan	Dest Addr	Src Addr	Frame...	Timestamp
	1		Cnd	Beacon Req	Disabled	189	N	N	Y	Broadcast	IntaPan.0000000000002222	19	2007-08-02 오후 4:32:03...
	2	00:00:01.9950	Bcn		Disabled	68	N	N	N		2420.0000	28	2007-08-02 오후 4:32:05...
	3	00:00:00.9811	Bcn		Disabled	69	N	N	N		2420.0000	28	2007-08-02 오후 4:32:06...
	4	00:00:00.9804	Bcn		Disabled	70	N	N	N		2420.0000	28	2007-08-02 오후 4:32:07...
	5	00:00:00.9811	Bcn		Disabled	71	N	N	N		2420.0000	28	2007-08-02 오후 4:32:08...
	6	00:00:00.9804	Bcn		Disabled	72	N	N	N		2420.0000	28	2007-08-02 오후 4:32:09...
	7	00:00:00.9811	Bcn		Disabled	73	N	N	N		2420.0000	28	2007-08-02 오후 4:32:10...
	8	00:00:00.9803	Bcn		Disabled	74	N	N	N		2420.0000	28	2007-08-02 오후 4:32:11...
	9	00:00:00.9811	Bcn		Disabled	75	N	N	N		2420.0000	28	2007-08-02 오후 4:32:12...

Frame 1: (Host) Len=19

MAC

MHR:

- Frame Control:
 - Sequence Number: 189
 - Destination PAN Identifier: Broadcast
 - Destination Address: Broadcast
 - Source Address: 0x00:00:00:00:00:22:22
- Radio Supplied Information:
 - Frame Check Sequence: 0x00cc
 - LQI: 255
- MAC Command:
 - MAC Command: Beacon request

Total Frames: 367 Frames Filtered In: 367 Frame #s Selected: 1: (1 total)

For Help Press F1

Figure 75. Steps 1 and 2 for Network Formation

Frame Display - FTS4ZB - Air

File Edit View Format Live Filter Options Window Help

Summary: MAC MAC with Auto-traverse

B...	Frame#	Delta	Type	MAC Cnd	Security	Seq#	Ack	Prd	InPan	Dest Addr	Src Addr	Frame...	Timestamp
	13	00:00:00.3210	Cnd	Assoc Req	Disabled	78	Y	N	N	2420.0000	Broadcast.0000000000005555	22	2007-08-02 오후 4:32:16...
	14	00:00:00.0007	Ack		Disabled	78	N	N	N			6	2007-08-02 오후 4:32:16...
	15	00:00:00.4926	Cnd	Data Req	Disabled	79	Y	N	Y	2420.0000	IntaPan.0000000000005555	19	2007-08-02 오후 4:32:16...
	16	00:00:00.0007	Ack		Disabled	79	N	Y	N			6	2007-08-02 오후 4:32:16...
	17	00:00:00.0013	Cnd	Assoc Rip	Disabled	191	Y	N	Y	2420.0000000000005555	IntaPan.0000	22	2007-08-02 오후 4:32:16...
	18	00:00:00.0006	Ack		Disabled	191	N	N	N			6	2007-08-02 오후 4:32:16...

Frame 13: (Host) Len=22

MAC

MHR:

- Frame Control:
 - Sequence Number: 78
 - Destination PAN Identifier: 0x2420
 - Destination Address: 0x00:00
 - Source PAN Identifier: Broadcast
 - Source Address: 0x00:00:00:00:00:55:55
- Radio Supplied Information:
 - Frame Check Sequence: 0x68da
 - LQI: 255
- MAC Command:
 - MAC Command: Association request
 - Capability Information:
 - Alternate PAN coordinator: No
 - Device type: Full Function Device
 - Power source: Mains
 - Receiver on when idle: Yes
 - Reserved OoD
 - Security capability: No
 - Allocate address: Yes

Total Frames: 367 Frames Filtered In: 367 Frame #s Selected: 13: (1 total)

For Help Press F1

Figure 76. Steps 3 for Network Formation

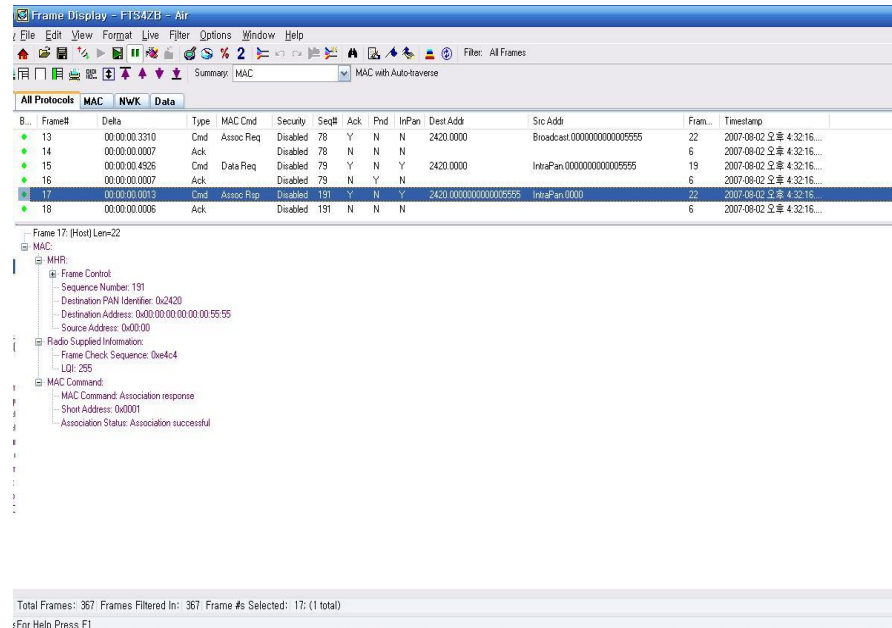


Figure 77. Step 4 of Network Formation

The succeeding steps in network formation are almost the same as 3-4. The only difference is that it could associate to the first router or to the coordinator.

Step 2: Address Resolution

As per standard:

- A broadcast request will be initiated by the source, if a node received the request with the matching ext. address; it will send back its own network address.

Actual Implementation:

- The end point broadcasts NWK resolution requests for a router with external address of 55.55.
- The router with external address 55.55 responds with its NWK address to the requesting end point.

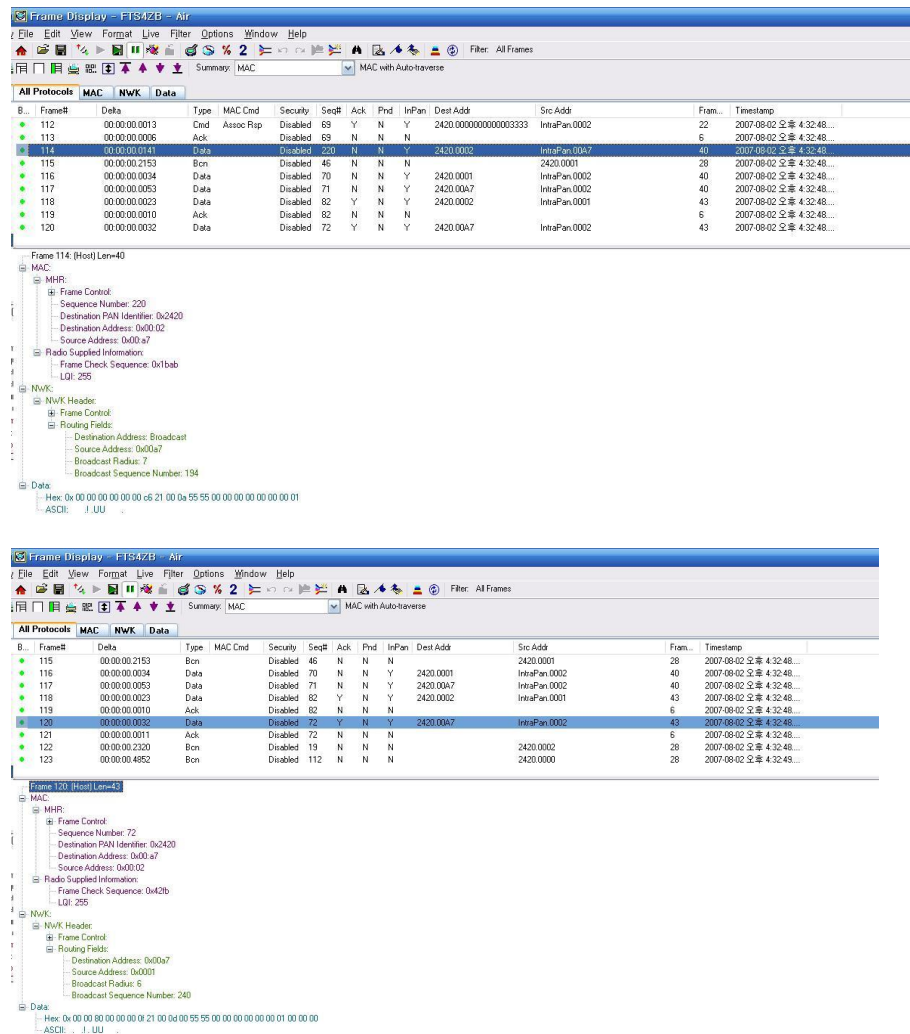


Figure 78. Address Resolution

Step 3: Sending Data

As per standard:

- Tree routing can be used as routing mechanism. Where in a child node can directly send data to/via its parent, using the nwk address of both source and destination.

Actual Implementation

- The end point uses the NWK address of 55.55 to send data.

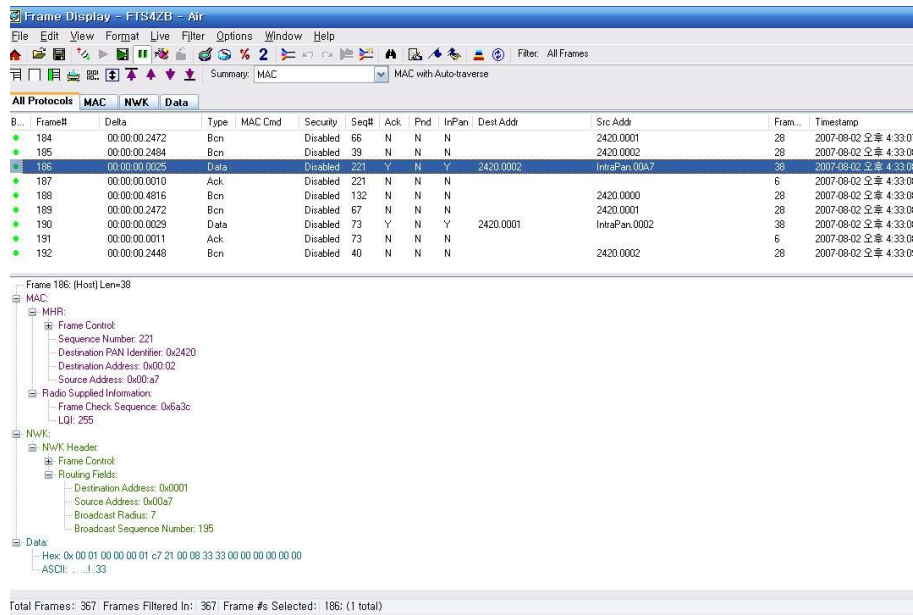


Figure 79. Sending Data

The test software described in the previous section is implemented in the application layer. The software used to get the necessary information in the network is called FTS4ZB Air Sniffer. This software includes a USB probing device.

9 Source Code

9.1 APL

9.1.1 APF

9.1.1.1 apf_format.c

```

/*****
//Module:
//FileName: apf_format
//Description: This file will implement the format of kvp/msg
/* *****/
/* Include files
/*****
#include "apf_format.h"
#include "app_framework.h"
/*****
/* Constant declaration
/*****

/*****
/* Type Definition
/*****
#define      SIZE_OF_MSG_TRANS_HR      2

/*****
/* Variable declaration
/*****

/*****
/* Static Function
/*****

/*****
/* Function
/*****

/*****
//@FunctionName: apf_buildIncomFrm
//@Description: This function will allocate memory and format the incom frm
/*****
AF_IMCOM_FRM* apf_buildIncomFrm(P_APS_FRM par_pFrm, UINT8 nDataLength, UINT8 SrcAddressMode,
ADDRESS* pSrcAddr, UINT8 LinkQuality, BYTE SecurityUse)
{
    /***** Variable declaration *****/
    AF_IMCOM_FRM* pFrm;
    /***** Function body *****/
    pFrm = mem_getMem(sizeof(AF_IMCOM_FRM)+nDataLength);
    if (pFrm)
    {
        pFrm->SrcAddr.AddrMode = SrcAddressMode;
        pFrm->SrcAddr.Addr = *pSrcAddr;
        pFrm->LinkQuality = LinkQuality;
        pFrm->Security = SecurityUse;

        pFrm->Broadcast = (par_pFrm->DeliveryMode == APS_DELIVERY_BROADCAST);
        pFrm->ClusterId = par_pFrm->ClusterId;
    }
}

```

```

    pFrm->SrcEp          = par_pFrm->SrcEndPoint;
    pFrm->CmdLen          = nDataLength;
    mem_cpy(pFrm->Cmd,&par_pFrm->Payload,nDataLength);
    return pFrm;
}
return NULL;
} // End of functions

/*****
//@FunctionName: apf_releaseInComFrm
//@Description:   EP will call this function to release the frame after using
*****/
void apf_releaseInComFrm(AF_IMCOM_FRM* pFrm)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    mem_freeMem(pFrm);
} // End of functions

/*-----
      PARSE OF INCOMMING AF FRAME
-----*/

/*****
//@FunctionName:   apf_getTypeOfFrm
*****/
AF_FRM_TYPE apf_getTypeOfFrm(AF_IMCOM_FRM* pFrm)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    return HI_UINT8(pFrm->Cmd[0]);
} // End of functions

/*****
//@FunctionName: apf_getNumOfTrans
*****/
UINT8 apf_getNumOfTrans(AF_IMCOM_FRM* pFrm)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    return LO_UINT8(pFrm->Cmd[0]);
} // End of functions

/*****
//@FunctionName: apf_getMSGTrans
//@Description:   This function will allocate memory and copy the requested
//               transaction into it. Release mem after use is required
*****/
BOOL apf_getMSGTrans(AF_IMCOM_FRM* pFrm, AF_MSG* pMsg, UINT8 nCount)
{
    /***** Variable declaration *****/
    UINT8      nIndex;
    UINT8      nOffset;
    AF_FRM_TYPE FrmType;
    /***** Function body *****/

```

```

FrmType = HI_UINT8(pFrm->Cmd[0]);
if (nCount > LO_UINT8(pFrm->Cmd[0]))
    return FALSE;

/*Build MSG*/
if (FrmType == AF_FRM_MSG)
{
    nOffset = 1;
    for ( nIndex = 0; nIndex < nCount-1; nIndex++)
    {
        pMsg = &pFrm->Cmd[nOffset];
        nOffset += 1+1;          /*Sequence number+data length*/
        nOffset += pMsg->DataLen; /*Data length*/
    }
    /*Reach the destination*/
    pMsg->TransSeqNumber = pFrm->Cmd[nOffset++];
    pMsg->DataLen        = pFrm->Cmd[nOffset++];
    pMsg->pData           = &pFrm->Cmd[nOffset++];
    return TRUE;
}
return FALSE;
} // End of functions

/*****
//@FunctionName:  apf_getKVPTrans
/*****
AF_KVP* apf_getKVPTrans(AF_IMCOM_FRM* pFrm, UINT8 nCount)
{
    /***** Variable declaration *****/
    UINT8      nIndex;
    UINT8      nOffset;
    AF_FRM_TYPE  FrmType;
    AF_MSG*     pMsg;
    /***** Function body *****/
    FrmType = HI_UINT8(pFrm->Cmd[0]);
    nCount = LO_UINT8(pFrm->Cmd[0]);
    /*Build MSG*/
    if (FrmType == AF_FRM_KVP)
    {
        /*DO it latter*/
    }
    return NULL;
} // End of functions

/*-----
BUILD OF AF FRAME BEFORE SEND
-----*/
/*****
//@FunctionName:  apf_initFrm
//@Description:    This function will init a frm before format it with data
/*****
AF_FRM*  apf_initFrm(AF_FRM_TYPE  par_FrmType)
{
    /***** Variable declaration *****/
    AF_FRM*  pFrm;

```

```

    /***** Function body *****/
    pFrm = mem_getMem(sizeof(AF_FRM));
    if (pFrm)
    {
        pFrm->nCurrOffset    = 1; /*reserver for header*/
        pFrm->TransCount     = 0;
        pFrm->nFrmSize       = 0;
        pFrm->FrmType        = par_FrmType;
        pFrm->pData          = NULL;
    }
    return pFrm;
} // End of functions

/*****
//@FunctionName: apf_addMsg
//@Description:   This function will add new MSG to the FRM after formatting frm
*****/
BOOL apf_addMsg(AF_FRM* par_pFrm, AF_MSG* par_pMsg)
{
    /***** Variable declaration *****/
    BYTE* pBuf;
    /***** Function body *****/
    if (par_pFrm == NULL)
        return FALSE;

    pBuf = par_pFrm->pData;
    par_pFrm->pData = mem_getMem(par_pFrm->nCurrOffset+SIZE_OF_MSG_TRANS_HR+par_pMsg-
>DataLen);
    if (par_pFrm->pData == NULL)
    {
        par_pFrm->pData = pBuf;
        return FALSE;
    }
    else
    {
        par_pFrm->TransCount++;
        mem_cpy(par_pFrm->pData,pBuf,par_pFrm->nCurrOffset);
        par_pFrm->pData[par_pFrm->nCurrOffset++] = par_pMsg->TransSeqNumber;
        par_pFrm->pData[par_pFrm->nCurrOffset++] = par_pMsg->DataLen;
        mem_cpy(&par_pFrm->pData[par_pFrm->nCurrOffset],par_pMsg->pData, par_pMsg->DataLen);
        par_pFrm->nCurrOffset += par_pMsg->DataLen;
        if (pBuf)
            mem_freeMem(pBuf);

        return TRUE;
    }
    return FALSE;
} // End of functions

/*****
//@FunctionName: apf_releaseFrm
//@Description:   This function will release an allocated frm
*****/
void apf_releaseFrm(AF_FRM* par_pFrm)
{
    /***** Variable declaration *****/

```



```

    /***** Function body *****/
    mem_freeMem(par_pFrm->pData);
    mem_freeMem(par_pFrm);
} // End of functions

```

9.1.1.2 app_framework.c

```

/*****
Module:
File Name: app_framework
Description: This file implements the application framework for zigbee app.
-----*/

/*****
Include files
-----*/
#include "zconf.h"
#include "apf_format.h"
#include "app_framework.h"
#include "queue.h"
#include "schedul.h"
#include "aps.h"
#include "ep_conf.h"

/*****
Constant declaration
-----*/

/*****
Type Definition
-----*/

/*****
Variable declaration
-----*/

/*****
Static Function
-----*/

/*****
Function
-----*/
//@FunctionName: app_finEp
/*****
ENDPOINT_DESCRIPTOR* app_finEp(UINT8 Epld)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    for (UINT8 i = 0; i < ZIGBEE_NUM_EP; i++)
        if (EndpointDescriptors[i].SimpleDescriptor.EndPoint == Epld)
            return &EndpointDescriptors[i];
    return NULL;
} // End of functions

```

```

/*****
//@FunctionName:   apf_dispatchEvent
//@Description:    Manages Event handling in application framework
//@Param   par_pMsg - ZAPP_MSG*
*****/
void apf_dispatchEvent(ZAPP_MSG*   par_pMsg)
{
    /***** Variable declaration *****/
    /***** Function body *****/

    /* Incoming data is broadcasting to endpoint */
    if (par_pMsg->EndPointId == EP_ID_BROADCAST)
    {
        /* Send message to application msg queue */
        for (int i=1;i<ZIGBEE_NUM_EP;i++)
        {
            par_pMsg = (ZAPP_MSG*)par_pMsg;
            par_pMsg->EndPointId = EndpointDescriptors[i].SimpleDescriptor.EndPoint;
            schedul_sendMsg(TASK_APER_APP,par_pMsg);
        }
    }
    /* Search for endpoint */
    else if (par_pMsg->EndPointId == EP_ID_ZDO)
        zdo_processEvent(par_pMsg);
    else
        /* Send message to application msg queue */
        schedul_sendMsg(TASK_APER_APP,par_pMsg);
} // End of functions

/*****
//@FunctionName:   apf_sendFrm
//@Description:    This function will send a apf frame
*****/
Z_STATUS apf_sendFrm(   AF_FRM* pFrm, UINT8 DestAddrMode,ADDRESS* pDestAddr, BYTE DstEndpoint,
                        BYTE ClusterId, BYTE SrcEndpoint,BYTE TxOptions)
{
    /***** Variable declaration *****/
    ENDPOINT_DESCRIPTOR* pEp;
    /***** Function body *****/
    pEp = app_finEp(SrcEndpoint);
    pFrm->pData[0] = BUILD_UINT8(pFrm->FrmType,pFrm->TransCount);
    return apsdeDataRequest(DestAddrMode,pDestAddr, DstEndpoint, pEp->SimpleDescriptor.AppProfId,
                            ClusterId, SrcEndpoint,pFrm->nCurrOffset,
                            pFrm->pData,TxOptions, FALSE, DEFAULT_RADIUS);
} // End of functions

/*****
//@FunctionName: apf_sendSingleMsg
//@Description:   This function will build a single msg and send it using
//               services in APS
*****/
Z_STATUS   apf_sendSingleMsg(BYTE* pData, UINT8 nDataLen,UINT8 TransSeq ,
                             UINT8 DestAddrMode,ADDRESS* pDestAddr,
                             BYTE DstEndpoint, BYTE ClusterId, BYTE SrcEndpoint,

```

```

                                BYTE TxOptions)
{
    /****** Variable declaration *****/
    BYTE*      pBuffer;
    BYTE        *pTemp;
    BYTE        nBufLen;
    Z_STATUS    Status;
    ENDPOINT_DESCRIPTOR* pEp;
    /****** Function body *****/
    /*Calculate length of single msg*/
    nBufLen = 1 + 1 + 1 + nDataLen;
    pBuffer = mem_getMem(nBufLen);
    if (pBuffer)
    {
        /*Header */
        pTemp = pBuffer;
        *pTemp++ = BUILD_UINT8(AF_FRM_MSG,1);
        *pTemp++ = TransSeq;
        *pTemp++ = nDataLen;
        mem_cpy(pTemp,pData,nDataLen);
        pEp = app_finEp(SrcEndpoint);
        Status = apsdeDataRequest(DestAddrMode,pDestAddr, DstEndpoint, pEp->SimpleDescriptor.AppProfil,
                                ClusterId,      SrcEndpoint,nBufLen,pBuffer,TxOptions,      FALSE,
                                DEFAULT_RADIUS);
        mem_freeMem(pBuffer);
        return Status;
    }
    else
        return TRANSACTION_OVERFLOW;
} // End of functions

/******
//@FunctionName:   apsdeDataConfirm
//@Description:    Implements Data Confirm primitive
//@Param: DestAddrMode - Destination Address Mode
//          DestAddr - Destination Address
//          DstEndpoint - Destination Endpoint
//          SrcEndpoint - Source Endpoint
//          Status - the result of the request
*/
void apsdeDataConfirm(UINT8 DestAddrMode,ADDRESS* DestAddr,
                     BYTE DstEndpoint, BYTE SrcEndpoint,
                     APS_STATUS Status)
{
    /****** Variable declaration *****/
    ZAPP_MSG      appMsg;
    /****** Function body *****/
    appMsg.EndPointId = SrcEndpoint;
    appMsg.MsgType = APF_DATA_CONFIRM;
    appMsg.Data = DstEndpoint;
    apf_dispatchEvent(&appMsg);
} // End of functions

/******
//@FunctionName:   apsdDataIndication
//@Description:    Implements the Data Indication primitive

```

```

/*****
void apsdDataIndication(P_APS_FRM par_pFrm, UINT8 nDataLength, UINT8 SrcAddressMode, ADDRESS*
SrcAddr,
                        UINT8 LinkQuality, BYTE SecurityUse )
{
    /***** Variable declaration *****/
    ZAPP_MSG      appMsg;
    AF_IMCOM_FRM* pAfFrm;
    /***** Function body *****/
    pAfFrm = apf_buildIncomFrm(par_pFrm, nDataLength, SrcAddressMode,
                              SrcAddr, LinkQuality, SecurityUse);

    if (pAfFrm)
    {
        appMsg.EndPointId = par_pFrm->DestEndpoint;
        appMsg.MsgType    = APF_DATA_INDICATION;
        appMsg.Data       = pAfFrm;
        apf_dispatchEvent(&appMsg);
    }
    /*Omit if can not build*/
} // End of functions

```

9.1.2 APS

9.1.2.1 aps.c

```

/*****
//Module: APL
//File Name: APS.C
//Description: This file implements the Application Data Entity Primitives
//-----*/

/*****
/*      Include files      */
/*****
#include "types.h"
#include "utils.h"
#include "aps.h"
#include "aps_ib.h"
#include "app_framework.h"
#include "zdo.h"
#include "data_queue.h"
#include "ucos_ii.h"
#include "schedul.h"
/*****
//      Constant declaration
/*****
#define      SIZE_OF_FULL_APS_HDR      7
/*****
/*      Type Definition      */
/*****
#define      APS_INVALID_HANDLE      MAX_APS_DATA_QUEUE_SIZE

/*Status of data in the handler*/
typedef enum
{
    APS_DATA_BUF      = 0x01,
    APS_DATA_WAIT_ACK
}APS_DATA_STATE;

```

typedef struct

```

{
    void*    pData;
    UINT8    nLen;
    UINT8    SourceEp;
    UINT8    DestEp;
    UINT16    ShortDesAddr;
    UINT8    Radius:6;
    UINT8    DiscoverRoute:2;
}ACK_RECORD;
/*****
/*      Variable declaration      */
*****/
/*Data queue*/
DATA_REC    ApsRecList[MAX_APS_DATA_QUEUE_SIZE];
DATA_QUEUE  ApsDataHandler;
UINT8       ApsCounter;
/*****
//@FunctionName:  aps_init
//@Description:    This function will initialize the APS. Called by ZDO //
*****/
void aps_init()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /*init data queue*/
    dataqueue_init(&ApsDataHandler,ApsRecList,MAX_APS_DATA_QUEUE_SIZE);
    /*Init aps counter*/
    ApsCounter = ramdom_getRandomByte();

    /*Init ib*/
    ib_initAddrMap();
    ib_initBindingTbl();

    aps_checkAck(NULL,NULL);
} // End of functions

/*****
/*      Static Function      */
*****/
//@FunctionName:      aps_checkAck
*****/
void aps_checkAck(void* arg1, void* arg2)
{
    /***** Variable declaration *****/
    NWK_MSG    Msg;
    /***** Function body *****/
    /*Send msg to the queue*/
    Msg.MsgType = APS_ACK_TIMER;
    mtimer_creatTmr(&Msg,
                    (UINT32)(1000*apscMaxAckWaitDureation),
                    TRUE,TASK_APER_NWK);
} // End of functions

```

```

/*****/
//@FunctionName: aps_resend
/*****/
void aps_resend(UINT8 nHandle)
{
    /***** Variable declaration *****/
    ACK_RECORD*      pRec;
    /***** Function body *****/
    pRec = ApsDataHandler.Reclist[nHandle].pData;
    nIdeDataRequest(pRec->ShortDesAddr,pRec->nLen, pRec->pData,nHandle,
                    pRec->Radius,pRec->DiscoverRoute,FALSE);
    /*Do not care the status, just request*/
} // End of functions

/*****/
/*                               Function                               */
/*****/

/*****/
//@FunctionName:  apsdeDataRequest
/*****/
UINT8          tHandle;;
Z_STATUS apsdeDataRequest( UINT8 DestAddrMode,ADDRESS* pDestAddr, BYTE DstEndpoint,
                           UINT16 ProfileID, BYTE ClusterId, BYTE SrcEndpoint,
                           BYTE asduLength, BYTE *asdu, BYTE TxOptions,
                           BYTE DiscoverRoute, BYTE RadiusCounter )
{
    /***** Variable declaration *****/
    P_APS_FRM      par_pFrm;
    UINT8          nLen;
    ADDR_MAP_ENTRY* pAddrMap;
    Z_STATUS        Status;
    ACK_RECORD*     pAckRec;
    /***** Function body *****/

    /*packet memory is kept*/

    /* Keep the transmission to evaluate ACK or resend*/
    tHandle = dataqueue_getFreeSlot(&ApsDataHandler);
    if (tHandle==ApsDataHandler.Size)
        return TRANSACTION_OVERFLOW;

    Status = SUCCESS;
    if (DestAddrMode == APS_ADDR_INDIRECT)
    {
        Status = APS_NO_BOUND_DEVICE;
    }
    else
    {
        /*Format the frame*/
        par_pFrm=
        (P_APS_FRM)mem_getMem(SIZE_OF_FULL_APS_HDR+NWK_NPDU_HEADER_SIZE+asduLength);
        if (par_pFrm)
        {
            par_pFrm = (BYTE*)par_pFrm+NWK_NPDU_HEADER_SIZE;          /*Data convention for
simplification*/

```

```

    par_pFrm->DeliveryMode      = APS_DELIVERY_NORMAL;
    par_pFrm->FrmType           = APS_FRM_DATA;
    par_pFrm->IndirectAddMode    = 0; /*Not care in direct transit */
    par_pFrm->AckRequest         = (TxOptions&APS_TX_ACK_REQ);
    par_pFrm->Sercurity          = FALSE;

    par_pFrm->DestEndpoint       = DstEndpoint;
    par_pFrm->SrcEndPoint        = SrcEndpoint;
    par_pFrm->ProfileId          = ProfileId;
    par_pFrm->ClusterId          = ClusterId;
    par_pFrm->ApsCounter         = ApsCounter++;

    nLen = SIZE_OF_FULL_APS_HDR;
    mem_cpy((BYTE*)par_pFrm + nLen,asdu, asduLength);
    nLen += asduLength;
  }
  else
    Status = TRANSACTION_OVERFLOW;
}

/* Address mode */
if (Status == SUCCESS)
{
    if(DestAddrMode == APS_ADDR_SHORT)
        Status = nldeDataRequest(pDestAddr->Short,nLen,par_pFrm,tHandle,
                                RadiusCounter,DiscoverRoute,FALSE);
    else if(DestAddrMode == APS_ADDR_EXT)
    {
        pAddrMap = ib_searchAddrEntry(0x00,pDestAddr);
        if (pAddrMap)
        {
            /*Put data in the handler*/
            Status = nldeDataRequest(pAddrMap->ShortAddr,nLen,par_pFrm,tHandle,
                                    RadiusCounter,DiscoverRoute,FALSE);
            pDestAddr->Short = pAddrMap->ShortAddr; /*For latter use*/
        }
        else
            Status = APS_NO_BOUND_DEVICE;
    }
}
#ifdef ZIGBEE_STUB
    nldeDataIndication(DestAddrMode, pDestAddr, nLen, par_pFrm, 0x00);
#endif

if (Status == SUCCESS)
{
    /*Add packet to the queue*/
    pAckRec = mem_getMem(sizeof(ACK_RECORD));
    if (pAckRec)
    {
        pAckRec->SourceEp      = SrcEndpoint;
        pAckRec->DestEp         = DstEndpoint;
        pAckRec->DiscoverRoute  = DiscoverRoute;
        pAckRec->Radius         = RadiusCounter;
        pAckRec->ShortDesAddr   = pDestAddr->Short;
        pAckRec->nLen            = nLen;
        pAckRec->pData          = par_pFrm;
        dataqueue_addRec(&ApsDataHandler,pAckRec,sizeof(ACK_RECORD),tHandle,
                        par_pFrm->ApsCounter,APS_DATA_BUF);
    }
}

```

```

    /*If ACK*/
    if (TxOptions&APS_TX_ACK_REQ)
        ApsDataHandler.RecList[tHandle].Status      = APS_DATA_WAIT_ACK;

        ApsDataHandler.RecList[tHandle].Reserved    = 0;          /*The first time, init
it to 0*/

        Status                                       = SUCCESS;
    }
    else
        Status = TRANSACTION_OVERFLOW;
    }
    if (Status != SUCCESS)
        /*Release the memory*/
        mem_freeMem((BYTE*)par_pFrm-NWK_NPDU_HEADER_SIZE);
}

return Status;
} // End of functions

/*****
//@FunctionName:      nldeDataIndication
*****/
void nldeDataIndication(UINT8 SrcAddrMode, ADDRESS* SrcAddress, BYTE NsduLength, BYTE *Nsdu,
BYTE LinkQuality)
{
    /****** Variable declaration *****/
    P_APS_FRM      pFrm;
    P_APS_FRM      pACK;
    ACK_RECORD*    pRec;
    UINT8          index;

    /****** Function body *****/
    pFrm = (P_APS_FRM)Nsdu;
    switch(pFrm->FrmType)
    {
    case APS_FRM_ACK:
        if (pFrm->DeliveryMode == APS_DELIVERY_NORMAL)
        {
            pACK = pFrm;
            index = dataqueue_findRec(&ApsDataHandler,pACK->ApsCounter);
            if (index != APS_INVALID_HANDLE)
            {
                pRec = ApsDataHandler.RecList[index].pData;
                /*Check the ack*/
                if (pACK->SrcEndPoint == pRec->DestEp)
                {
                    /*Inform the requester*/
                    apsdeDataConfirm(APS_ADDR_INDIRECT,NULL,pRec->DestEp,pRec-
>SourceEp,SUCCESS);

                    /*remove the frames and ack record*/
                    mem_freeMem((BYTE*)(pRec->pData) - NWK_NPDU_HEADER_SIZE);/*Data
convention: pay attention*/
                    mem_freeMem(pRec);

                    /*Remove data handler*/
                    dataqueue_removeRec(&ApsDataHandler,index);

```



```

    }
  }
}
break;
case APS_FRM_CMD:
  break;
case APS_FRM_DATA:
  if (pFrm->DeliveryMode == APS_DELIVERY_NORMAL)
  {
    pFrm = (P_APS_FRM)Nsdu;
    if (app_finEp(pFrm->DestEndpoint))
    {
      apsdDataIndication(pFrm, NsduLength - SIZE_OF_FULL_APS_HDR,
        (SrcAddrMode==AM_SHORT_16)?APS_ADDR_SHORT:APS_ADDR_EXT,
          SrcAddress,LinkQuality,FALSE);

      if (pFrm->AckRequest)
      {
        /* Send the ACK*/
        pACK = (P_APS_FRM)mem_getMem(sizeof(P_APS_FRM));
        if (pACK)
        {
          mem_cpy(pACK,pFrm,SIZE_OF_FULL_APS_HDR);
          pACK->FrmType = APS_FRM_ACK;
          nldeDataRequest(SrcAddress-
            >Short,SIZE_OF_FULL_APS_HDR,pACK,APS_INVALID_HANDLE,
              DEFAULT_RADIUS,FALSE,FALSE);
        }
      }
    }
  }
}
else if (pFrm->DeliveryMode == APS_DELIVERY_INDIRECT)
{
  /*TBD: different frame format */
}
else if (pFrm->DeliveryMode == APS_DELIVERY_BROADCAST)
{
  /*TBD: different frame format */
}
break;
default:
  break;
}
} // End of functions
/*****
**@FunctionName:      nldeDataConfirm
**/
void nldeDataConfirm(BYTE NsduHandle, NWK_STATUS Status)
{
  /***** Variable declaration *****/
  ACK_RECORD*        pRec;
  P_APS_FRM          par_pFrm;
  /***** Function body *****/
  /*Check the handler*/
  if (NsduHandle == APS_INVALID_HANDLE) /*This is non-ack packet*/
  {

```

```

    /*Omit it*/
}
else if(NsduHandle < APS_INVALID_HANDLE)
{
    pRec = ApsDataHandler.RecList[NsduHandle].pData;
    par_pFrm = pRec->pData;
    if (par_pFrm->DeliveryMode == APS_DELIVERY_INDIRECT)
    {
        /*Consider later*/
    }
    else if (par_pFrm->DeliveryMode == APS_DELIVERY_NORMAL)
    {
        if (par_pFrm->AckRequest) /*Hold to resend if need*/
        {
            if (Status == SUCCESS)
                ApsDataHandler.RecList[NsduHandle].Status = APS_DATA_WAIT_ACK;
            else
                ApsDataHandler.RecList[NsduHandle].Status = APS_DATA_WAIT_ACK;
        }
        else /*Release the memory*/
        {
            /*Inform the requester*/
            apsdeDataConfirm(0,NULL,pRec->DestEp,pRec->SourceEp,SUCCESS);

            /*remove the frames and ack record*/
            mem_freeMem((BYTE*)(pRec->pData) - NWK_NPDU_HEADER_SIZE);/*Data convention:
pay attention*/
            mem_freeMem(pRec);

            /*Remove data handler*/
            dataqueue_removeRec(&ApsDataHandler,NsduHandle);
        }
    }
    else if (par_pFrm->DeliveryMode == APS_DELIVERY_BROADCAST)
    {
        mem_freeMem((BYTE*)par_pFrm-NWK_NPDU_HEADER_SIZE) ; /*Data convention*/
    }
    else /*A-normal case*/
    {
#ifdef ZIGBEE_DEBUG
        err(ERR_UNRECOVERABLE,0,0,0);
#endif
    }
}
} // End of functions

/*****
//@FunctionName: nwk_processApsEvent
*****/
void nwk_processApsEvent(NWK_MSG* pMsg, void* arg)
{
    /****** Variable declaration *****/
    UINT8 Index;
    DATA_QUEUE* pQueue;

```

```

ACK_RECORD*          pRec;
P_APS_FRM            pFrm;

/****** Function body *****/
if (pMsg->MsgType == APS_ACK_TIMER)
{
    /*check the handler queue*/
    pQueue          = &ApsDataHandler;
    Index           = pQueue->FirstIndex;
    while (Index != pQueue->Size && pQueue->RecList[Index].pData) /* is the valid packet*/
    {
        pRec = pQueue->RecList[Index].pData;
        if (pQueue->RecList[Index].Status == APS_DATA_WAIT_ACK)
        {
            if (pQueue->RecList[Index].Reserved++ == apscMaxFrameRetries)
            {
                /*Inform the requester*/
                apsdeDataConfirm(APS_ADDR_INDIRECT, NULL, pRec->DestEp, pRec-
>SourceEp, SUCCESS);

                /*remove the frames and ack record*/
                mem_freeMem((BYTE*)(pRec->pData) - NWK_NPDU_HEADER_SIZE); /*Data
convention: pay attention*/
                mem_freeMem(pRec);

                /*Remove data handler*/
                dataqueue_removeRec(&ApsDataHandler, Index);
            }
            else
            {
                aps_resend(Index); /*resend*/

                pQueue->RecList[Index].Status = APS_DATA_WAIT_ACK;
                pQueue->RecList[Index].Reserved++; /*Increase the count*/
            }
        }
        Index = pQueue->RecList[Index].NextIndex;
    }
}
} // End of functions

```

9.1.2.2 aps_ib.c

```

/******
//Module: APL
//File Name: aps_ib
//Description: This file will implement the internal information block of APS
//-----*/

/******
/*      Include files      */
/******
#include "types.h"
#include "zconf.h"
#include "utils.h"
#include "aps_ib.h"
#include "linklist.h"

```

```

/*****/
/*                                     Constant declaration
*/
/*****/
#define      ADDRMAP_SHORT_ADDR      0x02
#define      ADDRMAP_EXT_ADDR        0x03

/*****/
/*                                     Type Definition
*/
/*****/
/*                                     Variable declaration
*/
/* Address mapping
*/
LINK_LIST      AddrMap;
UINT8          nNumofAddrEntry;
/* Binding table
*/
LINK_LIST      BindingTbl;
UINT8          nNumofBindingEntry;
BINDING_ENTRY* pCurBindingEntry;
/*****/
/*                                     Static Function
*/
/*****/
//@FunctionName: ib_cmpAddrMap
//@Description:   This function will compare two address entry
/*****/
BOOL      ib_cmpAddrMap(ADDR_MAP_ENTRY* pAddrEntry, void* Sample,UINT8 cmpOption )
{
    /***** Variable declaration *****/

    /***** Function body *****/
    if(cmpOption == ADDRMAP_SHORT_ADDR)
        return (pAddrEntry->ShortAddr == *(UINT16*)Sample);
    if (cmpOption == ADDRMAP_EXT_ADDR)
        return (pAddrEntry->ExtAddr == *(QWORD*)Sample);
    return FALSE;
} // End of functions

/*****/
//@FunctionName:   ib_cmpBindingEntry
//@Description:     This function will compare to find an appropriate binding
//                  entry. Value to compare will be corresponding to cmpOption
/*****/
BOOL      ib_cmpBindingEntry(BINDING_ENTRY* pBindEntry, void* pSample, UINT8 cmpOption)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    return FALSE;
} // End of functions

/*****/
/*                                     Function
*/

```

```

/*****/

/*-----
                                ADDRESS MAPPING FUNCTION
-----*/
/*****/

//@FunctionName: ib_initAddrMap
//@Description:   This function will initialize the addr mapping
/*****/

void    ib_initAddrMap()
{
    /***** Variable declaration *****/
    /***** Function body *****/
    llist_init(&AddrMap);
    nNumofAddrEntry= 0;
} // End of functions


/*****/

//@FunctionName: ib_addAddrEntry
//@Description:   This function will add a new entry in the addr mapping
/*****/

BOOL    ib_addAddrEntry(UINT16  ShortAddr,QWORD*   pExtAddr)
{
    /***** Variable declaration *****/
    ADDR_MAP_ENTRY*   pEntry;
    /***** Function body *****/
    pEntry  = (ADDR_MAP_ENTRY*)mem_getMem(sizeof(ADDR_MAP_ENTRY));
    if(pEntry)
    {
        pEntry->ExtAddr    = *pExtAddr;
        pEntry->ShortAddr  = ShortAddr;
        llist_add(&AddrMap,pEntry);
        nNumofAddrEntry++;
        return TRUE;
    }
    return FALSE;
} // End of functions


/*****/

//@FunctionName:  ib_searchAddrEntry
//@Description:    This function will search a entry in the addr mapping
/*****/

ADDR_MAP_ENTRY*   ib_searchAddrEntry(UINT16  ShortAddr,QWORD*   pExtAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    if (ShortAddr != 0)
        return llist_search(&AddrMap,ib_cmpAddrMap,&ShortAddr,ADDRMAP_SHORT_ADDR);
    if (pExtAddr != NULL)
        return llist_search(&AddrMap,ib_cmpAddrMap,pExtAddr,ADDRMAP_EXT_ADDR);
    return NULL;
} // End of functions


/*****/

//@FunctionName:  ib_removeAddrEntry
//@Description:    This function will remove a entry in the addr mapping

```

```

/*****
void    ib_removeAddrEntry(UINT16  ShortAddr,QWORD*   pExtAddr)
{
    /***** Variable declaration *****/
    ADDR_MAP_ENTRY*   pEntry;
    /***** Function body *****/
    if (ShortAddr != 0)
    {
        pEntry = llist_remove(&AddrMap,ib_cmpAddrMap,(void*)&ShortAddr,ADDRMAP_SHORT_ADDR);
        if (pEntry)
        {
            nNumofAddrEntry--;
            mem_freeMem(pEntry);
        }
    }
    if (pExtAddr != NULL)
    {
        pEntry = llist_remove(&AddrMap,ib_cmpAddrMap,pExtAddr,ADDRMAP_EXT_ADDR);
        if (pEntry)
        {
            nNumofAddrEntry--;
            mem_freeMem(pEntry);
        }
    }
} // End of functions

```

```

/*-----
                                BINDING TABLE
-----*/
/*****
//@FunctionName:   ib_initBindingTbl
//@Description:     This function will initialize the binding table
/*****
void    ib_initBindingTbl()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    llist_init(&BindingTbl);
    nNumofBindingEntry = 0;
    pCurBindingEntry = NULL;
} // End of functions

/*****
//@FunctionName:   ib_addBindingEntry
//@Description:     This function will add a binding entry into the binding table */
/*****
BOOL ib_addBindingEntry(QWORD*   srcAddr,          UINT8 srcEP,
                        QWORD*   dstAddr,          UINT8 dstEP,
                        BYTE      numClusterIds,    BYTE*   clusterIds)
{
    /***** Variable declaration *****/
    BINDING_ENTRY*   pBindingEntry;
    /***** Function body *****/
    if (numClusterIds > MAX_BINDING_CLUSTER_IDS)
        return FALSE;

```

```

pBindingEntry = mem_getMem(sizeof(BINDING_ENTRY));
if (pBindingEntry)
{
    pBindingEntry->srcAddr = *srcAddr;
    pBindingEntry->srcEP = srcEP;
    pBindingEntry->dstAddr = *dstAddr;
    pBindingEntry->dstEP = dstEP;
    pBindingEntry->numClusterIds = numClusterIds;
    for (int i = 0; i < numClusterIds; i++)
        pBindingEntry->clusterIdList[i] = clusterIds[i];
    llist_add(&BindingTbl, pBindingEntry);
    return TRUE;
}
return FALSE;
} // End of functions

/*****
//@FunctionName: ib_removeBindingEntry
//@Description: This function will remove and binding entry in the binding
//              table
*****/
void ib_removeBindingEntry(BINDING_ENTRY* pEntry)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    llist_removeRec(&BindingTbl, (void*)pEntry);
} // End of functions

/*****
//@FunctionName: ib_searchIterativeBindingEntry
//@Description: This function will search an binding entry matched the given
//              source addr and end point. If srcAddr & srcEP matched again
//              the last call of this function, the next entry will be returned
*****/
BINDING_ENTRY* ib_searchIterativeBindingEntry(QWORD* srcAddr, UINT8 srcEP)
{
    /***** Variable declaration *****/
    BINDING_ENTRY* pCurrentMsg;
    /***** Function body *****/
    /* If this search matched the previous search */
    if (pCurBindingEntry != NULL
        && pCurBindingEntry->srcAddr == *srcAddr
        && pCurBindingEntry->srcEP == srcEP)
    {
        /* Continue the search */
        pCurrentMsg = (BINDING_ENTRY*)pCurBindingEntry;
    }
    else
    {
        /* Restart the search */
        pCurrentMsg = (BINDING_ENTRY*)BindingTbl.pHead;
    }

    while (pCurrentMsg != NULL) /* Loop through the queue */
    {
        if (pCurrentMsg->srcAddr == *srcAddr
            && pCurrentMsg->srcEP == srcEP)
        {
            pCurBindingEntry = pCurrentMsg;
            return pCurrentMsg;
        }
    }
}

```

```

    }
    pCurrentMsg = pCurrentMsg->pNext;
  }
  return NULL;
} // End of functions

/*****
//@FunctionName: ib_restartIterativeBindingEntry
//@Description: This function will restart iterative search procedure
*****/
void ib_restartIterativeBindingEntry()
{
    /***** Variable declaration *****/
    /***** Function body *****/
    pCurBindingEntry = NULL;
} // End of functions

```

9.1.3 ZDO

9.1.3.1 zdo.c

```

/*****
//Module: APL/ZDO
//File Name: zdo.c
//Description: This file implement the behaviors of the ZDO of Coordinator,
// Router or End Device in zigbee network
//-----*/

/*****
/* Include files */
*****/
#include "types.h"
#include "app_framework.h"
#include "zdp.h"
#include "zdo.h"
#include "aps.h"
#include "zdo_app_interface.h"
#include "zconf.h"
#include "schedul.h"
#include "ucos_ii.h"

/*****
/* Constant declaration */
*****/

/*****
/* Type Definition */
*****/
typedef enum
{
    DEV_INIT = 0x01, /* Start device from beginning, in the initialization mode*/
    DEV_STARTED /* Device started, in the normal mode*/
}DEV_STATE;

/*****
/* Variable declaration */
*****/

```



```

/*Network descriptor in Nwk layer*/
extern NWK_DESC*          nwk_desc[MAX_NWK_DESC_SIZE];

/* Configuration Attribute Definitions
*/
/*----Config_NWK_Mode_and_Params----*/
UINT8      confDeviceType      =  ZIGBEE_DEVICE_TYPE;
DWORD      confChannelList     =  conf_CHANNEL_LIST;
UINT8      confScanDuration    =  conf_SCAN_DURATION;
WORD       confPandId          =  conf_PAN_ID;
UINT8      confProtocolVers    =  conf_PROTOCOL_VERS;
WORD       confStackProfile    =  conf_PROFILE_ID;
UINT8      confBeaconOrder     =  conf_BEACON_ORDER;
UINT8      confSuperFrmOrder   =  conf_SUP_FRM_ORDER;
BOOL       confBattLifeExt     =  conf_BATT_EXT;
UINT8      confScanAttempts    =  conf_SCAN_ATTPEMPT;
/*--Config_NWK_Time_btwn_Scans--*/
UINT8      confTimeBetweenScans=  conf_TIME_BWT_SCANS;
/*--Config_Max_Bind--*/
UINT8      confMaxBind         =  conf_MAX_BINDING;
/*--Config_EndDev_Bind_Timeout--*/
UINT8      confBindingTimeout  =  conf_BINDING_TIMEOUT;
/*Config_Permit_Join_Duration*/
UINT8      confPermitJoinDuration= conf_PERMIT_JOINT_DUR;
/*Config_NWK_Leave_removeChildren*/
BOOL       confLeaveRemoveChildrens=conf_REMOVE_CHILDREN;
/*Config_NWK_BroadcastDeliveryTime*/
UINT8      confBroacastDelivTime;
/*Config_NWK_TransactionPersistenceTime*/
UINT8      confTransPersistenceTime=conf_TRANSACTION_PERSISTENCE_TIME;
/*Config_NWK_indirectPollRate*/
UINT8      confIndirectPollRate  =conf_INDIRECT_POLL_RATE;
/*Config_Max_Assoc*/
UINT8      confMaxAssoc          =conf_MAX_ASSOCIATION;
/*Config_NWK_Join_Direct_Addrs*/
void*      confJoinDirectAddrs   =NULL;

/* Device state
*/
DEV_STATE   DevState;
UINT8      NunRetries;
/*****/

/*      Static Function      */
/*****/

/*****/

/*      Function      */
/*****/

#if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR
/*****/
/*@FunctionName:  zdo_startCoord
*/

```

```

//@Description:
//
//@Param
//
/*****
void zdo_startCoord(ZAPP_MSG*      pMsg)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    switch(pMsg->MsgType)
    {
        case MSG_ZDO_START:
            /* ZDO now is in INITIALIZATION STATE */
            nlmeResetRequest();           // Call NWK reset
            break;
        case MSG_ZDO_CONFIRM_RESET:
            nlmeNetworkFormationRequest(confChannelList & VALID_CHANNELS, confScanDuration,
            confBeaconOrder, confSuperFrmOrder, confPandId,
            confBattLifeExt);
            break;
        case MSG_ZDO_CONFIRM_FORMATION:
            /* Setting permit join */
            nlmePermitJoiningRequest(confPermitJoinDuration);
            /* Set internal parameter to nwk */
            nlmeSetRequest(NWKNETWORKBROADCASTDELIVERYTIME, sizeof(confBroadcastDelivTime),
            &confTransPersistenceTime);
            nlmeSetRequest(NWKTRANSACTIONPERSISTENCETIME, sizeof(confTransPersistenceTime),
            &confTransPersistenceTime);

            /* Device started */
            aps_init();           /* Initialize APS */
            zdo_app_DeviceStart(); /* Inform other endpoint */

            /* ZDO now is in NORMAL STATE */
            DevState = DEV_STARTED;
            break;
        default:
            break;
    }
} // End of functions
#endif //ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR

#if ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER
/*****
//@FunctionName: zdo_startRouter
/*****
void zdo_startRouter(ZAPP_MSG*      pMsg)
{
    /***** Variable declaration *****/
    DISCOR_CONFIRM*      pConf;
    /***** Function body *****/
    switch(pMsg->MsgType)
    {
        case MSG_ZDO_START:
            /* ZDO now is in INITIALIZATION STATE */
            nlmeResetRequest();           // Call NWK reset
            break;

```

```

case MSG_ZDO_CONFIRM_RESET:
    nlmeNetworkDiscoveryRequest(confChannelList & VALID_CHANNELS, confScanDuration);

    break;
case MSG_ZDO_CONFIRM_DIS:
    /*TBD: rejoin or join ? depends on starting mode*/
    pConf = (DISCOR_CONFIRM*)pMsg->Data;
    if (pConf->Status == nwk_SUCCESS)
    {
        if (nwk_checkNwk(confPandId, confChannelList))
        {
            nlmeJoinRequest(confPandId, TRUE, FALSE, confChannelList, confScanDuration,
                !NodePowerDescriptor.CurrentPowerSource,
                ((NodeDescriptor.CapabilityFlags & 0x08) == 0x08),
                CAP_SECURITY_CAPABLE);
            break;
        }
    }
    zdo_retryInit(NULL, NULL);

    break;
case MSG_ZDO_CONFIRM_JOIN:
    if (pMsg->Data != nwk_SUCCESS)
        zdo_retryInit(NULL, NULL);
    else
        nlmeStartRouterRequest(confBeaconOrder, confSuperFrmOrder, confBattLifeExt);
    break;
case MSG_ZDO_CONFIRM_START_ROUTER:
    if (pMsg->Data != nwk_SUCCESS)
        zdo_retryInit(NULL, NULL);
    else
    {
        nlmeSyncRequest(TRUE);

        /* Device started */
        aps_init();          /* Initialize APS */
        zdo_app_DeviceStart(); /* Inform other endpoint */

        /* ZDO now is in NORMAL STATE */
        DevState = DEV_STARTED;
    }
    break;
default:
    break;
    }
} // End of functions
#endif // !ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER

#if ZIGBEE_DEVICE_TYPE == ZIGBEE_ENDPOINT
/*****
// @FunctionName: zdo_startEndpoint
*****/
void zdo_startEndpoint(ZAPP_MSG* pMsg)
{
    /***** Variable declaration *****/
    DISCOR_CONFIRM* pConf;
    /***** Function body *****/

```

```

switch(pMsg->MsgType)
{
case MSG_ZDO_START:
    /* ZDO now is in INITIALIZATION STATE */
    nlmeResetRequest(); // Call NWK reset
    break;
case MSG_ZDO_CONFIRM_RESET:
    nlmeNetworkDiscoveryRequest(confChannelList & VALID_CHANNELS, confScanDuration);

    break;
case MSG_ZDO_CONFIRM_DIS:
    /*TBD: rejoin or join ? depends on starting mode*/
    pConf = (DISCOR_CONFIRM*)pMsg->Data;
    if (pConf->Status == nwk_SUCCESS)
    {
        if (nwk_checkNwk(confPandId, confChannelList))
        {
            nlmeJoinRequest(confPandId, FALSE, FALSE, confChannelList, confScanDuration,
                !NodePowerDescriptor.CurrentPowerSource,
                ((NodeDescriptor.CapabilityFlags&0x08) == 0x08),
                CAP_SECURITY_CAPABLE);
            break;
        }
    }
    zdo_retryInit(NULL, NULL);

    break;
case MSG_ZDO_CONFIRM_JOIN:
    if (pMsg->Data != nwk_SUCCESS)
        zdo_retryInit(NULL, NULL);
    else
    {
        if (confBeaconOrder != NON_BEACON_NWK)
            nlmeSyncRequest(TRUE);

        /* Device started */
        aps_init(); /* Initialize APS */
        zdo_app_DeviceStart(); /* Inform other endpoint */

        /* ZDO now is in NORMAL STATE */
        DevState = DEV_STARTED;
    }
    break;
break;
default:
    break;
}

} // End of functions
#endif //ZIGBEE_DEVICE_TYPE == ZIGBEE_ENDPOINT

/*****
//@FunctionName:      zdo_startDevice
*****/
void      zdo_startDevice(ZAPP_MSG*      pMsg)
{
    #if ZIGBEE_DEVICE_TYPE == ZIGBEE_ENDPOINT

```

```

    zdo_startEndpoint(pMsg);
#endif
#if ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER
    zdo_startRouter(pMsg);
#endif
#if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR
    zdo_startCoord(pMsg);
#endif
} // End of functions

/*****
//@FunctionName:      zdo_init
*****/
void zdo_init(void*   pParam,void*   arg)
{
    /***** Variable declaration *****/
    ZAPP_MSG  Msg;
    /***** Function body *****/
    /* Initialize ZDO */
    Msg.MsgType      = MSG_ZDO_START;
    Msg.EndPointId   = EP_ID_ZDO;
    schedul_sendMsg(TASK_APER_NWK,&Msg);
    TOGGLE_GLED();
} // End of functions

/*****
//@FunctionName:      zdo_retryInit
//@Description:       This function will restart zigbee initialization after a
//                   predefined duration
*****/
void zdo_retryInit(void*   pOstimer,void*   pParam)
{
    /***** Variable declaration *****/
    UINT8   nErr;
    /***** Function body *****/
    if (!pOstimer)
    {
        if (NunRetries++ == ZIGBEE_MAX_RETRIES)
        {
            err(ERR_UNRECOVERABLE,NULL,NULL,0); /*Report error: error in starting*/
        }
        pOstimer
        OSTmrCreate(0,ZIGBEE_RETRY_DELAY*3125,OS_TMR_OPT_ONE_SHOT,zdo_retryInit,NULL,NULL,&nErr);
        OSTmrStart(pOstimer,&nErr);
    }
    else
    {
        OSTmrDel(pOstimer,&nErr);
        zdo_init(NULL,NULL);
    }
} // End of functions

/*****
//@FunctionName:      zdo_processData
*****/
void zdo_processData(ZAPP_MSG*   pMsg)
{
    /***** Variable declaration *****/

```

```

AF_IMCOM_FRM*   pAfFrm;
AF_MSG          AfMsg;
/***** Function body *****/
pAfFrm = pMsg->Data;
if (!apf_getMSGTrans(pAfFrm,&AfMsg,1))
    return;

switch(pAfFrm->ClusterId)
{
    /*Response*/
    case NWK_addr_rsp:
        /*Check the status*/
        if (AfMsg.pData[0] == SUCCESS)
            ib_addAddrEntry(*(WORD*)(AfMsg.pData+1/*Status*/+ADDR_LEN_EXTENDED_64/*Ext addr*/),
                            (QWORD*)&AfMsg.pData[1]/*Status*/);

        break;
    case IEEE_addr_rsp:
        break;
    /*Request*/
    case NWK_addr_req:
        if (*(QWORD*)AfMsg.pData == aExtendedAddress)
        {
            zdp_NwkAddrRes(FALSE, &pAfFrm->SrcAddr, SUCCESS,&aExtendedAddress,NwkInfo.NwkAddr,
                           ZDP_ADDR_REQ_SINGLE, 0, 0,NULL,FALSE);
        }

        break;
    case IEEE_addr_req:
        break;
    default:
        break;
}
apf_releaseInComFrm(pAfFrm);
} // End of functions

/*****
//@FunctionName:  zdo_processEvent
*****/
void zdo_processEvent(ZAPP_MSG*   pMsg)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    if (pMsg->MsgType == MSG_ZDO_START)
        DevState = DEV_INIT;
    if (DevState == DEV_INIT)
        zdo_startDevice(pMsg); /* Start device from beginning*/
    else
    {
        switch(pMsg->MsgType)
        {
            case MSG_ZDO_START:
                break;
            case MSG_ZDO_CONFIRM_RESET:
            case MSG_ZDO_CONFIRM_FORMATION:
            case MSG_ZDO_CONFIRM_JOIN:
                err(ERR_UNRECOVERABLE,NULL,0);
                break;
            case MSG_ZDO_CONFIRM_DIS:
                break;
            case MSG_ZDO_INDI_JOIN:
                mem_freeMem(pMsg->Data);
                break;
            case APF_DATA_INDICATION:
                zdo_processData(pMsg);
        }
    }
}

```

```

        break;
    default:
        break;
    }
}

} // End of functions

```

9.1.3.2 zdo_app_interface.c

```

/*****
//Module: APL/ZDO
//File Name: zdo_app_interface.c
//Description: This file implement the interface of event from ZDO
//              to application
*****/

/*****
/*          Include files          */
*****/
#include "zdo_app_interface.h"
#include "app_framework.h"
#include "zconf.h"

/*****
/*          Constant declaration    */
*****/

/*****
/*          Type Definition          */
*****/

/*****
/*          Variable declaration    */
*****/

/*****
/*          Static Function         */
*****/

/*****
/*          Function                */
*****/

/*****
//@FunctionName:  zdo_app_DeviceStart
*****/
void zdo_app_DeviceStart()
{
    /***** Variable declaration *****/
    ZAPP_MSG      Msg;
    /***** Function body *****/
    Msg.EndPointId = EP_ID_BROADCAST;
    Msg.MsgType    = ZDO_DEVICE_STARTED;
}

```

```

    apf_dispatchEvent(&Msg);
} // End of functions

```

9.4.1 ZDP

```

/*****
//Module: APP/ZDP
//File Name: zdp
//Description: This file will implement ZDP functionality of Zigbee stack
*****/

/*****
/*      Include files      */
*****/
#include "zdp.h"
#include "app_framework.h"
#include "apf_format.h"
#include "zdo.h"

/*****
/*      Constant declaration      */
*****/

/*****
/*      Type Definition      */
*****/

/*****
/*      Variable declaration      */
*****/

/*****
/*      Static Function      */
*****/

UINT8      zdp_trans = 0;

/*****
/*      Function      */
*****/

/*****
//@FunctionName: zdp_NwkAddrReq
//@Description: This function will send msg to request NwkAddr corresponding
//              to a Ext address.
*****/
Z_STATUS zdp_NwkAddrReq( QWORD *pIEEEAddress, BYTE ReqType,
                        UINT8 StartIndex, BOOL SecurityUse)
{
    /***** Variable declaration *****/
    AF_MSG      AfMsg;
    AF_FRM*     pAfFrm;
    WORD        BroadAddr;

    /***** Function body *****/

    /*Init af msg*/
    AfMsg.pData = mem_getMem(16);
    if (AfMsg.pData == NULL)
        return TRANSACTION_OVERFLOW;
}

```



```

    AfMsg.TransSeqNumber = zdp_trans++;
    mem_cpy(AfMsg.pData, pIEEEAddress, ADDR_LEN_EXTENDED_64);
    AfMsg.pData[ADDR_LEN_EXTENDED_64] = ReqType;
    AfMsg.pData[ADDR_LEN_EXTENDED_64 + 1] = StartIndex;
    AfMsg.DataLen = ADDR_LEN_EXTENDED_64 + 1 + 1;

    pAfFrm = apf_initFrm(AF_FRM_MSG);
    if (pAfFrm)
    {
        if (apf_addMsg(pAfFrm, &AfMsg))
        {
            /* Send message to the destination */
            BroadAddr = BROADCAST_ADDR;
            apf_sendFrm(pAfFrm, APS_ADDR_SHORT, &BroadAddr, 0,
                        0, EP_ID_ZDO, 0x00);
        }
        apf_releaseFrm(pAfFrm);
        mem_freeMem(AfMsg.pData);
        return SUCCESS;
    }
    return TRANSACTION_OVERFLOW;
} // End of functions

/*****
// @FunctionName: zdp_NwkAddrRes
// @Description: This function will send a response to a address request
*****/
void zdp_NwkAddrRes(    BOOL    bIEEE, AF_ADDR* pDstAddr, BYTE Status,
                        BYTE    *IEEEAddrRemoteDev, UINT16 nwkAddr, BYTE ReqType,
                        BYTE    NumAssocDev, BYTE    StartIndex, UINT16 *NWKAddrAssocDevList,
                        BOOL    SecurityEnable )
{
    BYTE    nTemp;
    BYTE    *buf;
    BYTE    *pBuf;
    BYTE    bufLen;
    Z_STATUS stat;

    // Calculate buffer length needed
    bufLen = 1 + ADDR_LEN_EXTENDED_64 + 1 + 1 + 2;
    /* Status + IEEEAddrRemoteDev + nwkAddr + NumAssocDev + StartIndex */

    /* Check kind of request */
    if (ReqType == ZDP_ADDR_REQ_EXTENDED )
        bufLen += ((NumAssocDev - StartIndex) * sizeof( UINT16 ));
    else
        NumAssocDev = 0;

    // Allocate the buffer
    buf = mem_getMem( bufLen );

    if ( buf )
    {
        // Fill in the buffer
        pBuf = buf;

```

```

    *pBuf++ = Status;

    mem_cpy( pBuf, IEEEAddrRemoteDev, ADDR_LEN_EXTENDED_64);
    pBuf    += ADDR_LEN_EXTENDED_64;

    *pBuf++ = LO_UINT16( nwkAddr );
    *pBuf++ = HI_UINT16( nwkAddr );

    *pBuf++ = NumAssocDev;
    *pBuf++ = StartIndex;

    if ( ReqType == ZDP_ADDR_REQ_EXTENDED )
    {
        // Move past the start index
        for ( nTemp = 0; nTemp < StartIndex; nTemp++ )
            NWKAddrAssocDevList++;

        for ( nTemp = 0; nTemp < (NumAssocDev - StartIndex); nTemp++ )
        {
            *pBuf++ = LO_UINT16( *NWKAddrAssocDevList );
            *pBuf++ = HI_UINT16( *NWKAddrAssocDevList );
            NWKAddrAssocDevList++;
        }
    }

    // Send the message
    nTemp = (bIEEE)? IEEE_addr_rsp : NWK_addr_rsp;
    stat = apf_sendSingleMsg(buf,bufLen, zdp_trans, pDstAddr->AddrMode,
                            &pDstAddr->Addr, EP_ID_ZDO, nTemp,
                            EP_ID_ZDO, 0x00 );

    // Free the memory
    mem_freeMem(buf);
}
else
    stat = TRANSACTION_OVERFLOW;

return stat;
}

```

9.2 APP

```

/*****/
//Module: APP
//File Name: zigbee_app
//Description: This file will implement a simple application on zigbee network
/*
- Scenario:
    + Endpoint device will search for Nwk-Address of a
    + a device with external address is defined in aExDesAddr
    + After received the nwk address of the destination it will
    + send data to that node
- To customize
    + aExDesAddr for the destination coordinator
    + Change parameter of debugapp_sendData for the data to send
    + Change body of function debugapp_processReceivedData to process
    data

```

NOTE: Memory of the system is limited check the memory after you add any static variables.

To check it,

- open the file zigbee.sym in the folder that contains makefile
- Check the location of variable before __bss_end. The location must be smaller than 008010ff.

```

*/
//-----*/

/*****/
#ifndef DEBUGAPP_H
#define DEBUGAPP_H
/*****/
/*      Include files      */
/*****/
#include "types.h"
#include "schedul.h"
#include "app_framework.h"
#include "aps_ib.h"
#include "zdp.h"
#include "apf_format.h"
#include "aps.h"
#include "debugapp.h"
#include "hal_cc2420.h"
#include "zdo.h"
#include "ep_conf.h"
#include "ucos_ii.h"
/*****/
/*      Constant declaration      */
/*****/
#define          DATA_SENDING_INTERVAL          10 /*Second*/
/*****/
/*      Type Definition      */
/*****/
typedef enum
{
    APP_INIT          = 0x01,
    APP_SEARCH_ADDR,
    APP_SEND_DATA
}APP_STATE;
/*****/
/*      Variables Declaration      */
/*****/
UINT8          nTransSeq    = 0;
WORD           aDesShorAddr= INVALID_SHORT_ADDR;
UINT8          AppState     = APP_INIT;

/*Customizable variable: Change this value for the destination coord*/
QWORD          aExDesAddr   = 0x5555;

/*****/
/*      Static Function      */
/*****/

/*****/
/*      Function      */
/*****/

```



```

/*****/
//@FunctionName: debugapp_processReceivedData

//@Description: This function will be call to process the received data that
//              is sent by others DebugApp
//@Param       Note: buffer will be removed after calling of this function
//              To store data, copy it to other location
/*****/
void    debugapp_processReceivedData(void* pBuffer, UINT8 nLen)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /*Change this to customize data processing*/
    if (*(QWORD*)pBuffer == 0x3333)
        TOGGLE_OLED(); /*Just for demo: toggle led if it is addr of device*/
} // End of functions

/*****/
//@FunctionName: debugapp_sendData
//@Description: This function will send data from endpoint the coordinators */
/*****/
void    debugapp_sendData(void* pBuffer, UINT8 nLen)
{
    /***** Variable declaration *****/
    AF_MSG AfMsg;
    AF_FRM* pAfFrm;
    /***** Function body *****/
    /*Init af msg*/
    AfMsg.DataLen = nLen;
    AfMsg.TransSeqNumber = nTransSeq++;
    AfMsg.pData = pBuffer; /**/

    pAfFrm = apf_initFrm(AF_FRM_MSG);
    if (pAfFrm)
    {
        if (apf_addMsg(pAfFrm,&AfMsg))
        {
            /* Send message to the destination */

            apf_sendFrm(pAfFrm,APS_ADDR_SHORT,&aDesShorAddr,EP_ID_DEBUGAPP,0,EP_ID_DEBUGAPP,0x00);
        }
        apf_releaseFrm(pAfFrm);
    }
} // End of functions

/*****/
//@FunctionName: debugapp_createTimer
/*****/
void    debugapp_createTimer(void* pTimer, void* arg2)
{
    /***** Variable declaration *****/

```

```

ZAPP_MSG      Msg;
/***** Function body *****/
Msg.EndPointId = EP_ID_DEBUGAPP;
Msg.MsgType    = DAPP_SEND_DATA;
/*Send control the app*/
mtimer_creatTmr(&Msg,(UINT32)DATA_SENDING_INTERVAL*1000,TRUE,TASK_APER_APP);
} // End of functions

/*****
//@FunctionName: debugapp_searchCoord
*****/
void debugapp_searchCoord(BOOL bCreateTimer)
{
/***** Variable declaration *****/
ADDR_MAP_ENTRY* pAddMap;
/***** Function body *****/
if (aDesShorAddr != INVALID_SHORT_ADDR)
    return;

/*Search for coord.*/
aDesShorAddr = nwk_getShortAddrInNT(&aExDesAddr);
if (aDesShorAddr != INVALID_SHORT_ADDR)
{
    AppState = APP_SEND_DATA;
    return;
}
else /*Search in the APS*/
{
    pAddMap = ib_searchAddrEntry(0,&aExDesAddr);
    if (pAddMap)
    {
        TOGGLE_OLED();
        AppState = APP_SEND_DATA;
        aDesShorAddr = pAddMap->ShortAddr;
        return;
    }
}

/*Search addr*/
AppState = APP_SEARCH_ADDR;
zdp_NwkAddrReq(&aExDesAddr,ZDP_ADDR_REQ_SINGLE,1,1);

if (bCreateTimer)
    debugapp_createTimer(NULL,NULL);          /*To check later*/
} // End of functions

/*****
//@FunctionName: debugapp_task
//@Description: This is the main function of Application Task
*****/
void debugapp_task(void* arg1, void* arg2)
{
/***** Variable declaration *****/
ZAPP_MSG* pMsg;
AF_IMCOM_FRM* pAfFrm;
/***** Function body *****/
pMsg = (ZAPP_MSG*)arg1;

```

```

switch(pMsg->MsgType)
{
    /*Process general event */
    case ZDO_DEVICE_STARTED:
#ifdef Z_SOURCE
        debugapp_searchCoord(TRUE); /*Created periodical timer*/
#endif
        break;
    case APF_DATA_INDICATION:
        pAfFrm = pMsg->Data;
        debugapp_processReceivedData(&pAfFrm->Cmd[1+1+1/*Header*/,
                                                &pAfFrm->Cmd[2/*Lenght of Msg*/]);

        apf_releaseInComFrm(pAfFrm);
        break;
    case APF_DATA_CONFIRM:
        break;

    /*Application definition msg
    */
    case DAPP_SEND_DATA:
        switch(AppState)
        {
            case APP_SEARCH_ADDR:
                debugapp_searchCoord(FALSE); /*Timer created*/
                break;
            case APP_SEND_DATA:
                /*Change this input parameter to customize data to send*/
                debugapp_sendData(&aExtendedAddress, sizeof(QWORD));
                break;
            default:
                break;
        }
        break;
    default:
        break;
}
} // End of functions

/*****
//@FunctionName: debugapp_init
//@Description:      This function is called to initialize application at th
//                  first timer
*****/
void debugapp_init()
{
    /*Initialize of the application */
} // End of functions

#endif //DEBUGAPP_H

```




```

    {0,0},          /* Manufacturer Code */
    MAX_BUFFER_SIZE, /* Maximum Buffer Size */
    {0, 0}          /* Maximum Transfer Size*/
};

/* Node power descriptor
*/
NODE_POWER_DESCRIPTOR      NodePowerDescriptor =
{
    #if ZIGBEE_DEVICE_FFD == 1
        0x00,          /* Power mode: Always ON*/
        0x00,          /* main power sources*/
        0x00,          /* current power source: main power source */
        0x0C           /* Power Level */
    #else /* End Device*/
        0x01,          /* Power mode: Periodically ON*/
        0x02,          /* available power sources: battery*/
        0x02,          /* current power source: Battery */
        0x08           /* Power Level: 66% */
    #endif
};

/* device address*/
QWORD      aExtendedAddress = 0x1325;

```

9.3.2 Ztest.c

```

/*****
//Module: Common
//File Name: ztest.c
//Description: zigbee test scenario file
*****/

#include "mac_state.h"
#include "nwk.h"
#include "nwk_msg.h"
#include "app_framework.h"
#include "zdo.h"
#include "schedul.h"
#include "static_timer.h"
#include "phy_primitives.h"
#include "hal.h"
#include "utils.h"
#include "ucos_ii.h"
#include "mac_types.h"
#include "mac_beacon.h"
#include "aps.h"
#include "nwk_common.h"
#include "aps_ib.h"
#include "zdp.h"
#if ZIGBEE_DEVICE_FFD == TRUE
void test_coord()
{
    /***** Variable declaration *****/
    volatile BOOL      gotAssociateRequest;
    volatile WORD      myShortAddr;

```



```

WORD                panId;                // The PAN ID
#define PANID        0x2420
#define COORD_SHORT  0x1975
#define CHANNEL      19
BYTE                pBeaconPayload[8];    // The beacon payload (used initialized)
BYTE                setAttributeValue;
BOOL                isCoordinator;

/***** Function body *****/
// Initialize variables
isCoordinator = TRUE;
gotAssociateRequest = FALSE;
aExtendedAddress = 0x0000004722958919;
myShortAddr = COORD_SHORT;
panId = PANID;

// Reset the MAC layer
mlmeResetRequest(TRUE);

// Modify necessary PIB attributes
setAttributeValue = TRUE;
mlmeSetRequest(MAC_ASSOCIATION_PERMIT, &setAttributeValue);
mlmeSetRequest(MAC_RX_ON_WHEN_IDLE, &setAttributeValue);
mlmeSetRequest(MAC_SHORT_ADDRESS, (BYTE*) &myShortAddr);
setAttributeValue = 0;
mlmeSetRequest(MAC_BEACON_PAYLOAD_LENGTH, &setAttributeValue);
mlmeSetRequest(MAC_BEACON_PAYLOAD, pBeaconPayload);

// Start as a PAN coordinator of a beacon-enabled PAN, with superframe and beacon order 4
mlmeStartRequest(panId, CHANNEL, 4, 4, TRUE, FALSE, FALSE, FALSE, 0);

/* Wait for the data from the device */
while (TRUE);
} // End of functions
#endif

/*****
//@FunctionName: test_IndirectPkt
*****/
#if ZIGBEE_DEVICE_FFD == TRUE
void test_IndirectPkt()
{
    /***** Variable declaration *****/
    P_MPDU_TX        pPkt;
    ADDRESS           Addr;

    /***** Function body *****/
    mlmeResetRequest(TRUE);
    indirectTx_init();

    pPkt = mem_getFreeTxPkt();
    pPkt->Mpdu.dstAddr.Extended = 0x001;
    pPkt->Mpdu.FrmControl.DestAddMode = AM_EXTENDED_64;

    //indirectTx_addPkt(pPkt);

    beacon_initBeacon();

    indirectTx_controlIndirectPkt(NULL, NULL);

```

```

//   pPkt1 = indirectTx_getPkt(&Addr,AM_EXTENDED_64);

//   indirectTx_addPkt(pPkt);
//   indirectTx_addPkt(pPkt);
//   indirectTx_addPkt(pPkt);
indirectTx_addPkt(pPkt);

Addr.Extended = 0x001;

indirectTx_getPkt(&Addr,AM_EXTENDED_64);

indirectTx_removePkt(pPkt);

indirectTx_addPkt(pPkt);
indirectTx_addPkt(pPkt);

indirectTx_removePkt(pPkt);
} // End of functions
#endif
/*****
//@FunctionName:      test_formatBeacon
*****/
#if ZIGBEE_DEVICE_FFD == TRUE
void test_formatBeacon()
{
    /***** Variable declaration *****/
    P_MPDU_TX      pPkt;
    ADDRESS        Addr;
    /***** Function body *****/
    mlmeResetRequest(TRUE);
    indirectTx_init();
    Addr.Extended = 0x001;

    pPkt = mem_getFreeTxPkt();
    pPkt->Mpdu.dstAddr.Extended = 0x001;
    indirectTx_addPkt(pPkt);

    Addr.Extended = 0x003;

    pPkt = mem_getFreeTxPkt();
    pPkt->Mpdu.dstAddr.Extended = 0x003;
    indirectTx_addPkt(pPkt);

    // Start as a PAN coordinator of a beacon-enabled PAN, with superframe and beacon order 4
    mlmeStartRequest(0x00, CHANNEL, 4, 4, TRUE, FALSE, FALSE, FALSE,0);
} // End of functions
#endif
/*****
//@FunctionName:      test_transData
*****/
void test_transData()
{
    /***** Variable declaration *****/
    ADDRESS        srcAddr;
    ADDRESS        dstAddr;

```

```

    BYTE      Data[2];
    /***** Function body *****/
    mlmeResetRequest(TRUE);
    srcAddr.Short      = 0x111;
    dstAddr.Extended   = 0x222;
    Data[0]=2;Data[1]=5;
    mcpsDataRequest(0x02,0x2420, &srcAddr,
                    0x02,0x2420,&dstAddr,
                    0x01,Data, 0x00, 0x01);
} // End of functions

/*****
//@FunctionName: test_rx
*****/
void test_rx()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    mlmeResetRequest(TRUE);
    mac_setTrxState(PHY_RX_ON);
    wait(65000);
    while (TRUE)
    {
        while (CCA_IS_ACTIVE())
            wait(650);

        TOGGLE_RLED();
        wait(6500);
    }
} // End of functions

#if ZIGBEE_DEVICE_FFD == TRUE
/*****
//@FunctionName: test_nwkNT
*****/
MLME_BEACON_NOTIFY_INDICATION BeaconNotif;
void test_nwkNT()
{
    /***** Variable declaration *****/
    PENDING_ADDR_SPEC*      pPendingAddr;
    SUPER_FRAME*           pSuperFrm;
    ADDRESS                 Addr;
    CAPACITY_INFO           Cap;

    /***** Function body *****/
    BeaconNotif.AddrList[0].Short = 0x01;
    pPendingAddr = &BeaconNotif.PendAddrSpec;
    pPendingAddr->NumofShortAddr = 1;

    BeaconNotif.PanDescriptor.CoordAddrMode      = AM_EXTENDED_64;
    BeaconNotif.PanDescriptor.CoordAddress.Extended = 0x02;
    BeaconNotif.PanDescriptor.CoordPANId         = 0x2420;
    BeaconNotif.PanDescriptor.LogicalChannel     = 0x13;
    pSuperFrm = &BeaconNotif.PanDescriptor.SuperframeSpec;
    pSuperFrm->AssoPermit      = 1;

```

```

pSuperFrm->SuperFrmOrder    = 2;
pSuperFrm->BeaconOrder      = 4;
pSuperFrm->PanCoord          = TRUE;

BeaconNotif.Payload.DeviceDepth = 13;
BeaconNotif.Payload.RouterCapacity = 1;
BeaconNotif.Payload.EndDeviceCapacity = 1;

nwk_updateBeacon(&BeaconNotif);

nwk_updateBeacon(&BeaconNotif);

Addr.Extended    = 0x013;
Cap.DeviceType   = ZIGBEE_ROUTER;
Cap.RxOnWhenIdle = TRUE;
Cap.PwrSrc       = TRUE;
nwk_addNewChild(&Addr, 0x025, Cap, FALSE, FALSE);

Addr.Extended    = 0x14;
nwk_addNewChild(&Addr, 0x025, Cap, FALSE, FALSE);

nwk_checkNwkExisted(0x2420);

nwk_removeChild(&Addr);

NwkInfo.NwkJoinedDeviveType = ZIGBEE_PAN_COOR;
nwk_findRouterInNT(0x2420, 0);

```

```
} // End of functions
```

```

/*****
//@FunctionName: test_sendData
*****/

extern UINT8 CurrAperTask;
void test_sendData()
{
    /***** Variable declaration *****/
    ADDRESS      aDesExAddr;
    CAPACITY_INFO Cap;

    /***** Function body *****/
    NwkInfo.NwkState = NWK_STATE_NORMAL_MODE;
    nwk_ib.nwkUseTreeRouting = TRUE;
    aDesExAddr.Extended = 0x013;
    mlmeResetRequest(TRUE);
    aps_init();
    NwkInfo.pParent = mem_getMem(sizeof(NWK_NEIGHBOR_NODE));
    NwkInfo.pParent->NetworkAddress = 0x01;
    NwkInfo.pParent->RxOnWhenIdle = TRUE;
    nwk_neighborTable[0] = NwkInfo.pParent;
    CurrAperTask = TASK_APER_NWK;
    /* Send message to the destination */
    //debugapp_sendData();
    nlmeJoinRequest(confPandId, FALSE, FALSE, confChannelList, confScanDuration,
        !NodePowerDescriptor.CurrentPowerSource,

```

```

CAP_SECURITY_CAPABLE);
} // End of functions

```

((NodeDescriptor.CapabilityFlags&0x08) == 0x08),

```

/*****
//@FunctionName: test_mem
*****/
void test_mem()
{
    /***** Variable declaration *****/
    void* pMem;
    /***** Function body *****/
    pMem = mem_getMem(33);
    mem_freeMem(pMem);
} // End of functions
#endif

/*****
//@FunctionName: test_scheduler
*****/
void test_scheduler()
{
    /***** Variable declaration *****/
    UINT8 Index;
    /***** Function body *****/
    schedul_init();
    schedul_addTask(test_scheduler,TASK_PER_MAC_TX,Index++);
    schedul_addTask(test_scheduler,TASK_PER_MAC_TX,Index++);
    schedul_addTask(test_scheduler,TASK_PER_MAC_TX,Index++);
    schedul_removeTask(TASK_PER_MAC_TX);
    schedul_addTask(test_scheduler,TASK_PER_MAC_TX,Index++);
    schedul_removeTask(TASK_PER_MAC_TX);
    schedul_removeTask(TASK_PER_MAC_TX);
    schedul_removeTask(TASK_PER_MAC_TX);
    schedul_removeTask(TASK_PER_MAC_TX);
} // End of functions

/*****
//@FunctionName: test_timer
*****/
void test_timer()
{
    /***** Variable declaration *****/
    OS_TMR* pTestTimer;
    UINT8 nErr;
    /***** Function body *****/
    pTestTimer = OSTmrCreate(0,10,OS_TMR_OPT_ONE_SHOT,test_timer,NULL,NULL,&nErr);
    OSTmrStart(pTestTimer,&nErr);
    OSTmrStart(pTestTimer,&nErr);
} // End of functions

/*****
//@FunctionName: test_indiData
*****/
NPDU npdu;

```

```

MCPS_DATA_INDICATION indiData;
void test_indiData()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    npdu.DesAddr = 0;
    npdu.FrmType = NWK_FRM_DATA;
    indiData.pMsdu = &npdu;
    mcpsDataIndication(&indiData);
} // End of functions

/*****
//@FunctionName: test_app
*****/
extern UINT8 CurrAperTask;
void test_app()
{
    /***** Variable declaration *****/
    ADDRESS aDesExAddr;
    CAPACITY_INFO Cap;

    /***** Function body *****/
    NwkInfo.NwkState = NWK_STATE_NORMAL_MODE;
    nwk_ib.nwkUseTreeRouting= TRUE;
    aDesExAddr.Extended = 0x013;
    mlmeResetRequest(TRUE);
    aps_init();
    NwkInfo.pParent = mem_getMem(sizeof(NWK_NEIGHBOR_NODE));
    NwkInfo.pParent->NetworkAddress = 0x01;
    NwkInfo.pParent->RxOnWhenIdle = TRUE;
    nwk_neighborTable[0] = NwkInfo.pParent;
    CurrAperTask = TASK_APER_NWK;
    /* Send message to the destination */
#ifndef TYPE_COOR
    //debugapp_searchCoord(TRUE);
    zdp_NwkAddrRes(FALSE, 0x01, SUCCESS,&aExtendedAddress,NwkInfo.NwkAddr,
                    ZDP_ADDR_REQ_SINGLE, 0, 0,NULL,FALSE);
#endif
} // End of functions

/*****
//@FunctionName: test_addrMap
*****/
void test_addrMap()
{
    /***** Variable declaration *****/
    BOOL Status;
    UINT16 Short;
    ADDR_MAP_ENTRY* pMap;

    /***** Function body *****/
    aps_init();
    Short = 0x1325;
    Status = ib_addAddrEntry(Short,&aExtendedAddress);
    pMap = ib_searchAddrEntry(Short,NULL);
    pMap = ib_searchAddrEntry(0,&aExtendedAddress);
    Short = 0x1327;
} // End of functions
  
```

9.3.3 Zigbee.c

```

/*****/
//Module: Common
//File Name: zigbee.c
//Description: This component is the main() program to test the MAC layer
//             - zigbee_task is the task for NWK, APS, and ZDO
//             Here are main characteristics of this task
/*
    - This is aperiodic task
    - Message in queue contains the indication and confirms
      from MAC layer
    - To inform the task in upper layer (Application layers) a
      message will be send to their queues
    - Application task call directly functions in the NWK, APS,
      and ZDO layers
    NWM, APS, and ZDO have to take this mechanism into account to
    manage the coherence in their states.
    - app_task is the task for endpoint of zigbee. This task
      will activate handler function of each registered endpoint */
/*****/

/*****/
/*      Include files      */
/*****/
#include "mac_state.h"
#include "nwk.h"
#include "nwk_msg.h"
#include "app_framework.h"
#include "zdo.h"
#include "schedul.h"
#include "static_timer.h"
#include "phy_primitives.h"
#include "hal.h"
#include "utils.h"
#include "ucos_ii.h"
#include "ep_conf.h"
#include "nv.h"
#include "aps.h"
#include "mac_beacon.h"
#include "mac_tx.h"
/*****/
/*      Constant declaration      */
/*****/

/*****/
/*      Type Definition      */
/*****/
#define OS_GLOBALS
#define STACK_SIZE_MAIN 450
/*****/
/*      Variable declaration      */
/*****/
BYTE AppStk[STACK_SIZE_MAIN];
/*****/
/*      Static Function      */
/*****/

```

```

/*****
/*                               Function                               */
*****/

/*****
//@FunctionName: putData
//@Description:   Logging
*****/

/*-----
                STUFF FUNCTION
-----*/

UINT8  nCount =0;
#ifdef ZIGBEE_DEBUG
BYTE*   DataBuf;
void putData(BYTE* data, UINT8 size)
{
    mem_cpy(DataBuf+nCount,data, size);
    nCount+=size;
    if (nCount == 12)
    {
        ENABLE_GLOBAL_INT();
    }
}
#endif // _DEBUG

//UINT32  BosList[20];
void recordBosCounter(UINT32 bos)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    //  BosList[nCount] = bos;
    nCount++;
    nCount=nCount%20;
} // End of functions

void programEEPROM()
{
    ADDRESS    addr;
    addr.Extended = 0x2222;
    nv_write(EXT_ADDR_ADDRESS,&addr,EXT_ADDR_LEN);
    while (TRUE)
        SET_RLED();
}

/*****
//@FunctionName: err
*****/
void err(ERROR    Type,
          P_FNC    FncLocation,
          BYTE*    pData,
          UINT8    nLength)
{
    /***** Variable declaration *****/
    UINT8    a = 1;
    /***** Function body *****/

```



```

switch(Type)
{
case ERR_UNRECOVERABLE:
    /*Record the error*/
    while (a)    /*Blink red Led*/
    {
        wait(50000);
        TOGGLE_RLED();
    }
    break;
case ERR_DEBUG:
    break;
default:
    break;
}
} // End of functions

/*-----
                                APP TASK
-----*/

/*****
//@FunctionName: app_task
//@Description:   This is the main function of Application Task
*****/
void    app_task(void* arg1, void* arg2)
{
    /****** Variable declaration *****/
    ZAPP_MSG*    pMsg;
    ENDPOINT_DESCRIPTOR*    temp;
    /****** Function body *****/
    pMsg = (ZAPP_MSG*)arg1;

    /* Call appropriate end point */
    for (int i=1;i<ZIGBEE_NUM_EP;i++)
    {
        temp = &EndpointDescriptors[i];
        if (EndpointDescriptors[i].SimpleDescriptor.EndPoint == pMsg->EndPointId)
            EndpointDescriptors[i].pHandler(pMsg,NULL);
    }
} // End of functions

/*****
//@FunctionName:   app_init
//@Description:     This function is called to initialize application at the
//                  first timer
*****/
void    app_init()
{
    /****** Variable declaration *****/
    /****** Function body *****/

    /* Initialize all the endpoint */
    for (int i=0;i<ZIGBEE_NUM_EP;i++)
        EndpointDescriptors[i].pInit(NULL,NULL);
} // End of functions

```

```

/*-----
ZIGBEE TASK
-----*/

/*****
//@FunctionName: zigbee_init
//@Description: This function will be call to first time initialization of
// Zigbee
*****/
void zigbee_init()
{
    /***** Variable declaration *****/
    /***** Function body *****/

#ifdef ZIGBEE_DEBUG
    /*NV initialization*/
    nv_read(EXT_ADDR_ADDRESS,&aExtendedAddress,EXT_ADDR_LEN);
    if ( aExtendedAddress == 0xFFFFFFFFFFFFFFF ) /*EEPROM is erased*/
        //err(ERR_UNRECOVERABLE,NULL,NULL,0);
        aExtendedAddress = 0x9999; /*Default address*/
#endif
    /* Initialize NWK layer */
    nwk_init();

    /* Initialize Mac layer */
    mac_init();

} // End of functions

/*****
//@FunctionName: zigbee_task
*****/
ZAPP_MSG* pMsg;
void zigbee_task(BYTE* par_pMsg, void* args)
{
    /***** Variable declaration *****/

    /***** Function body *****/

    /* Dispatch msg to specific handler
    */
    pMsg = par_pMsg;
    if (par_pMsg[0] < MSG_NWK)
        nwk_processNwkEvent(par_pMsg,NULL);
    else if (par_pMsg[0] < MSG_ZDO)
        apf_dispatchEvent(par_pMsg);
    else if (par_pMsg[0] < MSG_APS)
        nwk_processApsEvent(par_pMsg,NULL);

} // End of functions

/*-----
MAIN
-----*/

```

```

/*****/
//@FunctionName:  main
//@Description:   Startup sequence and main loop.
/*****/
void main(void)
{
#ifdef ZIGBEE_TEST
    UINT32      capRequired;
    UINT32      nOffset;
    UINT8       nTemp;
    UINT32      nBeaconDuration;
    ZAPP_MSG    Msg;
#endif

    /*Set stack for main program, Timer INT will override this area*/
    SP          = &AppStk[STACK_SIZE_MAIN - 1];

    OS_ENTER_CRITICAL();

    OSTmr_Init(); /* initialize os. */
    OS_MemInit();

    schedul_init(); /* initialize scheduler */
    mem_init();     /* initialize memory partitions */
    mtimer_reset(); /* initialize msg timer*/

    zigbee_init(); /* initialize zigbee stack */           /*BUG, do not change the order of this initialization*/
#ifdef ZIGBEE_TEST
    app_init(); /* initialize app. */
#endif
    random_init(); /* initialize random generator */
    timer_init(); /* initialize timer */
    OS_EXIT_CRITICAL();

#ifdef ZIGBEE_TEST

    while(TRUE)
    {
        /* run a-periodical task*/
        schedul_runAperTasks();
    }

#else
    test_app();
    schedul_runAperTasks();
#endif
} // main

```

9.4 MAC

9.4.1 Beacons

```

/*****/
//Module: MAC
//File Name: mac_beacon
//Description: This component will define the MAC layer behavior for beacon

```

```

//          related function. For RFD, that includes:
//          + Process the received beacon and update information related
//          to beacon
//          + For the beacon network, periodically handle the beacon and
//          inform the upper layer: beacon loss...
//          + Defines functions for beacon-request primitive
/*****/

/*****/
/*          Include files          */
/*****/

#include "mac_beacon.h"

#include "types.h"
#include "mac_types.h"
#include "utils.h"

#include "mac_primitives.h"
#include "phy_primitives.h"
#include "phy_support.h"

#include "mac_tx.h"
#include "mac_mem.h"
#include "mac_packet.h"

#include "schedul.h"
#include "static_timer.h"
#include "ucos_ii.h"
/*****/
/*          Constant declaration          */
/*****/

/*****/
/*          Type Definition          */
/*****/

/*****/
/*          Variable declaration          */
/*****/
BEACON_INFO      BeaconInfo;
SLOT_TIME_INFO  SlotInfo;

/* This is for temporarily used
   - Indication packet
*/
MLME_BEACON_NOTIFY_INDICATION  BeaconNof;
/*****/
/*          Static Function          */
/*****/

//-----
// void mbcnGetBeaconMargin(void)
//
// DESCRIPTION:
//   Because of the crystal inaccuracy, a margin must be added before the beacon that will
//   be received. The result is that:

```

```
//      - RX must be turned on a short while before the beacon is expected to be received.
//      - TX must stop before this margin is reached
//
//      The Excel worksheet called "Beacon reception margin.xls" can be used to calculate these
//      constants.
//
// RETURN VALUE:
//      UINT8
//      The margin as a number of backoff periods
//-----
/*****
//@FunctionName: beacon_transmitConflictNotif
*****/
void beacon_transmitConflictNotif()
{
    /***** Variable declaration *****/
    P_MPDU_TX      pTxMpdu;
    ADDRESS*       pCoorAddr;
    UINT8          coordAddrMode;
    /***** Function body *****/

    /* Prepare the frame request */
    pTxMpdu = mem_getFreeTxPkt();
    if (!pTxMpdu)
        return;
    mem_set(&pTxMpdu->Mpdu.FrmControl, 2);

    /* Prepare the payload of the POLL.Reg */
    pTxMpdu->Mpdu.PayLoad[0] = CMD_PAN_ID_CONFLICT_NOTIFICATION;

    /* Set other information */
    pTxMpdu->PktInfo.txMode      = TX_MODE_NO_CSMACA;
    pTxMpdu->PktInfo.nRetryLeft = aMaxFrameRetries;

    if (mpib.macCoordShortAddress >= VALID_SHORT_ADDR)
    {
        pCoorAddr      = &mpib.macCoordExtendedAddress;
        coordAddrMode  = AM_EXTENDED_64;
    }
    else
    {
        pCoorAddr      = &mpib.macCoordShortAddress;
        coordAddrMode  = AM_SHORT_16;
    }

    mac_formatMpdu(pTxMpdu,
        MAC_FRM_CMD,
        AM_EXTENDED_64,
        mpib.macPANId,
        (ADDRESS *) &aExtendedAddress,
        coordAddrMode,
        mpib.macPANId,
        (ADDRESS *) pCoorAddr,
        0,
        pTxMpdu->Mpdu.PayLoad,
        1);
}
```

```

    if(!tx_trans(pTxMpdu)) /* Transmission */
    {
        mem_releaseTxPkt(pTxMpdu);
    }
} // End of functions

/*****
//@FunctionName: beacon_calBeaconMargin
//@Description:   Because of the crystal inaccuracy, a margin must be added
//                before the beacon that will be received. The result is that:
//                - RX must be turned on a short while before the beacon is
//                expected to be received.
//                - TX must stop before this margin is reached
*/
//Reused from C2420
/***** */
UINT8 beacon_calBeaconMargin(void) {
    switch (mpib.macBeaconOrder) {
        case 7: return 2;
        case 8: return 3;
        case 9: return 5;
        case 10: return 10;
        case 11: return 20;
        case 12: return 40;
        case 13: return 79;
        case 14: return 158;
        default: return 1;
    }
} // beacon_calBeaconMargin

/*****
//@FunctionName: beacon_notifyBeacon
//@Description: This function will make a notification packet and notify the
//                upper layer the appearance of new beacon
/*****
void beacon_notifyBeacon(P_MPDU_RX par_pRxMpdu);

/*****
//@FunctionName: beacon_createSyncBeaconTask
/*****
void beacon_synsBeaconTask(void* arg1, void* arg2);
void beacon_createSyncBeaconTask(void* arg1, void* arg2)
{
    schedul_addTask(beacon_synsBeaconTask, TASK_PER_MAC_PERIODICAL, NULL); /*Periodically track
beacon*/
}

/*****
//@FunctionName: beacon_createTimeoutSyncBncTask
/*****
void beacon_timeoutSyncBeacon(void* arg1, void* arg2);
void beacon_createTimeoutSyncBncTask(void* arg1, void* arg2)
{
    schedul_addTask(beacon_timeoutSyncBeacon, TASK_PER_MAC_PERIODICAL, NULL); /*Periodically track
beacon*/
}

```

```

/*****
*/
Function
*/
/*****

/*****
//@FunctionName: beacon_initBeacon
/*****
#if ZIGBEE_DEVICE_FFD == TRUE
void beacon_initBeacon()
{
    /***** Variable declaration *****/
    P_MPDU_TX pBeacon;
    /***** Function body *****/
    /* Prepare the beacon */
    pBeacon = BeaconInfo.pBeacon;
    if(!pBeacon)
    {
        BeaconInfo.pBeacon = BeaconInfo.Beacon; /*Allocate the beacon*/
        pBeacon = BeaconInfo.pBeacon;
    }

    //set other information
    pBeacon->PktInfo.txMode = TX_MODE_NO_CSMACA;
    pBeacon->PktInfo.nRetryLeft = aMaxFrameRetries;

    if (mpib.macShortAddress >= VALID_SHORT_ADDR)
        mac_formatMpdu(pBeacon,
                        MAC_FRM_BEACON,
                        AM_EXTENDED_64,
                        mpib.macPANId,
                        (ADDRESS *) &aExtendedAddress,
                        AM_NONE,
                        0,
                        NULL,
                        0,
                        NULL,
                        0);
    else
        mac_formatMpdu(pBeacon,
                        MAC_FRM_BEACON,
                        AM_SHORT_16,
                        mpib.macPANId,
                        (ADDRESS *) &mpib.macShortAddress,
                        AM_NONE,
                        0,
                        NULL,
                        0,
                        NULL,
                        0);
} // End of functions
#endif
/*****
//@FunctionName: beacon_synSlotTime
/*****
void beacon_synSlotTime()
{

```

```

DISABLE_GLOBAL_INT();

//      // Delay the next timer tick
//      OCR1A = (MAC_TIMER_OVERFLOW_VALUE + SlotInfo.captureTcnt)
//      - (MAC_TIMER_BACKOFF_SLOT_OFFSET + 322 - 1);

/*Sync bos counter*/
SlotInfo.bosCounter -= SlotInfo.captureBosCounter;
// Clear the compare(A) interrupt flag
TIFR = BM(OCF1A);
// Set the beacon "time stamp"
SlotInfo.bosCounterAdjustTime += (SlotInfo.captureBosCounter * (UINT32)
MAC_SYMBOL_DURATION * (UINT32)
aUnitBackoffPeriod)
+ ((UINT32) SlotInfo.captureTcnt
/ CLOCK_SPEED_MHZ);

ENABLE_GLOBAL_INT();
}

/*****
//@FunctionName: beacon_updateNewBeacon
//@Description: This function will update MAC with new received beacon
//              parameters in the beacon will be updated to PIB and Mac states
*****/
void beacon_updateNewBeacon(P_MPDU_RX par_pRxMpdu)
{
    /***** Variable declaration *****/
    SUPER_FRAME* pSuperframe;
    PENDING_ADDR_SPEC* pPending;
    UINT8 nOldBeaconOrder;
    UINT8 nPendAddrOffset;
    BOOL bPendData;
    UINT8 nTemp;
    /***** Function body *****/

    /*Check conflict PAN ID: for a PAN coordinator*/
    if ((macInfo.DeviceType == ZIGBEE_PAN_COOR))
    {
        if ((par_pRxMpdu->Mpdu.Payload[1] & (0x4000 >> 8)) /*other coor. with the same PanID*/
            && (par_pRxMpdu->Mpdu.srcPanId == mpib.macPANId))
            mlmeSyncLossIndication(PAN_ID_CONFLICT);
    }

    /*Check conflict PAN ID: for a router*/
    nTemp = par_pRxMpdu->Mpdu.FrmControl.SourceAddMode;
    if (((nTemp == AM_SHORT_16)&&(par_pRxMpdu->Mpdu.srcAddr.Short != mpib.macCoordShortAddress))
        || ((nTemp == AM_EXTENDED_64)&&(par_pRxMpdu->Mpdu.srcAddr.Extended !=
mpib.macCoordExtendedAddress)))
    {
        /*Transmit conflict command*/
        //beacon_transmitConflictNotif();
    }
    else if (((nTemp == AM_SHORT_16)&&(par_pRxMpdu->Mpdu.srcAddr.Short ==
mpib.macCoordShortAddress))
        || ((nTemp == AM_EXTENDED_64)&&(par_pRxMpdu->Mpdu.srcAddr.Extended ==
mpib.macCoordExtendedAddress)))
    {
        TOGGLE_RLED();
    }
}

```




```

/*Not conflict*/
pSuperframe = par_pRxMpdu->Mpdu.Payload;

//update the PIB
nOldBeaconOrder      = mpib.macBeaconOrder;
mpib.macSuperframeOrder = pSuperframe->SuperFrmOrder;
mpib.macBeaconOrder    = pSuperframe->BeaconOrder;
BeaconInfo.nFinalCapSlot = pSuperframe->FinalCap;
BeaconInfo.bBatteryLifeExt = pSuperframe->BatLifeExt;

// Calculate the beacon duration (BOS) including IFS
BeaconInfo.nBeaconDuration = msupCalcPacketDuration(2
+ 1 + 2
+ ((par_pRxMpdu-
== AM_EXTENDED_64)
? 8
: 2)
+
mac_getMpduLength(&par_pRxMpdu->Mpdu),
FALSE);

//align the timer with the beacon.
if (mpib.macBeaconOrder != 15)
    beacon_synSlotTime();

//If the type of network change from non-beacon to beacon
if (nOldBeaconOrder == 15 && mpib.macBeaconOrder != 15)
{
    //do nothing for now,
}
else if (nOldBeaconOrder != 15 && mpib.macBeaconOrder == 15)
    //update the delayed PIB attr. now
    mac_updateDelayedPib();

//transmit auto request data
if (mpib.macAutoRequest) /* Search for the address to send the data */
{
    bPendData = FALSE;
    // Find offset to Pending Address Specification in the payload
    // < SFS + GTS > <----- GTS list ----->
    nPendAddrOffset = 2 + 1 + 3 * (par_pRxMpdu->Mpdu.Payload[2] & 0x07);
    pPending = &par_pRxMpdu->Mpdu.Payload[nPendAddrOffset++]; /* Get the Pending
Address Specification*/
    nTemp = pPending->NumofShortAddr;
    while (nTemp-->0) /* Short addresses are located first
*/
    {
        if (mpib.macShortAddress == *((WORD*) (par_pRxMpdu->Mpdu.Payload +
nPendAddrOffset))) {
            bPendData = TRUE;
            break;
        }
        nPendAddrOffset += 2;
    }

    // Then check extended addresses, see if anyone has pending data!
    if (!bPendData)

```

```

        nTemp          = pPending->NumofExtAddr;
        while (nTemp--) {
            if (aExtendedAddress == *(QWORD*) (par_pRxMpdu->Mpdu.Payload +
nPendAddrOffset)) {
                bPendData = TRUE;
                break;
            }
            nPendAddrOffset += 8;
        }

        /* Send data request */
        if (bPendData)
            mlme_autoPoll();
    }

    /*make resume for the delayed transmission task, if exist
    if (MacTx.bUsedCsmaca && MacTx.sCsmaca.bWaitBeaconForTx)
        tx_resumeAfterBeacon(FALSE);

    BeaconInfo.bBeaconLost = FALSE;

    /* Mark that we received the beacon */
    if (BeaconInfo.bTrackBeacon || BeaconInfo.bTrackBeaconReq)
        beacon_beaconReceivedOnTracking();
}
} // End of functions

/*****
//@FunctionName: beacon_synsBeaconTask
//@Description: This task will create a timer to enable Rx to track beacon
//              If necessary, it will create an other timer to track beacon
//              periodically
*****/
void beacon_synsBeaconTask(void* Param, void* arg1)
{
    /****** Variable declaration *****/
    BYTE* pTem;
    /****** Function body *****/
    pTem = &Param;

    if (pTem[0] == 0x00) /* Periodically track beacon */
    {

        /*setup a timer to process beacon timeout
        timer_startTimer(createTimeoutSyncBncTask, NULL, ((UINT32)(aBaseSuperframeDuration /
aUnitBackoffPeriod) << mpib.macBeaconOrder));

        /*if the maximum of beacon loss is reached
        if(BeaconInfo.nCurrNumOfLoss == 0)
        {
            /*Beacon is loss, remove the current mode
            BeaconInfo.bTrackBeacon = FALSE;
            BeaconInfo.bBeaconLost = TRUE;
            /*Beacon loss indication
            mlmeSyncLossIndication(BEACON_LOSS);
        }
    }

```

```

}
else /* Sync beacon request from upper */
{
    //Check the current state of syn.-beacon
    if (BeaconInfo.bTrackBeacon)
    {
        //if currently, beacon is tracked and already syn., cancel the current tracking task
        timer_endTimer(createSyncBeaconTask);
        timer_endTimer(createTimeoutSyncBncTask);
    }

    //reset the state
    BeaconInfo.bTrackBeaconReq = TRUE;
    BeaconInfo.bTrackBeacon = pTem[1];
    BeaconInfo.nCurrNumOfLoss = aMaxLostBeacons;
    BeaconInfo.bBeaconLost = FALSE;

    //Set the current channel and other necessary parameters
    plmeSET_request(PHY_PIB_CURRENT_CHANNEL, pTem[0]);

    //Add a timer to track beacon searching process in a beacon period
    timer_startTimer(createTimeoutSyncBncTask, NULL, (UINT32) (aBaseSuperframeDuration /
aUnitBackoffPeriod) << (UINT32) mpib.macBeaconOrder);
}

//start RX to receive the beacon
mac_setTrxState(PHY_RX_ON);

//remove the current task
schedul_removeTask(TASK_PER_MAC_PERIODICAL);
} // End of functions

/*****
//FunctionName: beacon_beaconReceivedOnTracking
*****/
void beacon_beaconReceivedOnTracking()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    if (BeaconInfo.bTrackBeaconReq)
        BeaconInfo.bTrackBeaconReq = FALSE;

    /*End timer*/
    timer_endTimer(createTimeoutSyncBncTask);
    timer_endTimer(createSyncBeaconTask);

    BeaconInfo.nCurrNumOfLoss = aMaxLostBeacons;
    /* Continue to track the beacon */
    if (BeaconInfo.bTrackBeacon)
        timer_startTimer(createSyncBeaconTask, NULL,
((UINT32) (aBaseSuperframeDuration / aUnitBackoffPeriod) << (UINT32)
mpib.macBeaconOrder) - 4*beacon_calBeaconMargin());
} // End of functions

/*****/

```

```

//@FunctionName: beacon_timeoutSyncBeacon
/*****
void beacon_timeoutSyncBeacon(void* args,void* par_pParam)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /* End of initial search, lost beacon*/
    if (BeaconInfo.bTrackBeaconReq) /* Beacon are being searching*/
    {
        BeaconInfo.bTrackBeaconReq = FALSE;

        //turn off the Rx
        mac_setTrxState(PHY_TRX_OFF);

        //Beacon loss indication
        mlmeSyncLossIndication(BEACON_LOSS);

        return;
    }

    /* Time out tracking periodically beacon*/
    if (BeaconInfo.bTrackBeacon)
    {
        BeaconInfo.nCurrNumOfLoss--; /* reduce beacon loss counter */

        timer_endTimer(createSyncBeaconTask);
        /* Continue to track the beacon */
        timer_startTimer(createSyncBeaconTask,NULL,
            ((UINT32) (aBaseSuperframeDuration / aUnitBackoffPeriod) << (UINT32)
mpib.macBeaconOrder)
            - 2*beacon_calBeaconMargin());
    }
    schedul_removeTask(TASK_PER_MAC_PERIODICAL);
} // End of functions

/*****
//@FunctionName: beacon_notifyBeacon
//@Description: This function will make a notification packet and notify the
//              upper layer the appearance of new beacon
*****/
void beacon_notifyBeacon(P_MPDU_RX par_pRxMpdu)
{
    /***** Variable declaration *****/
    MLME_BEACON_NOTIFY_INDICATION* pBeaconNof;
    PENDING_ADDR_SPEC* pPending;
    UINT8 nOffset;
    UINT8 nTemp;
    UINT8 NumPendingAddr;
    /***** Function body *****/
    pBeaconNof = &BeaconNof;

    //Make up a beacon notification
    pBeaconNof->PanDescriptor.LogicalChannel = PhyInfo.PhyPIB.phyCurrentChannel;
    pBeaconNof->PanDescriptor.LinkQuality = par_pRxMpdu->Mpdu.Fcs.LinkQuality;
    pBeaconNof->PanDescriptor.SuperframeSpec = *((WORD *)
                                                    par_pRxMpdu->Mpdu.Payload);

```

```

pBeaconNof->PanDescriptor.GTSPermit = !!(par_pRxMpdu->Mpdu.Payload[2] & 0x80);
pBeaconNof->PanDescriptor.SecurityUse = par_pRxMpdu->Mpdu.FrmControl.SecurityEnable;
pBeaconNof->PanDescriptor.SecurityFailure = par_pRxMpdu->Mpdu.FrmControl.SecurityEnable; //TBD
pBeaconNof->PanDescriptor.CoordAddress = par_pRxMpdu->Mpdu.srcAddr;
pBeaconNof->PanDescriptor.CoordAddrMode = par_pRxMpdu->Mpdu.FrmControl.SourceAddMode;
pBeaconNof->PanDescriptor.CoordPANId = par_pRxMpdu->Mpdu.srcPanId;
//pBeaconNof->PanDescriptor.ACLEntry, Timestamp, ACLEntry = TBD
pBeaconNof->PanDescriptor.TimeStamp = OSTmrTime;

//Gts information field
nOffset = (par_pRxMpdu->Mpdu.Payload[2] & 0x07);
if (nOffset)
    nOffset = 3 + 1 + 3 * nOffset;
else
    nOffset = 3; //GTS not present;

//Pending address
pBeaconNof->PendAddrSpec = par_pRxMpdu->Mpdu.Payload[nOffset];
pPending = &pBeaconNof->PendAddrSpec;

nOffset++; /*Pending spec*/

    nTemp = 0;
    //copy the address list: Short-Addr
    NumPendingAddr = pPending->NumofShortAddr;
    while (NumPendingAddr--) {
        nTemp++;
        pBeaconNof->AddrList[nTemp].Short = *((WORD*) (par_pRxMpdu->Mpdu.Payload + nOffset /* Super
frame and GTS */
                                                    + nTemp*sizeof(WORD)));
    }
    nOffset += pPending->NumofShortAddr*sizeof(WORD);

    //copy the address list: Ext-Addr
    NumPendingAddr = pPending->NumofExtAddr;
    while (NumPendingAddr--) {
        nTemp++;
        pBeaconNof->AddrList[nTemp].Extended = *((QWORD*) (par_pRxMpdu->Mpdu.Payload + nOffset /*
Super frame and GTS */
                                                    + nTemp*sizeof(QWORD)));
    }
    nOffset += pPending->NumofExtAddr*sizeof(QWORD);

    //copy the payload
    pBeaconNof->sduLength = par_pRxMpdu->Mpdu.nLength-nOffset; /* Super frame and GTS and pending
addrs.*/
    mem_cpy(&pBeaconNof->Payload, par_pRxMpdu->Mpdu.Payload+nOffset, pBeaconNof->sduLength);

    //notify the upper layer
    mlmeBeaconNotifyIndication(pBeaconNof);

} // End of functions

#if ZIGBEE_DEVICE_FFD == TRUE
/*****
//@FunctionName: beacon_formatBeacon
*****/

```

```

void beacon_formatBeacon()
{
    /***** Variable declaration *****/
    SUPER_FRAME*      sSuperFrm;
    BYTE*              pPayload;
    P_MPDU_TX          pBeacon;
    /***** Function body *****/
    pBeacon            = BeaconInfo.pBeacon;
    /* Update the beacon */
    pBeacon->Mpdu.SeqNum = mpib.macBSN++;

    /* Format the beacon data */
    pPayload = BeaconInfo.pBeacon->Mpdu.PayLoad;
    /* Superframe */
    sSuperFrm = (SUPER_FRAME*)pPayload;
    sSuperFrm->AssoPermit      = mpib.macAssociationPermit;
    sSuperFrm->BatLifeExt      = mpib.macBattLifeExt;
    sSuperFrm->BeaconOrder     = mpib.macBeaconOrder;
    sSuperFrm->SuperFrmOrder   = mpib.macSuperframeOrder;
    sSuperFrm->PanCoord        = (macInfo.DeviceType == ZIGBEE_PAN_COOR);
    sSuperFrm->FinalCap        = 0; /*TBD*/

    /*GTS*/
    pPayload[sizeof(SUPER_FRAME)] = 0;

    /*Beacon payload*/
    mem_cpy(pPayload+sizeof(SUPER_FRAME) + 1/*GTS*/ + BeaconInfo.nPendingAddrLen,
            mpib.macBeaconPayload, mpib.macBeaconPayloadLength);
    BeaconInfo.pBeacon->Mpdu.nLength = sizeof(SUPER_FRAME) + 1/*GTS*/+
        BeaconInfo.nPendingAddrLen + mpib.macBeaconPayloadLength;

    /*Beacon duration*/
    BeaconInfo.nBeaconDuration = msupCalcPacketDuration(BeaconInfo.pBeacon->Mpdu.nLength, FALSE);
} // End of functions

/*****
//@FunctionName: beacon_transBeacon
*****/
void beacon_transBeacon()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /* Update the indirect queue: Update TTL and remove expired packet */
    indirectTx_controlIndirectPkt(NULL, NULL); /* remove the expired packet */

    /* Make beacon */
    beacon_formatBeacon();

    /* Transmit the beacon */
    tx_transBeacon(BeaconInfo.pBeacon);

    /**/
    DISABLE_GLOBAL_INT();
    mpib.macBeaconTxTime = OSTmrTime;
    ENABLE_GLOBAL_INT();
}

```

```

    tx_resumeAfterTxBeacon();
} // End of functions

/*****
//@FunctionName: beacon_transBeaconTask
*****/
void beacon_createBeaconTxTask(void* arg1, void* arg2);
void beacon_transBeaconTask(void* par_pParam, void* args)
{
    /***** Variable declaration *****/
    UINT32 nBeaconDuration;
    /***** Function body *****/
    /*Update the buffered PIB */
    mac_updateDelayedPib();

    /*Schedule for the next trans*/
    schedul_removeTask(TASK_PER_MAC_PERIODICAL); /* Remove */

    /*Schedule beacon trans with adjusted time*/
    nBeaconDuration = ((UINT32)(aBaseSuperframeDuration / aUnitBackoffPeriod) << mpib.macBeaconOrder);
    if ((BeaconInfo.nBeaconOffset != 0)) /*This will be sync by parent*/
    {
        /*Sync the beacon with the parent*/
        if (BeaconInfo.nBeaconOffset <= SlotInfo.bosCounter%nBeaconDuration) /*Beacon is delayed*/
        {
            nBeaconDuration = nBeaconDuration - (SlotInfo.bosCounter%nBeaconDuration -
BeaconInfo.nBeaconOffset);
        }
        else /*Beacon is faster than the parent*/
        {
            nBeaconDuration = nBeaconDuration + (BeaconInfo.nBeaconOffset -
SlotInfo.bosCounter%nBeaconDuration);
        }
    }

    /*Sync the beacon*/
    timer_startTimer(createBeaconTxTask, NULL, /* And call-back later */
nBeaconDuration);

    /*Transmit beacon*/
    beacon_transBeacon();

    /* set up the end of CAP*/
    if (!mpib.macAssociationPermit)
    {
        if (mpib.macRxOnWhenIdle)
        {
            timer_startTimer(forceOffTrx, NULL, (UINT32)(beacon_calBeaconMargin()+
(UINT32)((aBaseSlotDuration / aUnitBackoffPeriod) * (UINT32) (BeaconInfo.nFinalCapSlot +
(UINT32) 1)) << (UINT32) mpib.macSuperframeOrder));
        }
        else
            mac_setTrxState(PHY_TRX_OFF);
    }

    TOGGLE_RLED(); /*Beacon*/
}

```

```

} // End of functions

/*****
//@FunctionName: beacon_createBeaconTxTask
*****/
void beacon_createBeaconTxTask(void* arg1, void* arg2)
{
    schedul_addTask(beacon_transBeaconTask, TASK_PER_MAC_PERIODICAL, NULL);
}

/*****
//@FunctionName: beacon_startBeacon
*****/
void beacon_startBeacon(void* par_pTimer, void* par_pParam)
{
    /***** Variable declaration *****/
    UINT8 nErr;
    /***** Function body *****/
    if (par_pTimer != NULL)
        OSTmrDel(par_pTimer, &nErr);

    beacon_initBeacon();
    beacon_createBeaconTxTask(NULL, NULL);
    /*Mark the beacon transmission point*/
    if (macInfo.DeviceType == ZIGBEE_ROUTER)
        BeaconInfo.nBeaconOffset = SlotInfo.bosCounter % ((UINT32)(aBaseSuperframeDuration /
aUnitBackoffPeriod) << mpib.macBeaconOrder);
} // End of functions

#endif // ! ZIGBEE_DEVICE_FFD == TRUE

```

9.4.2 Common

9.4.2.1 mac_mem.c

```

/*****
//Module: MAC/common
//File Name: mac_task
//Description: Mac memory management funtions
*****/

/*****
/* Include files */
*****/
#include "mac_mem.h"
#include "ucos_ii.h"

/*****
/* Type Definition */
*****/

/*****
/* Constant declaration */
*****/

/*****
/* Variable declaration */
*****/

```



```

BYTE      MacRxPktPool[MAC_RX_POOL_SIZE][sizeof(MPDU_RX)];
BYTE      MacTxPktPool[MAC_TX_POOL_SIZE][sizeof(MPDU_TX)];
  
```

```

OS_MEM*    MacRxMem;
OS_MEM*    MacTxMem;
  
```

```

/*****/
//@FunctionName:mem_initialize
//@Description:   Initialize memory management component of the MAC
/*****/
void mem_initialize()
{
    /***** Variable declaration *****/
    UINT8      nErr;
    /***** Function body *****/

    //create memory partition
    MacRxMem = OSMemCreate(MacRxPktPool,
                          MAC_RX_POOL_SIZE,
                          sizeof(MPDU_RX),
                          &nErr);
    MacTxMem = OSMemCreate(MacTxPktPool,
                          MAC_TX_POOL_SIZE,
                          sizeof(MPDU_TX),
                          &nErr);
} // End of functions

/*****/
//@FunctionName: mem_release
//@Description:   Release all the memory managed by the MAC
/*****/
void mem_release()
{
    /***** Variable declaration *****/
    UINT8      nErr;
    /***** Function body *****/

    //release memory partition
    OSMemDelete(MacRxMem,&nErr);
    OSMemDelete(MacTxMem,&nErr);
} // End of functions

/*****/
//@FunctionName:mem_getFreeTxPkt
//@Description:   Get a free tx packet
/*****/
P_MPDU_TX mem_getFreeTxPkt()
{
    /***** Variable declaration *****/
    UINT8      nErr;
    /***** Function body *****/
    return (P_MPDU_TX)(OSMemGet(MacTxMem, &nErr));
} // End of functions
  
```

```

/*****/
//@FunctionName: mem_releaseTxPkt
//@Description: Release a tx packet
/*****/
void mem_releaseTxPkt(P_MPDU_TX par_pPacket)
{
    /***** Variable declaration *****/
    UINT8 nErr;
    /***** Function body *****/
    nErr = OSMemPut(MacTxMem, (void*)(par_pPacket));
} // End of functions

/*****/
//@FunctionName: mem_getFreeRxPkt
//@Description: Get a free Rx packet
/*****/
P_MPDU_RX mem_getFreeRxPkt()
{
    /***** Variable declaration *****/
    UINT8 nErr;
    /***** Function body *****/
    return (P_MPDU_RX)(OSMemGet(MacRxMem, &nErr));
} // End of functions

/*****/
//@FunctionName: mem_releaseRxPkt
//@Description: Release a tx packet
/*****/
void mem_releaseRxPkt(P_MPDU_RX par_pPacket)
{
    /***** Variable declaration *****/
    UINT8 nErr;
    /***** Function body *****/
    nErr = OSMemPut(MacRxMem, (void*)((void*)par_pPacket));
} // End of functions

```

9.4.2.1 mac_packet.c

```

/*****/
//Module: MAC/common
//File Name: mac_packet.c
//Description: This component define the format of packet used internally by
//             MAC and functions to format the packet.
/*****/

/*****/
/* Include files */
/*****/
#include "mac_packet.h"
/*****/
/* Constant declaration */
/*****/

/*****/
/* Type Definition */
/*****/

```

```

/*****/
/*      Variable declaration      */
/*****/

/*****/
/*      Static Function      */
/*****/

/*****/
/*      Function      */
/*****/

/*****/
//@FunctionName: msupCalcPacketDuration
//@Description: Calculates the number of backoff slots required to transmit
//a packet, including (turnaround time, acknowledgment and) inter-frame spacing.*/
/*****/
inline UINT8 msupCalcPacketDuration(UINT8 length, BOOL ackRequest) {
    UINT8 totalByteCount, lastBosByteCount;

    // Calculate the number of bytes in the whole packet (not including the preamble, but including
    // SFD, length-byte and PSDU)
    totalByteCount = 1 + 1 + length;

    // Acknowledgment request
    if (ackRequest) {

        // The number of bytes in the last backoff slot
        lastBosByteCount = totalByteCount % (aUnitBackoffPeriod / 2);

        // Round up the total count to N * (aUnitBackoffPeriod / 2), and add an extra slot if there isn't enough
        // turnaround time before the ack (12-32 symbols)
        if (lastBosByteCount) totalByteCount += (aUnitBackoffPeriod / 2) - lastBosByteCount;

        // Add the bytes in the acknowledgment frame
        totalByteCount += (aUnitBackoffPeriod / 2) + (1 + 1 + 5);
    }

    // Add interframe spacing
    if (length > aMaxSIFSFrameSize) {
        totalByteCount += (aMinLIFSPeriod / 2);
    } else {
        totalByteCount += (aMinSIFSPeriod / 2);
    }

    // Round up
    return ((BYTE) (totalByteCount + ((aUnitBackoffPeriod / 2) - 1))) / (aUnitBackoffPeriod / 2);
} // msupCalcPacketDuration

/*****/
//@FunctionName: mac_formatMpduHr
//@Description: This function will format the MPDU

```

```

/*****
inline MAC_STATUS mac_formatMpdu(P_MPDU_TX par_pTxMpdu,
    BYTE type,
    BYTE srcAddrMode,
    WORD srcPanId,
    ADDRESS* pSrcAddr,
    BYTE desAddrMode,
    WORD destPanId,
    ADDRESS* pDestAddr,
    BYTE txOptions,
    BYTE* pPayload,
    UINT8 nLen)
{
    /***** Variable declaration *****/
    MPDU* pMpdu;
    /***** Function body *****/
    pMpdu = &par_pTxMpdu->Mpdu;

    mem_set(&pMpdu->FrmControl,2);

    pMpdu->FrmControl.SourceAddMode = srcAddrMode;
    pMpdu->srcPanId = srcPanId;
    pMpdu->srcAddr = *pSrcAddr;
    pMpdu->FrmControl.DestAddMode = desAddrMode;
    pMpdu->dstPanId = destPanId;
    pMpdu->dstAddr = *pDestAddr;

    //payload
    pMpdu->nLength = nLen;
    mem_cpy((BYTE*)pMpdu->PayLoad,(BYTE*)pPayload,nLen);

    //Control frame
    pMpdu->SeqNum = mpib.macDSN++; //sequence number
    pMpdu->FrmControl.FrameType = type; //frame type
    if (txOptions & TX_OPT_ACK_REQ)
        pMpdu->FrmControl.ACKRequest = 1;
    if ((srcAddrMode && desAddrMode) && (srcPanId == destPanId))
        pMpdu->FrmControl.IntraPAN = 1;

    //get the packet length
    nLen = mac_getMpduLength(pMpdu);
    //calculate the packet duration
    par_pTxMpdu->PktInfo.nDuration = msupCalcPacketDuration(nLen, txOptions & TX_OPT_ACK_REQ);
    par_pTxMpdu->PktInfo.txOptions = txOptions;

    if (desAddrMode == AM_SHORT_16)
        par_pTxMpdu->PktInfo.bToCoord = ((mpib.macPANId == destPanId) &&
            (pDestAddr->Short == mpib.macCoordShortAddress));

    if (desAddrMode == AM_EXTENDED_64)
        par_pTxMpdu->PktInfo.bToCoord = ((mpib.macPANId == destPanId) &&
            (pDestAddr->Extended ==
mpib.macCoordExtendedAddress));

    return SUCCESS;
} // End of functions
*****/

```

```

/*@FunctionName: mac_getMpduHrLength
/*@Description: This function will give the length of a valid MPDU. If the
//          MPDU is not valid, return value will be 0
*/

/*****
inline UINT8 mac_getMpduHrLength(MPDU* par_pMpdu)
{
    /***** Variable declaration *****/
    UINT8    nLength = 0;
    /***** Function body *****/
    /*Frame control + Seq*/
    nLength+= sizeof(FRAME_CONTROL)+sizeof(UINT8);

    /*Src Address*/
    if(par_pMpdu->FrmControl.SourceAddMode > 1)
        nLength+= sizeof(WORD) + (par_pMpdu->FrmControl.SourceAddMode)*6-10;

    /*Des Address*/
    if(par_pMpdu->FrmControl.DestAddMode > 1)
        nLength+= sizeof(WORD) + (par_pMpdu->FrmControl.DestAddMode)*6-10;

    if (par_pMpdu->FrmControl.IntraPAN)
    {
        nLength-=sizeof(WORD); /*without source PanId*/
    }
    return nLength;
} // End of functions

/*****
/*@FunctionName: mac_getMpduLength
/*@Description: This function will give the length of a valid MPDU. If the
//          MPDU is not valid, return value will be 0
*/
inline UINT8 mac_getMpduLength(MPDU* par_pMpdu)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    return par_pMpdu->nLength+mac_getMpduHrLength(par_pMpdu);
} // End of functions

```

9.4.3 MAC Primitive

9.4.3.1 mac_mcps.c

```

/*****
//Module: MAC/mac_primitive
//File Name: mac_mcps.c
//Description: This file implement the mcps-sap of the MAC
//-----*/
//Revision History:
//Date          Programmer          Details
*/
//02/10/2007    pnthai              Create
//-----*/
// Copyright (C) 2006 - All Rights Reserved
/*****

```

```

/*****
/*                                     Include files
*/
/*****
#include "mac_primitives.h"
#include "mac_mem.h"
#include "mac_packet.h"
#include "mac_tx.h"
#include "phy_primitives.h"
/*****
/*                                     Constant declaration
*/
/*****

/*****
/*                                     Type Definition
*/
/*****

/*****
/*                                     Variable declaration
*/
/*****

/*****
/*                                     Static Function
*/
/*****

/*****
/*                                     Function
*/
/*****

//*****
//@FunctionName: mcpsDataRequest
//@Description:
//      Issued by local SSCS to MLME
//@Param
//      *pMCPS_Data_Request -      MCPS_DATA_REQUEST structure
//*****
MAC_STATUS mcpsDataRequest(  UINT8   SrcAddrMode,      // The source addressing mode for this
                           primitive and subsequenst MPDU. (00-03)
                           UINT16   SrcPANId,          // The 16bit PAN identifier of the entity from which
the MSDU is being tranferred
                           ADDRESS* pSrcAddr,          // The individual device address of the entity
from which the MSDU is being transferred
                           UINT8     DstAddrMode,      // The destination addressing mode for this
primitive
                           UINT16    DstPANId,          // The 16bit PAN identifier of the entity to which the
MSDU is being tranferred
                           ADDRESS* pDstAddr,          // The individual device address to which the
MSDU is being tranferred
                           UINT8     msduLength,       // The number of octets forming the MSDU

```

```

        BYTE    *msdu,           // The set of octets forming the MSDU
        UINT8    msduHandle,     // The handle associated with the MSDU to
transmitted by the MAC sublayer entity
        BYTE    TxOptions)       // The transmission options for this MSDU
{

    P_MPDU_TX      pTxMPDU;
    MAC_STATUS     macStatus;

    #if      ZIGBEE_DEVICE_FFD == FALSE
        return UNSUPPORTED_ATTRIBUTE;
    #endif

    /* Sercurity option */
    if (TxOptions & TX_OPT_SECURITY_ENABLE)
        return UNSUPPORTED_ATTRIBUTE;

    /* GTS option */
    //The indirect transmission bit is
    //ignored if the GTS transmission bit is set to 1, the destination address is not present, or the MAC sublayer
    //receiving this primitive is not the MAC sublayer of a coordinator.
    //GTS transmission is required or not required
    //TBD
    if (TxOptions & TX_OPT_GTS)
        return UNSUPPORTED_ATTRIBUTE;

    //Get a free MPDU
    pTxMPDU = mem_getFreeTxPkt();
    if(pTxMPDU == NULL)           /* If no available packet, try later*/
        return TRANSACTION_OVERFLOW;
    mem_set(&pTxMPDU->Mpdu.FrmControl,2);

    //Building the MPDU frame
    macStatus = mac_formatMpdu(pTxMPDU, MAC_FRM_DATA, SrcAddrMode,SrcPANId,
                                pSrcAddr, DstAddrMode, DstPANId, pDstAddr,
                                TxOptions, msdu, msduLength);

    if (macStatus != SUCCESS)
        return FRAME_TOO_LONG; /* Can not build the frame */

    //set other information
    pTxMPDU->PktInfo.nRetryLeft = aMaxFrameRetries;
    pTxMPDU->PktInfo.msduHandle = msduHandle;
    pTxMPDU->PktInfo.txMode     = TX_MODE_CSMACA;

    /* Indirect transmission option */
    //If the TxOptions parameter specifies that an indirect transmission is required and this primitive is received
    //by the MAC sublayer of a coordinator
    #if      ZIGBEE_DEVICE_FFD == TRUE
        if (TxOptions & TX_OPT_INDIRECT)
            macStatus = indirectTx_addPkt(pTxMPDU);           /*Indirect transmission */
        else
    #endif
    //transmit the packet
    macStatus = tx_trans(pTxMPDU);
    if(!macStatus)

```

```

    mem_releaseTxPkt(pTxMPDU);

#ifdef ZIGBEE_STUB
    MacTx.pPacket = pTxMPDU;
    tx_endTrans(pTxMPDU,SUCCESS);
#endif

    return (macStatus == TRUE)?SUCCESS:TRANSACTION_OVERFLOW;
} //mcpsDataRequest

/*****
//@FunctionName: mlmePurgeRequest
//@Description:
//      Optional for FFD & RFD
//@Param
//      UINT8    msduHandle: handle to the transaction that will be purge
*****/
void mcpsPurgeRequest(UINT8    msduHandle)
{
    /***** Variable declaration *****/
    MAC_TX_PACKET *pIndirectPacket;
    /***** Function body *****/
    // For each packet in the queue ...
restart:
    packetIndex = miqInfo.firstIndirectPacket;
    while (packetIndex != NO_PACKET) {
        pIndirectPacket = &pMtxPacketPool[packetIndex];
        if ((pIndirectPacket->msduHandle == msduHandle) && (pIndirectPacket->type == FT_DATA)) {
            if (pIndirectPacket->requested) {
                goto restart;
            } else {
                pIndirectPacket->timeToLive = MIQ_PACKET_PURGED;
                mschAddTask(mschReserveTask(), MAC_TASK_PRI_MEDIUM, miqExpireIndirectPacketsTask,
                    (WORD) pIndirectPacket);
                return SUCCESS;
            }
        }
        packetIndex = pIndirectPacket->nextPacket;
    }

    return INVALID_HANDLE;
} // End of functions

```

9.4.3.2 mac_mlme.c

```

/*****
//Module: MAC/mac_primitive
//File Name: mac_mlme.c
//Description: This file implement the mlme-sap of the MAC
// REUSED and changed from C2430
//-----*/
//Revision History:
//Date          Programmer    Details
//02/10/2007    pnthai        Create
//-----*/
// Copyright (C) 2006 - All Rights Reserved

```



```

/*****/

/*****/
/*                               Include files                               */
/*****/
#include "types.h"
#include "mac_primitives.h"
#include "mac_mem.h"
#include "mac_packet.h"
#include "mac_tx.h"
#include "phy_primitives.h"
#include "phy_support.h"
#include "schedul.h"
#include "static_timer.h"
#include "mac_state.h"
#include "mac_beacon.h"
#include "mac_tx.h"
#include "mac_mlme.h"
#include "utils.h"
#include "ucos_ii.h"
/*****/
/*                               Constant declaration                               */
/*****/

/* A bit-mask containing the valid channels for 2.4 GHz devices
   (channels 11 to 26)
*/
#define MSC_VALID_CHANNELS                0x07FFF800

/*****/
/*                               Type Definition                               */
/*****/
/* Information for Association transaction */
typedef struct
{
    void*          pParam;
    MAC_STATUS     macStatus;
}ASSOCIATION_INFO;

/* Information for Rx Enable Request */
typedef struct
{
    UINT16         nDuration;
}RXENABLE_INFO;

/*****/
/*                               Variable declaration                               */
/*****/

/* Scan information
*/
MAC_SCAN_INFO*    pScanInfo;

/* Scan result to report to upper layer
*/
MAC_SCAN_RESULT*  pScanResult;
  
```

```

/*****/
/*                               Static Function                               */
/*****/

/*****/
//@FunctionName:                                                         */
//@Description: This function will handle association request transition      */
//                                                         */
//@Param                                                         */
//                                                         */
/*****/
void    mlme_handleAssocTransition(void* par_pParam1,
                                   void* par_Param2);

/*****/
/*                               Function                               */
/*****/

/*****/
//@FunctionName:                                                         */
//@Description:  MLME-SET.request structure is generated by the next higher
//               layer and issued to its MLME to write the indicated MAC PIB attribute
//@Param                                                         */
//                                                         */
/*****/
MAC_STATUS mlmeSetRequest(MAC_PIB_ATTR_ID   PIBAttribute,
                          void* pPibAttributeValue)
{
    BOOL    isBeaconEnabledPan  = (mpib.macBeaconOrder < 15);
    WORD    pibAttributeValue;

    // Store the word, not always used but saves memory space
    pibAttributeValue = *((WORD *) pPibAttributeValue);

    switch (PIBAttribute)
    {
        // Category 1 attributes (read-only)
        case MAC_ACK_WAIT_DURATION:
            if ((UINT8) pibAttributeValue == 54)
                break;
            return INVALID_PARAMETER;
        case MAC_BATT_LIFE_EXT_PERIODS:
            if ((UINT8) pibAttributeValue == 6)
                break;
            return INVALID_PARAMETER;
        case MAC_GTS_PERMIT:
        case MAC_PROMISCUOUS_MODE:
            if ((BOOL) pibAttributeValue == FALSE)
                break;
            return INVALID_PARAMETER;

        // Category 2 attributes (direct write)
        case MAC_ASSOCIATION_PERMIT:
            mpib.macAssociationPermit = (BOOL) pibAttributeValue;
            break;
        case MAC_AUTO_REQUEST:
    }
}

```

```

    mpib.macAutoRequest = (BOOL) pibAttributeValue;
    break;
case MAC_BEACON_PAYLOAD_LENGTH:
    if (((UINT8) pibAttributeValue) > aMaxBeaconPayloadLength)
    {
        return INVALID_PARAMETER;
    }
    else
    {
        mpib.macBeaconPayloadLength = (UINT8) pibAttributeValue;
    }
    break;
case MAC_BSN:
    mpib.macBSN = (UINT8) pibAttributeValue;
    break;
case MAC_DSN:
    mpib.macDSN = (UINT8) pibAttributeValue;
    break;
case MAC_MAX_CSMA_BACKOFFS:
    if (((UINT8) pibAttributeValue) > 5)
    {
        return INVALID_PARAMETER;
    }
    else
    {
        mpib.macMaxCSMABackoffs = (UINT8) pibAttributeValue;
    }
    break;
case MAC_MIN_BE:
    if (((UINT8) pibAttributeValue) > 3)
    {
        return INVALID_PARAMETER;
    }
    else
    {
        mpib.macMinBE = (UINT8) pibAttributeValue;
    }
    break;

    // Category 3 attributes (disable interrupts)
case MAC_BEACON_PAYLOAD:
    DISABLE_GLOBAL_INT();
    mpib.macBeaconPayload = (BYTE *) pPibAttributeValue;
    ENABLE_GLOBAL_INT();
    break;
case MAC_BEACON_TX_TIME:
    DISABLE_GLOBAL_INT();
    mem_cpy(&mpib.macBeaconTxTime,
            (UINT32 *) pPibAttributeValue,
            sizeof(UINT32));
    ENABLE_GLOBAL_INT();
    break;
case MAC_COORD_EXTENDED_ADDRESS:
    DISABLE_GLOBAL_INT();
    mem_cpy(&mpib.macCoordExtendedAddress,
            (QWORD *) pPibAttributeValue,
            sizeof(QWORD));

```



```

        ENABLE_GLOBAL_INT();
        break;
    case MAC_COORD_SHORT_ADDRESS:
        DISABLE_GLOBAL_INT();
        mpib.macCoordShortAddress = pibAttributeValue;
        ENABLE_GLOBAL_INT();
        break;
    case MAC_PAN_ID:
        DISABLE_GLOBAL_INT();
        mpib.macPANId = pibAttributeValue;
        phy_WriteRam((BYTE *) &mpib.macPANId, CC2420RAM_PANID, 2, FALSE);
        ENABLE_GLOBAL_INT();
        break;
    case MAC_SHORT_ADDRESS:
        DISABLE_GLOBAL_INT();
        mpib.macShortAddress = pibAttributeValue;
        phy_WriteRam((BYTE *) &mpib.macShortAddress, CC2420RAM_SHORTADDR, 2, FALSE);
        ENABLE_GLOBAL_INT();
        break;
    case MAC_TRANSACTION_PERSISTENCE_TIME:
        DISABLE_GLOBAL_INT();
        mpib.macTransactionPersistenceTime = pibAttributeValue;
        ENABLE_GLOBAL_INT();
        break;

    // Category 4 attributes (buffered until beacon transmission / handled carefully)
    case MAC_BATT_LIFE_EXT:
        //TBD
        break;

    case MAC_BEACON_ORDER:
        /*TBD: Handle beacon network change*/
        mpib.macBeaconOrder = pibAttributeValue;
        break;
    case MAC_RX_ON_WHEN_IDLE:
        DISABLE_GLOBAL_INT();
        if (isBeaconEnabledPan) {
            macInfo.pibTempBuffer.macRxOnWhenIdle = (BOOL) pibAttributeValue;
            macInfo.pibTempBuffer.updateMask |= MPIB_UPD_RX_ON_WHEN_IDLE_BM;
        }
        else /* Update directly the value */
            mpib.macRxOnWhenIdle = (BOOL) pibAttributeValue;
        ENABLE_GLOBAL_INT();
        break;
    case MAC_SUPERFRAME_ORDER:
        DISABLE_GLOBAL_INT();
        if (((UINT8) pibAttributeValue) > 15)
        {
            return INVALID_PARAMETER;
        }
    #if ZIGBEE_DEVICE_FFD == TRUE
        else if ((isBeaconEnabledPan
            && BeaconInfo.bTransmitBeacon) || BeaconInfo.bTrackBeaconReq)
    #else
        else if (BeaconInfo.bTrackBeaconReq)
    #endif
        //! ZIGBEE_DEVICE_FFD == TRUE
        {

```

```

        macInfo.pibTempBuffer.macSuperframeOrder = (UINT8)
                                pibAttributeValue;
        macInfo.pibTempBuffer.updateMask |= MPIB_UPD_SUPERFRAME_ORDER_BM;
    }
    else
    {
        mpib.macSuperframeOrder = (UINT8) pibAttributeValue;
    }
    ENABLE_GLOBAL_INT();
    break;

default:
    return UNSUPPORTED_ATTRIBUTE;
}
return SUCCESS;
}

//-----
// MLME-GET.request structure
// requests information about a given PIB attribute
//-----
MAC_STATUS mlmeGetRequest(MAC_PIB_ATTR_ID      PIBAttribute,          // The identifier of the PIB
                           sttribute to read
                           void* pPibAttributeValue) // The value of the PIB sttribute);
{
    /***** Variable declaration *****/

    /***** Function body *****/
    switch (PIBAttribute)
    {
    case MAC_ACK_WAIT_DURATION:
        *((BYTE *) pPibAttributeValue) = mpib.macAckWaitDuration; break;
    case MAC_BATT_LIFE_EXT_PERIODS:
        *((BYTE *) pPibAttributeValue) = mpib.macBattLifeExtPeriods; break;
    case MAC_GTS_PERMIT:
        *((BOOL *) pPibAttributeValue) = mpib.macGTSPermit; break;
    case MAC_PROMISCUOUS_MODE:
        *((BOOL *) pPibAttributeValue) = mpib.macPromiscuousMode; break;

    case MAC_ASSOCIATION_PERMIT:
        *((BOOL *) pPibAttributeValue) = mpib.macAssociationPermit; break;
    case MAC_AUTO_REQUEST:
        *((BOOL *) pPibAttributeValue) = mpib.macAutoRequest; break;
    case MAC_BEACON_PAYLOAD_LENGTH:
        *((BYTE *) pPibAttributeValue) = mpib.macBeaconPayloadLength; break;
    case MAC_BSN:
        *((BYTE *) pPibAttributeValue) = mpib.macBSN; break;
    case MAC_DSN:
        *((BYTE *) pPibAttributeValue) = mpib.macDSN; break;
    case MAC_MAX_CSMA_BACKOFFS:
        *((BYTE *) pPibAttributeValue) = mpib.macMaxCSMABackoffs; break;
    case MAC_MIN_BE:
        *((BYTE *) pPibAttributeValue) = mpib.macMinBE; break;

    case MAC_BEACON_PAYLOAD:
        DISABLE_GLOBAL_INT();

```

```

    *((BYTE **) pPibAttributeValue) = mpib.macBeaconPayload;
    ENABLE_GLOBAL_INT();
    break;
case MAC_BEACON_TX_TIME:
    DISABLE_GLOBAL_INT();
    mem_cpy(pPibAttributeValue, &mpib.macBeaconTxTime, sizeof(UINT32));
    ENABLE_GLOBAL_INT();
    break;
case MAC_COORD_EXTENDED_ADDRESS:
    DISABLE_GLOBAL_INT();
    mem_cpy(pPibAttributeValue, &mpib.macCoordExtendedAddress, sizeof(QWORD));
    ENABLE_GLOBAL_INT();
    break;
case MAC_COORD_SHORT_ADDRESS:
    DISABLE_GLOBAL_INT();
    *((WORD *) pPibAttributeValue) = mpib.macCoordShortAddress;
    ENABLE_GLOBAL_INT();
    break;
case MAC_PAN_ID:
    DISABLE_GLOBAL_INT();
    *((WORD *) pPibAttributeValue) = mpib.macPANId;
    ENABLE_GLOBAL_INT();
    break;
case MAC_SHORT_ADDRESS:
    DISABLE_GLOBAL_INT();
    *((WORD *) pPibAttributeValue) = mpib.macShortAddress;
    ENABLE_GLOBAL_INT();
    break;
case MAC_TRANSACTION_PERSISTENCE_TIME:
    DISABLE_GLOBAL_INT();
    *((WORD *) pPibAttributeValue) = mpib.macTransactionPersistenceTime;
    ENABLE_GLOBAL_INT();
    break;

    // Category 4 attributes (buffered until beacon transmission / handled carefully)
case MAC_BATT_LIFE_EXT:
    //      DISABLE_GLOBAL_INT();
    //      if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_BATT_LIFE_EXT_BM) {
    //          *((BOOL *) pPibAttributeValue) = macInfo.pibTempBuffer.macBattLifeExt;
    //      } else {
    //          *((BOOL *) pPibAttributeValue) = mpib.macBattLifeExt;
    //      }
    //      ENABLE_GLOBAL_INT();
    break;
case MAC_BEACON_ORDER:
    //      DISABLE_GLOBAL_INT();
    //      if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_BEACON_ORDER_BM) {
    //          *((BOOL *) pPibAttributeValue) = macInfo.pibTempBuffer.macBeaconOrder;
    //      } else {
    //          *((BOOL *) pPibAttributeValue) = mpib.macBeaconOrder;
    //      }
    //      ENABLE_GLOBAL_INT();
    break;
case MAC_RX_ON_WHEN_IDLE:
    //      DISABLE_GLOBAL_INT();
    //      if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_RX_ON_WHEN_IDLE_BM) {
    //          *((BOOL *) pPibAttributeValue) = macInfo.pibTempBuffer.macRxOnWhenIdle;

```

```

//      } else {
//          *((BOOL*) pPibAttributeValue) = mpib.macRxOnWhenIdle;
//      }
//      ENABLE_GLOBAL_INT();
break;
case MAC_SUPERFRAME_ORDER:
//      DISABLE_GLOBAL_INT();
//      if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_SUPERFRAME_ORDER_BM) {
//          *((BOOL*) pPibAttributeValue) = macInfo.pibTempBuffer.macSuperframeOrder;
//      } else {
//          *((BOOL*) pPibAttributeValue) = mpib.macSuperframeOrder;
//      }
//      ENABLE_GLOBAL_INT();
break;
default:
return UNSUPPORTED_ATTRIBUTE;
}
return SUCCESS;
} // End of functions

//-----
//---                               MLME-ASSOCIATE
//-----
// MLME-ASSOCIATE.request structure
// generated by the next higher of an unassociated device and issued to its MLME to request an association
// with a coordinator
//-----
void mlmeAssociateRequest(UINT8    LogicalChannel, // The logical channel on which to attempt
                           // association
                           UINT8    CoordAddrMode, // The coordinator addressing mode for
                           // this primitive and subsequent MPDU
                           UINT16    CoordPANId,   // The identifier of PAN with which to
                           // associate
                           ADDRESS*  pCoordAddress, // The address of the coordinator
                           // with which to associate
                           BYTE       CapabilityInformation, // Specifies the operational capabilities of the
                           // associating device
                           BOOL       SecurityEnable) // TRUE if enabled)
{
    /***** Variable declaration *****/
    PHY_STATUS  phyStatus;
    MAC_STATUS  macStatus;
    P_MPDU_TX   pMpdu;
    ASSOCIATION_INFO* pAssolInfo; /* Information for association */
    /***** Function body *****/

    //validate parameters
    if ((!(LogicalChannel >= 11) && (LogicalChannel <= 26))
        || ((CoordAddrMode != AM_SHORT_16)
            && (CoordAddrMode != AM_EXTENDED_64)))
    {
        mlmeAssociateConfirm(0xFFFF,INVALID_PARAMETER);
        return;
    }
    /* Checking MAC status */
    if(! mac_requestState(MAC_TRANSAC_ASSOCIATION, MAC_ASSOC_REQUEST))
    {

```

```

    mlmeAssociateConfirm(0xFFFF, UNDEFINED);
    return;
}

//Update PHY current channel
phyStatus = plmeSET_request(PHY_PIB_CURRENT_CHANNEL, LogicalChannel);

//update the macPANId
macStatus = mlmeSetRequest(MAC_PAN_ID, &CoordPANId);
if (CoordAddrMode == AM_SHORT_16)
{
    mpib.macCoordShortAddress = pCoordAddress->Short;
    mpib.macCoordExtendedAddress = INVALID_EXT_ADDR;
}
else
{
    mpib.macCoordShortAddress = RESERVED_SHORT_ADDR;
    mpib.macCoordExtendedAddress = pCoordAddress->Extended;
}
macInfo.DeviceType = ZIGBEE_ENDPOINT; /*Default device*/

if (phyStatus != PHY_SUCCESS || macStatus != SUCCESS)
{
    mlmeAssociateConfirm(0xFFFF, INVALID_PARAMETER);
    return;
}

//reserve the packet
pMpdu = mem_getFreeTxPkt();
mem_set(&pMpdu->Mpdu.FrmControl, 2);
if (!pMpdu)
{
    mlmeAssociateConfirm(0xFFFF, TRANSACTION_OVERFLOW);
    return;
}

//Prepare the payload of the ASSO-CMD
//Command identifier

pMpdu->Mpdu.Payload[0] = CMD_ASSOCIATION_REQUEST;
pMpdu->Mpdu.Payload[1] = CapabilityInformation;

//set other information
pMpdu->PktInfo.txMode = TX_MODE_CSMA|TX_MODE_FORCE_UNSLOTTED;
pMpdu->PktInfo.nRetryLeft = aMaxFrameRetries;

macStatus = mac_formatMpdu(pMpdu,
    MAC_FRM_CMD,
    AM_EXTENDED_64,
    0xFFFF,
    (ADDRESS *) &aExtendedAddress,
    CoordAddrMode,
    CoordPANId,
    pCoordAddress,
    TX_OPT_ACK_REQ,
    pMpdu->Mpdu.Payload,
    2);

```



```

if (macStatus != SUCCESS)
{
    mlmeAssociateConfirm(0xFFFF,INVALID_PARAMETER);
    return;
}

//set other information
MacTx.bUsedCsmaca = TRUE;
pMpdu->PktInfo.nRetryLeft = aMaxFrameRetries;

pAssolInfo = mem_getMem(sizeof(ASSOCIATION_INFO));
pAssolInfo->macStatus = SUCCESS;
pAssolInfo->pParam = pMpdu;
schedul_addTask(mlme_handleAssocTransition,TASK_PER_MAC_TRANSACTION,pAssolInfo);
} // End of functions

/*****
**@FunctionName: mlme_notifyAssociation
***/
P_MPDU_RX pRxMpdu;
void mlme_notifyAssociation(MAC_STATUS par_macStatus,
                           void* par_pParam)
{
    /***** Variable declaration *****/
    ASSOCIATION_INFO* pAssolInfo; /* Information for association */
    MAC ASSO_STATE par_State;
    /***** Function body *****/
    pRxMpdu = (P_MPDU_RX)par_pParam;
    pAssolInfo = (ASSOCIATION_INFO*)schedul_getParam(TASK_PER_MAC_TRANSACTION);

    pAssolInfo->pParam = par_pParam;
    pAssolInfo->macStatus = par_macStatus;
    if (par_macStatus == SUCCESS)
    {
        if (pRxMpdu->Mpdu.Payload[0]==CMD_ASSOCIATION_REQUEST)
        {
            if (macInfo.sState.nTransacState == MAC_ASSOC_REQUEST_SENT)
                par_State = MAC_ASSOC_DATA_REQUEST; /* Will send data cmd*/
            else
                par_State = MAC_ASSOC_REQUEST_SENT; /* Sent ass-request cmd*/
        }

        if (pRxMpdu->Mpdu.Payload[0]==CMD_DATA_REQUEST)
            par_State = MAC_ASSOC_DATA_REQUEST_SENT;
        if (pRxMpdu->Mpdu.Payload[0] == CMD_ASSOCIATION_RESPONSE)
        {
            par_State = MAC_ASSOC_END;
            /* update the PIB information*/
            if (pRxMpdu->Mpdu.FrmControl.SourceAddMode == AM_EXTENDED_64)
                mpib.macCoordExtendedAddress = pRxMpdu->Mpdu.srcAddr.Extended;
            else
                mpib.macCoordShortAddress = pRxMpdu->Mpdu.srcAddr.Short;
            mlmeSetRequest(MAC_PAN_ID,&pRxMpdu->Mpdu.srcPanId);
            mlmeSetRequest(MAC_SHORT_ADDRESS,(WORD *) &pRxMpdu->Mpdu.Payload[1]);
        }
    }
}

```

```

    }
    else
        par_State = MAC_ASSOC_END;

        mac_requestState(MAC_TRANSAC_ASSOCIATION, par_State);
        schedul_activate(TASK_PER_MAC_TRANSACTION);
    } // End of functions

    /* Set next state */
    /* active association task */

    /*****
    //@FunctionName: mlme_timeoutAssoDataRes
    */
    //@Description: This function will clean the association request transition
    // if the association response timeout
    */
    /*****
    void mlme_timeoutAssoDataRes(void* args, void* par_pParam)
    {
        mlme_notifyAssociation(NO_DATA, par_pParam);
    } // End of functions

    void mlme_runAssoDataReq(void* args, void* par_param)
    {
        mlme_notifyAssociation(SUCCESS, (void *) par_param);
    } // End of functions

    /*****
    //@FunctionName:
    */
    //@Description: This function will handle association request transition
    */
    /*****
    void mlme_handleAssocTransition(void* par_pParam,
                                    void* args)
    {
        /***** Variable declaration *****/
        P_MPDU_RX    pRxMpdu;
        UINT8        CommandID;
        MAC_STATUS    macStatus;
        ASSOCIATION_INFO* pAssolInfo;    /* Information for association */
        /***** Function body *****/
        pAssolInfo = (ASSOCIATION_INFO*)par_pParam;

        switch (macInfo.sState.nTransacState)
        {
            case MAC_ASSOC_REQUEST:
                /*transmit the packet
                if(tx_trans((P_MPDU_TX) pAssolInfo->pParam) == FALSE)
                {
                    mlme_notifyAssociation(CHANNEL_ACCESS_FAILURE, NULL);
                    mem_releaseTxPkt((P_MPDU_TX) pAssolInfo->pParam);
                    mac_requestState(MAC_TRANSAC_ASSOCIATION, MAC_ASSOC_END);
                state */
                    break;
                }
                schedul_suspend(TASK_PER_MAC_TRANSACTION);
                break;
            case MAC_ASSOC_REQUEST_SENT:
                /*set up timer to send data request
                timer_startTimer(runAssoDataReq,

```

```

        pAssolInfo->pParam,

#ifdef ZIGBEE_STUB
        20
#else
        (UINT32)aResponseWaitTime / aUnitBackoffPeriod
#endif
    );
    schedul_suspend(TASK_PER_MAC_TRANSACTION);

    break;
case MAC_ASSOC_DATA_REQUEST:
    CommandID = CMD_DATA_REQUEST;
    //prepare the data request
    if (mpib.macCoordShortAddress >= VALID_SHORT_ADDR) // extended address
    {
        macStatus = mac_formatMpdu((P_MPDU_TX) pAssolInfo->pParam,
            MAC_FRM_CMD,
            AM_EXTENDED_64,
            mpib.macPANId,
            (ADDRESS *) &aExtendedAddress,
            AM_EXTENDED_64,
            mpib.macPANId,
            (ADDRESS *) &mpib.macCoordExtendedAddress,
            TX_OPT_ACK_REQ,
            &CommandID,
            1);
    }
    else
    {
        macStatus = mac_formatMpdu((P_MPDU_TX) pAssolInfo->pParam,
            MAC_FRM_CMD,
            AM_EXTENDED_64,
            mpib.macPANId,
            (ADDRESS *) &aExtendedAddress,
            AM_SHORT_16,
            mpib.macPANId,
            (ADDRESS *) &mpib.macCoordShortAddress,
            TX_OPT_ACK_REQ,
            &CommandID,
            1);
    }

    if (macStatus != SUCCESS)
    {
        mlmeAssociateConfirm(0xFFFF,INVALID_PARAMETER);
        return;
    }

    /* other information do not need to be set*/
    //transmit the packet
    if(!tx_trans((P_MPDU_TX) pAssolInfo->pParam))
    {
        mem_releaseTxPkt((P_MPDU_TX) pAssolInfo->pParam);
        mac_requestState(MAC_TRANSAC_ASSOCIATION, MAC_ASSOC_END);        /* Set next
state */
    }

```

```

    /* wait to be weak up*/
    schedul_suspend(TASK_PER_MAC_TRANSACTION);

    break;
case MAC_ASSOC_DATA_REQUEST_SENT:
    /*release the packet
    mem_releaseTxPkt((P_MPDU_TX) pAssolInfo->pParam);

    /*wait for the response from the coord
    timer_startTimer(timeoutAssoDataRes,
        NULL,
#ifdef ZIGBEE_STUB
        200
    #else
        (UINT32)aMaxFrameResponseTime / aUnitBackoffPeriod
    #endif
    );
    schedul_suspend(TASK_PER_MAC_TRANSACTION);
    break;
case MAC_ASSOC_END:

    /* Reset mac state */
    mac_requestState(MAC_TRANSAC_NO,MAC_TRANSAC_NO);

    /*received the response
    pRxMpdu = (P_MPDU_RX) pAssolInfo->pParam;

    if (pAssolInfo->macStatus == SUCCESS)
    {
        /*cancel the timer
        timer_endTimer(timeoutAssoDataRes);

        /*call to upper layer
        mlmeAssociateConfirm(mpib.macShortAddress,SUCCESS);
    }
    else
    {
        mlmeAssociateConfirm(0xFFFF,pAssolInfo->macStatus);
        if (pAssolInfo->macStatus != NO_DATA )
            timer_endTimer(timeoutAssoDataRes); /*cancel the timer*/
    }

    /*release the packet
    if (pAssolInfo->macStatus != NO_DATA
        && pAssolInfo->macStatus != SUCCESS)
    {
        mem_releaseRxPkt(pRxMpdu);
    }

    mac_setTrxState(PHY_TRX_OFF);
    mem_freeMem(pAssolInfo);
    schedul_removeTask(TASK_PER_MAC_TRANSACTION);
    break;
default:
    break;
}

```



```

    return;
} // End of function

#if ZIGBEE_DEVICE_FFD == 1
/*****
//@FunctionName: mlmeAssociateResponse
void mlmeAssociateResponse(ADDRESS* pDeviceAddress, WORD ShortAddress,
                           UINT8 status, BOOL securityEnable)
{
    /***** Variable declaration *****/
    P_MPDU_TX    pMpduTx;
    /***** Function body *****/
    pMpduTx = mem_getFreeTxPkt(); /*Allocate the packet*/
    if (!pMpduTx)
    {
        return;
    }

    //set other information
    pMpduTx->PktInfo.txMode
TX_MODE_NO_CSMACA; //TX_MODE_CSMACA|TX_MODE_FORCE_UNSLOTTED;
    pMpduTx->PktInfo.nRetryLeft = aMaxFrameRetries;

    pMpduTx->Mpdu.Payload[0] = CMD_ASSOCIATION_RESPONSE;
    mem_cpy(pMpduTx->Mpdu.Payload+1, &ShortAddress, sizeof(WORD));
    pMpduTx->Mpdu.Payload[3] = status; /* status */
    pMpduTx->Mpdu.nLength = 1+ sizeof(WORD) + 1;

    if (mpib.macShortAddress >= VALID_SHORT_ADDR)
        mac_formatMpdu(pMpduTx,
                        MAC_FRM_CMD,
                        AM_EXTENDED_64,
                        mpib.macPANId,
                        (ADDRESS *) &aExtendedAddress,
                        AM_EXTENDED_64,
                        mpib.macPANId,
                        pDeviceAddress,
                        TX_OPT_INDIRECT|TX_OPT_ACK_REQ,
                        pMpduTx->Mpdu.Payload,
                        4);
    else
        mac_formatMpdu(pMpduTx,
                        MAC_FRM_CMD,
                        AM_SHORT_16,
                        mpib.macPANId,
                        (ADDRESS *) &mpib.macShortAddress,
                        AM_EXTENDED_64,
                        mpib.macPANId,
                        pDeviceAddress,
                        TX_OPT_INDIRECT|TX_OPT_ACK_REQ,
                        pMpduTx->Mpdu.Payload,
                        4);

    indirectTx_addPkt(pMpduTx);
} // End of functions

```



```
#endif //ZIGBEE_DEVICE_FFD == 1
//-----
//--- MLME-DISASSOCIATE
//-----
// MLME-DISASSOCIATE.request structure
// is used by an associated device to notify the coordinator of its intent to leave the PAN
//-----
//-----
void mlmeDisassociateRequest(QWORD DeviceAddress, // The address of the device to which to
send the disassociation notification command
                        UINT8 DisassociateReason, // The reason for disassociation
                        BOOL SecurityEnable) // TRUE if enabled for this transfer
{
    /***** Variable declaration *****/
    P_MPDU_TX pMpdu;
    BYTE Payload[2];
    MAC_STATUS macStatus;
    /***** Function body *****/
    //if it is NOT "to coord."
    if((DeviceAddress==mpib.macCoordExtendedAddress)
        || mac_requestState(MAC_TRANSAC_DISASSOCIATION,MAC_TRANSAC_DISASSOCIATION))
    {
        mlmeDisassociateConfirm(UNDEFINED);
        return;
    }
    else
    {
        //else, send the CMD to the coord.
        //prepare the packet
        //reserver a packet
        pMpdu = mem_getFreeTxPkt();
        mem_set(&pMpdu->Mpdu.FrmControl,2);
        if (pMpdu == NULL)
        {
            mlmeDisassociateConfirm(TRANSACTION_OVERFLOW);
            return;
        }

        Payload[0] = CMD_DISASSOCIATION_NOTIFICATION;
        Payload[1] = DisassociateReason;

        //prepare the data request
        macStatus = mac_formatMpdu(pMpdu,
                                MAC_FRM_CMD,
                                AM_EXTENDED_64,
                                mpib.macPANId,
                                (ADDRESS *) &aExtendedAddress,
                                AM_EXTENDED_64,
                                mpib.macPANId,
                                (ADDRESS *) &mpib.macCoordExtendedAddress,
                                TX_OPT_ACK_REQ,
                                Payload,
                                2);

        if (macStatus != SUCCESS)
        {
            mlmeAssociateConfirm(0xFFFF,INVALID_PARAMETER);
        }
    }
}
```

```

    return;
}

//set other information
MacTx.bUsedCsmaca = TRUE;
pMpdu->PktInfo.nRetryLeft = aMaxFrameRetries;
//pMpdu->PktInfo.msduHandle = msduHandle;

//transmit the packet
if(!tx_trans(pMpdu))
    mem_releaseTxPkt(pMpdu);
}

return;
} // End of functions

/*****
//@FunctionName: mlme_handleDisassoTransition
*****/
void mlme_handleDisassoTransition(MAC_STATUS par_macStatus,
                                  void* par_Param)
{
    /***** Variable declaration *****/
    P_MPDU_TX    pMpdu;
    /***** Function body *****/

    //release the packet
    pMpdu = (P_MPDU_TX) par_Param;
    mem_releaseTxPkt(pMpdu);

    //call confirm primitive
    mlmeDisassociateConfirm(par_macStatus);
} // End of functions

/*****
//@FunctionName:    mlme_handleDisassoReq
//@Description: This function will handle the disassociation req*/
//
//@Param
//
*****/
void mlme_handleDisassoReq(MAC_STATUS par_macStatus,
                           void* par_Param)
{
    mlmeDisassociateConfirm(par_macStatus);
} // End of functions

//-----
//---                MLME-DISASSOCIATE
//-----
// MLME-RESET.request generated by the next higher layer and issued to its MLME
// to request a reset of the MAC sublayer to its initial conditions
//-----
/*****
//@FunctionName:
//@Description:
*****/

```

```

//                                                                    */
//@Param                                                                */
//      SetDefaultPIB: If true the MACsublayer is reset and all MAC
//      PIB attributes are set to their default values);                */
/*****/
MAC_STATUS mlmeResetRequest(BOOL      SetDefaultPIB)
{
    /***** Variable declaration *****/
    WORD      oldRegVal;
    /***** Function body *****/

    /* Disable the FIFOP interrupt, and also T1_COMPA,
    which would otherwise turn the FIFOP interrupt back on */
    DISABLE_T1_COMPA_INT();
    DISABLE_T1_CAPTURE_INT();

    /* This is the first not time initialization */
    if(macInfo.sState.nTransacType != MAC_INITIALIZATION)
    {
        mac_setTrxState(PHY_TRX_OFF); /* turn off TRX */
        mem_release();               /* release the memory in MAC*/
    }

    //reset beacon info
    BeaconInfo.nFinalCapSlot    = 15;
    BeaconInfo.bBatteryLifeExt  = FALSE;
    BeaconInfo.bTrackBeacon     = FALSE;
    BeaconInfo.bTrackBeaconReq  = FALSE;
    BeaconInfo.nBeaconDuration  = 0;
    #if      ZIGBEE_DEVICE_FFD == TRUE
        BeaconInfo.pBeacon      = NULL;
    #endif    /* ZIGBEE_DEVICE_FFD == TRUE

    // Reset time stamps and counters
    SlotInfo.bosCounter = 0;
    SlotInfo.captureBosCounter = 0;
    SlotInfo.captureTcnt = 0;
    SlotInfo.captureTime = 0;
    SlotInfo.bosCounterAdjustTime = 0;

    //phy reset
    PhyInfo.bFifopIntOff = FALSE;
    PhyInfo.eState        = PHY_RX_STATE_PHR;
    PhyInfo.nQueueingPkt = 0;

    //set PIB to the default value
    if (SetDefaultPIB)
    {
        PhyInfo.PhyPIB.phyCurrentChannel = 11;
        PhyInfo.PhyPIB.phyTransmitPower = 0x12;
        PhyInfo.PhyPIB.phyCcaMode = 1;

        mpib.macAckWaitDuration = 54;
        mpib.macAssociationPermit = FALSE;
        mpib.macAutoRequest = TRUE;
        mpib.macBattLifeExt = FALSE;
        mpib.macBattLifeExtPeriods = 6;
    }
}

```



```

    mpib.macBeaconPayload = NULL;
    mpib.macBeaconPayloadLength = 0;
    mpib.macBeaconOrder = 15;
    mpib.macBeaconTxTime = 0;
    mpib.macBSN = random_getRandomByte();
    mpib.macCoordShortAddress = INVALID_SHORT_ADDR;
    mpib.macDSN = random_getRandomByte();
    mpib.macGTSPermit = TRUE;
    mpib.macMaxCSMABackoffs = 4;
    mpib.macMinBE = 3;
    mpib.macPANId = 0xFFFF;
    mpib.macPromiscuousMode = FALSE;
    mpib.macRxOnWhenIdle = TRUE;
    mpib.macShortAddress = INVALID_SHORT_ADDR;
    mpib.macSuperframeOrder = 15;
    mpib.macTransactionPersistenceTime = 0x0004; //0x01F4
    mpib.macCoordExtendedAddress = 0xFFFF;
}

// Configure CC2420 registers
// Write default register values
FASTSPI_SETREG(CC2420_MDMCTRL1, 0x0500);
FASTSPI_SETREG(CC2420_IOCFG0, 0x0003);
FASTSPI_SETREG(CC2420_SECCTRL0, CC2420_SECCTRL0_NO_SECURITY);
FASTSPI_GETREG(CC2420_RXCTRL1, oldRegVal);
oldRegVal |= 0x2000;
FASTSPI_SETREG(CC2420_RXCTRL1, oldRegVal);

// PAN ID, extended and short addresses
phy_WriteRam(&mpib.macPANId, CC2420RAM_PANID, 2, FALSE);
phy_WriteRam(&aExtendedAddress, CC2420RAM_IEEEADDR, 8, FALSE);
phy_WriteRam(&mpib.macShortAddress, CC2420RAM_SHORTADDR, 2, FALSE);

// set the current channel
plmeSET_request(PHY_PIB_CURRENT_CHANNEL, PhyInfo.PhyPIB.phyCurrentChannel);

//initialize Tx
tx_init();

#if ZIGBEE_DEVICE_FFD == TRUE
    indirectTx_init();
#endif

//initialize the task queue, MAC memory
schedul_resetPerTask();
mem_initialize(); /* Reset packet management */

PhyInfo.sCurPacket.pCurRxPacket = mem_getFreeRxPkt();

/* Ready state */
macInfo.DeviceType = 0; /* Device default*/
macInfo.pibTempBuffer.updateMask = 0;
macInfo.sState.nTransacType = MAC_TRANSAC_NO;

mlmeResetConfirm(SUCCESS);

```

```

// Turn the interrupts back on
ENABLE_T1_COMPA_INT();
ENABLE_T1_CAPTURE_INT();

return SUCCESS;
} // End of functions

//-----
// MLME-SCAN.request structure
// is used to initiate a channel scan over a given list of channels
//-----
/*****/
//@FunctionName:
                                */
//@Description:   This function will handle scan procedure          */
//
//                                     */
//@Param
//                                     */
//                                     */
/*****/
void mlme_handleScanTransition(void* par_pParam, void* args);

/*****/
//@FunctionName:
                                */
//@Description:   This function will handle timeout in scanning procedure  */
//               + Passive scan procedure
//
//                                     */
//@Param
//                                     */
//                                     */
/*****/
void mlme_scanTimeout()
{
    macInfo.sState.nTransacState = SCAN_FINISHED_CHANNEL;
} // End of functions

/*****/
//@FunctionName:
                                */
//@Description:   This is the main function of mlme.scanrequest
//
//                                     */
//@Param
//                                     */
//                                     */
/*****/
void mlmeScanRequest(  UINT8      ScanType,      // Indicates the type of scan
                      DWORD      ScanChannels,   // The 5 MSBs are reserved. The 27 LSBs
                      // indicate which channels are to be scanned for each 27 valid channels
                      UINT16      ScanDuration)  // A value used to calculate the length of time to
// spend scanning each channel for ED, active, and passive scans
{
    /***** Variable declaration *****/

    /***** Function body *****/

    /* Control state for new transition*/
    if(! mac_requestState(MAC_TRANSAC_SCAN, SCAN_INITIALIZATION))

```

```

{
    mlmeScanConfirm(UNDEFINED, NULL);
    return;
}

pScanInfo = mem_getMem(sizeof(MAC_SCAN_INFO));
if (pScanInfo==NULL)
{
    mlmeScanConfirm(UNDEFINED, NULL);
    return;
}

if (ScanType != ORPHAN_SCAN)
{
    pScanResult = mem_getMem(sizeof(MAC_SCAN_RESULT));
    if (pScanResult==NULL)
    {
        mlmeScanConfirm(UNDEFINED, NULL);
        return;
    }
}

/* Store the scan information in to the variable */
pScanInfo->oldPhyCurrentChannel = PhyInfo.PhyPIB.phyCurrentChannel;
pScanInfo->scanType = ScanType;
pScanInfo->scanChannels = ScanChannels;
pScanInfo->scanDuration = ScanDuration;

/* Create task to scan*/
schedul_addTask(mlme_handleScanTransition, TASK_PER_MAC_TRANSACTION, NULL);

} // End of functions

/*****
//FunctionName:
*/
//@Description: This function will handle scan procedure
//
//@Param
//
//
*****/
void mlme_handleScanTransition(void* par_pParam, void* args)
{
    /***** Variable declaration *****/
    WORD iocfg0;
    DWORD bitmask = 0x00000001;
    DWORD panId;
    BOOL bContinue = FALSE;
    MAC_STATUS macStatus;
    UINT8 n;
    /***** Function body *****/
    switch(macInfo.sState.nTransacState)
    {
    case SCAN_INITIALIZATION:

        /* Configure CC2420 for scanning */
        if (pScanInfo->scanType != ORPHAN_SCAN) {

```

```

DISABLE_GLOBAL_INT();
panId = 0xFFFF;
FASTSPI_WRITE_RAM_LE(&panId, CC2420RAM_PANID, 2, n);

// Set the BCN_ACCEPT bit
FASTSPI_GETREG(CC2420_IOCFG0, iocfg0);
iocfg0 |= 0x0800;
FASTSPI_SETREG(CC2420_IOCFG0, iocfg0);
ENABLE_GLOBAL_INT();
}
/* Prepare the packet for orphan scan */
if ((pScanInfo->scanType == ORPHAN_SCAN) || (pScanInfo->scanType == ACTIVE_SCAN))
{
    pScanInfo->pScanCMD = mem_getFreeTxPkt();
    pScanInfo->pScanCMD->Mpdu.Payload[0] =
        (pScanInfo->scanType == ACTIVE_SCAN)?
        CMD_BEACON_REQUEST:CMD_ORPHAN_NOTIFICATION;
    macStatus = mac_formatMpdu(pScanInfo->pScanCMD,
                                MAC_FRM_CMD,
                                AM_EXTENDED_64,
                                mpib.macPANId,
                                (ADDRESS *) &aExtendedAddress,
                                AM_SHORT_16,
                                mpib.macPANId,
                                (ADDRESS *) &mpib.macCoordShortAddress,
                                0,
                                pScanInfo->pScanCMD->Mpdu.Payload,
                                1);

    //set other information
    pScanInfo->pScanCMD->PktInfo.txMode = TX_MODE_NO_CSMACA;
    pScanInfo->pScanCMD->PktInfo.nRetryLeft = aMaxFrameRetries;
}

/* Initialize the scan result */
pScanInfo->scanChannels &= MSC_VALID_CHANNELS;
pScanInfo->currentChannel = 0;
pScanResult->unscannedChannels = pScanInfo->scanChannels;
pScanResult->resultListSize = 0;
pScanResult->scanType = pScanInfo->scanType;

switch(pScanInfo->scanType)
{
case ED_SCAN:
case ACTIVE_SCAN:
case ORPHAN_SCAN:
    pScanInfo->bRealigned = FALSE;
    break;
case PASSIVE_SCAN:
    break;
default:
    break;
}

/* Set next state */
macInfo.sState.nTransacState = SCAN_INIT_NEW_CHANNEL;

```

```

    break;
case SCAN_INIT_NEW_CHANNEL:

    /* Condition to get next channel */
    switch(pScanInfo->scanType)
    {
    case ED_SCAN:
        break;
    case ORPHAN_SCAN:
        /* No condition check for orphan scan */
        break;
    case ACTIVE_SCAN:
    case PASSIVE_SCAN:
        if (pScanResult->resultListSize >= MAC_MAX_PAN_DESCRIPTOR)
        {
            macInfo.sState.nTransacState = SCAN_FINISHED;          /* Finish scan */
            break;
        }
        break;
    default:
        break;
    }

    /* Get next channel */
    pScanInfo->currentChannel++;
    for (int n = 0; n < 27; n++) {
        if (n >= pScanInfo->currentChannel) {
            if (bitmask & pScanInfo->scanChannels) {
                pScanInfo->currentChannel = n;
                pScanResult->unscannedChannels &= ~bitmask;
                bContinue = TRUE;
                break;
            }
        }
        bitmask = bitmask << 1;
    }

    if (bContinue) /*not finished*/
    {
        /* Set channel */
        plmeSET_request(PHY_PIB_CURRENT_CHANNEL, (UINT32)pScanInfo->currentChannel);
        macInfo.sState.nTransacState = SCAN_SCANNING;

        switch(pScanInfo->scanType)
        {
        case ED_SCAN:
            break;
        case ACTIVE_SCAN:
            /* Send the orphan notification */
            tx_trans(pScanInfo->pScanCMD);
            break;
        case ORPHAN_SCAN:
            /* Send the orphan notification */
            tx_trans(pScanInfo->pScanCMD);
            break;
        case PASSIVE_SCAN:

```

```

        /* Set timer to check the timeout */
        timer_startTimer(scanTimeout,NULL,
            ((UINT32) aBaseSuperframeDuration / (UINT32) aUnitBackoffPeriod) * (((UINT32) 1 <<
pScanInfo->scanDuration) + 2));
        mac_setTrxState(PHY_RX_ON);                /*Turn on the Rx*/
    default:
        break;
    }

}

else
    macInfo.sState.nTransacState = SCAN_FINISHED; /*Finish scan */
        break;

case SCAN_SCANNING:
    break;
case SCAN_ORPHAN_REALIGNED:
    break;
case SCAN_FINISHED_CHANNEL:
    switch(pScanInfo->scanType)
    {
        case ED_SCAN:
        case ACTIVE_SCAN:
        case ORPHAN_SCAN:
        case PASSIVE_SCAN:
            /*Finish scan on this channel*/
            macInfo.sState.nTransacState = SCAN_INIT_NEW_CHANNEL;
        }
        break;
case SCAN_FINISHED:

    switch(pScanInfo->scanType)
    {
        case ED_SCAN:
            break;
        case ORPHAN_SCAN:
            mem_releaseTxPkt(pScanInfo->pScanCMD); /* release the cmd */
            if (pScanInfo->bRealigned)
                mlmeScanConfirm(SUCCESS,NULL);
            else
                mlmeScanConfirm(NO_BEACON,NULL);
            break;
        case ACTIVE_SCAN:
        case PASSIVE_SCAN:
            /* Generate a scan confirm */
            if (pScanResult->resultListSize > 0)
                macStatus = SUCCESS;
            else
                macStatus = NO_BEACON;
            mlmeScanConfirm(macStatus, pScanResult);
            break;
    }

    // Return to normal state
    mac_requestState(MAC_TRANSAC_NO, MAC_TRANSAC_NO);

    // Restore CC2420 settings

```

```

    if ((pScanInfo->scanType != ORPHAN_SCAN)
        || (pScanInfo->bRealigned == FALSE)) {
        DISABLE_GLOBAL_INT();
        FASTSPI_WRITE_RAM_LE(&mpib.macPANId, CC2420RAM_PANID, 2, n);

        // Clear the BCN_ACCEPT bit
        FASTSPI_GETREG(CC2420_IOCFG0, iocfg0);
        iocfg0 &= ~0x0800;
        FASTSPI_SETREG(CC2420_IOCFG0, iocfg0);
        ENABLE_GLOBAL_INT();

        /* Restore the earlier RX state */
        plmeSET_request(PHY_PIB_CURRENT_CHANNEL, pScanInfo->oldPhyCurrentChannel);
    }

    /* Turn off Rx */
    mac_setTrxState(PHY_TRX_OFF);
    mem_freeMem(pScanInfo);
    pScanInfo = NULL;
    /* Remove the current task */
    schedul_removeTask(TASK_PER_MAC_TRANSACTION);

    break;
default:
    break;
}

} // End of functions

/*****
//FunctionName: mlme_handlePktInScan
*/
//@Description: This function will attract the data from the beacon before
//              it is past to upper layer. Information will be store in the
//              SCAN_RESULT structure
//
//@Param
//
/*****
void mlme_handlePktInScan(P_MPDU_RX par_pRxPkt)
{
    /***** Variable declaration *****/
    PAN_DESCRIPTOR* pPanDesc;
    BOOL blgnore;
    /***** Function body *****/

    switch(pScanInfo->scanType)
    {
    case ED_SCAN:
        break;
    case ORPHAN_SCAN:
        macInfo.sState.nTransacState = SCAN_FINISHED;
        pScanInfo->bRealigned = TRUE;
        break;
    case ACTIVE_SCAN: /*Update with beacon */
    case PASSIVE_SCAN:

```

```

/* Check current available list*/
if (pScanResult->resultListSize >= MAC_MAX_PAN_DESCRIPTOR)
{
    /* Cancel the timeout for the current channel */
    timer_endTimer(scanTimeout);

    /*Set end-state */
    macInfo.sState.nTransacState = SCAN_FINISHED_CHANNEL;
    break;    /* Just ignore it*/
}
blgnore = FALSE;
/* Check if beacon is existing in the list */
for (int i = 0; i < pScanResult->resultListSize; i++) {
    pPanDesc = &pScanResult->pPANDescriptorList[i];
    if ((par_pRxPkt->Mpdu.FrmControl.SourceAddrMode == pPanDesc->CoordAddrMode) &&
        (par_pRxPkt->Mpdu.srcPanId == pPanDesc->CoordPANId) &&
        (pScanInfo->currentChannel == pPanDesc->LogicalChannel)) {

        if (par_pRxPkt->Mpdu.FrmControl.SourceAddrMode == AM_EXTENDED_64) {
            if (par_pRxPkt->Mpdu.srcAddr.Extended == pPanDesc->CoordAddress.Extended)
            {
                blgnore = TRUE;
                break;    /* Just ignore it*/
            }
        } else
        if (par_pRxPkt->Mpdu.srcAddr.Short == pPanDesc->CoordAddress.Short)
        {
            blgnore = TRUE;
            break;    /* Just ignore it*/
        }
    }
}

if(!blgnore)
{
    /* Finished the current channel */
    pScanResult->resultListSize++;

    pPanDesc = &pScanResult->pPANDescriptorList[pScanResult->resultListSize-1];
    pPanDesc->CoordAddrMode = par_pRxPkt->Mpdu.FrmControl.SourceAddrMode;
    pPanDesc->CoordPANId = par_pRxPkt->Mpdu.srcPanId;
    pPanDesc->CoordAddress = par_pRxPkt->Mpdu.srcAddr;
    pPanDesc->LogicalChannel = pScanInfo->currentChannel;
    pPanDesc->SuperframeSpec = *((WORD *)par_pRxPkt->Mpdu.Payload);
    pPanDesc->LinkQuality = par_pRxPkt->Mpdu.Fcs.LinkQuality;
    pPanDesc->GTSPermit = !!(par_pRxPkt->Mpdu.Payload[2] & 0x80);
    pPanDesc->SecurityFailure = par_pRxPkt->Mpdu.FrmControl.SecurityEnable;
}
break;
}

} // End of functions

//-----

//-----

```



```

//---
//----- MLME-POLL
//-----
// The MLME-POLL.request primitive prompts the device to request data from the coordinator.
//-----
MAC_STATUS mlmePollRequest(BYTE coordAddrMode, ADDRESS *coordAddress, WORD coordPANId, BOOL
securityEnable)
{
    /****** Variable declaration *****/
    P_MPDU_TX          pTxMpdu;
    MAC_STATUS          macStatus;
    /****** Function body *****/

    /* Checking MAC status */
    if (mac_requestState(MAC_TRANSAC_POLL, POLL_INIT))
        return TRANSACTION_OVERFLOW;

    /* validate parameters */
    if (((coordAddrMode != AM_SHORT_16)
        && (coordAddrMode != AM_EXTENDED_64)))
        return INVALID_PARAMETER;

    /* Prepare the frame request */
    pTxMpdu = mem_getFreeTxPkt();
    mem_set(&pTxMpdu->Mpdu.FrmControl, 2);
    if (!pTxMpdu)
        return TRANSACTION_OVERFLOW;

    /*Prepare the payload of the POLL.Reg
    pTxMpdu->Mpdu.PayLoad[0] = CMD_DATA_REQUEST;

    /*Set other information
    pTxMpdu->PktInfo.txMode      = TX_MODE_NO_CSMACA;
    pTxMpdu->PktInfo.nRetryLeft = aMaxFrameRetries;

    macStatus = mac_formatMpdu(pTxMpdu,
                                MAC_FRM_CMD,
                                AM_EXTENDED_64,
                                coordPANId, /*TBD*/
                                (ADDRESS *) &aExtendedAddress,
                                coordAddrMode,
                                coordPANId,
                                (ADDRESS *) coordAddress,
                                TX_OPT_ACK_REQ,
                                pTxMpdu->Mpdu.PayLoad,
                                1);

    if (macStatus != SUCCESS)
        mlmePollConfirm(INVALID_PARAMETER);

    if(tx_trans(pTxMpdu)) /* Transmission */
        mem_releaseTxPkt(pTxMpdu);

    return SUCCESS;
} // End of functions
/*****
//@FunctionName: mlme_autoPollTask
*/

```

```

//@Description:
//
//@Param
//
/*****
void    mlme_autoPollTask(void* arg1,void* arg2)
{
    /***** Variable declaration *****/
    MAC_STATUS    Status;
    /***** Function body *****/
    if (mpib.macCoordShortAddress >= VALID_SHORT_ADDR)
        Status = mlmePollRequest(AM_SHORT_16, mpib.macPANId,
        (ADDRESS*)&mpib.macCoordShortAddress, FALSE);
    else
        Status = mlmePollRequest(AM_EXTENDED_64, mpib.macPANId,
        (ADDRESS*)&mpib.macCoordExtendedAddress, FALSE);

    if(Status == SUCCESS)
        schedul_removeTask(TASK_PER_MAC_TRANSACTION);
    /*if not success try later*/
} // End of functions
/*****
//@FunctionName:
//
//@Description:
//
//@Param
//
/*****
void    mlme_autoPoll(){
    schedul_addTask(mlme_autoPollTask,TASK_PER_MAC_TRANSACTION,NULL);
}
/*****
//@FunctionName: mlme_handlePollTransition(
/*****
void    mlme_handlePollTransition(MAC_STATUS    par_macStatus, void* par_pParam)
{
    /***** Variable declaration *****/
    P_MPDU_RX    pRxMpdu;
    /***** Function body *****/
    switch(macInfo.sState.nTransacState)
    {
    case POLL_INIT:
        switch(par_macStatus)
        {
        case INVALID_PARAMETER:
        case CHANNEL_ACCESS_FAILURE:
        case NO_ACK:
            mlmePollConfirm(par_macStatus);
            mac_requestState(MAC_TRANSAC_NO, MAC_TRANSAC_NO);
            break;
        case SUCCESS:
            if (MacTx.bFrmPending == 1)
            {
                /* Set to state to received the response, keep RX on*/
                mac_requestState(MAC_TRANSAC_POLL, POLL_SENT_REQUEST);
                timer_startTimer(timeoutPoll,NULL, (UINT32)aMaxFrameResponseTime /

```

```

aUnitBackoffPeriod);
    }
    else /* Received ACK but no data */
    {
        mlmePollConfirm(NO_DATA);
        mac_requestState(MAC_TRANSAC_NO, MAC_TRANSAC_NO);
    }
    break;
}
mem_releaseTxPkt((P_MPDU_TX)par_pParam);          /* Release the packet */
break;
case POLL_SENT_REQUEST:                            /* Do nothing */
    break;
case POLL_FINISHED:                                /* received the data */
    pRxMpdu = (P_MPDU_RX)par_pParam;
    /* Check the received frame */
    if (pRxMpdu == NULL)                            /* Time out */
    {
        mlmePollConfirm(NO_DATA);
        mac_requestState(MAC_TRANSAC_NO, MAC_TRANSAC_NO);
    }
    else                                             /* Received the data */
    {
        if ((pRxMpdu->Mpdu.FrmControl.FrameType == MAC_FRM_CMD)
            || (pRxMpdu->Mpdu.nLength == 0))          /* Received cmd or zero length */
        {
            mlmePollConfirm(NO_DATA);
        }
        if (pRxMpdu->Mpdu.nLength != 0)              /* Received a frame */
        {
            mlmePollConfirm(SUCCESS);
        }
        timer_endTimer(timeoutPoll); /* End timer */
        mac_requestState(MAC_TRANSAC_NO, MAC_TRANSAC_NO); /* End the transaction */
    }
    break;
default:
    break;
}

} // End of functions

/*****
//@FunctionName:  mlme_timeoutPoll
*/
//@Description:
//
//@Param
//
*****/
void mlme_timeoutPoll(void* args1, void* args2)
{
    mac_requestState(MAC_TRANSAC_POLL, POLL_FINISHED);
    mlme_handlePollTransition(NO_DATA, NULL);
} // End of functions

```

```
//-----
//---                               MLME-RX-ENABLE
//-----
// The MLME-RX-ENABLE.request primitive is generated by the next higher layer and issued to the MLME
// to enable the receiver for a fixed duration, at a time relative to the start of the current or next superframe on
// a beacon-enabled PAN or immediately on a nonbeacon-enabled PAN. The receiver is enabled exactly once
// per primitive request
//-----
/*****/
//@FunctionName:
                                */

//@Description:
//
                                */
//@Param
                                */
//
                                */
/*****/
void mlme_handleRxEnableTask(void* par_pParam,
                             void* args)
{
    /***** Variable declaration *****/
    RXENABLE_INFO* pRxEndableInfo;
    /***** Function body *****/
    if (mpib.macBeaconOrder == 15)
    {
        switch(macInfo.sState.nTransacState)
        {
            case RX_ENABLE:
                /* Enable RX only when TX finished the current transmission */
                if(MacTx.eState == MAC_TX_IDLE)
                {
                    pRxEndableInfo = (RXENABLE_INFO*)par_pParam;
                    timer_startTimer(handleRxEnableTask, (UINT32)par_pParam,pRxEndableInfo->nDuration);

/*Timer*/

                    macInfo.sState.nTransacState = RX_NORMAL;
                    schedul_suspend(TASK_PER_MAC_TRANSACTION); /*Keep task in queue */
                    mac_setTrxState(PHY_RX_ON);
                }
                /* Not IDLE, try next */
                break;
            case RX_NORMAL:
                mac_setTrxState(PHY_TRX_OFF);
                macInfo.sState.nTransacType = MAC_TRANSAC_NO;
                macInfo.sState.nTransacState = MAC_TRANSAC_NO;
                schedul_removeTask(TASK_PER_MAC_TRANSACTION); /*Remove task in queue */
                break;
            default:
                break;
        }
    }
    else {} //TBD
} // End of functions

/*****/
//@FunctionName:
                                */

//@Description:
//
                                */
                                */
```

```

//@Param                                                                    */
//                                                                    */
/***** Variable declaration *****/
void mlmeRxEnableRequest(BOOL DeferPermit, UINT32 RxOnTime, UINT32 RxOnDuration)
{
    /***** Variable declaration *****/
    RXENABLE_INFO* pRxEndableInfo;
    /***** Function body *****/
    /*In the non-beacon network */
    if (mpib.macBeaconOrder == 15)
    {
        /* RX Enable request should be a transaction, add task to transaction queue */
        pRxEndableInfo = mem_getMem(sizeof(RXENABLE_INFO));
        pRxEndableInfo->nDuration = (RxOnDuration + (aUnitBackoffPeriod - 1)) / aUnitBackoffPeriod;
        mac_requestState(MAC_TRANSAC_RXENABLE, RX_ENABLE);
        schedul_addTask(mlme_handleRxEnableTask, TASK_PER_MAC_TRANSACTION, pRxEndableInfo);
    }
    else {} //TBD
} // End of functions

//-----
//---                               MLME-SYNC.request
//-----
// MLME-SYNC.request is generated by the next higher layer of */
// a device on a beacon-enabled PAN and issued to its MLME to */
// synchronize with the coordinator
// This request will change the current status of TRx then it
// must be sync with other task using Trx, ex. trans task. This
// request must be queue in TASK_TX to be sync
//-----
void beacon_synsBeaconTask(void* Param, void* arg1);
MAC_STATUS mlmeSyncRequest(UINT8 LogicalChannel, // The logical channel on which to attempt
coordinator synchronization
                        BOOL TrackBeacon) // TRUE --> if the MLME is to synchronize with
the next beacon and attempt to track all future beacons
{
    /***** Variable declaration *****/
    BYTE* pTem;
    WORD DataTask;
    /***** Function body *****/
    pTem = &DataTask;
    pTem[0] = LogicalChannel;
    pTem[1] = TrackBeacon;
    schedul_addTask(beacon_synsBeaconTask, TASK_PER_MAC_PERIODICAL, (DWORD*)DataTask); /*Type cast
from WORD to pointer*/
    return SUCCESS;
} // End of functions

#if ZIGBEE_DEVICE_FFD == TRUE
//-----
//---                               MLME-START.request
//-----
// The MLME-START.request primitive makes a request for the device to start using a new superframe
// configuration.
//-----
void beacon_startBeacon(void* par_pTimer, void* par_pParam);

```

```

void mlmeStartRequest(WORD PanId, UINT8 LogicalChannel, UINT8 BeaconOrder,
                     UINT8 SuperframeOrder, BOOL PanCoordinator, BOOL BatteryLifeExtension,
                     BOOL CoordRealignment, BOOL SecurityEnable,
                     UINT32 StartTime)
{
    /****** Variable declaration *****/
    WORD      mdmctrl0;
    MAC_STATUS returnStatus;
    OS_TMR*    pTimer;
    /****** Function body *****/
    returnStatus = SUCCESS;
    /* Check parameters */
    if (!((LogicalChannel >= 11) && (LogicalChannel <= 26))
        || (BeaconOrder > 15)
        || ((SuperframeOrder > BeaconOrder) && (SuperframeOrder != 15)))
        returnStatus = INVALID_PARAMETER;
    /* Check the address */
    if (mpib.macShortAddress >= VALID_SHORT_ADDR)
        returnStatus = NO_SHORT_ADDRESS;
    /* Check if need to set macSuperframeOrder to 15 */
    if (BeaconOrder == 15)
    {
        BeaconInfo.bTransmitBeacon = FALSE;
        mpib.macSuperframeOrder    = 15;
    }
    else
    {
        mpib.macBeaconOrder        = BeaconOrder;
        mpib.macSuperframeOrder    = SuperframeOrder;
    }

    mlmeSetRequest(MAC_BATT_LIFE_EXT, &BatteryLifeExtension);
    mlmeSetRequest(MAC_PAN_ID, &PanId);
    plmeSET_request(PHY_PIB_CURRENT_CHANNEL, LogicalChannel);
    /* Set the PAN ID */
    /* Change the RF channel */

    mdmctrl0 = 0x0AE2;
    if (PanCoordinator)
    {
        macInfo.DeviceType = ZIGBEE_PAN_COOR;
        mdmctrl0 |= 0x1000; // Set the MDMCTRL0.PAN_COORDINATOR
    }
    else
        macInfo.DeviceType = ZIGBEE_DEVICE_FFD;

    DISABLE_GLOBAL_INT();
    FASTSPI_SETREG(CC2420_MDMCTRL0, mdmctrl0);
    ENABLE_GLOBAL_INT();

    /* Broadcast the coordinator realignment cmd */
    if (CoordRealignment)
    {
        /* TBD */
    }

    /* If transmitting the beacon, the change of parameter will be applied later */
    /* If not, transmit the beacon with super-frame configuration now */
    if (BeaconOrder == 15)

```

```

{
    /* If it is transmitting, stop it*/
}
else /* Transmit the beacon */
{
    if (StartTime == 0)
    {
        BeaconInfo.nBeaconOffset = 0;
        beacon_startBeacon(NULL,NULL);
    }
    else
    {
        pTimer = OSTmrCreate(0,StartTime,OS_TMR_OPT_ONE_SHOT,beacon_startBeacon,
                            NULL,NULL,&mdmctrl0);
        OSTmrStart(pTimer,&mdmctrl0);
    }
}

mlmeStartConfirm(returnStatus);
} // End of functions

//-----
//--- The MLME-ORPHAN.response
//-----
//The MLME-ORPHAN.response primitive allows the next higher layer of a coordinator to respond to the
//MLME-ORPHAN.indication primitive.
//-----
void mlmeOrphanResponse(QWORD* pOrphanAddress, WORD shortAddress,
                        BOOL associatedMember, BOOL securityEnable)
{
    /****** Variable declaration *****/
    P_MPDU_TX      pMpduTx;
    BYTE*          pPayload;
    MAC_STATUS      macStatus;
    /****** Function body *****/

    /*Ignore this response if the associatedMember field is FALSE*/
    if (associatedMember) {
        pMpduTx = mem_getFreeTxPkt();
        pPayload = pMpduTx->Mpdu.Payload;

        /*Format the payload of the realignment cmd*/
        *(pPayload++) = CMD_COORDINATOR_REALIGNMENT;
        // Generate the packet payload
        *(pPayload++) = (BYTE) mpib.macPANId;
        *(pPayload++) = (BYTE) ((mpib.macPANId & 0xFF00) >> 8);
        *(pPayload++) = (BYTE) mpib.macShortAddress;
        *(pPayload++) = (BYTE) ((mpib.macShortAddress & 0xFF00) >> 8);
        *(pPayload++) = PhyInfo.PhyPIB.phyCurrentChannel;
        *(pPayload++) = (BYTE) shortAddress;
        *(pPayload) = (BYTE) ((shortAddress & 0xFF00) >> 8);

        macStatus = mac_formatMpdu(pMpduTx,
                                   MAC_FRM_CMD,
                                   AM_EXTENDED_64,
                                   mpib.macPANId,
                                   (ADDRESS *) &aExtendedAddress,

```

```

        AM_SHORT_16,
        mpib.macPANId,
        (ADDRESS *) &mpib.macCoordShortAddress,
        0,
        pMpduTx->Mpdu.Payload,
        8);

    //set other information
    pMpduTx->PktInfo.txMode      = TX_MODE_NO_CSMACA;
    pMpduTx->PktInfo.nRetryLeft = aMaxFrameRetries;
}
} // End of functions

#endif // ! ZIGBEE_DEVICE_FFD == TRUE

```

9.4.3.3 mac_state.c

```

/*****
//Module: MAC/mac_primitive
//File Name: mac_state.c
//Description: This files will provide function to manage MAC state
//-----*/
//Revision History:
//Date          Programmer          Details
*/
//01/30/2007      pnthai          Create
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
/*          Include files
*/

/*****
#include "mac_state.h"
#include "mac_types.h"
#include "mac_primitives.h"
#include "phy_support.h"
#include "mac_beacon.h"
#include "mac_tx.h"
#include "phy_primitives.h"
#include "mac_mime.h"
*****/

/*          Constant declaration
*/

/*****
/*          Type Definition
*/

/*****
/*          Variable declaration

```



```

    */
/*****/

//variable holds MAC PIB
MAC_PIB      mpib;

//Variable holds MAC info.
MAC_INFO     macInfo;

/*****/
/*          Static Function          */
/*****/

/*****/
/*          Function          */
/*****/

/*****/
//@FunctionName: mac_init
    */
//@Description:    Call this function to first time initialization of MAC    */
//
//@Param          */
//
/*****/
void mac_init()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    macInfo.sState.nTransacType = MAC_INITIALIZATION; /* initialization */
    phytrx_setPwrMode(PHY_PWR_ON); /* Initialize at first time power-on */

} // End of functions

/*****/
//@FunctionName: mac_requestState
    */
//@Description:    This function will control state transition of MAC    */
//                Currently, only one transaction is allowed at tim TBD: check this
//
//@Param          */
//
/*****/
BOOL    mac_requestState(UINT8 par_nTransacType, UINT8 par_nTransacState)
{
    /***** Variable declaration *****/
    BOOL    bResult;
    /***** Function body *****/
    bResult = TRUE;

    switch(par_nTransacType)
    {

```

```

case MAC_TRANSAC_NO:
    bResult = TRUE;
    break;
case MAC_TRANSAC_ASSOCIATION:
    switch(par_nTransacState)
    {
        case MAC_ASSOC_REQUEST:
            bResult = ((macInfo.sState.nTransacType == MAC_TRANSAC_NO)
                || (macInfo.sState.nTransacType == MAC_ASSOC_REQUEST));
            break;
        case MAC_ASSOC_REQUEST_SENT:
            bResult = (macInfo.sState.nTransacType == MAC_TRANSAC_ASSOCIATION
                && macInfo.sState.nTransacState == MAC_ASSOC_REQUEST);
            break;
        case MAC_ASSOC_DATA_REQUEST:
            bResult = (macInfo.sState.nTransacType == MAC_TRANSAC_ASSOCIATION
                && macInfo.sState.nTransacState == MAC_ASSOC_REQUEST_SENT);
            break;
        case MAC_ASSOC_DATA_REQUEST_SENT:
            bResult = (macInfo.sState.nTransacType == MAC_TRANSAC_ASSOCIATION
                && macInfo.sState.nTransacState == MAC_ASSOC_DATA_REQUEST);
            break;
        default:
            break;
    }

    break;
case MAC_TRANSAC_SCAN:
    switch(par_nTransacState)
    {
        case SCAN_INITIALIZATION:
            bResult = (macInfo.sState.nTransacType == MAC_TRANSAC_NO);
            break;
        case SCAN_INIT_NEW_CHANNEL:
        case SCAN_SCANNING:
        case SCAN_FINISHED_CHANNEL:
            break;
        case SCAN_ORPHAN_REALIGNED:
            bResult = (macInfo.sState.nTransacType == SCAN_SCANNING);
            break;
        case SCAN_FINISHED:
            break;
        default:
            break;
    }
    break;
case MAC_TRANSAC_RXENABLE:
    bResult = (macInfo.sState.nTransacType == MAC_TRANSAC_NO
        && macInfo.sState.nTransacState == MAC_TRANSAC_NO);
    break;
    default:
        break;
    }

    if (bResult)
    {
        macInfo.sState.nTransacType = par_nTransacType;
    }
  
```

```

    macInfo.sState.nTransacState = par_nTransacState;
}

return bResult;
} // End of functions

/*****
**@FunctionName: mac_validateMacState
**@Description:
**@Param
**
*****/
BOOL mac_validateMacState(MAC_COMMAND_TYPE parCmd, MAC_FRM_TYPE par_Frm)
{
    /***** Variable declaration *****/
    BOOL    bResult;

    /***** Function body *****/
    bResult = TRUE;
    switch(par_Frm)
    {
    case MAC_FRM_CMD:
        switch(parCmd)
        {
        case CMD_ASSOCIATION_RESPONSE:

            bResult = (macInfo.sState.nTransacType==MAC_TRANSAC_ASSOCIATION
                        && macInfo.sState.nTransacState==MAC_ASSOC_DATA_REQUEST_SENT);

            break;
        case CMD_DISASSOCIATION_NOTIFICATION:
            bResult = (macInfo.sState.nTransacType==MAC_TRANSAC_DISASSOCIATION);
            break;
        case CMD_COORDINATOR_REALIGNMENT:
            bResult = ((macInfo.DeviceType == ZIGBEE_ROUTER) || (macInfo.DeviceType ==
            ZIGBEE_ENDPOINT));
            break;
        case CMD_DATA_REQUEST:
        case CMD_PAN_ID_CONFLICT_NOTIFICATION:
        case CMD_ORPHAN_NOTIFICATION:
        case CMD_BEACON_REQUEST:
        case CMD_GTS_REQUEST:
            bResult = ((macInfo.DeviceType == ZIGBEE_ROUTER) || (macInfo.DeviceType ==
            ZIGBEE_PAN_COOR));
            break;
        default:
            break;
        }

        break;
    case MAC_FRM_DATA:

        bResult = TRUE;

```

```

    break;
case MAC_FRM_BEACON:
    bResult = TRUE;
    break;
case MAC_FRM_ACK:

    bResult = (MacTx.eState==MAC_TX_WAIT_ACK);

    break;
default:
    break;
}

return bResult;

} // End of functions

/*****
//@FunctionName: mac_validateFrmData
//
//@Description: This function will validate the coherence of data with the
//               current data of MAC
//
//@Param
//
*****/
BOOL mac_validateFrmData(P_MPDU_RX par_pRxMpdu)
{
    /***** Variable declaration *****/
    BOOL    bReturn;
    /***** Function body *****/
    bReturn = TRUE;
    switch(par_pRxMpdu->Mpdu.FrmControl.FrameType)
    {
    case MAC_FRM_CMD:
        switch(par_pRxMpdu->Mpdu.PayLoad[0])
        {
        case CMD_ASSOCIATION_REQUEST:
            bReturn = mpib.macAssociationPermit;
            break;
        case CMD_ASSOCIATION_RESPONSE:
            break;
        case CMD_DISASSOCIATION_NOTIFICATION:
            break;
        case CMD_COORDINATOR_REALIGNMENT:
            bReturn = FALSE;
            if (par_pRxMpdu->Mpdu.FrmControl.SourceAddMode == AM_EXTENDED_64)
                if(((macInfo.sState.nTransacType == MAC_TRANSAC_SCAN)
                    && (par_pRxMpdu->Mpdu.FrmControl.DestAddMode == AM_EXTENDED_64))
                    || (par_pRxMpdu->Mpdu.srcAddr.Extended == mpib.macCoordExtendedAddress))
                    bReturn = TRUE;

            break;
        case CMD_DATA_REQUEST:
            break;
        case CMD_PAN_ID_CONFLICT_NOTIFICATION:
            break;

```

```

    case CMD_ORPHAN_NOTIFICATION:
        break;
    case CMD_BEACON_REQUEST:
        break;
    case CMD_GTS_REQUEST:
        break;
    default:
        break;
}
case MAC_FRM_DATA:
    break;
case MAC_FRM_BEACON:
    break;
case MAC_FRM_ACK:
    break;
default:
    break;
}

return bReturn;
} // End of functions

/*****
//@FunctionName: mac_updateDelayedPib
*/
//@Description: This function will update the delayed Pib. It will be called
//               at starting of a new super-frame
//
//
//@Param
//
//
*****/
void mac_updateDelayedPib()
{
    DISABLE_GLOBAL_INT();
    if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_BATT_LIFE_EXT_BM)    mpib.macBattLifeExt =
macInfo.pibTempBuffer.macBattLifeExt;
    if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_BEACON_ORDER_BM)    mpib.macBeaconOrder =
macInfo.pibTempBuffer.macBeaconOrder;
    if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_RX_ON_WHEN_IDLE_BM)    mpib.macRxOnWhenIdle =
macInfo.pibTempBuffer.macRxOnWhenIdle;
    if (macInfo.pibTempBuffer.updateMask & MPIB_UPD_SUPERFRAME_ORDER_BM)
mpib.macSuperframeOrder = macInfo.pibTempBuffer.macSuperframeOrder;
    macInfo.pibTempBuffer.updateMask = 0x00;
    ENABLE_GLOBAL_INT();
}

/*****
//@FunctionName: mac_manageTrxAfterTx
*/
//@Description:
//
//
//@Param
//
//
*****/
void mac_manageTrxAfterTx(P_MPDU_TX    par_pMpdu, MAC_STATUS    par_macStatus)
{

```

```

/***** Variable declaration *****/

/***** Function body *****/
/* Depend on transaction */
switch(macInfo.sState.nTransacType)
{
case MAC_TRANSAC_ASSOCIATION:
    if (macInfo.sState.nTransacState != MAC_ASSOC_DATA_REQUEST)
        phyrx_turnOffTrx(NULL);
    else
        phyrx_initOnRx();
    break;
case MAC_TRANSAC_SCAN:
    phyrx_initOnRx(); /* Auto start RX to receive the ACK*/
default:
    if (mpib.macRxOnWhenIdle || BeaconInfo.bTrackBeaconReq)
    {
        phyrx_initOnRx(); /* Auto start RX */
    }
    else
    {
        phyrx_turnOffTrx(NULL);
    }
    break;
}
} // End of functions

/*****
//@FunctionName: mac_setTrxState
*/
//@Description: This function will control the TRX state
//              - Turn on Rx while Tx is on, do nothing
//              - Turn off Rx while Tx is on, do nothing
//
//
//@Param
//
//
*****/
PHY_STATUS mac_setTrxState(PHY_STATUS par_State)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    //cause PHY change state
    if (((PhyInfo.PhyState == PHY_RX_ON) || (PhyInfo.PhyState == PHY_TRX_OFF)
        || (PhyInfo.PhyState == PHY_TX_ON)) && (PhyInfo.PhyState == par_State))
    {
    }
    else if (par_State == PHY_FORCE_TRX_OFF && PhyInfo.PhyState != PHY_TRX_OFF)
        phyrx_turnOffTrx(NULL); /* change state now !*/
    else if (par_State == PHY_TX_ON)
        phyt_x_turnOnTx();
    else if (par_State == PHY_TRX_OFF)
    {
        if (!mpib.macRxOnWhenIdle && !BeaconInfo.bTrackBeaconReq)
        {
            phyrx_turnOffTrx(NULL);
        }
    }
}

```

```

    }
    else if ((PhyInfo.PhyState == PHY_TX_ON)&& (par_State == PHY_RX_ON))
        phyrx_initOnRx();
    else if (par_State == PHY_TX_ON)
        phytx_turnOnTx(NULL);
    else if ((par_State == PHY_RX_ON))
        phyrx_turnOnRx(NULL);

    return PHY_SUCCESS;
} // End of functions

/*****
//FunctionName: mac_forceOffTrx
*/
//Description:
//
//@Param
//
/*****
void mac_forceOffTrx(void* arg1, void* arg2)
{
    phyrx_turnOffTrx(NULL);
}

```

9.4.4 MAC RX

```

/*****
//Module: MAC/mac_rx
//File Name: mac_rx.c
//Description: This file implements packet reception mechanisms
//-----*/
// Copyright (C) 2006 - All Rights Reserved
/*****

/*****
/*
                                Include files
                                */

/*****
#include "mac_rx.h"
#include "mac_packet.h"
#include "mac_primitives.h"
#include "mac_beacon.h"
#include "phy_primitives.h"
#include "mac_tx.h"
#include "mac_mem.h"
#include "mac_mlme.h"
#include "schedul.h"
#include "utils.h"

/*****
/*
                                Constant declaration
                                */

/*****

/*****
/*
                                Type Definition
                                */

```

```

/*****/

/*****/
/*                                     Variable declaration
*/
/*****/

/*****/
/*                                     Static Function                               */
/*****/

/*****/
/*                                     Function                                   */
/*****/

/*****/
//@FunctionName:  macrx_ProcessMpdu
*/
//@Description:   this function will process the received packet                */
//
//                                     */
//@Param
//                                     */
/*****/
MCPS_DATA_INDICATION* pIndiData;
void macrx_ProcessMpdu(P_MPDU_RX par_pRxMpdu)
{
    /***** Variable declaration *****/
    BYTE*      pPayload;
    BOOL       bEnded;
    FRAME_CONTROL* pFrameCtrl;
    /***** Function body *****/
    bEnded = FALSE;

    /* If ACK */
    if(par_pRxMpdu->Mpdu.FrmControl.FrameType == MAC_FRM_ACK)
    {
        /* For ACK no Rx packet is created */
        tx_notifyAck(par_pRxMpdu);
        return;
    }

    pFrameCtrl = &par_pRxMpdu->Mpdu.FrmControl;
    /* Transaction oriented command processing */
    switch(macInfo.sState.nTransacType)
    {
    case MAC_TRANSAC_NO:
        break;
    case MAC_TRANSAC_ASSOCIATION:
        if (par_pRxMpdu->Mpdu.FrmControl.FrameType == MAC_FRM_CMD
            && par_pRxMpdu->Mpdu.Payload[0] == CMD_ASSOCIATION_RESPONSE)
        {
            mlme_notifyAssociation(SUCCESS, (void*)par_pRxMpdu);
        }
    }
}

```



```

    if (par_pRxMpdu->Mpdu.FrmControl.FrameType == MAC_FRM_DATA
        && par_pRxMpdu->Mpdu.nLength == 0)
    {
        mlme_notifyAssociation(NO_DATA, (void*)par_pRxMpdu);
    }
    break;
case MAC_TRANSAC_DISASSOCIATION:
    break;
case MAC_TRANSAC_POLL:
    mac_requestState(MAC_TRANSAC_POLL, POLL_FINISHED);          /* In the poll transition */
    mlme_handlePollTransition(SUCCESS, par_pRxMpdu);
    break;
case MAC_TRANSAC_SCAN:
    switch(pScanInfo->scanType)
    {
        case ED_SCAN:
            break;
        case ACTIVE_SCAN:
        case PASSIVE_SCAN:
            /* If it is in scanning state */
            if (macInfo.sState.nTransacType == MAC_TRANSAC_SCAN)
                mlme_handlePktInScan(par_pRxMpdu);
            break;
        case ORPHAN_SCAN:
            if (par_pRxMpdu->Mpdu.PayLoad[0] == CMD_COORDINATOR_REALIGNMENT)
                mlme_handlePktInScan(par_pRxMpdu);
            break;
    }
default:
    break;
}

/* Command oriented processing */
switch(par_pRxMpdu->Mpdu.FrmControl.FrameType)
{
case MAC_FRM_CMD:
    switch(par_pRxMpdu->Mpdu.PayLoad[0])
    {
        case CMD_COORDINATOR_REALIGNMENT:

            /* Process the command: Update data */
            pPayload = par_pRxMpdu->Mpdu.PayLoad;
            mlmeSetRequest(MAC_COORD_EXTENDED_ADDRESS, (WORD*) &par_pRxMpdu-
                >Mpdu.srcAddr);
            mlmeSetRequest(MAC_PAN_ID, (WORD*) &pPayload[1]);
            mlmeSetRequest(MAC_COORD_SHORT_ADDRESS, (WORD*) &pPayload[3]);
            plmeSET_request(PHY_PIB_CURRENT_CHANNEL, pPayload[5]);

            /* For orphan responses, with extended destination address,
            update the short address from the realignment frame */
            if (par_pRxMpdu->Mpdu.FrmControl.DestAddMode == AM_EXTENDED_64) {
                mlmeSetRequest(MAC_SHORT_ADDRESS, (WORD*) &pPayload[6]);
            }
            mlmeSyncLossIndication(REALIGNMENT);
            break;
    }
}

#if ZIGBEE_DEVICE_FFD == TRUE
case CMD_ASSOCIATION_REQUEST:

```

```

    mlmeAssociateIndication(&par_pRxMpdu->Mpdu.srcAddr, par_pRxMpdu-
>Mpdu.Payload[1], FALSE, 0);
    break;
case CMD_DATA_REQUEST:
    indirectTx_RespondDataReq(par_pRxMpdu);
    break;
case CMD_PAN_ID_CONFLICT_NOTIFICATION:
    /* The PAN Coordinator has received a PAN ID conflict notification
    command from a device on its own PAN */
    if (par_pRxMpdu->Mpdu.srcPanId == mpib.macPANId)
        mlmeSyncLossIndication(PAN_ID_CONFLICT);
    break;
case CMD_DISASSOCIATION_NOTIFICATION:
    if ((pFrameCtrl->SourceAddMode == AM_EXTENDED_64) &&
        (pFrameCtrl->DestAddMode == AM_EXTENDED_64)) {
        mlmeDisassociateIndication(&par_pRxMpdu->Mpdu.srcAddr.Extended,
par_pRxMpdu-
>Mpdu.Payload[1],
FALSE, FALSE);
    }
    break;
case CMD_ORPHAN_NOTIFICATION:
    if ((pFrameCtrl->SourceAddMode == AM_EXTENDED_64) &&
        (pFrameCtrl->DestAddMode == AM_SHORT_16)) {

        // Must have broadcast source and destination PAN
        if ((par_pRxMpdu->Mpdu.srcPanId == 0xFFFF) &&
            (par_pRxMpdu->Mpdu.dstPanId == 0xFFFF))
            mlmeOrphanIndication(&par_pRxMpdu->Mpdu.srcAddr.Extended,
pFrameCtrl->SecurityEnable, 0x00);
    }
    break;
case CMD_BEACON_REQUEST:      /* Only FFD, device will omit */
    beacon_transBeacon();
    break;
#endif // !ZIGBEE_DEVICE_FFD == TRUE
case CMD_GTS_REQUEST:
default:
    break;
}
break;
case MAC_FRM_DATA:
    /* Format indication data */
    plndiData = (MCPS_DATA_INDICATION*)mem_getMem(sizeof(MCPS_DATA_INDICATION));
    if (plndiData)
    {
        plndiData->srcAddrMode      = pFrameCtrl->SourceAddMode;
        plndiData->srcPanId         = par_pRxMpdu->Mpdu.srcPanId;
        plndiData->srcAddr          = par_pRxMpdu->Mpdu.srcAddr;
        plndiData->dstAddrMode      = pFrameCtrl->DestAddMode;
        plndiData->dstPanId         = par_pRxMpdu->Mpdu.dstPanId;
        plndiData->dstAddr          = par_pRxMpdu->Mpdu.dstAddr;
        plndiData->mpduLinkQuality = par_pRxMpdu->Mpdu.Fcs.LinkQuality;
        plndiData->securityUse      = FALSE;
        plndiData->aclEntry         = 0;
        plndiData->msduLength       = par_pRxMpdu->Mpdu.nLength;
        plndiData->pMsdu = mem_getMem(par_pRxMpdu->Mpdu.nLength);
        if (plndiData->pMsdu)

```

```

    {
        mem_cpy(pIndiData->pMsdu, par_pRxMpdu->Mpdu.PayLoad , pIndiData->msduLength);
        //call to DATA.indication
        mcpsDataIndication(pIndiData);
    }
    else
        mem_freeMem(pIndiData);
}
/*Omit packet if not enough memory*/
break;
case MAC_FRM_BEACON:
    if(macInfo.sState.nTransacType == MAC_TRANSAC_SCAN)
    {
        //Make notification to upper layer
        beacon_notifyBeacon(par_pRxMpdu);
    }
    else if (par_pRxMpdu->Mpdu.srcPanId == mpib.macPANId)
    {
        beacon_updateNewBeacon(par_pRxMpdu);          /* Notify the beacon */
    }

    break;
case MAC_FRM_ACK:
    break;
default:
    break;
}

//remove the packet
mem_releaseRxPkt(par_pRxMpdu);
} // End of functions

```

9.4.5 MAC TX

9.4.5.1 mac_tx.c

```

/*****
Module: MAC\mac_tx
File Name: mac_tx.c
Description: Handles functions for packet transmissions
Copyright (C) 2006 - All Rights Reserved
*****/

/*****
*
*
*
*
*****/

/* Include files
*****/

#include "mac_tx.h"
#include "csma-ca.h"
#include "mac_mlme.h"
#include "mac_primitives.h"

#include "mac_packet.h"
#include "mac_mlme.h"
#include "phy_primitives.h"

#include "phy_support.h"
#include "static_timer.h"

```

```

#include "schedul.h"
#include "mac_beacon.h"
/*****
 *      Constant declaration      */
*****/

/*****
 *      Type Definition          */
*****/

/*****
 *      Variable declaration      */
*****/
MAC_TX          MacTx;
/* Scan information
 */
/*****
 *      Static Function          */
*****/
/*@FunctionName: tx_transTask
 */
/*@Description:   This is task to trans packet   */
*****/
void tx_transTask(void*,void* par_pParam);

/*****
 *      Function                  */
*****/
/*@FunctionName: tx_send
 */
*****/
BOOL tx_send(P_MPDU_TX      par_pPacket);

/*****
 *      Function                  */
*****/
/*@FunctionName: tx_endTrans
 */
/*@Description:   Procedure to end trans. of a packet: cmd, data...   */
*****/
void tx_endTrans(P_MPDU_TX      par_pMpdu, MAC_STATUS      par_macStatus);

/*****
 *      Function                  */
*****/
/*@FunctionName: tx_timeoutAck
 */
*****/
void tx_timeoutAck(void*, void* par_pParam);

/*****
 *      Function                  */
*****/
/*@FunctionName: tx_init
 */
*****/
void tx_init()
{
    MacTx.eState      = MAC_TX_IDLE;
    MacTx.pPacket      = NULL;
}

/*****
 *      Function                  */
*****/

```

```

//@FunctionName: Tx_Trans
*/
//@Description:      This function will create an task to handle packet trans
//                  until successful or expired. Task created is queued in the
//                  task queue
/*****
BOOL tx_trans(P_MPDU_TX      par_pMpdu)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    par_pMpdu->bTransmitting = FALSE; /* This packet is not
current pkt*/

    return schedul_addTask(tx_transTask, TASK_PER_MAC_TX, (DWORD*)par_pMpdu); /* Add task to trans*/
} // End of functions

/*****
//@FunctionName:Tx_notifyAck
*/
//@Description:      This function will notify the Trans layer a ACK received
//                  Return will be TRUE if it is a valid ACK and current transTask
//                  will be stopped, else return false and ACK will be discarded
/*****
BOOL tx_notifyAck(P_MPDU_RX par_pRxMpdu)
{
    /***** Variable declaration *****/
    BOOL      bReturn;
    /***** Function body *****/
    bReturn = FALSE;

    if (MacTx.pPacket != NULL)
    {
#ifdef ZIGBEE_STUB
        par_pRxMpdu->Mpdu.SeqNum = MacTx.pPacket->Mpdu.SeqNum;
#endif
        if (MacTx.pPacket->Mpdu.SeqNum == par_pRxMpdu->Mpdu.SeqNum)
        {
            bReturn = TRUE;
            MacTx.eState = MAC_TX_ACKED;
            MacTx.bFrmPending = par_pRxMpdu->Mpdu.FrmControl.FramePending;
        }
    }
    return bReturn;
} // End of functions

/*****
//@FunctionName: Tx_send
//@Description:      This function will handle sending a packet using CSMA-CA
/*****
BOOL tx_send(P_MPDU_TX      par_pMpdu)
{
    /***** Variable declaration *****/
    BOOL      eTxState;
    BYTE      nTemp;
    /***** Function body *****/

```

```

eTxState = FALSE;

/* If using Csmaca */
if (MacTx.bUsedCsmaca)
{
    // Wait for the back-off slot boundary before transmitting
    if (MacTx.sCsmaca.bSlotted)
        WAIT_FOR_BOUNDARY();
    else
    {
#ifdef ZIGBEE_STUB
//          do {
//              FASTSPI_UPD_STATUS(nTemp);
//          } while (!(nTemp & CC2420_RSSI_VALID_BM));
#endif
    }

    pddata_request(&par_pMpdu->Mpdu);
    return TRUE;
} // End of functions

/***** Function Declaration *****/
// @FunctionName: tx_transTask
// @Description: This is task to trans packet
/***** Function Definition *****/
void tx_transTask(void* par_pParam, void* args)
{
    /***** Variable declaration *****/
    P_MPDU_TX      pMpdu;
    CSMACA_RESULT  eResult;
    BOOL           bSlotted;
    PHY_STATUS     sStatus;
    UINT8          nTemp;

    /***** Function body *****/

    /* Set up for initialize transmission */
    pMpdu=(P_MPDU_TX)par_pParam;

    if (pMpdu->bTransmitting == FALSE)
    {
        MacTx.eState          = MAC_TX_INIT;
        MacTx.bUsedCsmaca     = ( pMpdu->PktInfo.txMode & TX_MODE_CSMACA);
        MacTx.pPacket         = pMpdu;
        pMpdu->bTransmitting   = TRUE;
    }

    switch(MacTx.eState)
    {
    case MAC_TX_INIT:

        //set internal value of the MPDU
        MacTx.eState          = MAC_TX_PROCESSING;

        //if used csmaca, initialize it.
        if (MacTx.bUsedCsmaca)

```

```

    {
        //when using csma-ca transmission
        bSlotted = !((mpib.macBeaconOrder == 15) || (pMpdu->PktInfo.txMode &
TX_MODE_FORCE_UNSLOTTED)
        || (BeaconInfo.bBeaconLost == TRUE));
        csmaca_initCsmaca(&MacTx.sCsmaca, &pMpdu->PktInfo,bSlotted);

        //next state
        MacTx.eState = MAC_TX_PROCESSING;
    }
    else
    {
        //set the initial state for Trans-layer
        MacTx.eState = MAC_TX_TRANS;
    }

    break;
case MAC_TX_PROCESSING:
    //run CSMA-CA procedure
    csmaca_process(&MacTx.sCsmaca);
    eResult = MacTx.sCsmaca.eCurrentResult;
    if (eResult == CSMACA_RESULT_SUCCESS)
    {
        MacTx.eState = MAC_TX_TRANS;
    }
    else if ((eResult == CSMACA_RESULT_FAILURE)
    || (eResult == CSMACA_RESULT_SHORT_TIME))
    {
        //failed, end the transmission
        tx_endTrans(pMpdu, CHANNEL_ACCESS_FAILURE);
        break;
    }
    else if (MacTx.sCsmaca.bWaitBeaconForTx == TRUE)
    {
        schedul_suspend(TASK_PER_MAC_TX); /*Suspend the task*/
        break;
    }
    else
        //no result now, try again
        break;

case MAC_TX_TRANS:
    //start TX
    sStatus = mac_setTrxState(PHY_TX_ON);
    if (sStatus == PHY_SUCCESS)
    {
        if (! tx_send(pMpdu))
        {
            //try again, do it later
        }
        else
        {
            //set timer to check ACK
            if(pMpdu->Mpdu.FrmControl.ACKRequest)
            {
                /* Auto start RX to receive the ACK*/
                phyrx_initOnRx();
            }
        }
    }

```

```

        MacTx.eState = MAC_TX_WAIT_ACK;
        timer_startTimer(timeoutAck, (void*)pMpdu,
                          (UINT32)(pMpdu->PktInfo.nDuration+2));
    }
    else
    {
        /*End of transmission*/
        MacTx.eState = MAC_TX_TRANSMITTING;
    }
}
}
break;
case MAC_TX_TRANSMITTING:
    FASTSPI_UPD_STATUS(nTemp);
    if (!(nTemp & CC2420_TX_ACTIVE_BM))
    {
        tx_endTrans(pMpdu, SUCCESS);
    }
    break;
case MAC_TX_WAIT_ACK:
    /*Waiting for ACK*/
    break;
case MAC_TX_ACKED:

    //remove the timer
    if (MacTx.pPacket->Mpdu.FrmControl.ACKRequest)
        timer_endTimer(timeoutAck);

    //end of transmission successfully
    tx_endTrans(MacTx.pPacket, SUCCESS);

    break;
case MAC_TX_NO_ACK:
    //no retry left, ACK not received yet
    tx_endTrans(pMpdu, NO_ACK);

    break;
default:
    break;
}

} // End of functions

/*****
//@FunctionName: tx_timeoutAck
*****/
void tx_timeoutAck(void* args, void* par_pParam)
{
    /***** Variable declaration *****/
    P_MPDU_TX    pMpdu;

    /***** Function body *****/
    pMpdu = (P_MPDU_TX)par_pParam;

    //not received ACK yet, decrease nRetryLeft
    pMpdu->PktInfo.nRetryLeft--;

```



```

//and retry
if (pMpdu->PktInfo.nRetryLeft > 0)
{
    MacTx.eState = MAC_TX_INIT;
}
else
{
    MacTx.eState = MAC_TX_NO_ACK;
}

} // End of functions

/*****
**@FunctionName: tx_endTrans
**@Description: Procedure to end trans. of a packet: cmd, data...
*****/
void tx_endTrans(P_MPDU_TX par_pMpdu, MAC_STATUS par_macStatus)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    mac_manageTrxAfterTx(par_pMpdu, par_macStatus);

    /*Indirect packet*/
    #if ZIGBEE_DEVICE_FFD == TRUE
    if (par_pMpdu->PktInfo.txOptions & TX_OPT_INDIRECT)
    {
        if (par_macStatus == SUCCESS)
        {
            indirectTx_removePkt(par_pMpdu);
        }
    }
    #endif
    if (!(par_pMpdu->PktInfo.txOptions & TX_OPT_INDIRECT))
    /* Call to appropriate transaction */
    switch (par_pMpdu->Mpdu.FrmControl.FrameType)
    {
        case MAC_FRM_CMD:
            switch (macInfo.sState.nTransacType)
            {
                case MAC_TRANSAC_ASSOCIATION:
                    mlme_notifyAssociation(par_macStatus, (void*)par_pMpdu);
                    break;
                case MAC_TRANSAC_DISASSOCIATION:
                    mlme_handleDisassoTransition(par_macStatus, (void*)par_pMpdu);
                    break;
                case MAC_TRANSAC_POLL:
                    mlme_handlePollTransition(par_macStatus, par_pMpdu);
                    break;
                case MAC_TRANSAC_SCAN:
                    if (par_macStatus != SUCCESS) /* Failure to transmit on the channel, finished the channel */
                    {
                        macInfo.sState.nTransacState = SCAN_FINISHED_CHANNEL; /*Finish scan current
channel*/
                        break;
                    }
            }
    }

```

```

    }
    switch(pScanInfo->scanType)      /* If success in transmitting */
    {
    case ED_SCAN:
        break;
    case ACTIVE_SCAN:
        /* Set timer to check the timeout */
        timer_startTimer(scanTimeout,NULL,
            ((UINT32) aBaseSuperframeDuration / (UINT32) aUnitBackoffPeriod) * (((UINT32) 1 <<
pScanInfo->scanDuration) + 2));
        mem_releaseTxPkt(MacTx.pPacket);
        break;
    case PASSIVE_SCAN:
        break;
    case ORPHAN_SCAN:
        timer_startTimer(scanTimeout,NULL,                                /* Set timer to check
the timeout */
            (UINT16) aResponseWaitTime / aUnitBackoffPeriod);
        break;
    }
    break;
case MAC_TRANSAC_NO:
    mem_releaseTxPkt(MacTx.pPacket);
    break;
default:
    mem_releaseTxPkt(MacTx.pPacket);
    break;
}
break;
case MAC_FRM_DATA:
    if (!(MacTx.pPacket->PktInfo.txMode&TX_MODE_MAC_INTERNAL))
        mcpsDataConfirm(MacTx.pPacket->PktInfo.msduHandle,par_macStatus);
    mem_releaseTxPkt(MacTx.pPacket);
    break;
case MAC_FRM_BEACON:
case MAC_FRM_ACK:
    break;
default:
    break;
}

/*Finished trans*/
MacTx.eState = MAC_TX_IDLE;

//remove this task,
schedul_removeTask(TASK_PER_MAC_TX);
} // End of functions

/*****
//@FunctionName:tx_transBeacon
*****/
void tx_transBeacon(P_MPDU_TX par_pBeacon)
{
    phytx_turnOnTx();
    pddata_request(&par_pBeacon->Mpdu);
    mac_setTrxState(PHY_RX_ON);
} // End of functions

```

```

/*****/
//@FunctionName: tx_resumeAfterBeaconTxTask
/*****/
void tx_resumeAfterBeaconTxTask(void* arg, void* arg1)
{
    tx_resumeAfterBeacon(TRUE);
    schedul_removeTask(TASK_PER_MAC_PERIODICAL);
} // End of functions
void tx_resumeAfterTxBeacon()
{
    if (macInfo.DeviceType == ZIGBEE_PAN_COOR)
        SlotInfo.bosCounter = BeaconInfo.nBeaconDuration + 2;
    schedul_addTask(tx_resumeAfterBeaconTxTask, TASK_PER_MAC_PERIODICAL, NULL);
} // End of functions

/*****/
//@FunctionName: tx_resumeAfterBeacon
/*****/
UINT32 capRequired;
UINT32 capLeft;
UINT32 nCap;
void tx_resumeAfterBeacon(BOOL bTransBeacon)
{
    if (MacTx.sCsmaca.bWaitBeaconForTx == TRUE)
    {
        if (bTransBeacon) /* Beacon transmitted */
        {
            if (!MacTx.pPacket->PktInfo.bToCoord) /* To child or others */
            {
                nCap = (((aBaseSlotDuration / aUnitBackoffPeriod) * (UINT32) (BeaconInfo.nFinalCapSlot
+ (UINT32) 1)) << (UINT32) mpib.macSuperframeOrder);
                /* <- The CAP duration > <----- Used CAP -----> <-- Beacon
margin --> */
                capLeft = nCap - (BeaconInfo.nBeaconDuration + 1) -
beacon_calBeaconMargin();
            }
            else
                capLeft = 0;
        }
        else /* Beacon received */
        {
            /* Cancel location timeout */
            timer_endTimer(locateBeaconTimeout);

            if (MacTx.pPacket->PktInfo.bToCoord) /* To parent */
            {
                nCap = (((aBaseSlotDuration / aUnitBackoffPeriod) * (UINT32) (BeaconInfo.nFinalCapSlot
+ (UINT32) 1)) << (UINT32) mpib.macSuperframeOrder);
                // <- The CAP duration > <----- Used CAP -----> <-- Beacon margin --
>
                capLeft = nCap - (SlotInfo.bosCounter + 1) -
beacon_calBeaconMargin();
            }
            else
                capLeft = 0;
        }
    }
}

```

```

    }

    /* Calculate the CAP required (number of backoff slots) */
    if (BeaconInfo.bBatteryLifeExt) {
        /*Need to be corrected*/
        //          <----- The CAP duration ----->    <----- Used CAP ----->
        capLeft          = (BeaconInfo.nBeaconDuration + mpib.macBattLifeExtPeriods) -
(SlotInfo.bosCounter + 1);
        capRequired = MacTx.sCsmaca.nRandomBackoff + 2 + 1 + MacTx.sCsmaca.bSlotted;
    }
    else
        capRequired = MacTx.sCsmaca.nRandomBackoff + 2 +
            MacTx.pPacket->PktInfo.nDuration;

    /* Enough time left in the CAP */
    if (capLeft >= capRequired)
    {
        schedul_activate(TASK_PER_MAC_TX);
        MacTx.sCsmaca.bWaitBeaconForTx = FALSE;
    }
    else
    {
        /*Adjust the time remained*/
        if (capLeft > 0)
        {
            MacTx.sCsmaca.nRandomBackoff = MAX(0, MacTx.sCsmaca.nRandomBackoff - capLeft);
        }
    }
}
} // End of functions

```

9.4.5.2 csma-ca.c

```

/*****
Module: MAC\mac_tx
File Name: csma-ca.c
Description: Implements csma-ca mechanism
*/

//-----*/
//Revision History:
//Date          Programmer          Details
*/
//01/08/2007
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
*          Include files
*/
*****/

#include "csma-ca.h"
#include "mac_primitives.h"
#include "phy_primitives.h"
#include "schedul.h"
#include "static_timer.h"

```

```

#include "phy_support.h"
#include "mac_beacon.h"
#include "mac_tx.h"
/*****
/*                                     Constant declaration
*/
/*****
#define MAC_CSMA_CW          1

/*****
/*                                     Type Definition
/*
/*****

/*****
/*                                     Variable declaration
/*
/*****

/*****
/*                                     Static Function
/*                                     */
/*****
/*@FunctionName:  csmaca_locateBeaconTimeout
/*@Description:
//
/*@Param
//
/*****
void    csmaca_locateBeaconTimeout(void* args, void*    par_pParam);

/*****
/*@FunctionName: csmaca_setupBackOffs
/*
/*@Description:
//
/*@Param
//
/*****
BOOL csmaca_setupBackOffs(CSMACA_INFO*    par_csmaca);

/*****
/*@FunctionName: csmaca_restartFromDelay
/*
/*@Description:
//
/*@Param
//
/*****
void csmaca_restartFromDelay(void* args1, void* args2)
{
    //restart the TX task
    schedul_activate(TASK_PER_MAC_TX);
}

```

```

/*****/
//@FunctionName: csmaca_setupBackoffCallback
*/
//@Description: */
// */
//@Param */
// */
/*****/
BOOL csmaca_setupBackoffCallback(CSMACA_INFO* par_pCsmaca)
{
    timer_startTimer( restartFromDelay, NULL,
                      (UINT32)((par_pCsmaca->nRandomBackoff==0)?1:par_pCsmaca->nRandomBackoff));
    //suspend, and restart next time by timers
    schedul_suspend(TASK_PER_MAC_TX);
    return TRUE;
}

/*****/
/*          Function          */
/*****/

/*****/
//@FunctionName: csmaca_initCsmaca
*/
//@Description: */
// */
//@Param */
// */
/*****/
void csmaca_initCsmaca(CSMACA_INFO* par_pCsmaca, PKT_INFO* par_pPktInfo, BOOL par_bSlotted)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    //reset internal value of csma-ca
    par_pCsmaca->nb = 0;
    par_pCsmaca->cw = MAC_CSMA_CW;
    par_pCsmaca->be = mpib.macMinBE;
    par_pCsmaca->bSlotted = par_bSlotted;
    par_pCsmaca->pPktInfo = par_pPktInfo;
    par_pCsmaca->bWaitBeaconForTx = FALSE;
    par_pCsmaca->nRandomBackoff = 0xFF;

    if (par_bSlotted && mpib.macBattLifeExt) {
        par_pCsmaca->be = MIN(2, par_pCsmaca->be);
    }

    par_pCsmaca->eState = CSMACA_STATE_INIT;
    par_pCsmaca->eCurrentResult = CSMACA_RESULT_NO_ERR;
} // End of functions

/*****/
//@FunctionName:csmaca_process
*/

```

```

//@Description:      Main task to perform csma-ca procedure          */
//
//@Param              */
//
/*****
void csmaca_process(CSMACA_INFO* par_pcsmaca)
{
    /***** Variable declaration *****/
    BOOL          bEnoughTime;
    /***** Function body *****/
    switch(par_pcsmaca->eState)
    {
    case CSMACA_STATE_UNDEFINED:
        //do nothing here
        break;
    case CSMACA_STATE_INIT:

        if (par_pcsmaca->nRandomBackoff == 0xFF) /*Initialization*/
            //locate back-off period
            bEnoughTime = csmaca_setupBackOffs(par_pcsmaca);

        else
            bEnoughTime = TRUE;                /*Recovered from delay*/

        /* Check time to transmit in this CAP */
        if (bEnoughTime == TRUE)
        {
            //Starting of delay period
            csmaca_setupBackoffCallback(par_pcsmaca);
            par_pcsmaca->eCurrentResult = CSMACA_RESULT_NO_ERR;
            par_pcsmaca->eState          = CSMACA_STATE_RANDOM_DELAY;
        }
        else
        {
            // CSMA-CA should not be resumed in the next super-frame for indirect packets!
            if (par_pcsmaca->pPktInfo->txOptions & TX_OPT_INDIRECT)
            {
                par_pcsmaca->eCurrentResult = CSMACA_RESULT_SHORT_TIME;
                return;
            }
            else
            {
                //Request to locate beacon if beacon is not tracked
                if (! BeaconInfo.bTrackBeaconReq //beacon is not locating
                    && ! BeaconInfo.bTrackBeacon ) //or not tracked beacon
                    mac_setTrxState(PHY_RX_ON);

                if (par_pcsmaca->pPktInfo->bToCoord) /*Locate beacon only if to coord.*/
                    /* Start timer to wait */
                    timer_startTimer(locateBeaconTimeout,(void*)par_pcsmaca,
                                     ((UINT32) (aBaseSuperframeDuration / aUnitBackoffPeriod) <<
                                     (UINT32) mpib.macBeaconOrder));

                par_pcsmaca->bWaitBeaconForTx = TRUE;
                par_pcsmaca->eCurrentResult = CSMACA_RESULT_NO_ERR;
                par_pcsmaca->eState          = CSMACA_STATE_INIT;
            }
        }
    }
}

```

```

    break;
case CSMACA_STATE_RANDOM_DELAY:
    //after the random delay of 2expBE-1, performing the CCA
case CSMACA_STATE_TURNON_RX:
    // Make sure that the FIFO interrupt is off during the CSMA-CA procedure
    // and the timing-critical TX FIFO access
    phy_disableFIFOInt(TRUE);

    // Turn on RX to be able to do the CCA check in the CSMA-CA algorithm
    mac_setTrxState(PHY_RX_ON);

    par_pcsmaca->eState          = CSMACA_STATE_PERFORM_CCA;
    par_pcsmaca->eCurrentResult = CSMACA_RESULT_NO_ERR;

    break;
case CSMACA_STATE_PERFORM_CCA: //perform CCA
    if (par_pcsmaca->bSlotted == TRUE) //slotted
    {
        // CCA check on the back-off slot boundary
        WAIT_FOR_BOUNDARY();
        if (plmeCCA_request() == PHY_IDLE)
        {
            par_pcsmaca->cw--;
            if(par_pcsmaca->cw == 0)
                par_pcsmaca->eCurrentResult = CSMACA_RESULT_SUCCESS;
        }
        else
        {
            par_pcsmaca->nb++;
            par_pcsmaca->cw = MAC_CSMA_CW;
            par_pcsmaca->be = MIN(par_pcsmaca->be + 1, aMaxBE);

            if (par_pcsmaca->nb > mpib.macMaxCSMABackoffs)
                par_pcsmaca->eCurrentResult = CSMACA_RESULT_FAILURE; /*channel access
failure*/
            else
            {
                /* Try again from beginning*/
                par_pcsmaca->eState          = CSMACA_STATE_INIT;
                par_pcsmaca->nRandomBackoff = 0xFF;
                par_pcsmaca->eCurrentResult = CSMACA_RESULT_NO_ERR;
            }
        }
    }
    //un-slotted, check CCA immediately
    else
    {
        if (plmeCCA_request() == PHY_IDLE)
            par_pcsmaca->eCurrentResult = CSMACA_RESULT_SUCCESS;
        else
        {
            par_pcsmaca->nb++;
            par_pcsmaca->be = MIN(par_pcsmaca->be + 1, aMaxBE);

            if (par_pcsmaca->nb > mpib.macMaxCSMABackoffs)
                par_pcsmaca->eCurrentResult = CSMACA_RESULT_FAILURE; /*channel access

```



```

failure*/
    else
    {
        /* Try again from beginning*/
        par_pcsmaca->eState = CSMACA_STATE_INIT;
        par_pcsmaca->nRandomBackoff = 0xFF;
        par_pcsmaca->eCurrentResult = CSMACA_RESULT_NO_ERR;
    }
}

mac_setTrxState(PHY_TRX_OFF);
//End of CCA, enable the FIFO interrupt
phy_disableFIFOInt(FALSE);

break;
default:
break;
}

} // End of function

/*****
//@FunctionName: csmaca_setupBackOffs
*/
//@Description:
//
//@Param
//
*****/
BOOL csmaca_setupBackOffs(CSMACA_INFO* par_csmaca)
{
    /****** Variable declaration *****/
    BOOL bReturn;
    UINT32 nCap;
    INT32 bosLeft;
    UINT8 randomBackoff;
    /****** Function body *****/

    // Check for validity of indirect packets
    //TBD

    // Calculate the random back-off period
    randomBackoff = random_getRandomByte() & (BM(par_csmaca->be) - 1);

    // Calculate the number of FULL back-off slots left in the CAP (including the two/three periods required to
    prepare for transmission
    if (mpib.macSuperframeOrder == 15 || !par_csmaca->bSlotted ) {
        bosLeft = 0;
    } else if (BeaconInfo.bBatteryLifeExt && par_csmaca->pPktInfo->bToCoord) {
        // <----- The CAP duration -----> <----- Used CAP -----> < Delay +
        overhead + duration >
        bosLeft = (BeaconInfo.nBeaconDuration+ mpib.macBattLifeExtPeriods) - (SlotInfo.bosCounter + 1) -
        (randomBackoff + 2 + 1);
    } else
    {

```

```

    nCap      = (((aBaseSlotDuration / aUnitBackoffPeriod) * (UINT32) (BeaconInfo.nFinalCapSlot +
(UINT32) 1)) << (UINT32) mpib.macSuperframeOrder);
    if (MacTx.pPacket->PktInfo.bToCoord) /*To parent*/
    {
        //      <- The CAP duration > <----- Used CAP ----->      <----- Delay + overhead + duration
----->      <- Beacon margin -->
        bosLeft = nCap - (SlotInfo.bosCounter + 1) - (randomBackoff + 2 +
par_csmaca->pPktInfo->nDuration) - beacon_calBeaconMargin();
    }
    else /*To child or other*/
    {
        /*      <- The CAP duration >      <----- Used CAP ----->      <----- Delay + overhead
+ duration ----->      <- Beacon margin --> */
        bosLeft = nCap - (BeaconInfo.nBeaconDuration+ 1) - (randomBackoff + 2
+ par_csmaca->pPktInfo->nDuration) - beacon_calBeaconMargin();
    }
}

// Subtract an extra back-off period if slotted CSMA-CA is used (2 CCA checks instead of 1)
if (par_csmaca->bSlotted) bosLeft--;

// Default values...
par_csmaca->bWaitBeaconForTx = FALSE;

if (bosLeft >= 0)
    bReturn = TRUE; /*Enough time to transmit*/
else
    bReturn = FALSE;

par_csmaca->nRandomBackoff = randomBackoff;

return bReturn;
} // csmaSetStartupTime

/*****
//@FunctionName:  csmaca_locateBeaconTimeout
*/
//@Description:
//
//@Param
//
*****/
void  csmaca_locateBeaconTimeout(void* args,void*  par_pParam)
{
    /***** Variable declaration *****/
    CSMACA_INFO*  pCsmaca;
    /***** Function body *****/
    pCsmaca = (CSMACA_INFO*)par_pParam;

    if(BeaconInfo.bTrackBeaconReq)
    {
        //Locate beacon.
        timer_startTimer(locateBeaconTimeout,(void*)pCsmaca,
((UINT32) (aBaseSuperframeDuration / aUnitBackoffPeriod) << (UINT32)
mpib.macBeaconOrder));
    }
}

```

```

else
{
    //the beacon is lost
    pCsmaca->bWaitBeaconForTx = FALSE;
    schedul_activate(TASK_PER_MAC_TX);
    //switch back to un-slotted Csmaca
    pCsmaca->bSlotted = FALSE;
    pCsmaca->nb = 0;
    pCsmaca->be = mpib.macMinBE;
    pCsmaca->bSlotted = FALSE;
    BeaconInfo.bBeaconLost = TRUE;
}

} // End of functions
  
```

9.4.5.3 indirect_tx..c

```

/*****
Module: MAC\mac_tx
File Name: indirect_tx.c
Description: This component implement the indirect transmission from coor.
*****/

/*****
* Include files
*****/

#include "indirect_tx.h"
#include "linklist.h"
#include "mac_beacon.h"

#if ZIGBEE_DEVICE_FFD == TRUE
/*****
* Constant declaration
*****/

/*****
* Type Definition
*****/
typedef struct
{
    UINT8      TTL;
    P_MPDU_TX  pPacket;
    UINT8      NextIndex; /*Index of next packet in the list*/
}INDIRECT_PKT;

/* Pool of indirect packet */
INDIRECT_PKT IndirectPkts[MAC_TX_POOL_SIZE];
UINT8 FirstPkt;
UINT8 LastPkt;

/*****
* Variable declaration
*****/

/*****
* Static Function
*****/
  
```

```

//@FunctionName: indirectTx_releasePkt
//@Description: This function will be called to release a pkt in the indirect
//              packet list:
//              - Data packet: inform the nwk layer
//              - CMD packet: release without informing */
/*****
void indirectTx_releasePkt(P_MPDU_TX par_pPacket, MAC_STATUS Status)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    if (par_pPacket == NULL)
        return;

    if (par_pPacket->Mpdu.FrmControl.FrameType == MAC_FRM_DATA)
    {
        /*Call to upper*/
        mcpsDataConfirm(par_pPacket->PktInfo.msduHandle, Status);
    }

    /*remove packet*/
    mem_releaseTxPkt(par_pPacket);
} // End of functions
/*****
/*              Function */
/*****
//@FunctionName: indirectTx_init
/*****
void indirectTx_init()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    for (UINT8 index = 0; index < MAC_TX_POOL_SIZE; index++)
    {
        IndirectPkts[index].pPacket      = NULL;
        IndirectPkts[index].NextIndex    = MAC_TX_POOL_SIZE;
        FirstPkt                         = MAC_TX_POOL_SIZE;
        LastPkt                          = MAC_TX_POOL_SIZE;
    }
} // End of functions

/*****
//@FunctionName: This function will add packet into the indirect packet queue */
//@Description: */
// */
//@Param      TRUE: If packet is added into the queue
//            FALSE: If packet is not added into the queue */
// */
/*****
BOOL indirectTx_addPkt(P_MPDU_TX par_pPacket)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /*Search for free slot */

```

```

DISABLE_GLOBAL_INT();
for (UINT8 index = 0; index < MAC_TX_POOL_SIZE; index++)
{
    if (IndirectPkts[index].pPacket == NULL)
    {
        IndirectPkts[index].pPacket          = par_pPacket;
        IndirectPkts[index].TTL              = mpib.macTransactionPersistenceTime;
        par_pPacket->PktInfo.IndirectPktIndex = index;

        /*Update next index*/
        IndirectPkts[LastPkt].NextIndex = index;
        LastPkt = index;
        /*First packet*/
        if (FirstPkt == MAC_TX_POOL_SIZE)
            FirstPkt = index;

        ENABLE_GLOBAL_INT();
        return TRUE;
    }
}
ENABLE_GLOBAL_INT();
return FALSE;
} // End of functions

/*****
//FunctionName: indirectTx_controllIndirectPkt
//@Description:   This function will control the indirect packet
/*
    - Update the TTL
    - Remove the expired pkt
    - Update the pending addr to the beacon
*/
*****/
void indirectTx_controllIndirectPkt(void* arg1, void* arg2)
{
    /***** Variable declaration *****/
    PENDING_ADDR_SPEC* pPendingAddrSpec;
    UINT8 nNumExtAddr;
    UINT8 Index;
    UINT8 PrevIndex;

    /***** Function body *****/
    nNumExtAddr = 0;
    BeaconInfo.nPendingAddrLen = sizeof(PENDING_ADDR_SPEC);
    DISABLE_GLOBAL_INT();
    Index = FirstPkt;
    while (Index != MAC_TX_POOL_SIZE && IndirectPkts[Index].pPacket) /* is the valid packet*/
    {
        if (!IndirectPkts[Index].TTL--)
        {
            indirectTx_releasePkt(IndirectPkts[Index].pPacket, TRANSACTION_EXPIRED);
            IndirectPkts[Index].pPacket = NULL;

            /*If we removed the first packet*/
            if (Index == FirstPkt)
                FirstPkt = IndirectPkts[Index].NextIndex;

            /*If not the first packet*/

```

```

else
    IndirectPkts[PrevIndex].NextIndex = IndirectPkts[Index].NextIndex;

    /*Last packet is removed */
    if (Index == LastPkt)
        LastPkt = PrevIndex;
}
else
{
    /* Omit nwk addr */
    if (IndirectPkts[Index].pPacket->Mpdu.FrmControl.DestAddrMode == AM_EXTENDED_64)
    {
        *(QWORD*)(BeaconInfo.pBeacon->Mpdu.Payload+                2+                1+
sizeof(PENDING_ADDR_SPEC)+
        nNumExtAddr*sizeof(QWORD)) = IndirectPkts[Index].pPacket-
>Mpdu.dstAddr.Extended;
        nNumExtAddr++;
    }
    Index = IndirectPkts[Index].NextIndex;
    if (FirstPkt == Index)
        PrevIndex = MAC_TX_POOL_SIZE;
    else
        PrevIndex = Index;
}
/*implemented ext add only, please do if need to complete with ex-add*/
pPendingAddrSpec = (PENDING_ADDR_SPEC*)(BeaconInfo.pBeacon-
>Mpdu.Payload+2+1);
pPendingAddrSpec->NumofExtAddr = nNumExtAddr;
pPendingAddrSpec->NumofShortAddr = 0;
BeaconInfo.nPendingAddrLen += nNumExtAddr*sizeof(QWORD);
ENABLE_GLOBAL_INT();
} // End of functions

/***** Variable declaration *****/
//@FunctionName: indirectTx_getPkt
/***** Function body *****/
P_MPDU_TX indirectTx_getPkt(ADDRESS *par_pDestAddr, UINT8 par_Addrmode)
{
    /****** Variable declaration *****/
    UINT8 Index;
    /****** Function body *****/
    DISABLE_GLOBAL_INT();
    Index = FirstPkt;
    while (Index != MAC_TX_POOL_SIZE && IndirectPkts[Index].pPacket) /* is the valid packet*/
    {
        if ((par_pDestAddr->Extended == IndirectPkts[Index].pPacket->Mpdu.dstAddr.Extended &&
par_Addrmode == AM_EXTENDED_64) ||
(par_pDestAddr->Short == IndirectPkts[Index].pPacket->Mpdu.dstAddr.Short && par_Addrmode ==
AM_SHORT_16))
        {
            ENABLE_GLOBAL_INT();
            return IndirectPkts[Index].pPacket;
        }
        Index = IndirectPkts[Index].NextIndex;
    }
    ENABLE_GLOBAL_INT();
}

```

```

    return NULL;
} // End of functions

/*****
//@FunctionName: indirectTx_removePkt
*****/
void indirectTx_removePkt(P_MPDU_TX par_pMpdTx)
{
    /***** Variable declaration *****/
    UINT8      Index;
    UINT8      removedIndex;
    /***** Function body *****/
    DISABLE_GLOBAL_INT();
    Index      = FirstPkt;
    removedIndex = par_pMpdTx->PktInfo.IndirectPktIndex;
    /*First packet is removed */
    if (removedIndex == FirstPkt)
    {
        FirstPkt = IndirectPkts[FirstPkt].NextIndex;
        indirectTx_releasePkt(par_pMpdTx, SUCCESS);
        IndirectPkts[removedIndex].pPacket = NULL;
        IndirectPkts[removedIndex].NextIndex = MAC_TX_POOL_SIZE;

        /*Last packet is removed */
        if (removedIndex == LastPkt)
            LastPkt = MAC_TX_POOL_SIZE;
    }
    else
    {
        while (Index != MAC_TX_POOL_SIZE && IndirectPkts[Index].pPacket) /* is the valid packet*/
        {
            if (IndirectPkts[Index].NextIndex == removedIndex)
                break;
            Index = IndirectPkts[Index].NextIndex;
        }

        if (removedIndex == IndirectPkts[Index].NextIndex)
        {
            IndirectPkts[Index].NextIndex = IndirectPkts[removedIndex].NextIndex;

            indirectTx_releasePkt(par_pMpdTx, SUCCESS);
            IndirectPkts[removedIndex].pPacket = NULL;
            IndirectPkts[removedIndex].NextIndex = MAC_TX_POOL_SIZE;

            /*Last packet is removed */
            if (removedIndex == LastPkt)
                LastPkt = Index;
        }
    }
    ENABLE_GLOBAL_INT();
    return;
} // End of functions

/*****
//@FunctionName: indirectTx_RespondDataReq
*****/

```

```

void indirectTx_RespondDataReq(P_MPDU_RX par_pMpduTx)
{
    /***** Variable declaration *****/
    P_MPDU_TX      pTxMpdu;
    ADDRESS*       pCoorAddr;
    UINT8          coordAddrMode;
    /***** Function body *****/
    pTxMpdu = indirectTx_getPkt(&par_pMpduTx->Mpdu.srcAddr, par_pMpduTx->Mpdu.FrmControl.SourceAddMode);
    if (pTxMpdu)
    {
        tx_trans((P_MPDU_TX)pTxMpdu);
        return;
    }
    else
    {
        /* Prepare the frame request */
        pTxMpdu = mem_getFreeTxPkt();
        if (!pTxMpdu)
        {
            return;
        }
        mem_set(&pTxMpdu->Mpdu.FrmControl, 2);

        //Set other information
        pTxMpdu->PktInfo.txMode = TX_MODE_CSMACA|TX_MODE_MAC_INTERNAL;
        pTxMpdu->PktInfo.nRetryLeft = 1;

        mac_formatMpdu(pTxMpdu,
            MAC_FRM_DATA,
            AM_SHORT_16,
            mpib.macPANId,
            (ADDRESS *) &mpib.macShortAddress,
            par_pMpduTx->Mpdu.FrmControl.SourceAddMode,
            par_pMpduTx->Mpdu.srcPanId,
            (ADDRESS *) &par_pMpduTx->Mpdu.srcAddr,
            TX_OPT_ACK_REQ,
            NULL,
            0);

        if(!tx_trans(pTxMpdu)) /* Transmission */
            mem_releaseTxPkt(pTxMpdu);
    }
} // End of functions

#endif //! ZIGBEE_DEVICE_FFD == anTRUE

```

9.4.6 PHY Primitive

9.4.6.1 trx_interrupt.c

```

/*****
//Module: MAC/phy_primitives
//File Name: trx_interrupt.c
//Description: This file handles interrupts fom packet transmission
*****/

```



```

/*****/
/*                                     Include files
*/
/*****/
#include "mac_mem.h"
#include "phy_primitives.h"
#include "phy_support.h"
#include "mac_beacon.h"
#include "zconf.h"
#include "ucos_ii.h"
#include "mac_packet.h"
/*****/
/*                                     Constant declaration
*/
/*****/

/*****/
/*                                     Type Definition
*/
/*****/

/*****/
/*                                     Variable declaration
*/
/*****/
#define          RX_STACK_SIZE          0
#define          TMR_STACK_SIZE        0
#define          MAX_QUEUEING_RX_PKT    3
/*****/
/*                                     Variable declaration
*/
/*****/

volatile UINT8    TmrIntNesting;
volatile UINT8    RxIntNesting;
/*****/
/*                                     Local Function
*/
/*****/

/*@FunctionName: phyrx_receiveUnsecured
*/
/*@Description:    This function will receive the packet in un-secured mode
//
//
//
//
//
//
*/
void phyrx_receiveUnsecured();

/*****/
/*@FunctionName: phyrx_receivePayload
*/
/*@Description:
//
//
//
//
//
//
*/

```

```

//@Param
//
/*****
BOOL phyrx_receivePayload();

/*****
/*
Function
/*****

/*****
//@FunctionName: SIGNAL
*/

//@Description: Timer 1 interrupt to process queued timer
//
//@Param
//
/*****
SIGNAL(SIG_OUTPUT_COMPARE1A)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    //Disable global interrupt
    DISABLE_T1_COMPA_INT();

    TmrIntNesting++;

    /* Activate timer and periodical task */
    OSTmr_Task(NULL);
    schedul_runPerTasks();

    /* Other execution related to MAC layer */
    SlotInfo.bosCounter++; /* update the BOS counter */

    TmrIntNesting--;

    //End of interrupt disability
    ENABLE_T1_COMPA_INT();
} // End of functions

/*****
//@FunctionName: SIGNAL(SIG_INPUT_CAPTURE1)
//@Description: This interrupt is triggered when the CC2420 SFD pin goes high
//
//
//
//@Param
//Reused from C2420
/*****
SIGNAL(SIG_INPUT_CAPTURE1) {
    SlotInfo.captureTcnt = ICR1;
    SlotInfo.captureBosCounter = SlotInfo.bosCounter;
    if (TCNT1 < SlotInfo.captureTcnt) {
        SlotInfo.captureBosCounter++;
    }
    SlotInfo.captureTime = SlotInfo.bosCounterAdjustTime + (

```

```
SlotInfo.captureBosCounter * MAC_SYMBOL_DURATION * aUnitBackoffPeriod) +
(UINT32) SlotInfo.captureTcnt / CLOCK_SPEED_MHZ;
```

```
#ifdef ZIGBEE_DEBUG
```

```
#endif
```

```
} // SIGNAL(SIG_INPUT_CAPTURE1)
```

```

/*****
//@FunctionName: SIGNAL(SIG_OUTPUT_COMPARE1C)
//@Description:   Used by the FIFOP interrupt to make sure that is enabled a
//               short while after it has returned.This allows for timer ticks
//               in between, which has a lower priority than the FIFOP interrupt.
//@Param
//Reused from C2420
*****/

```

```
SIGNAL(SIG_OUTPUT_COMPARE1C) {
} // SIGNAL(SIG_OUTPUT_COMPARE1C)
```

```

/*****
//@FunctionName: SIGNAL(SIG_INTERRUPT0)
//@Description:   This interrupt will step by step build Physic layer SDU a
//               and necessary information. When finish, it will call
//               PD_DATA.Indication() to dispatch the packet to MAC layer
//@Param
*****/

```

```
SIGNAL(SIG_INTERRUPT0)
{
```

```
    /***** Variable declaration *****/
```

```
    /***** Function body *****/
```

```
    // Turn off the main MAC interrupts
    DISABLE_T1_COMPA_INT();
    DISABLE_FIFOP_INT();
```

```
#ifdef ZIGBEE_DEBUG
```

```
#endif
```

```

/* just implemented unsecured reception*/
while ((PhyInfo.nQueueingPkt < MAX_QUEUEING_RX_PKT)
      && FIFOP_IS_ACTIVE()
      &&PhyInfo.PhyState == PHY_RX_ON)
{
    phyrx_receiveUnsecured(&PhyInfo);
}

```

```

/* flush the Rx when
- Invalid state
- Too many packet
- Overflow */

```

```

    if((PhyInfo.PhyState != PHY_RX_ON)
        || (PhyInfo.nQueueingPkt >= MAX_QUEUEING_RX_PKT)
        || (FIFOP_IS_ACTIVE()&&!FIFO_IS_ACTIVE()))
    {
        phyrx_ResetRx();
    }

#ifdef ZIGBEE_DEBUG

#endif

    ENABLE_FIFOP_INT();

    /* Enable timer */
    if (!TmrIntNesting)
    {
        ENABLE_T1_COMPA_INT();
    }
} // End of functions

/*****
//@FunctionName: phyrx_receiveUnsecured
*/
//@Description: This function will receive the packet in un-secured mode */
//
//@Param */
//
/*****
void phyrx_receiveUnsecured()
{
    /****** Variable declaration *****/
    P_MPDU_RX          pRxPacket;
    FRAME_CONTROL*     pFrmCtrl;
    WORD                readCount;
    BYTE*               pTemp;

    /****** Function body *****/

#ifdef ZIGBEE_STUB
//    /* At the first time, ControlVariable always be 0*/
//    if (ControlVariable == 0)
//    {
//        test_initBeaconPkt();
//        ControlVariable++;
//    }
//    else if (ControlVariable == 1)
//    {
//        /* Generate ACK */
//        test_initAckForAssReqPkt();
//        ControlVariable++;
//    }
//    else if (ControlVariable == 2)
//    {
//        /* Generate ACK for DATA request */
//        test_initAckForAssReqPkt();

```

```

//      ControlVariable++;
//    }
//    else if(ControlVariable == 3)
//    {
//      /* Generate Ass response*/
//      test_initAssResPkt();
//      ControlVariable++;
//    }

#endif

/* Allocate the pkt*/
if (PhyInfo.sCurPacket.pCurRxPacket == 0)
{
    PhyInfo.sCurPacket.pCurRxPacket = mem_getFreeRxPkt();
}

if (PhyInfo.sCurPacket.pCurRxPacket == 0)
{
    PhyInfo.eState = PHY_RX_STATE_RESET;
}
else
{
    pRxPacket = PhyInfo.sCurPacket.pCurRxPacket;
    pFrmCtrl = (FRAME_CONTROL*)&PhyInfo.sCurPacket.pCurRxPacket->Mpdu;
}

switch (PhyInfo.eState) {
case PHY_RX_STATE_PHR:

#ifdef ZIGBEE_DEBUG
    SET_YLED();
#endif

    //receive the length of the MPDU
    pTemp = (BYTE*)&PhyInfo.sCurPacket.nLength;
    readCount = 1;
    ReadFifo(pTemp,readCount);
    //receive the header
    pTemp = (BYTE*)&PhyInfo.sCurPacket.pCurRxPacket->Mpdu;
    readCount = MAC_HR_LEN;
    ReadFifo(pTemp,readCount);

    /* validate the packet */
    PhyInfo.sCurPacket.nHrLength = mac_getMpduHrLength(pFrmCtrl);
    if (PhyInfo.sCurPacket.nLength < PhyInfo.sCurPacket.nHrLength+MAC_FCS_LEN)
    {
        PhyInfo.eState = PHY_RX_STATE_RESET;
        break;
    }

    // This packet will be acknowledged if the ack request bit is set
    phyrx_checkAck(pRxPacket);

    /* Check if the frame has payload : ACK */
    if (PhyInfo.sCurPacket.nLength == MAC_HR_LEN+MAC_FCS_LEN

```

```

        && pFrmCtrl->FrameType == MAC_FRM_ACK)
        {
            PhyInfo.eState = PHY_RX_STATE_FCS;
        }
    else
    {
        //set threshold to receive the payload
        PhyInfo.eState = PHY_RX_STATE_PAYLOAD;
        SetFifopThreshold(readCount,PhyInfo.sCurPacket.nLength-MAC_HDR_LEN-MAC_FCS_LEN-1);
    }

#ifdef ZIGBEE_DEBUG
#endif // _DEBUG

    /* If data is ready, read it*/
    if (IFIFOP_IS_ACTIVE() && PhyInfo.eState == PHY_RX_STATE_PAYLOAD)
    {
        break;
    }

    case PHY_RX_STATE_PAYLOAD:

        //receive the payload of the MPDU
        phyrx_receivePayload();

        /* Next state */
        PhyInfo.eState = PHY_RX_STATE_FCS;
        break;
    default:
        break;
}

/* Read the FCS */
if (PhyInfo.eState == PHY_RX_STATE_FCS)
{
    //receive the FCS of the MPDU
    pTemp = (BYTE*)&pRxPacket->Mpdu.Fcs;
    readCount = MAC_FCS_LEN;
    ReadFifo(pTemp,readCount);

    //Check CRC state
    if(pRxPacket->Mpdu.Fcs.CRCCheck)
    {
        //CRC check is not ok, discard this packet.
        PhyInfo.eState = PHY_RX_STATE_DISCARD;
    }
    //else //crc check is ok,
    {
        PhyInfo.eState = PHY_RX_STATE_SUCCESS;
    }
}

/* End of the receiving process, indicate the data or clean up*/
switch (PhyInfo.eState) {
    case PHY_RX_STATE_SUCCESS:

```

```

//Call PD-DATA.indication, LinkQuality will be calculated on the upper layer
pddata_indication(pRxPacket,0);

//next state
PhyInfo.eState = PHY_RX_STATE_PHR;
//Next state will receive a new packet
SetFifopThreshold(readCount,MAC_HR_LEN+1);

#ifdef ZIGBEE_DEBUG
    CLR_YLED();
#endif

    break;
case PHY_RX_STATE_RESET:
    phyrx_ResetRx();
    /*No break*/
case PHY_RX_STATE_DISCARD:

#ifdef ZIGBEE_DEBUG
    CLR_YLED();
#endif

//next state
PhyInfo.eState = PHY_RX_STATE_PHR;
//Next state will receive a new packet
SetFifopThreshold(readCount,MAC_HR_LEN+1);

    break;
default:
    break;
}

} // End of functions

/*****
//@FunctionName:  phyrx_receivePayload
*****/

/*****
BOOL phyrx_receivePayload()
{
    /***** Variable declaration *****/
    FRAME_CONTROL*      pFrameCtrl;
    MPDU*                pMpdu;
    BYTE*                pTemp;
    UINT8                TemLen;
    /***** Function body *****/

    pFrameCtrl      = (FRAME_CONTROL*)&PhyInfo.sCurPacket.pCurRxPacket->Mpdu;
    pMpdu           = &PhyInfo.sCurPacket.pCurRxPacket->Mpdu;

    //check the des address mode
    if (pFrameCtrl->DestAddMode)
    {
        //copy  PAN ID and address.
        pTemp  = &pMpdu->dstPanId;
    }
}

```

```

    TemLen = MAC_PAN_ID_LEN;
    ReadFifo(pTemp, TemLen);

    pTemp = &pMpdu->dstAddr;
    if (pFrameCtrl->DestAddMode == AM_SHORT_16)
    {
        TemLen = ADDR_LEN_SHORT_16;

    }
    else if (pFrameCtrl->DestAddMode == AM_EXTENDED_64)
    {
        TemLen = ADDR_LEN_EXTENDED_64;
    }
    else
        return FALSE;

    ReadFifo(pTemp, TemLen);
}

//check the src address mode
if(pFrameCtrl->SourceAddMode)
{
    //copy src PAN ID and address.
    if (!pFrameCtrl->IntraPAN)
    {
        pTemp = &pMpdu->srcPanId;
        TemLen = MAC_PAN_ID_LEN;
        ReadFifo(pTemp, TemLen);
    }

    pTemp = &pMpdu->srcAddr;
    if (pFrameCtrl->SourceAddMode == AM_SHORT_16)
    {
        TemLen = ADDR_LEN_SHORT_16;
    }
    else if (pFrameCtrl->SourceAddMode == AM_EXTENDED_64)
    {
        TemLen = ADDR_LEN_EXTENDED_64;
    }
    else
        return FALSE;

    ReadFifo(pTemp, TemLen);
}

/*Set the length*/
pMpdu->nLength = PhyInfo.sCurPacket.nLength - PhyInfo.sCurPacket.nHrLength - MAC_FCS_LEN;

pTemp = (BYTE*)pMpdu->PayLoad;
TemLen = pMpdu->nLength;
ReadFifo(pTemp, TemLen);

//here, alway return true
return TRUE;

```



```
} // End of functions
```

9.4.6.2 phy_primitives.c

```
/*
*****
//Module: MAC/phy_primitives
//File Name: phy_primitives.c
//Description: Implements PHY function primitives
*****
*/

/*
*****
Include files*/
*****

#include "phy_primitives.h"
#include "phy_support.h"

#include "mac_rx.h"
#include "mac_packet.h"
#include "mac_mem.h"

#include "schedul.h"

/*
*****
Constant declaration
*****
*/

/*
*****
Type Definition
*****
*/

/*
*****
Variable declaration
*****
*/

PHY_INFO          PhyInfo;

/*
*****
*/
```

```
/*                      Static Function                      */
/*****/

/*****/

/*                      Function                      */
/*****/

/*****/

//@FunctionName: pddata_request
//@Description:      The PD-DATA.request primitive requests the transfer of an
//                  MPDU (i.e., PSDU) from the MAC sublayer to                      */
//                  the local PHY entity.                      */
/*****/

PHY_STATUS pddata_request(MPDU* par_pMpdu)
{
    /***** Variable declaration *****/
    PHY_STATUS      eStatus;
    UINT8           nLength;
    /***** Function body *****/
    eStatus = PHY_INVALID_PARAMETER;

    if ((PhyInfo.PhyState == PHY_RX_ON) || (PhyInfo.PhyState == PHY_TRX_OFF))
    {
        eStatus = PhyInfo.PhyState;
    }

    if (PhyInfo.PhyState == PHY_TX_ON)
    {
        // Write length and the payload to the TX FIFO
        nLength= mac_getMpduLength(par_pMpdu)+MAC_FCS_LEN;
        phy_WriteFifo(&nLength, 1);

        /*Write the header and SEQ*/
    }
}
```



```
phy_WriteFifo(&par_pMpdu->FrmControl, sizeof(FRAME_CONTROL)+sizeof(UINT8));

/*Write the des. address*/
if (par_pMpdu->FrmControl.DestAddMode >= AM_SHORT_16)
{
    nLength = sizeof(WORD) + (par_pMpdu->FrmControl.DestAddMode)*6-10;
    phy_WriteFifo(&par_pMpdu->dstPanId, nLength);
}

/*Write the source address*/
if (par_pMpdu->FrmControl.SourceAddMode >= AM_SHORT_16)
{
    if (!par_pMpdu->FrmControl.IntraPAN)
    {
        nLength = sizeof(WORD)+(par_pMpdu->FrmControl.SourceAddMode)*6-10;
        phy_WriteFifo(&par_pMpdu->srcPanId, nLength);
    }
    else
    {
        nLength = (par_pMpdu->FrmControl.SourceAddMode)*6-10;
        phy_WriteFifo(&par_pMpdu->srcAddr, nLength);
    }
}

/*Send the payload*/
phy_WriteFifo(&par_pMpdu->PayLoad, par_pMpdu->nLength);

eStatus = PHY_SUCCESS;
}

return eStatus;

} // End of functions

/*****/
//@FunctionName: pddata_indicationTask
//@Description: The PD-DATA.indication primitive indicates the transfer of an
```

```

//          MPDU (i.e., PSDU) from the PHY to the          */
//          local MAC sublayer entity.                      */
//          Don't care Link quality                          */
/*****/

void pddata_indicationTask(void* par_pParam, void* args)
{
    /***** Variable declaration *****/
    P_MPDU_RX      pMpduRx;
    /***** Function body *****/
    pMpduRx = (P_MPDU_RX)par_pParam;

    //call to appropriate function to process the incoming packet
    macrx_ProcessMpdu(pMpduRx);

    /* Decrease number of queuing pkt*/
    PhyInfo.nQueueingPkt--;

    schedul_removeTask(TASK_PER_MAC_RX);    /* Remove this task */

} // End of functions

/*****/

/*@FunctionName: pddata_indication
    */

/*@Description:      This indication will create a task to process the packet    */
/*****/

void pddata_indication(P_MPDU_RX pRxMpdu,  UINT8 pPduLinkQuality)
{
    /***** Variable declaration *****/
    FRAME_CONTROL*  pFrmCtrl;
    BOOL            bResult;
    /***** Function body *****/

    pFrmCtrl = (FRAME_CONTROL*)&pRxMpdu->Mpdu;

    /*Validate state*/

```

```

bResult = mac_validateMacState(pRxMpdu->Mpdu.Payload[0],pFrmCtrl->FrameType)
    && mac_validateFrmData(pRxMpdu);

/* Omit the packet if not valid */
if(!bResult)
{
    return;
}

/* Check the data frame */
if (pFrmCtrl->FrameType == MAC_FRM_ACK)
{
    /* It's ACK, call direct to TX */
    macrx_ProcessMpdu(pRxMpdu);
}
else /* For other type create task to process */
{
    bResult = schedul_addTask(pddata_indicationTask, TASK_PER_MAC_RX, (WORD*)pRxMpdu);
    if (bResult)
    {
        /* Increment the number of queuing pkt*/
        PhyInfo.nQueueingPkt++;

        PhyInfo.sCurPacket.pCurRxPacket = mem_getFreeRxPkt();

        return; /* Create task */
    }
}

} // End of functions

/*****/
//@FunctionName: plmeCCA_request
//@Description: The PLME-CCA.request primitive requests that the PLME perform
//              a CCA as defined in 6.7.9
//
/*****/

```

```
PHY_STATUS    plmeCCA_request(void)
{
    /****** Variable declaration *****/

    PHY_STATUS    eReturn;

    /****** Function body *****/

    eReturn = INVALID_PARAMETER;

    if ((PhyInfo.PhyState == PHY_TX_ON) || (PhyInfo.PhyState == PHY_TRX_OFF))
    {
        eReturn = PhyInfo.PhyState;
    }
    if (PhyInfo.PhyState == PHY_RX_ON)
    {
        if (CCA_IS_ACTIVE())
        {
            eReturn = PHY_IDLE;
        }
        else
        {
            eReturn = PHY_BUSY;
#ifdef ZIGBEE_STUB
            eReturn = PHY_IDLE;
#endif
        }
    }
    return eReturn;
}

/*****
//@FunctionName:    plmeED_request
//@Description:      The PLME-ED.request primitive requests that the PLME perform
//                   an ED measurement (see 6.7.7).
*/
/*****
PHY_STATUS plmeED_request (UINT8* par_EnergyLevel)
{

```

```

    /***** Variable declaration *****/
    INT8      maxRssi, rssi;
    PHY_STATUS eStatus;

    /***** Function body *****/
    eStatus = INVALID_PARAMETER;

    if(PhyInfo.PhyState == PHY_RX_ON)
    {
        maxRssi = -128;
        DISABLE_GLOBAL_INT();
        FASTSPI_GETREG(CC2420_RSSI, rssi);
        ENABLE_GLOBAL_INT();

        // ... and detect the peak
        maxRssi = MAX(rssi, maxRssi);

        *par_EnergyLevel = RSSI_2_ED(maxRssi);
        eStatus = PHY_SUCCESS;
    }

    if (PhyInfo.PhyState == PHY_TX_ON || PhyInfo.PhyState == PHY_TRX_OFF)
    {
        eStatus = PhyInfo.PhyState;
    }

    return eStatus;

} // End of functions

/*****
**@FunctionName:   plmeGET_request
**@Description:    The PLME-GET.request primitive requests information about
**                a given PHY PIB attribute.
**
*****/
```

PHY_STATUS plmeGET_request (PHY_PIB_ATT_TYPE par_PIBAttType, UINT32* par_pPIBAttributeValue)

```
{
    /***** Variable declaration *****/

    PHY_STATUS      eStatus;

    /***** Function body *****/

    switch(par_PIBAttType)
    {
    case PHY_PIB_CURRENT_CHANNEL:
        *par_pPIBAttributeValue = PhyInfo.PhyPIB.phyCurrentChannel;
        eStatus = PHY_SUCCESS;

        break;

    case PHY_PIB_CHANNEL_SUPPORTED:
        *par_pPIBAttributeValue = PhyInfo.PhyPIB.phyChannelsSupported;
        eStatus = PHY_SUCCESS;

        break;

    case PHY_PIB_TRANSMIT_POWER:
        *par_pPIBAttributeValue = PhyInfo.PhyPIB.phyTransmitPower;
        eStatus = PHY_SUCCESS;

        break;

    case PHY_PIB_CCA_MODE:
        *par_pPIBAttributeValue = PhyInfo.PhyPIB.phyCcaMode;
        eStatus = PHY_SUCCESS;

        break;

    default:
        eStatus = PHY_UNSUPPORTED_ATTRIBUTE;

        break;

    }

    return eStatus;
} // End of functions

/*****
//@FunctionName: plmeSET_request
//@Description:   The PLME-SET.request primitive attempts to set the indicated
//                PHY PIB attribute to the given value.
*****/
```




/*****/

PHY_STATUS plmeSET_request (PHY_PIB_ATT_TYPE par_PIBAttType, UINT32 par_PIBAttributeValue)

{

/****** Variable declaration *****/

PHY_STATUS eStatus;

UINT16 newFreq, oldFreq;

BOOL sfdWasActiveBeforeStrobe;

/****** Function body *****/

eStatus = INVALID_PARAMETER;

switch(par_PIBAttType)

{

case PHY_PIB_CURRENT_CHANNEL:

if ((par_PIBAttributeValue >= 0) && (par_PIBAttributeValue <= 26))

{

PhyInfo.PhyPIB.phyCurrentChannel = par_PIBAttributeValue;

// Derive frequency programming from the given channel number

newFreq = (UINT16) (par_PIBAttributeValue - 11); // Subtract the base channel

newFreq = newFreq + (newFreq << 2); // Multiply with 5, which is the channel spacing

newFreq = newFreq + 357 + 0x4000; // 357 is 2405-2048, 0x4000 is LOCK_THR = 1

// Do we need to change the frequency at all?

DISABLE_GLOBAL_INT();

FASTSPI_GETREG(CC2420_FSCTRL, oldFreq);

ENABLE_GLOBAL_INT();

if ((newFreq ^ oldFreq) & 0x03FF)

{

DISABLE_GLOBAL_INT();

// Force RX off

sfdWasActiveBeforeStrobe = SFD_IS_ACTIVE();

FASTSPI_STROBE(CC2420_SRFOFF);

// Clean up if we interrupted the receiver

```
        if (sfdWasActiveBeforeStrobe)
            phyrx_ResetRx();

        //update the frequency register
        FASTSPI_SETREG(CC2420_FSCTRL, newFreq);

        // Return to the previous RX state
        FASTSPI_STROBE(CC2420_SRXON);

        ENABLE_GLOBAL_INT();
    }
    eStatus = PHY_SUCCESS;
}
break;
case PHY_PIB_CHANNEL_SUPPORTED:
    //TBD??? too complicated to check this param
    PhyInfo.PhyPIB.phyChannelsSupported = par_PIBAttributeValue;
    eStatus = PHY_SUCCESS;
    break;
case PHY_PIB_TRANSMIT_POWER:
    if ((par_PIBAttributeValue >= 0) && (par_PIBAttributeValue <= 0xbf))
    {
        PhyInfo.PhyPIB.phyTransmitPower = par_PIBAttributeValue;
        eStatus = PHY_SUCCESS;
    }
    break;
case PHY_PIB_CCA_MODE:
    if((par_PIBAttributeValue >= 1) && (par_PIBAttributeValue <= 3))
    {
        PhyInfo.PhyPIB.phyCcaMode = par_PIBAttributeValue;
        eStatus = PHY_SUCCESS;
    }
    break;
default:
    eStatus = PHY_UNSUPPORTED_ATTRIBUTE;
    break;
```

}

return eStatus;

}

9.4.6.3 phy_support.c

```

/*****
//Module: MAC/phy_primitives
//File Name: phy_support.c
//Description: This module will implement support function for manipulation on
// the phy
*****/

/*****
/* Include files */
*****/

#include "phy_support.h"
#include "phy_primitives.h"
#include "mac_state.h"
#include "zconf.h"
#include "mac_packet.h"
#include "indirect_tx.h"
*****/

/* Constant declaration */
*****/

/* The number of microseconds used to power up
the CC2420 voltage regulator
*/
#define MPM_VREG_TURN_ON_TIME 600
*****/

/* Type Definition */
*****/

/*****
/* Variable declaration */
*****/

/*****
/* Static Function */
*****/

/*****
/* Function */
*****/

/*****
//@FunctionName: phytrx_setPwrMode
//@Description: This function will be call to control power of the TRX
// Currently, only support on PWR_ON mode
*****/
void phytrx_setPwrMode(PHY_MODE par_Mode)
{

```

```

/***** Variable declaration *****/
BYTE          spiStatus;
/***** Function body *****/
switch(par_Mode)
{
case PHY_PWR_ON:

    /*Initialize*/
    MAC_PORT_INIT();
    SPI_INIT();
    FIFOP_INT_INIT();
    DB_PERIPHERAL_PORT_INIT();

    /* Initialize the C2420*/
    SET_RESET_ACTIVE();
    SET_VREG_ACTIVE();
    wait(MPM_VREG_TURN_ON_TIME);

    SET_RESET_INACTIVE();
    FASTSPI_STROBE(CC2420_SXOSCON);
#ifdef ZIGBEE_STUB
    /*no wait*/
#else
    /* Wait for oscillator become stable */
    do {
        DISABLE_GLOBAL_INT();
        FASTSPI_UPD_STATUS(spiStatus);
        ENABLE_GLOBAL_INT();
    } while (!(spiStatus & CC2420_XOSC16M_STABLE_BM));
#endif
    /*Setting PHY state */
    PhyInfo.bFifopIntOff = FALSE;
    PhyInfo.eState       = PHY_RX_STATE_PHR;
    PhyInfo.PhyState     = PHY_TRX_OFF;

    break;
default:
    break;
}
} // End of functions

/*****
//@FunctionName: phyrx_turnOffRx
//@Description:  Decrements the "RX on" counter, and turns off RX on a 1 -> 0 */
//              transition if SFD is low. If SFD is high, the turn-off
//              operation is delayed by one backoff period by using
//              a callback (TBD)
*****/
void phyrx_turnOffTrx(void* par_pParam)
{
    DISABLE_GLOBAL_INT();

    phyrx_ResetRx();

    FASTSPI_STROBE(CC2420_SRFOFF);

    /*Disable interrupt */

```

```

DISABLE_FIFOP_INT();

//reset state
PhyInfo.PhyState      = PHY_TRX_OFF;
PhyInfo.eState        = PHY_RX_STATE_PHR;

ENABLE_GLOBAL_INT();
} // phyrx_DecrOnCounter

/*****
//@FunctionName: phyrx_turnOnRx
//@Description:   Increments the "RX on" counter, and turns on RX on
//                a 0 -> 1 transition
*/
void phyrx_turnOnRx(void* par_pParam) {
    DISABLE_GLOBAL_INT();

    FASTSPI_STROBE(CC2420_SRXON);
    /*Disable interrupt */
    ENABLE_FIFOP_INT();

    PhyInfo.PhyState      = PHY_RX_ON;
    PhyInfo.eState        = PHY_RX_STATE_PHR;

    ENABLE_GLOBAL_INT();
} // phyrx_IncrOnCounter

void phyrx_initOnRx() {
    PhyInfo.PhyState      = PHY_RX_ON;
    PhyInfo.eState        = PHY_RX_STATE_PHR;
    /*Disable interrupt */
    ENABLE_FIFOP_INT();
} // phyrx_IncrOnCounter

/*****
//@FunctionName: phytx_turnOnTx
//@Description:   This function will turn on the TX
*/
BOOL phytx_turnOnTx()
{
    /****** Variable declaration *****/
    BOOL      bStatus;
    BYTE      nTemp;
    /****** Function body *****/
    DISABLE_GLOBAL_INT();

    // Flush the FIFO
    phyrx_ResetRx();
    FASTSPI_STROBE(CC2420_SFLUSHTX);

    // Start TX
    FASTSPI_STROBE(CC2420_STXON);

    // Was TX started?
    FASTSPI_UPD_STATUS(nTemp);

```

```

#ifdef ZIGBEE_STUB
    nTemp = 0xFF;
#endif
if (!(nTemp & CC2420_TX_ACTIVE_BM))
{
    bStatus = FALSE;
}
else
{
    bStatus = TRUE;
    PhyInfo.PhyState = PHY_TX_ON;
    PhyInfo.eState = PHY_RX_STATE_PHR;
}
ENABLE_GLOBAL_INT();

return bStatus;
} // End of functions

/*****
// @FunctionName: phyrx_ResetRx
// @Description: Resets the RX engine by clearing the RX FIFO
*****/
void phyrx_ResetRx()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    DISABLE_GLOBAL_INT();

    // Flush the RX FIFO twice
    FASTSPI_STROBE(CC2420_SFLUSHRX);
    FASTSPI_STROBE(CC2420_SFLUSHRX);

    // read the current packet from the C2420
    // TBD

    // Reset the RX engine state
    PhyInfo.eState = PHY_RX_STATE_PHR;

    DISABLE_GLOBAL_INT();
} // End of functions

//-----
// void phy_WriteRam(void *pData, UINT16 address, UINT8 count, BOOL disableInterrupts)
//
// DESCRIPTION:
//     Write data to the CC2420 RAM-space
//     Used for setup of addresses and security
//
// PARAMETERS:
//     void *pData
//         Pointer to data to be written
//     UINT16 address
//         CC2420 RAM Address
//     UINT8 count
//         Number of bytes to be written
//     BOOL disableInterrupts

```

```
//          Disable global interrupts while writing?
//-----
void phy_WriteRam(void *pData, UINT16 address, UINT8 count, BOOL disableInterrupts) {
    if (count) {
        do {
            if (disableInterrupts) DISABLE_GLOBAL_INT();
            SPI_ENABLE();
            FASTSPI_TX(0x80 | (address & 0x7F)); \
            FASTSPI_TX((address >> 1) & 0xC0); \
            do {
                FASTSPI_TX(((BYTE*) pData));
                pData++;
            } while (--count & 0x07);
            SPI_DISABLE();
            if (disableInterrupts) ENABLE_GLOBAL_INT();

            address += 8;
        } while (count);
    }
} // msupWriteRam
```

```
//-----
// void phy_ReadFifo(void *pData, UINT8 count)
//
// DESCRIPTION:
//     Reads data from the CC2420 RX FIFO
//
// PARAMETERS:
//     void *pData
//         Pointer to data to be read
//     UINT8 count
//         Number of bytes to be read
//-----
PHY_FIFO_STATE phy_ReadFifo(void *pData, UINT8 count, UINT8* readCount) {
    DISABLE_GLOBAL_INT();
    SPI_ENABLE();
    FASTSPI_RX_ADDR(CC2420_RXFIFO);
    *readCount = count;
    if (count) {
        do {
            FASTSPI_RX(((BYTE*) pData));
            pData++;
        } while (--count);
    }
    /*Check number of read bytes*/
    *readCount -= count;

    SPI_DISABLE();
    ENABLE_GLOBAL_INT();

    return PHY_FIFO_READ_SUCCESS;
} // phy_ReadFifo

//-----
```

```

// void phy_WriteFifo(void *pData, UINT8 count)
//
// DESCRIPTION:
//     Write data to the CC2420 TX FIFO
//
// PARAMETERS:
//     void *pData
//         Pointer to data to be written
//     UINT8 count
//         Number of bytes to be written
//-----
void phy_WriteFifo(void *pData, UINT8 count) {
    if (count) {
        do {
            DISABLE_GLOBAL_INT();
            SPI_ENABLE();
            FASTSPI_TX_ADDR(CC2420_TXFIFO);
            do {
                FASTSPI_TX*((BYTE*) pData));
                pData++;
            } while ((--count) & 0x07);
            SPI_DISABLE();
            ENABLE_GLOBAL_INT();
        } while (count);
    }
} // phy_WriteFifo

/*****
//@FunctionName:phyrx_checkAck
*/
//@Description:    This function will check the need for ACK, if need, it will
//                perform setting to send ACK
*/
void phyrx_checkAck(P_MPDU_RX par_pMpdu)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /*ACK request*/
    if (par_pMpdu->Mpdu.FrmControl.ACKRequest)
    {
        DISABLE_GLOBAL_INT();
        #if ZIGBEE_DEVICE_FFD == TRUE
            if (SFD_IS_ACTIVE())
            {
                if (PKT_IS_PENDING())
                {
                    FASTSPI_STROBE(CC2420_SACKPEND);
                }
                else
                    FASTSPI_STROBE(CC2420_SACK);
            }
        #else
            if (SFD_IS_ACTIVE())

```



```

    {
        FASTSPI_STROBE(CC2420_SACK);
    }
#endif    ///ZIGBEE_DEVICE_FFD == TRUE

    ENABLE_GLOBAL_INT();
}
} // End of functions

/*****
//@FunctionName: phy_disableFIFOInt
//
//@Description:
//
//@Param
//
*****/
void phy_disableFIFOInt(BOOL bDisable)
{
    DISABLE_GLOBAL_INT();
    if (bDisable)
    {
        PhyInfo.bFifopIntOff = bDisable;
        DISABLE_FIFOP_INT();
    }
    else
    {
        FASTSPI_STROBE(CC2420_SFLUSHRX);    // Flush the RX FIFO twice
        FASTSPI_STROBE(CC2420_SFLUSHRX);
        ENABLE_FIFOP_INT();
    }
    ENABLE_GLOBAL_INT();
}

```

9.5 NWK

9.5.1 NWK routing

```

/*****
//Module: NWK
//File Name: nwk_routing
//Description:    This file will implement the routing procedure defined for
//                Zigbee network
//
//-----*/
//Revision History:
//Date            Programmer            Details
//
//07/03/2007
//-----*/
//    Copyright (C) 2006 - All Rights Reserved
/*****

/*
                                Include files
*/

```

```

/*****/
#include "types.h"
#include "nwk_routing.h"
#include "nwk_common.h"
#include "mac_types.h"
#include "mac_primitives.h"
/*****/

/*                                     Constant declaration
*/

/*****/

/*                                     Type Definition
*/

/*****/

/*                                     Variable declaration
*/

/*****/
/* Code for FFD only */
#if ZIGBEE_DEVICE_FFD == 1
/* Cskip of the device for hierarchical routing
*/
UINT16          Cskip;
/*Index for router and device
*/
UINT16          RouterAllocAddrMap;
UINT16          DeviceAllocAddrMap;

// Declaration: Array of struct for Neighbor Table of size NWK_NEIGHBOR_TBLE_SIZE
extern NWK_NEIGHBOR_NODE* nwk_neighborTable[];
/*****/
/*                                     Static Function
*/

/*****/
/*                                     Function
*/

/* -----
ADDR ALLOCATION
-----*/
/*****/
//@FunctionName: routing_init
*/
//@Description:
//
//@Param
//
/*****/
void routing_init()
{
    /***** Variable declaration *****/
    UINT8      nTem;
    UINT16     nPower;
    INT16      n1;
    INT16      n2;

```

```

    }
    nTem = nwk_ib.nwkMaxDepth-NwkInfo.BcnPayload.DeviceDepth-1;
    nPower = 1;
    for (UINT8 i = 0; i<nTem; i++)
        nPower = nPower*nwk_ib.nwkMaxRouters;

    if (nwk_ib.nwkMaxRouters == 1)
        Cskip = 1 + nwk_ib.nwkMaxChildren*(nTem);
    else
    {
        if (nwk_ib.nwkMaxRouters == 0)
            Cskip = -1 - nwk_ib.nwkMaxChildren + nwk_ib.nwkMaxRouters + nwk_ib.nwkMaxChildren*nPower;
        else
        {
            n1 = nwk_ib.nwkMaxRouters + nwk_ib.nwkMaxChildren*nPower-1 - nwk_ib.nwkMaxChildren;
            n2 = nwk_ib.nwkMaxRouters-1;
            Cskip = (UINT16)n1/n2;
        }
    }

    if (Cskip == 0)
    {
        /*not available for association*/
        mlmeSetRequest(MAC_ASSOCIATION_PERMIT,&nTem);
    }

    RouterAllocAddrMap = 0x00;
    DeviceAllocAddrMap = 0x00;
} // End of functions

/***** Function body *****/
//@FunctionName: routing_allocDistAddr
//
//
//@Description: This function will implement the distributed addr assignment
//
//@Param
//
/*****
UINT16 routing_allocDistAddr(BOOL bRouter)
{
    /*Variable declaration */
    /*Function body */

    if (bRouter)
    {
        /*Search for free slot*/
        for (UINT8 i = 0; i<nwk_ib.nwkMaxRouters; i++)
        {
            if (!(1<<i)&RouterAllocAddrMap)
            {
                RouterAllocAddrMap = RouterAllocAddrMap | (1<<i);
                return NwkInfo.NwkAddr+1+i*Cskip;
            }
        }
    }
    else

```

```

{
    /*Search for free slot*/
    for (UINT8 i = 0; i < nwk_ib.nwkMaxChildren - nwk_ib.nwkMaxRouters; i++)
    {
        if (!((1 < i) & DeviceAllocAddrMap))
        {
            DeviceAllocAddrMap = DeviceAllocAddrMap | (1 < i);
            return NwkInfo.NwkAddr + nwk_ib.nwkMaxRouters * Cskip + i + 1;
        }
    }
    return 0; /*no address available*/
} // End of functions

/*****
//@FunctionName: routing_checkFreeAddr
*/
//
//
//
//@Description:      This function will implement the distributed addr assignment */
//                  Check the free addr
//
//
//@Param
//
//
/*****
BOOL routing_checkFreeAddr(BOOL bRouter)
{
    /****** Variable declaration *****/
    /****** Function body *****/

    if (bRouter)
    {
        /*Search for free slot*/
        for (UINT8 i = 0; i < nwk_ib.nwkMaxRouters; i++)
            if (!((1 < i) & RouterAllocAddrMap))
                return TRUE;
    }
    else
    {
        /*Search for free slot*/
        for (UINT8 i = 0; i < nwk_ib.nwkMaxChildren - nwk_ib.nwkMaxRouters; i++)
            if (!((1 < i) & DeviceAllocAddrMap))
                return TRUE;
    }
    return FALSE; /*no address available*/
} // End of functions

/*-----
ROUTING DECISION
-----*/
/*****
//@FunctionName: routing_getNextNodeInHier
*/
//
//@Description:      This function will apply hierarchical routing to find the
//                  next network node for a given destination
*/

```

```

//          Calculation is based on neighbors table          */
//
//@Param                                                    */
//                                                    */
/*****
NWK_NEIGHBOR_NODE* routing_getNextNodeInHier(UINT16    DesAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    /*Check if this addr is in range of child of the device*/
    if (DesAddr > NwkInfo.NwkAddr
        && DesAddr <= NwkInfo.NwkAddr+nwk_ib.nwkMaxRouters*Cskip+nwk_ib.nwkMaxChildren-
nwk_ib.nwkMaxRouters)
    {
        /*Search for appropriate child in the NT*/
        for (UINT8 index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
        {
            if ((nwk_neighborTable[index] != NULL)
                && (nwk_neighborTable[index]->Relationship == NWK_CHILD))
            {
                /*Destination is the child of the device*/
                if(nwk_neighborTable[index]->NetworkAddress == DesAddr)
                    return nwk_neighborTable[index];
                /*Destination not is child but within the tree of the device*/
                if ((nwk_neighborTable[index]->NetworkAddress < DesAddr)
                    && (DesAddr < nwk_neighborTable[index]->NetworkAddress + Cskip))
                    return nwk_neighborTable[index];
            }
        }
        /*Destination not found*/
        return NULL;
    }

    /*Destination is not in the range of children*/
    return NwkInfo.pParent;
} // End of functions

/*****
//@FunctionName: routing_getNextHop
*/
//@Description:      This function will apply routing table to find the
//                  next network node for a given destination
//                  Calculation is based on routing table
//
//@Param
//
/*****
QWORD* routing_getNextHop(UINT16    DesAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    return 0;
} // End of functions

#endif //!ZIGBEE_DEVICE_FFD == 1          /* Code for FFD only */

```

9.5.2 Data Queue

```

/*****
//Module: NWK
//File Name: data_queue.c
//Description: This file will implement function to handle the data queue
//            Will be used in NWK and APS
//-----*/
//Revision History:
//Date          Programmer          Details
*/
//07/16/2007
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
/*
                                Include files
*/
*****/
#include "data_queue.h"

/*****
/*
                                Constant declaration
*/
*****/

/*****
/*
                                Type Definition
*/
*****/

/*****
/*
                                Variable declaration
*/
*****/

/*****
/*
                                Static Function
*/
*****/

/*****
/*
                                Function
*/
*****/

//@FunctionName: dataqueue_init
//@Description: This function will init data queue
//
//@Param
//
*****/
void dataqueue_init(DATA_QUEUE* pQueue, DATA_REC* pRecList, UINT8 Size)
{
    /***** Variable declaration *****/

```

```

    /***** Function body *****/
    pQueue->RecList = pRecList;
    pQueue->Size      = Size;

    for (UINT8 index = 0; index < Size; index++)
    {
        pQueue->RecList[index].pData      = NULL;
        pQueue->RecList[index].NextIndex  = Size;
    }
    pQueue->FirstIndex      = Size;
    pQueue->LastIndex       = Size;
} // End of functions

/*****
//@FunctionName: dataqueue_getFreeSlot
*/
//@Description:   This function will add a new record into the queue
//
//@Param
//
/*****
UINT8 dataqueue_getFreeSlot(DATA_QUEUE* pQueue)
{
    /***** Variable declaration *****/
    UINT8      index;
    /***** Function body *****/
    /*Search for free slot */
    for (index = 0; index < pQueue->Size; index++)
        if (pQueue->RecList[index].pData == NULL)
            return index;

    /*Not available to add*/
    return pQueue->Size;
} // End of functions

/*****
//@FunctionName: dataqueue_addRec
//@Description:   This function will add a new record into the queue
//
//@Param
//
/*****
UINT8 dataqueue_addRec(DATA_QUEUE* pQueue, BYTE*  pData,
                      UINT8  nLength, UINT8 Index, UINT8 Handle,UINT8 Status)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    /*Search for free slot */
    if (pQueue->RecList[Index].pData == NULL)
    {
        pQueue->RecList[Index].pData      = pData;
        pQueue->RecList[Index].Handle     = Handle;
        pQueue->RecList[Index].nLength    = nLength;
        pQueue->RecList[Index].Status     = Status;
    }
}

```

```

    /*Update next index*/
    pQueue->RecList[pQueue->LastIndex].NextIndex = Index;
    pQueue->LastIndex = Index;

    /*First packet*/
    if (pQueue->FirstIndex == pQueue->Size)
        pQueue->FirstIndex = Index;

    return Handle;
}

/*Not available to add*/
return pQueue->Size;
} // End of functions

/*****
//@FunctionName: dataqueue_getRec
*/
//@Description: This function will get the oldest record from the queue
//
//@Param
//
/*****
UINT8 dataqueue_getRec(DATA_QUEUE* pQueue, UINT8 Status)
{
    /****** Variable declaration *****/
    UINT8 Index;
    /****** Function body *****/
    Index = pQueue->FirstIndex;
    while (Index != pQueue->Size && pQueue->RecList[Index].pData) /* is the valid packet*/
    {
        if (pQueue->RecList[Index].Status == Status)
            return Index;

        Index = pQueue->RecList[Index].NextIndex;
    }

    return pQueue->Size;
} // End of functions

/*****
//@FunctionName: dataqueue_removeRec
*/
//@Description: This function will remove a record from the queue
//
//@Param
//
/*****
BOOL dataqueue_removeRec(DATA_QUEUE* pQueue, UINT8 RemovedIndex)
{
    /****** Variable declaration *****/
    UINT8 Index;
    /****** Function body *****/
    Index = pQueue->FirstIndex;
    /*First packet is removed */
    if (RemovedIndex == pQueue->FirstIndex)

```



```

{
    pQueue->FirstIndex = pQueue->RecList[RemovedIndex].NextIndex;
    pQueue->RecList[RemovedIndex].pData = NULL;
    pQueue->RecList[RemovedIndex].NextIndex = pQueue->Size;

    /*Last packet is removed */
    if (RemovedIndex == pQueue->LastIndex)
        pQueue->LastIndex = pQueue->Size;

    return TRUE;
}
else
{
    while (Index != pQueue->Size && pQueue->RecList[Index].pData) /* is the valid data*/
    {
        if (pQueue->RecList[Index].NextIndex == RemovedIndex)
            break;
        Index = pQueue->RecList[Index].NextIndex;
    }

    if (RemovedIndex == pQueue->RecList[Index].NextIndex)
    {
        pQueue->RecList[Index].NextIndex = pQueue->RecList[RemovedIndex].NextIndex;

        pQueue->RecList[RemovedIndex].pData = NULL;
        pQueue->RecList[RemovedIndex].NextIndex = pQueue->Size;

        /*Last packet is removed */
        if (RemovedIndex == pQueue->LastIndex)
            pQueue->LastIndex = Index;

        return TRUE;
    }
}

return FALSE;
} // End of functions

/*****
//@FunctionName: dataqueue_findRec
*/
//@Description:
//
//@Param
//
*****/
UINT8 dataqueue_findRec(DATA_QUEUE* pQueue, UINT8 nHandler)
{
    /****** Variable declaration *****/
    UINT8 Index;
    /****** Function body *****/
    Index = pQueue->FirstIndex;
    while (Index != pQueue->Size && pQueue->RecList[Index].pData) /* is the valid packet*/
    {
        if (pQueue->RecList[Index].Handle == nHandler)
            return Index;
    }
}

```

```

    Index = pQueue->RecList[Index].NextIndex;
}

return pQueue->Size;
} // End of functions

```

9.5.3 NWK

```

/*****
//Module: NWK
//File Name: nwk.c
//Description: This file will implement intertask communication mechanism
//              that conformed with the scheduler mechanism detailed in AS-OS
//-----*/
//Revision History:
//Date          Programmer          Details
*/
//05/28/2007
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
/*          Include files
*/
*****/

#include "types.h"
#include "utils.h"
#include "mac_types.h"
#include "schedul.h"
#include "mac_primitives.h"
#include "nwk_common.h"
#include "nwk_msg.h"
#include "nwk_routing.h"
#include "nwk_data.h"
#include "data_queue.h"
/*****
/*          Constant declaration
*/
*****/

/*****
/*          Type Definition
*/
*****/

/*****
/*          Variable declaration
*/
*****/

/*****
/*          Static Function
*/
*****/

```

```

/*****
/*                               Function                               */
/*****
/*****
//@FunctionName:  nwk_init
/*
//@Description:
//
//@Param
//
/*****
void nwk_init()
{
    /***** Variable declaration *****/

    /***** Function body *****/

    /*Init nwk stata */
    NwkInfo.CurRequest = NWK_STATE_INIT;
    NwkInfo.NwkState   = NWK_STATE_INIT;

    /*Init data transmission*/
    nwk_dataInit();
} // End of functions

/*****
//@FunctionName: nwk_setNwkState
/*
//@Description:  This function will control transaction on nwk layer
//
//@Param
//
/*****
BOOL nwk_setNwkState(NWK_STATE      State)
{
    /***** Variable declaration *****/
    BOOL      bReturn;
    /***** Function body *****/
    if (NwkInfo.NwkState == NWK_STATE_NORMAL_MODE)
    {
        if(State == NWK_STATE_SYNC)
            bReturn = TRUE;
    }
    return FALSE;
} // End of functions

/*-----

Confirm/Indication function

-----*/

/*****
//@FunctionName:  nwk_sendStatMsg
/*
//@Description:
*/

```

```

//
//@Param
//
/*****
void nwk_sendStatMsg(NWK_MSG_TYPE      Type,
                    MAC_STATUS      Status,
                    WORD      Data)
{
    /***** Variable declaration *****/
    NWK_MSG      Msg;
    /***** Function body *****/
    Msg.MsgType = Type;
    Msg.Status  = Status;
    Msg.Data    = Data;
    schedul_sendMsg(TASK_APER_NWK, &Msg);
}

/*****
//      Function Name:      mcpsDataConfirm
//      Description:
//      Parameters:
//      Return:              nothing
/*****
void mcpsDataConfirm(BYTE      msduHandle,
                    MAC_STATUS status)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_DATA,status,msduHandle);
} // End of functions mcpsDataConfirm

/*****
//      Function Name:      mcpsDataIndication
//      Description:
//      Parameters:
//      Return:              nothing
/*****
void mcpsDataIndication(MCPS_DATA_INDICATION* par_pDIndic)
{
    /***** Variable declaration *****/
    NWK_MSG      Msg;
    /***** Function body *****/
    Msg.MsgType      = MSG_MAC_INDI_DATA;
    Msg.Data          = (WORD)par_pDIndic;
    schedul_sendMsg(TASK_APER_NWK, &Msg);
} // mcpsDataIndication

/*****
//      Function Name:      mlmeAssociateConfirm
//      Description:
//      Parameters:
//      Return:              nothing
/*****
void mlmeAssociateConfirm(WORD AssocShortAddress,UINT8 status)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_ASSOCIATION,status,AssocShortAddress);
} // mlmeAssociateConfirm

/*****

```

```
//      Function Name:      mlmeDisassociateConfirm
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeDisassociateConfirm(MAC_STATUS status)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_DISASSOCIATION,status,0);
}

/*****/
//      Function Name:      mlmeDisassociateIndication
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeDisassociateIndication(QWORD* par_pDevAddr, BYTE Reason,
                                BOOL SecurityUse, BOOL AclEntry)
{
    /***** Variable declaration *****/
    MSG_DISASSO_INDI_DATA*      pDisAssoData;
    /***** Function body *****/
    pDisAssoData = mem_getMem(sizeof(MSG_DISASSO_INDI_DATA));
    if (pDisAssoData)
    {
        pDisAssoData->devAddr      = *par_pDevAddr;
        pDisAssoData->Reason        = Reason;
        pDisAssoData->Security      = SecurityUse;
        pDisAssoData->AclEntry      = AclEntry;

        nwk_sendStatMsg(MSG_MAC_INDI_DISASSOC,SUCCESS,(WORD)pDisAssoData);
    }
} // End of functions

/*****/
//      Function Name:      mlmeResetConfirm
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeResetConfirm(MAC_STATUS status)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_RESET,status,0);
}

/*****/
//      Function Name:      mlmePollConfirm
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmePollConfirm(MAC_STATUS status)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_POLL,status,0);
} // End of functions

/*****/
```

```

//      Function Name:      mlmeScanConfirm
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeScanConfirm(MAC_STATUS status, MAC_SCAN_RESULT* par_pScanResult)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_SCAN,status,(WORD)par_pScanResult);
}

/*****/
//      Function Name:      mlmeSyncLossIndication
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeSyncLossIndication(MAC_STATUS lossReason)
{
    nwk_sendStatMsg(MSG_MAC_INDI_LOSS,lossReason,0);
}

/*****/
//      Function Name:      mlmeCommStatusIndication
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeCommStatusIndication(WORD panId, BYTE srcAddrMode, ADDRESS *pSrcAddr, BYTE dstAddrMode,
ADDRESS *pDstAddr,
                                BYTE status)
{
    MSG_COMM_STATUS_DATA*      pCommIndi;
    /***** Function body *****/
    pCommIndi = mem_getMem(sizeof(MSG_COMM_STATUS_DATA));
    if (pCommIndi)
    {
        pCommIndi->panId      = panId;
        pCommIndi->srcAddrMode = srcAddrMode;
        pCommIndi->SrcAddr     = *SecurityUse;
        pCommIndi->status      = status;

        nwk_sendStatMsg(MSG_MAC_INDI ASSO,SUCCESS,pCommIndi);
    }
}

/*****/
//      Function Name:      mlmeBeaconNotifyIndication
//      Description:
//      Parameters:
//      Return:              nothing
/*****/
void mlmeBeaconNotifyIndication(MLME_BEACON_NOTIFY_INDICATION *pBcnNot)
{
    nwk_sendStatMsg(MSG_MAC_INDI_BEACON,0,(WORD)pBcnNot);
}

#if      ZIGBEE_DEVICE_FFD == 1

```

```

/*****
//      Function Name:      mlmeStartConfirm
//      Description:
//      Parameters:
//      Return:              nothing
*****/
void mlmeStartConfirm(MAC_STATUS      status)
{
    nwk_sendStatMsg(MSG_MAC_CONFIRM_START,status,0);
}

/*****
//      Function Name:      mlmeStartConfirm
//      Description:
//      Parameters:
//      Return:              nothing
*****/
void mlmeAssociateIndication(ADDRESS* par_pDevAddr, BYTE CapInfor,
                             BOOL SecurityUse, UINT8 AclEntry)
{
    /***** Variable declaration *****/
    MSG ASSO_INDI_DATA*      pAssIndication;
    /***** Function body *****/
    pAssIndication = mem_getMem(sizeof(MSG ASSO_INDI_DATA));
    if (pAssIndication)
    {
        pAssIndication->devAddr      = par_pDevAddr->Extended;
        pAssIndication->CapacityInfor = CapInfor;
        pAssIndication->Security      = SecurityUse;
        pAssIndication->aclEntry      = AclEntry;

        nwk_sendStatMsg(MSG_MAC_INDI ASSO,SUCCESS,pAssIndication);
    }
} // End of functions

/*****
//@FunctionName:
*/

//@Description:
//
//@Param
//
*****/
void mlmeOrphanIndication(QWORD* pOrphanAddress, BOOL securityUse, UINT8 aclEntry)
{
    nwk_sendStatMsg(MSG_MAC_INDI_ORPHAN,SUCCESS,pOrphanAddress);
}
#endif //!ZIGBEE_DEVICE_FFD == 1

/*****
//@FunctionName:
*/

//@Description:
//
//@Param
//
*****/
void nwk_processAssConfirm(NWK_MSG*      pMsg)

```

```

{
    /***** Variable declaration *****/

    /***** Function body *****/
    if (pMsg->Status == SUCCESS){
    if (NwkInfo.CurRequest == NWK_STATE_JOIN)
    {
        if (pMsg->Data != INVALID_SHORT_ADDR)
        {
            /*UPdated on association*/
            NwkInfo.pParent->Relationship = NWK_PARENT; /*Set parent*/
            NwkInfo.pParent->PermitJoining = TRUE;
            NwkInfo.NwkAddr = pMsg->Data;
            NwkInfo.NwkState = NWK_STATE_JOIN_SUCCESS;
            nlmeGetRequest(MAC_COORD_EXTENDED_ADDRESS,&NwkInfo.pParent-
>ExtendedAddress);

            if (NwkInfo.NwkJoinedDeviveType == ZIGBEE_ENDPOINT)
                NwkInfo.NwkState = NWK_STATE_NORMAL_MODE; /*Ready*/

            nlmeJoinConfirm(NwkInfo.NwkPANId, nwk_SUCCESS);
            return;
        }
    }
}

NwkInfo.CurRequest = NWK_STATE_INIT;
NwkInfo.pParent->Relationship = NWK_NO_RELATION; /*Set no relationship*/
NwkInfo.pParent->PermitJoining = FALSE;
nlmeJoinConfirm(NwkInfo.NwkPANId, pMsg->Status);
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processScanConfirm(NWK_MSG* pMsg)
{
    /***** Variable declaration *****/
    MAC_SCAN_RESULT *pScanResult;
    /***** Function body *****/
    pScanResult = (MAC_SCAN_RESULT*)pMsg->Data;
    if (pMsg->Status == SUCCESS)
    {
        #if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR /* Code for PAN Coordinator only */
        if (NwkInfo.CurRequest == NWK_STATE_FORMATION)
            nlme_formNwk();
        #endif //! ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR

        if (NwkInfo.CurRequest== NWK_STATE_DISCOVERY)
        {

```



```

        nlmeNetworkDiscoveryConfirm(nwk_desc, nwk_SUCCESS);
    }

    if (NwkInfo.CurRequest == NWK_STATE_REJOIN)
    {
        nlmeJoinConfirm(NwkInfo.NwkPANId, nwk_SUCCESS);
        NwkInfo.NwkState = NWK_STATE_JOIN_SUCESS;
    }
}
else
{
    #if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR /* Code for PAN Coordinator only */
    if (NwkInfo.CurRequest == NWK_STATE_FORMATION)
        nlme_formNwk();
    #endif // !ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR
    if (NwkInfo.CurRequest == NWK_STATE_DISCOVERY)
        nlmeNetworkDiscoveryConfirm(0, NULL, pMsg->Status);

    if (NwkInfo.CurRequest == NWK_STATE_REJOIN)
        nlmeJoinConfirm(NwkInfo.NwkPANId, nwk_NO_NETWORKS);
}

/*Free the memory*/
mem_freeMem(pScanResult);
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processDisAssConfirm(NWK_MSG* pMsg)
{
    /****** Variable declaration *****/

    /****** Function body *****/
    if(pMsg->Status == SUCCESS){
        nlmeLeaveConfirm(NULL, nwk_SUCCESS); /*TBD*/
    }
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processPollConfirm(NWK_MSG* pMsg)
{
    /****** Variable declaration *****/
    /****** Function body *****/
    if (pMsg->Status == SUCCESS)
        nlmeSyncConfirm(nwk_SUCCESS);
}

```

```

    else
        nlmeSyncConfirm(nwk_SYNC_FAILURE);
} // End of functions

/*****
//@FunctionName:
                                */
//@Description:
//                                */
//@Param
//                                */
/*****/
#if ZIGBEE_DEVICE_FFD == TRUE
void nwk_processStartConfirm(NWK_MSG* pMsg)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    if (pMsg->Status == SUCCESS)
    {
        if (NwkInfo.CurRequest == NWK_STATE_FORMATION)
        {
            NwkInfo.NwkState = NWK_STATE_NORMAL_MODE;
            NwkInfo.NwkJoinedDeviveType = ZIGBEE_PAN_COOR;
            nlmeNetworkFormationConfirm(nwk_SUCCESS);
        }
        if (NwkInfo.CurRequest == NWK_STATE_START_ROUTER)
        {
            NwkInfo.NwkState = NWK_STATE_NORMAL_MODE;
            NwkInfo.NwkJoinedDeviveType = ZIGBEE_ROUTER;
            nlmeStartRouterConfirm(nwk_SUCCESS);
        }
    }
}
else
    nlmeNetworkFormationConfirm(nwk_STARTUP_FAILURE);
} // End of functions
#endif

/*****
//@FunctionName:
                                */
//@Description:
//                                */
//@Param
//                                */
/*****/
void nwk_processBeaconIndication(NWK_MSG* pMsg)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    if (NwkInfo.CurRequest == NWK_STATE_DISCOVERY)
        nwk_updateBeacon((MLME_BEACON_NOTIFY_INDICATION*)pMsg->Data);
} // End of functions

#if ZIGBEE_DEVICE_FFD == TRUE
/*****/
//@FunctionName:

```

```

                                                                    */
//@Description:                                                                    */
//                                                                    */
//@Param                                                                    */
//                                                                    */
/*****/
void nwk_processAssolIndication(NWK_MSG* pMsg)
{
    /***** Variable declaration *****/
    CAPACITY_INFO* pCap;
    MSG ASSO_INDI_DATA* pAssolIndi;
    WORD NwkAddr;
    NWK_STATUS Status;
    /***** Function body *****/
    pAssolIndi = (MSG ASSO_INDI_DATA*)pMsg->Data;
    /* Check if device can be associated */
    /* Add to neighbor table */

    pCap = &pAssolIndi->CapacityInfor;

    if (((pCap->DeviceType==FALSE) && !NwkInfo.BcnPayload.EndDeviceCapacity)
        || ((pCap->DeviceType==TRUE) && !NwkInfo.BcnPayload.RouterCapacity))
    {
        NwkAddr = INVALID_SHORT_ADDR;
        Status = DEV_NOT_PERMIT;
    }
    else
    {
        NwkAddr = nwk_getShortAddrInNT(&pAssolIndi->devAddr);
        if (NwkAddr == INVALID_SHORT_ADDR)
        {
            NwkAddr = routing_checkFreeAddr(pCap->DeviceType);
            if (NwkAddr)
            {
                NwkAddr = routing_allocDistAddr(pCap->DeviceType);
                Status = nwk_addNewChild( &pAssolIndi->devAddr, NwkAddr, *pCap,
                                         pAssolIndi->Security, pAssolIndi->aclEntry);
            }
            if(Status == nwk_TABLE_FULL || NwkAddr == INVALID_SHORT_ADDR)
            {
                NwkAddr = INVALID_SHORT_ADDR;
                Status = DEV_AT_CAP;
                nwk_inhibitJoin(TRUE);
            }
            else
            {
                nlmeJoinIndication(NwkAddr,&pAssolIndi->devAddr,pAssolIndi->CapacityInfor);
            }
        }
        else
            Status = nwk_SUCCESS;
    }

    mlmeAssociateResponse(&pAssolIndi->devAddr,(WORD)NwkAddr,Status,FALSE);
    /* Table full or no available nwk addr: Discard the association cmd */
    mem_freeMem(pMsg->Data);
} // End of functions

```



```
#endif //ZIGBEE_DEVICE_FFD == 1

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processNwkEvent(NWK_MSG* pMsg, void* arg)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    switch(pMsg->MsgType)
    {
        /* Confirmation */
        case MSG_MAC_CONFIRM_ASSOCIATION:
            nwk_processAssConfirm(pMsg);
            break;
        case MSG_MAC_CONFIRM_SCAN:
            nwk_processScanConfirm(pMsg);
            break;
        case MSG_MAC_CONFIRM_DATA:
            nwk_processDataConfirm(pMsg);
            break;
        case MSG_MAC_CONFIRM_DISASSOCIATION:
            nwk_processDisAssConfirm(pMsg);
            break;
        case MSG_MAC_CONFIRM_RESET:
            nlme_resetNwk(pMsg->Status);
            break;
        case MSG_MAC_CONFIRM_POLL:
            nwk_processPollConfirm(pMsg);
            break;

        case MSG_MAC_CONFIRM_START:
            #if ZIGBEE_DEVICE_FFD == TRUE
                nwk_processStartConfirm(pMsg);
            #endif
            break;
        /* Indication */
        case MSG_MAC_INDI_DATA:
            nwk_processDataIndication(pMsg);
            break;
        case MSG_MAC_INDI_BEACON:
            nwk_processBeaconIndication(pMsg);
            break;
        case MSG_MAC_INDI ASSO:
            #if ZIGBEE_DEVICE_FFD == TRUE
                nwk_processAssolIndication(pMsg);
            #endif
            break;
        case MSG_MAC_INDI_LOSS:
            if (pMsg->Data == BEACON_LOSS)
                nlmeSyncIndication();
            break;
    }
}
```

```

    case MSG_MAC_INDI_DISASSOC:
        /*TBD*/
        break;
    case MSG_BT_JITER_TIMEOUT:
    case MSG_BT_1s_TIMEOUT:
        nwk_processBtEvent(pMsg);
        break;
    default:
        break;
}
} // End of functions

```

9.5.4 NWK Common

```

/*****
//Module: NWK
//File Name: _nwk_common
//Description:

//-----*/
//Revision History:
//Date          Programmer          Details
*/
//06/14/2007
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
/*          Include files
*/

/*****
#include "nwk_common.h"
#include "nwk.h"
#include "utils.h"
#include "ucos_ii.h"
#include "zconf.h"
*****/

/*          Constant declaration
*/

/*****
/*          Type Definition
*/

/*****
/*          Variable declaration
*/

/* Network layer information*/
NWK_INFO          NwkInfo;
// Declaration: Network Information Base
NWK_IB            nwk_ib;

```

```

// Declaration: Network Descriptor
NWK_DESC*          nwk_desc[MAX_NWK_DESC_SIZE];
// Declaration: Array of struct for Neighbor Table of size NWK_NEIGHBOR_TBLE_SIZE
NWK_NEIGHBOR_NODE*  nwk_neighborTable[NWK_NEIGHBOR_TBLE_SIZE];

/* Current child index
*/
UINT8              nCurrentChild = NWK_NEIGHBOR_TBLE_SIZE;

/* From Zdo */
extern             UINT8          confPermitJoinDuration;
/*****
/*                               Static Function                               */
*****/

/*****
/*                               Function                               */
*****/

/*****
//@FunctionName: This function will be called in the scanning procedure to
//                               update the neighbor table and network table
//                               */
//@Description:
//                               */
//@Param
//                               */
*****/
void nwk_updateBeacon(MLME_BEACON_NOTIFY_INDICATION* BcnNotif)
{
    /***** Variable declaration *****/
    SUPER_FRAME*    superFrm;
    UINT8            index;
    /***** Function body *****/
    /*Check if neighbors exist */
    superFrm = (SUPER_FRAME*)&BcnNotif->PanDescriptor.SuperframeSpec;
    for (index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
    {
        if (nwk_neighborTable[index] != NULL)
        {
            if (((BcnNotif->PanDescriptor.CoordAddrMode == AM_SHORT_16) &&
                (BcnNotif->PanDescriptor.CoordAddress.Short == nwk_neighborTable[index]-
>NetworkAddress)) ||
                ((BcnNotif->PanDescriptor.CoordAddrMode == AM_EXTENDED_64) &&
                (BcnNotif->PanDescriptor.CoordAddress.Extended == nwk_neighborTable[index]-
>ExtendedAddress )))
                return; /*Existed*/
        }
    }

    /*Get free entry*/
    for (index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
        if (nwk_neighborTable[index] == NULL)
            break;

    /* Check */
    if (index == NWK_NEIGHBOR_TBLE_SIZE){

```

```

        // omit the beacon
    }
    else {
        //Update Neighbor Table
        nwk_neighborTable[index] =
        (NWK_NEIGHBOR_NODE*)mem_getMem(sizeof(NWK_NEIGHBOR_NODE));
        if (nwk_neighborTable[index])
        {
            nwk_neighborTable[index]->PanID = BcnNotif-
>PanDescriptor.CoordPANId; //required
            if (BcnNotif->PanDescriptor.CoordAddrMode == AM_SHORT_16)
            {
                nwk_neighborTable[index]->NetworkAddress = BcnNotif-
>PanDescriptor.CoordAddress.Short; //required
                nwk_neighborTable[index]->ExtendedAddress = INVALID_EXT_ADDR;
            }
            if (BcnNotif->PanDescriptor.CoordAddrMode == AM_EXTENDED_64)
            {
                nwk_neighborTable[index]->NetworkAddress = INVALID_SHORT_ADDR;
                nwk_neighborTable[index]->ExtendedAddress = BcnNotif-
>PanDescriptor.CoordAddress.Extended;
            }
            if (superFrm->PanCoord == TRUE)
                nwk_neighborTable[index]->DeviceType = ZIGBEE_PAN_COOR;
            else
                nwk_neighborTable[index]->DeviceType = ZIGBEE_ROUTER; //required

            //RxOnWhenIdle only router in nonbeacon or coord.
            nwk_neighborTable[index]->RxOnWhenIdle = TRUE;(((superFrm-
>BeaconOrder == 15) || (superFrm->PanCoord == TRUE));

            nwk_neighborTable[index]->Relationship = NWK_NO_RELATION;

            //required
            nwk_neighborTable[index]->Depth = BcnNotif-
>Payload.DeviceDepth;
            nwk_neighborTable[index]->BeaconOrder = superFrm->BeaconOrder;

            nwk_neighborTable[index]->PermitJoining = superFrm->AssoPermit;
            #if ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER
            nwk_neighborTable[index]->PotentialParent = ((BcnNotif->Payload.RouterCapacity ==
TRUE)
|| (BcnNotif-
>Payload.EndDeviceCapacity == TRUE));
            #elif ZIGBEE_DEVICE_TYPE == ZIGBEE_ENDPOINT
            nwk_neighborTable[index]->PotentialParent = (BcnNotif->Payload.EndDeviceCapacity ==
TRUE);
            #else
            nwk_neighborTable[index]->PotentialParent = FALSE; /*Pan coord.*/
            #endif

            nwk_neighborTable[index]->LogicalChannel = BcnNotif-
>PanDescriptor.LogicalChannel;
            nwk_neighborTable[index]->IncomingBeaconTimestamp = BcnNotif-
>PanDescriptor.TimeStamp;
            nwk_neighborTable[index]->BeaconTxTimeOffset = BcnNotif->Payload.TxOffset;

```

```

    }
  }

  /* If we are in the discovery */
  if (NwkInfo.CurRequest == NWK_STATE_DISCOVERY)
  {
      /* Update Network Description */
      for (index = 0; index < MAX_NWK_DESC_SIZE; index++)
      {
          if (nwk_desc[index] != NULL &&
              nwk_desc[index]->PanID == BcnNotif->PanDescriptor.CoordPANId)
          {
              /*Update the network*/
              if (superFrm->AssoPermit)
                  nwk_desc[index]->PermitJoining = TRUE;

              return;
          }
      }

      for (index = 0; index < MAX_NWK_DESC_SIZE; index++)
      {
          if (nwk_desc[index] == NULL)
              break;
          if (index < MAX_NWK_DESC_SIZE)
          {
              nwk_desc[index] = mem_getMem(sizeof(NWK_DESC));
              if (nwk_desc[index])
              {
                  nwk_desc[index]->PanID = BcnNotif->PanDescriptor.CoordPANId;
                  nwk_desc[index]->LogicalChannel = BcnNotif->
>PanDescriptor.LogicalChannel;
                  nwk_desc[index]->StackProfile = BcnNotif->Payload.StackProfile;
                  nwk_desc[index]->ZigbeeVersion = BcnNotif->
>Payload.nwkProtocolVersion;
                  nwk_desc[index]->BeaconOrder = superFrm->BeaconOrder;
                  nwk_desc[index]->SuperframeOrder = superFrm->SuperFrmOrder;
                  nwk_desc[index]->PermitJoining = superFrm->AssoPermit;
              }
          }
      }
  }

  } // End of functions

  /*****
  // @FunctionName: This function will check in the nwk Description table if
  // there exist such a network with the same PANID
  // @Description:
  //
  // @Param
  //
  *****/
  BOOL nwk_checkNwkExisted(UINT16 PanId)
  {
      /***** Variable declaration *****/
      UINT8 index;
      /***** Function body *****/

      for (index = 0 ; index < MAX_NWK_DESC_SIZE; index++){
          if (nwk_desc[index] != NULL &&
              (UINT16)nwk_desc[index]->PanID == (UINT16)PanId){

```



```

        return TRUE;
    }
}

return FALSE;
} // End of functions

/*****
//@FunctionName: This function will check if we should chose the channel for
//                a new network. The choice depends on implementation
//@Description:
//
//@Param
//
*****/
UINT8 nwk_choseChannel(UINT32 ScanChannel)
{
    /***** Variable declaration *****/
    return DEFAULT_CHANNEL;
    /***** Function body *****/
} // End of functions

/*****
//@FunctionName: This function will search in the neighbor table a router
//                to join
//
//@Description:
//
//@Param
//
*****/
NWK_NEIGHBOR_NODE* nwk_findRouterInNT(UINT16 PanId, DWORD ScanChannels)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    ScanChannels = 0;
    /*TBD: consider the link quality*/
    for (UINT8 index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++){
        if ( (nwk_neighborTable[index] != NULL)
            && (nwk_neighborTable[index]->PanID == PanId)
            && (nwk_neighborTable[index]->PermitJoining)
            && (nwk_neighborTable[index]->PotentialParent))
            return nwk_neighborTable[index];
    }
    return NULL;
} // End of functions

/*****
//@FunctionName: This function will search in the neighbor table a node that
//                has the same 64-bits address
//@Description:
//
//@Param
//
*****/
NWK_NEIGHBOR_NODE* nwk_findInNT(WORD NwkAddr)
{

```

```

/***** Variable declaration *****/
/***** Function body *****/
for (UINT8 counter = 0; counter < NWK_NEIGHBOR_TBLE_SIZE ; counter++)
{
    if ( nwk_neighborTable[counter]
    && nwk_neighborTable[counter]->NetworkAddress == NwkAddr){
        return nwk_neighborTable[counter];
    }
}

return NULL;
} // End of functions

/*****
//@FunctionName: This function will search in the neighbor table a node that
//                has the same 64-bits address
//@Description:
//
//@Param
//
*****/
NWK_NEIGHBOR_NODE* nwk_findNodeInNT(ADDRESS* pExAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    for (UINT8 counter = 0; counter < NWK_NEIGHBOR_TBLE_SIZE ; counter++)
    {
        if ( nwk_neighborTable[counter]
        && nwk_neighborTable[counter]->ExtendedAddress == pExAddr->Extended){
            return nwk_neighborTable[counter];
        }
    }

    return NULL;
} // End of functions

/*****
//@FunctionName:
//
//@Description:
//
//@Param
//
*****/
UINT16 nwk_findExtAddrInNT(ADDRESS* pExAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    for (UINT8 counter = 0; counter < NWK_NEIGHBOR_TBLE_SIZE ; counter++)
    {
        if ( nwk_neighborTable[counter]
        && nwk_neighborTable[counter]->ExtendedAddress == pExAddr->Extended){
            return nwk_neighborTable[counter]->NetworkAddress;
        }
    }

    return INVALID_EXT_ADDR;
} // End of functions

```

```

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
/*****
UINT8 nwk_findIndexInNT(UINT8 AddrMode, ADDRESS* pSrcAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    for (UINT8 counter = 0; counter < NWK_NEIGHBOR_TBLE_SIZE ; counter++)
    {
        if ( nwk_neighborTable[counter])
        {
            if ((AddrMode == AM_SHORT_16)
                &&(nwk_neighborTable[counter]->NetworkAddress == pSrcAddr->Short))
                return counter;
            if ((AddrMode == AM_EXTENDED_64)
                &&(nwk_neighborTable[counter]->ExtendedAddress == pSrcAddr->Extended))
                return counter;
        }
    }
    return NWK_NEIGHBOR_TBLE_SIZE;
} // End of functions

/*****
//@FunctionName: This function check if a node is parent
//@Description:
//
//@Param
//
/*****
BOOL nwk_IsParent(ADDRESS* pExAddr)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    for (UINT8 index = 0 ; index < NWK_NEIGHBOR_TBLE_SIZE; index++){
        if (nwk_neighborTable[index]
            && ((nwk_neighborTable[index]->ExtendedAddress == pExAddr->Extended) ||
                (nwk_neighborTable[index]->NetworkAddress == pExAddr->Short))
            && (nwk_neighborTable[index]->Relationship == NWK_PARENT))
            return TRUE;
    }

    return FALSE;
} // End of functions
#if ZIGBEE_DEVICE_FFD == 1 /* Code for FFD only */
/*****
//@FunctionName: This function check if a node is child
//@Description:
//
//@Param

```

```

//                                                                 */
/*****/
BOOL nwk_IsChild(ADDRESS* pExAddr)
{
    /***** Variable declaration *****/
    UINT8 index;
    /***** Function body *****/
    for (index = 0 ; index < NWK_NEIGHBOR_TBLE_SIZE; index++){
        if (nwk_neighborTable[index]
            && ((nwk_neighborTable[index]->ExtendedAddress == pExAddr->Extended) ||
              (nwk_neighborTable[index]->NetworkAddress == pExAddr->Short))
            && (nwk_neighborTable[index]->Relationship == NWK_CHILD))
            return TRUE;
    }

    return FALSE;
} // End of functions
#endif // !ZIGBEE_DEVICE_FFD == 1 /* Code for FFD only */
/*****/
//@FunctionName: This function will get extended addr from short add of
//               a node in the NT (neighbors table) */
//@Description:
// */
//@Param */
// */
/*****/
ADDRESS* nwk_getExAddInNT(UINT16 ShortAddr)
{
    /***** Variable declaration *****/
    UINT8 index;
    /***** Function body *****/
    for (index = 0 ; index < NWK_NEIGHBOR_TBLE_SIZE; index++){
        if (nwk_neighborTable[index]
            && nwk_neighborTable[index]->NetworkAddress == ShortAddr)
            return (ADDRESS*)&nwk_neighborTable[index]->ExtendedAddress;
    }

    return NULL;
} // End of functions

/*****/
//@FunctionName: This function will get extended addr from short add of
//               a node in the NT (neighbors table) */
//@Description:
// */
//@Param */
// */
/*****/
UINT16 nwk_getShortAddInNT(ADDRESS* pExAddr)
{
    /***** Variable declaration *****/
    UINT8 index;
    /***** Function body *****/
    for (index = 0 ; index < NWK_NEIGHBOR_TBLE_SIZE; index++){
        if (nwk_neighborTable[index]
            && nwk_neighborTable[index]->ExtendedAddress == pExAddr->Extended)

```

```

        return nwk_neighborTable[index]->NetworkAddress;
    }

    return INVALID_SHORT_ADDR;
} // End of functions

/*****
//@FunctionName: Set relationship to a node in the NT
//@Description:
//
//@Param
//
*****/
void nwk_setRelationship(QWORD* pExAddr, NWK_NEIG_RELATION Relationship)
{
    /***** Variable declaration *****/
    UINT8 index;
    /***** Function body *****/
    for (index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++){
        if (nwk_neighborTable[index]
            && nwk_neighborTable[index]->ExtendedAddress == *pExAddr)
        {
            nwk_neighborTable[index]->Relationship = Relationship;
            break;
        }
    }
} // End of functions

#if ZIGBEE_DEVICE_FFD == TRUE
/*****
//@FunctionName: This function will remove a child in the NT due to a
//               disassociation cmd
//@Description:
//
//@Param
//
*****/
NWK_STATUS nwk_removeChild(QWORD* pDevAddress)
{
    /***** Variable declaration *****/
    UINT8 index;
    /***** Function body *****/

    /*Get the child */
    for (index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
    {
        if (nwk_neighborTable[index] != NULL
            && nwk_neighborTable[index]->ExtendedAddress == *pDevAddress)
        {
            mem_freeMem(nwk_neighborTable[index]);
            nwk_neighborTable[index] = NULL;

            /* not FULL: Change permit joint */
            if (confPermitJoinDuration == 0x00)
            {
                index = 1;
            }
        }
    }
}

```

```

        mlmeSetRequest(MAC_ASSOCIATION_PERMIT,&index);
    }

    return nwk_SUCCESS;
}

return nwk_UNKNOWN_DEVICE;
} // End of functions

/*****
//@FunctionName: Add new child in the NT
//@Description:
//
//@Param
//
*****/
NWK_STATUS nwk_addNewChild(QWORD* pDevAddress, WORD NwkAddr,CAPACITY_INFO CapInfor,
                          BOOL securityUse, UINT8 aclEntry)
{
    /***** Variable declaration *****/
    UINT8 nFreeIndex;
    UINT8 index;
    /***** Function body *****/
    nFreeIndex = NWK_NEIGHBOR_TBLE_SIZE;

    /*Get free entry*/
    for (index = 0;index<NWK_NEIGHBOR_TBLE_SIZE;index++)
    {
        if (nFreeIndex == NWK_NEIGHBOR_TBLE_SIZE
            && nwk_neighborTable[index] == NULL)
            nFreeIndex = index;
        if (nwk_neighborTable[index] != NULL
            && nwk_neighborTable[index]->ExtendedAddress == *pDevAddress)
        {
            return nwk_ALREADY_PRESENT;
        }
    }
    /* Check */
    if (nFreeIndex < NWK_NEIGHBOR_TBLE_SIZE)
    {
        nwk_neighborTable[nFreeIndex] =
        (NWK_NEIGHBOR_NODE*)mem_getMem(sizeof(NWK_NEIGHBOR_NODE));
        if (nwk_neighborTable[nFreeIndex])
        {
            nwk_neighborTable[nFreeIndex]->PanID = NwkInfo.NwkPANId;
            nwk_neighborTable[nFreeIndex]->ExtendedAddress = *pDevAddress;
            //required
            nwk_neighborTable[nFreeIndex]->NetworkAddress = NwkAddr;
            //required
            if (CapInfor.DeviceType)
                nwk_neighborTable[nFreeIndex]->DeviceType =
                ZIGBEE_ROUTER; //required
            else
                nwk_neighborTable[nFreeIndex]->DeviceType = ZIGBEE_ENDPOINT;
            //required
        }
    }
}

```

```

        nwk_neighborTable[nFreeIndex]->RxOnWhenIdle          =
    CapInfor.RxOnWhenIdle;
        nwk_neighborTable[nFreeIndex]->Relationship          =   NWK_CHILD;
        nwk_neighborTable[nFreeIndex]->Depth                =
    NwkInfo.BcnPayload.DeviceDepth+1;
        nwk_neighborTable[nFreeIndex]->BeaconOrder           =   NwkInfo.NwkBeaconOrder;
        nwk_neighborTable[nFreeIndex]->PermitJoining          =   (CapInfor.DeviceType ==
TRUE); /*Need to check*/
        nwk_neighborTable[nFreeIndex]->PotentialParent       =   FALSE;
        nwk_neighborTable[nFreeIndex]->LogicalChannel         =   NwkInfo.NwkChannel;
        nwk_neighborTable[nFreeIndex]->IncomingBeaconTimestamp =   0x00;
/*Not used for child*/
        nwk_neighborTable[nFreeIndex]->BeaconTxTimeOffset     =   0x00;
/*Not used for child*/
        return nwk_SUCCESS;
    }
}

    return nwk_TABLE_FULL;
} // End of functions

/*****
**/
//@FunctionName: This function will return next child in the NT
//@Description:
//
//@Param
//
/*****
NWK_NEIGHBOR_NODE*      nwk_getFirstChild(BOOL bFirstChild)
{
    /***** Variable declaration *****/

    /***** Function body *****/
    /* Reset current child index */
    nCurrentChild = NWK_NEIGHBOR_TBLE_SIZE;
    for (UINT8 index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
    {
        if (nwk_neighborTable[index] != NULL
            && nwk_neighborTable[index]->Relationship == NWK_CHILD)
        {
            nCurrentChild = index;
            return nwk_neighborTable[index];
        }
    }
    return NULL;
} // End of functions

/*****
**/
//@FunctionName: This function will return next child in the NT
//@Description:
//
//@Param
//
/*****
NWK_NEIGHBOR_NODE*      nwk_getNextChild(BOOL bFirstChild)
{

```

```

    /***** Variable declaration *****/

    /***** Function body *****/
    for (UINT8 index = nCurrentChild; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
    {
        if (nwk_neighborTable[index] != NULL
            && nwk_neighborTable[index]->Relationship == NWK_CHILD)
        {
            nCurrentChild = index;
            return nwk_neighborTable[index];
        }
    }
    return NULL;
} // End of functions

#endif //ZIGBEE_DEVICE_FFD == TRUE

/*****
//@FunctionName: This function will check if there exist such a nwk in the
//                network descriptor. After that, it will release the memory
//                holding by nwk desc
*/
//@Description:
//
//@Param
//
/*****
BOOL nwk_checkNwk(WORD PanID, DWORD ChannelList)
{
    /***** Variable declaration *****/
    BOOL bReturn;
    /***** Function body *****/
    ChannelList = 0;
    bReturn = FALSE;
    for (UINT8 index = 0; index < MAX_NWK_DESC_SIZE; index++){
        if (nwk_desc[index] != NULL)
        {
            if(nwk_desc[index]->PanID == PanID)
                bReturn = TRUE;

            /*End of the discovery, remove the memory*/
            mem_freeMem(nwk_desc[index]);
        }
    }

    return bReturn;
} // End of functions

#if ZIGBEE_DEVICE_FFD == TRUE
/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
/*****

```



```

/*****
void nwk_inhibitJoin(BOOL bInhibit)
{
    /***** Variable declaration *****/
    UINT8 nValue;
    /***** Function body *****/
    NwkInfo.BcnPayload.EndDeviceCapacity = 0;
    NwkInfo.BcnPayload.RouterCapacity = 0;
    /* FULL: Change permit joint */
    nValue = 0;
    mlmeSetRequest(MAC_ASSOCIATION_PERMIT,&nValue);
} // End of functions

/*****
//@FunctionName:
                                */
//@Description:
//
//@Param
//
/*****
void nwk_schedulBeacon()
{
    /***** Variable declaration *****/
    UINT32 nOffset;
    UINT32 nBeaconDuration;
    UINT32 nSFDuration;
    UINT32 nNumSlot;
    UINT32 EarliestStamp;
    UINT8 nTemp;
    UINT32* Slots;
    /***** Function body *****/
    /*Calculate start time*/
    DISABLE_GLOBAL_INT();
    nOffset = OSTmrTime;
    ENABLE_GLOBAL_INT();

    Slots = (UINT32*)mem_getMem(NWK_NEIGHBOR_TBLE_SIZE*sizeof(UINT32));
    if (Slots == NULL) /*Unrecoverable error*/
        err(ERR_UNRECOVERABLE,NULL,NULL,0);

    /*Get the earliest beacon*/
    /*Calculate the current slot*/
    EarliestStamp = nOffset;
    for (UINT8 index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
    {
        if (nwk_neighborTable[index] != NULL
            && (nwk_neighborTable[index]->DeviceType == ZIGBEE_ROUTER ||
                nwk_neighborTable[index]->DeviceType == ZIGBEE_PAN_COOR ))
        {
            /*Offset from beginning of beacon of the parent*/
            if ( EarliestStamp >= nwk_neighborTable[index]->IncomingBeaconTimestamp)
                EarliestStamp = nwk_neighborTable[index]->IncomingBeaconTimestamp;
        }
    }
}

```

```

}

nBeaconDuration = ((UINT32)(aBaseSuperframeDuration / aUnitBackoffPeriod) << NwkInfo.NwkBeaconOrder);
nSFDuration      = ((UINT32)(aBaseSuperframeDuration / aUnitBackoffPeriod) <<
NwkInfo.NwkSuperFrmOrder);

/*Calculate the current slot*/
for (UINT8 index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
{
    if (nwk_neighborTable[index] != NULL
        && (nwk_neighborTable[index]->DeviceType == ZIGBEE_ROUTER ||
            nwk_neighborTable[index]->DeviceType == ZIGBEE_PAN_COOR ))
    {
        /*Offset from beginning of beacon of the parent*/
        Slots[index] = nwk_neighborTable[index]->IncomingBeaconTimestamp - EarliestStamp;
        Slots[index] = Slots[index]%nBeaconDuration;
        Slots[index] = Slots[index]/nSFDuration+1;
    }
    else
        Slots[index] = 0;
}

/*Search for free slot*/
nNumSlot = (UINT32)nBeaconDuration/nSFDuration;
if (nNumSlot == 0)
    err(ERR_UNRECOVERABLE,NULL,NULL,0);
for (UINT8 index = 1; index <= nNumSlot; index++)
{
    nTemp = index;
    for (UINT8 i = 0; i < NWK_NEIGHBOR_TBLE_SIZE; i++)
    {
        if (Slots[i] == nTemp) /*Slot used in the neighbors*/
        {
            nTemp = 0;
            break;
        }
    }
    if (nTemp != 0) /*Slot is not used*/
        break;
}

if (nTemp == 0)
    err(ERR_UNRECOVERABLE,NULL,NULL,0);

/*Get offset with earliest stamp*/
nOffset = nOffset - EarliestStamp;
nOffset = nOffset % nBeaconDuration;

/*Check current time with free slot*/
nTemp -=1;
if(nOffset > (UINT32)nSFDuration*nTemp)
    nOffset = (nBeaconDuration - nOffset) + (UINT32)nSFDuration*nTemp;
else
    nOffset = (UINT32)nSFDuration*nTemp - nOffset;

mem_freeMem(Slots);

```

```

    NwkInfo.BeaconOffset = nOffset;
} // End of functions
#endif    ///ZIGBEE_DEVICE_FFD == TRUE

/*****
//@FunctionName:
*/
//@Description:    Assume that maximum is 16 record in the NT
//
//@Param
//
*****/
UINT16 nwk_getNtBitMap()
{
    /***** Variable declaration *****/
    UINT16    Bitmap;
    /***** Function body *****/
    Bitmap = 0;
    for (UINT8    index = 0; index < NWK_NEIGHBOR_TBLE_SIZE; index++)
        if ((nwk_neighborTable[index] != NULL)
            && ((nwk_neighborTable[index]->Relationship == NWK_CHILD) ||
                (nwk_neighborTable[index]->Relationship == NWK_PARENT)))
            Bitmap |= (1<<index);

    return Bitmap;
} // End of functions

```

9.5.5 NWK Data

```

/*****
//Module: NWK
//File Name: nwk_data.c
//Description: This file will implement network data handler
//              - Transmission
//              - Relay data
//
//              - Broadcast data
//-----*/
//Revision History:
//Date              Programmer              Details
*/
//07/16/2007
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
/*              Include files
*/
*****/

#include "types.h"
#include "data_queue.h"
#include "nwk_common.h"
#include "nwk_data.h"
#include "zconf.h"
#include "nwk.h"

```

```

#include "utils.h"
#include "nwk_routing.h"
#include "nwk_msg.h"
#include "schedul.h"
/*****
/*
                                Constant declaration
                                */
/*****
#define APS_HANDLE_OFFSET      2*MAX_NWK_DATA_QUEUE_SIZE
#define NWK_HANDLE             MAX_NWK_DATA_QUEUE_SIZE
#define BT_HANDLE              (MAX_NWK_DATA_QUEUE_SIZE+1)
#define NWK_MAX_BTR            3
/*****
/*
                                Type Definition
                                */
/*****
typedef enum
{
    NWK_DATA_WAIT_SEND = 0x01,
    NWK_DATA_SENDING = 0x02
}NWK_DATA_STATUS;

/*Broadcasting record
*/
typedef struct
{
    P_NPDU      pNpdu;
    UINT8       nLength;
    UINT16      NTIndexBitMap;          /*Assume that neighbor table size < 32*/
    UINT8       SrcIndex;
    UINT8       nwkDeliveryTimeElapsed; /*In second*/
    UINT8       nwkAckElapsed;          /*In second*/
    UINT8       nwkBrcRetries:3;
    UINT8       Handle:5;
}BTR;

/* Broadcasting table
*/
typedef struct
{
    BTR*        Btr[NWK_MAX_BTR];
    UINT8       nCount;
}BTT;

/*****
/*
                                Variable declaration
                                */
/*****
/*Data queue*/
#if ZIGBEE_DEVICE_FFD == 1          /* Code for FFD only */
/* Data queue
*/
DATA_REC      RecList[MAX_NWK_DATA_QUEUE_SIZE];
DATA_QUEUE    NwkDataQueue;
#endif //ZIGBEE_DEVICE_FFD == 1    /* Code for FFD only */
/* Broadcasting table
*/

```

```

BTT                                NwkBtt;
/*****
/*                                Static Function                                */
/*****

/*****
/*                                Function                                */
/*****

/*****
//@FunctionName:
/*
//@Description:
//
//@Param
//
/*****
void                                nwk_dataInit()
{
    /***** Variable declaration *****/

    /***** Function body *****/
    #if ZIGBEE_DEVICE_FFD == 1          /* Code for FFD only */
        /*init data queue*/
        dataqueue_init(&NwkDataQueue,RecList,MAX_NWK_DATA_QUEUE_SIZE);
    #endif /*ZIGBEE_DEVICE_FFD == 1    /* Code for FFD only */
} // End of functions

/*-----

                                Data management function

-----*/

/*****
//@FunctionName: This function will
//
//                                - Format the NPDU data at the beginning of the Nsdu
//                                this memory already spared for NWK header
//
//                                - Get new memory block (32 bytes) for NPDU command, format
//                                it
//@Description:
//
//
//@Param
//
/*****
P_NPDU nwk_makeNpdu(NWK_FRM_TYPE FrmType, BYTE NsduLength, BYTE *Nsdu)
{
    /***** Variable declaration *****/
    P_NPDU npdu = NULL;
    /***** Function body *****/
    if (FrmType == NWK_FRM_DATA){
        npdu = (P_NPDU)((BYTE*)Nsdu-NWK_NPDU_HEADER_SIZE); /*Data convention: pay attention*/
        /* Header */
        npdu->FrmType      =      NWK_FRM_DATA;
        //PAYLOAD: already here

```

```

    return npdu;
  }
  if (FrmType == NWK_FRM_CMD)
  {
    /*TBD*/
  }
  return npdu;
} // End of functions

/*****
//@FunctionName: This function will responsible for sending a single packet */
//@Description: */
// */
//@Param */
// */
*****/
Z_STATUS      nwk_sendUni(P_NPDU pNpdu,UINT8 nLength,UINT8 npduHandle, NWK_NEIGHBOR_NODE*
pNextHop)
{
  /***** Variable declaration *****/
  UINT8      TxOption;
  Z_STATUS    Status;
  UINT16      nPanId;
  QWORD*      pAddr;
  UINT8      AddrMode;
  /***** Function body *****/
  if (pNextHop != NULL)
  {
    if (pNextHop->RxOnWhenIdle)
      TxOption = ACK_TX;
    else if(pNextHop->Relationship == NWK_CHILD)
      TxOption = ACK_TX|IND_TX;
    else
      return nwk_UNKNOWN_DEVICE;

    if (npduHandle == BT_HANDLE) /*No ACK*/
      TxOption &= ~ACK_TX;

    if (NwkInfo.NwkAddr != INVALID_SHORT_ADDR)
    {
      AddrMode    = AM_SHORT_16;
      pAddr       = &NwkInfo.NwkAddr;
    }
    else
    {
      AddrMode    = AM_SHORT_16;
      pAddr       = &aExtendedAddress;
    }

    mlmeGetRequest(MAC_PAN_ID,&nPanId);
    if (pNextHop->NetworkAddress != INVALID_SHORT_ADDR)
      Status = mcpsDataRequest ( AddrMode, nPanId, pAddr,
                                AM_SHORT_16, nPanId, (ADDRESS*)&pNextHop-
                                >NetworkAddress,
                                nLength, (BYTE*) pNpdu, npduHandle, TxOption);
    else

```

```

        Status = mcpsDataRequest ( AddrMode, nPanId, pAddr,
                                AM_EXTENDED_64, nPanId, (ADDRESS*)&pNextHop-
>ExtendedAddress,
                                nLength, (BYTE*) pNpdu, npduHandle, TxOption);

        return Status;
    }
    else
        return nwk_UNKNOWN_DEVICE;
} // End of functions
/*****
//@FunctionName:
                                */
//@Description:
                                */
//
                                */
//@Param
                                */
//
                                */
/*****/
void nwk_sendBroadcast(BTR* pBtr)
{
    /***** Variable declaration *****/
    UINT16 wTemp;
    UINT16 BitMap;
    /***** Function body *****/
    if (pBtr != NULL)
    {
        BitMap = nwk_getNtBitMap();
        for (UINT8 Index = 0; Index < NWK_NEIGHBOR_TBLE_SIZE; Index++)
        {
            wTemp = 1<<Index;
            if ((!(pBtr->NTIndexBitMap&wTemp)) && (BitMap&wTemp))
            {
                /*Check Source*/
                if (Index == pBtr->SrcIndex)
                {
                    wTemp = (UINT16)(1<<pBtr->SrcIndex);
                    pBtr->NTIndexBitMap |= wTemp;
                    nwk_sendUni(pBtr->pNpdu, pBtr->nLength,
                        BT_HANDLE,nwk_neighborTable[Index]);
                }
                else
                    nwk_sendUni(pBtr->pNpdu, pBtr->nLength,
                        BT_HANDLE,nwk_neighborTable[Index]);
            }
        }
    }
} // End of functions

/*****
//@FunctionName: This function will responsible for sending a single packet */
//
// - For relaying data: get a new slot handler and transmit
//
// - For sending data from APS: send it with handler from APS*/
//
// - With NsduHandle = 0: get queued relaying packet and send it*/
//@Description:
                                */
//
                                */
//@Param
                                */
//
                                */
/*****/

```

```

Z_STATUS      nwk_send(P_NPDU pNpdu,UINT8 nLength, UINT8      npduHandle)
{
    /***** Variable declaration *****/
    NWK_NEIGHBOR_NODE* pNextHop;
    /***** Function body *****/
    /*Routing procedure: if node has no routing capacity or
    no path, use tree routing if available: Need to correct this */
    if (nwk_ib.nwkUseTreeRouting)
    {
        #if ZIGBEE_DEVICE_FFD == 1                /* Code for FFD only */
            pNextHop = routing_getNextNodeInHier(pNpdu->DesAddr);
        #else
            pNextHop = NwkInfo.pParent;
        #endif
        if (pNextHop)
            return nwk_sendUni(pNpdu,nLength,npduHandle, pNextHop);
    }
    TOGGLE_OLED();
    /* TBD: Routing table, buffer to wait the route ...*/
    return nwk_UNKNOWN_DEVICE;
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//
//@Param
//
//
*****/
#if ZIGBEE_DEVICE_FFD == 1                /* Code for FFD only */
void nwk_transmitQueuedPkt()
{
    /***** Variable declaration *****/
    Z_STATUS      Status;
    UINT8          npduHandle;
    /***** Function body *****/
    npduHandle = dataqueue_getRec(&NwkDataQueue,NWK_DATA_WAIT_SEND);
    if (npduHandle < NwkDataQueue.Size)
    {
        Status = nwk_send(NwkDataQueue.Reclist[npduHandle].pData,
                           NwkDataQueue.Reclist[npduHandle].nLength,npduHandle/*Old packet*/);
        if (Status == SUCCESS)
            NwkDataQueue.Reclist[npduHandle].Status = NWK_DATA_SENDING;
    }
} // End of functions
#endif //!ZIGBEE_DEVICE_FFD == 1                /* Code for FFD only */

/*****
//@FunctionName: This function will handle broadcasting process
//@Description:
//
//
//@Param
//
//
*****/

```



```

/*****
Z_STATUS      nwk_broadcast(P_NPDU pNpdu,UINT8 nLength, UINT8 AddrMode, ADDRESS*  pSrcAddr, UINT8
nHandle)
{
    /***** Variable declaration *****/
    UINT8      i;
    UINT16     nTemp;
    UINT8      nFreeIndex;
    NWK_MSG     Msg;
    /***** Function body *****/
    /*This function is called for new incoming broadcast packet*/
    /*Check in the BTT*/
    nFreeIndex = NWK_MAX_BTR;
    for (i = 0; i < NWK_MAX_BTR; i++)
    {
        if ((NwkBtt.Btr[i] != NULL)
            && (NwkBtt.Btr[i]->SeqNumber == pNpdu->SeqNumber)
            && (NwkBtt.Btr[i]->SrcAddr == pNpdu->SrcAddr))
        {
            /*Btr existed*/
            break;
        }
        else if ((NwkBtt.Btr[i] == NULL)
            && (nFreeIndex == NWK_MAX_BTR))
            nFreeIndex = i;
    }

    if (i < NWK_MAX_BTR)
    {
        /*Mark device have relayed */
        nFreeIndex = nwk_findIndexInNT(AddrMode, pSrcAddr);
        nTemp = (UINT16)(1<<nFreeIndex);
        NwkBtt.Btr[i]->NTIndexBitMap |= nTemp;
        return nwk_ALREADY_PRESENT;
    }

    if ( (nFreeIndex < NWK_MAX_BTR)&&(i == NWK_MAX_BTR) )    /*record is not existed*/
    {
        /* Create new record*/
        if (pNpdu->Radius > 0)
        {
            NwkBtt.Btr[nFreeIndex] = mem_getMem(sizeof(BTR));
            if (NwkBtt.Btr[nFreeIndex])
            {
                /*Set data for new record*/
                NwkBtt.Btr[nFreeIndex]->nLength      = nLength;
                NwkBtt.Btr[nFreeIndex]->pNpdu        = pNpdu;
                NwkBtt.Btr[nFreeIndex]->NTIndexBitMap = 0;
                NwkBtt.Btr[nFreeIndex]->nwkAckElapsed = 0;
                NwkBtt.Btr[nFreeIndex]->nwkBrcRetries = 0;

                /*Handle*/
                NwkBtt.Btr[nFreeIndex]->Handle = nHandle;

                /*If from others*/
                if (pSrcAddr != NULL)
                    NwkBtt.Btr[nFreeIndex]->SrcIndex = nwk_findIndexInNT(AddrMode, pSrcAddr);
            }
        }
    }
}

```

```

    else
        NwkBtt.Btr[nFreeIndex]->SrcIndex = NWK_NEIGHBOR_TBLE_SIZE;

    /*Create timer*/
    Msg.MsgType = MSG_BT_JITER_TIMEOUT;
    Msg.Status = nFreeIndex;
    Msg.Data = 0;

    mtimer_creatTmr(&Msg,(UINT32)random_getRandomByte()%nwkMaxBroadcastJitter,FALSE,TASK_APER_NWK);
    NwkBtt.Btr[nFreeIndex]->nwkDeliveryTimeElapsed = 0;

    /*Create the periodical timer*/
    if (NwkBtt.nCount == 0)
    {
        Msg.MsgType = MSG_BT_1s_TIMEOUT;
        Msg.Data = 0;
        Msg.Status = 0;
        mtimer_creatTmr(&Msg,(UINT32)1000,TRUE,TASK_APER_NWK);
    }
    NwkBtt.nCount++;
    i= nFreeIndex;

    return SUCCESS; /*New broadcasting packet*/
}
else
    return TRANSACTION_OVERFLOW;
}
else if (pNpdu->Radius == 0)
    return nwk_NOT_PERMITTED;
}
else
    return TRANSACTION_OVERFLOW;
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
Z_STATUS nldeDataRequest(UINT16 DstAddr, BYTE NsduLength, BYTE *Nsdu,
                        BYTE NsduHandle, BYTE Radius, BYTE DiscoverRoute, BOOL
SecurityEnable)
{
    /****** Variable declaration *****/
    P_NPDU pNpdu;
    Z_STATUS Status;
    /****** Function body *****/
    if (NwkInfo.NwkState != NWK_STATE_NORMAL_MODE)
        return nwk_INVALID_REQUEST;
    else
    {
        // TBD: Check NIB if Security is enabled: if NIB = 0 then ignore SecurityEnable

        /* Form the NPDU */

```

```

pNpdu = nwk_makeNpdu(NWK_FRM_DATA, NsduLength, Nsdu);

    if (!pNpdu)
        return nwk_TABLE_FULL;

    /* Header */
    pNpdu->ProtocolVer = nwkProtocolVersion;
    pNpdu->DiscoverRoute = nwk_ib.nwkUseTreeRouting ?
NWK_DISCO_SUPPRESS:DiscoverRoute; /*Use tree routing*/
    if((nwkDefaultSecurityLevel == 0x00) || (NwkInfo.SecurityEnable == FALSE) ||
(SecurityEnable == FALSE)){
        pNpdu->Security = 0x00;
    }
    else {
        pNpdu->Security = 0x00; /* security not implemented */
    }

    //ADDRESS FIELD
    pNpdu->DesAddr = DstAddr;
    pNpdu->SrcAddr = NwkInfo.NwkAddr;
    pNpdu->Radius = Radius;
    pNpdu->SeqNumber = nwk_ib.nwkSequenceNumber++;

    mlmeGetRequest(MAC_SHORT_ADDRESS, &pNpdu->SrcAddr);
    pNpdu->DiscoverRoute = DiscoverRoute;
    pNpdu->Security = FALSE; /*Not implemented*/
    if (Radius)
        pNpdu->Radius = Radius;
    else
        pNpdu->Radius = 2*nwk_ib.nwkMaxDepth;

    if (DstAddr != BROADCAST_ADDR)
        Status = nwk_send(pNpdu, NsduLength+NWK_NPDU_HEADER_SIZE,
NsduHandle+APS_HANDLE_OFFSET);
    else if (DstAddr == BROADCAST_ADDR && pNpdu->Radius > 0)
    {
        Status = nwk_broadcast(pNpdu, NsduLength+NWK_NPDU_HEADER_SIZE, AM_SHORT_16,
NULL, NsduHandle+APS_HANDLE_OFFSET);
    }
    else
        Status = nwk_INVALID_PARAMETER;

    return Status;
    /*No memory handling needed here*/
}
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processDataIndication(NWK_MSG* pMsg)
{

```

```

/***** Variable declaration *****/
MCPS_DATA_INDICATION*      pIndiData;
P_NPDU                      pNpdu;
UINT8                       npduHandle;
Z_STATUS                    Status;
/***** Function body *****/
pIndiData = pMsg->Data;
pNpdu     = (P_NPDU)pIndiData->pMsdu;

/* Decrement the radius*/
pNpdu->Radius--;

/*send packet to upper layer*/
if(pNpdu->DesAddr == NwkInfo.NwkAddr)
{
    if (pNpdu->FrmType == NWK_FRM_DATA)
    {
        nldeDataIndication(AM_SHORT_16,&pNpdu->SrcAddr,          pIndiData->msduLength-
        NWK_NPDU_HEADER_SIZE,
                                &pNpdu->Payload, pIndiData->mpduLinkQuality);
    }
    else if(pNpdu->FrmType == NWK_FRM_CMD)
    {
    }
    mem_freeMem(pNpdu);
}
/*Broadcast the packet*/
else if(pNpdu->DesAddr == BROADCAST_ADDR)
{
    Status = nwk_broadcast( pNpdu,pIndiData->msduLength,
                            pIndiData->srcAddrMode, &pIndiData->srcAddr,NWK_HANDLE);

    if (Status == SUCCESS ||Status == nwk_NOT_PERMITTED)
    {
        if (pNpdu->FrmType == NWK_FRM_DATA)
            nldeDataIndication(AM_SHORT_16,&pNpdu->SrcAddr,          pIndiData->msduLength-
            NWK_NPDU_HEADER_SIZE,
                                    &pNpdu->Payload, pIndiData->mpduLinkQuality);
        else if(pNpdu->FrmType == NWK_FRM_CMD)
        {
            /*Later*/
        }
    }

    if(Status != SUCCESS)
        mem_freeMem(pIndiData->pMsdu); /*no resource or already existed, discard data*/
}
#if ZIGBEE_DEVICE_FFD == TRUE
else if (pNpdu->Radius > 0)
{
    /* Relay packet */
    npduHandle = dataqueue_getFreeSlot(&NwkDataQueue);
    if (npduHandle < NwkDataQueue.Size) /*Not available*/
        Status = nwk_send(pNpdu,pIndiData->msduLength,npduHandle);

    if (npduHandle == NwkDataQueue.Size)
        /*Can not relay packet, discard it*/
        mem_freeMem(pNpdu);
}

```

```

    else
    {
        Status = (Status == SUCCESS)? NWK_DATA_SENDING:NWK_DATA_WAIT_SEND;
        dataqueue_addRec(&NwkDataQueue,pNpdu,pIndiData-
>msduLength,npduHandle,NWK_HANDLE,Status);
    }
}
#else
else
    /*This is end-device, discard it*/
    mem_freeMem(pNpdu);
#endif
    mem_freeMem(pIndiData);/*Free indication msg*/
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processDataConfirm(NWK_MSG* pMsg)
{
    /***** Variable declaration *****/
    DATA_REC* pRec;
    P_NPDU pNpdu;
    /***** Function body *****/
    /*This packet is on nwk duty*/
    if (pMsg->Data < NWK_HANDLE)
    {
        if (pNpdu->SrcAddr == NwkInfo.NwkAddr
            && pNpdu->FrmType == NWK_FRM_CMD)
        {
            /* Process end of transmission of cmd */
            /*Cmd need to be handled in the handler tabld*/
            return;
        }
    }
    #if ZIGBEE_DEVICE_FFD == TRUE
        pRec = &NwkDataQueue.RecList[pMsg->Data];
        pNpdu = pRec->pData;
        /*if broadcast packet*/
        if (pNpdu->DesAddr == BROADCAST_ADDR)
        {
            /*Process broadcast packet*/
        }
        else if (pNpdu->SrcAddr != NwkInfo.NwkAddr
            && pNpdu->FrmType != NWK_FRM_CMD)
        {
            /* Finished relaying data */
            /*remove data handle*/
            dataqueue_removeRec(&NwkDataQueue,pMsg->Data);

            /*release data*/
            mem_freeMem(pNpdu);

```

```

    /* Transmit queued relayed packet*/
    nwk_transmitQueuedPkt();
  }
  else
  {
    /*release data*/
    mem_freeMem(pNpdu);
  }
#endif //ZIGBEE_DEVICE_FFD == TRUE
}
else /*This packet is on APS duty*/
  nldDataConfirm((UINT8)pMsg->Data-APS_HANDLE_OFFSET, pMsg->Status);
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nwk_processBtEvent(NWK_MSG* pMsg)
{
  /***** Variable declaration *****/
  UINT8 npduHandle;
  UINT8 index;
  NWK_MSG Msg;
  /***** Function body *****/
  if (pMsg->MsgType == MSG_BT_JITER_TIMEOUT)
  {
    nwk_sendBroadcast(NwkBtt.Btr[Msg.Status]);
  }
  if (pMsg->MsgType == MSG_BT_1s_TIMEOUT) /*Update the BTT*/
  {
    for (UINT8 i = 0; i < NWK_MAX_BTR; i++)
    {
      if (NwkBtt.Btr[i] != NULL)
      {
        NwkBtt.Btr[i]->nwkAckElapsed++;
        NwkBtt.Btr[i]->nwkDeliveryTimeElapsed++;

        if (NwkBtt.Btr[i]->nwkAckElapsed == nwk_ib.nwkPassiveAckTimeout)
        {
          /*resend*/
          nwk_sendBroadcast(NwkBtt.Btr[i]);
          NwkBtt.Btr[i]->nwkAckElapsed = 0;
          NwkBtt.Btr[i]->nwkBrcRetries++;
        }
        if ((NwkBtt.Btr[i]->nwkDeliveryTimeElapsed == nwk_ib.nwkBroadcastDeliveryTime)
            || (NwkBtt.Btr[i]->nwkBrcRetries == nwk_ib.nwkMaxBroadcastRetries))
          /* remove */
        {
          /*remove the record*/
          if (NwkBtt.Btr[i]->Handle >= APS_HANDLE_OFFSET)
            nldDataConfirm(NwkBtt.Btr[i]->Handle-APS_HANDLE_OFFSET,SUCCESS);
          else

```

```

        mem_freeMem(NwkBtt.Btr[i]->pNpdu);

        mem_freeMem(NwkBtt.Btr[i]);
        NwkBtt.Btr[i] = NULL;

        NwkBtt.nCount--;
        if (NwkBtt.nCount == 0) /*End the periodical timer*/
        {
            Msg.MsgType = MSG_BT_1s_TIMEOUT;
            Msg.Data     = 0;
            Msg.Status    = 0;
            mtimer_endTmr(&Msg);
        }
    }
}
} // End of functions

```

9.5.6 NWK Nlme

```

/*****
//Module: Network Layer
//File Name: nwk_nlme
//Description: This file implement the nwk management entity of the network
//-----*/
//Revision History:
//Date              Programmer          Details
*/
//02/26/2007        Create
//-----*/
// Copyright (C) 2006 - All Rights Reserved
*****/

/*****
/*                      Include files
*/

/*****
#include "nwk.h"
#include "mac_primitives.h"
#include "app_framework.h"
#include "nwk_msg.h"
#include "zdo.h"
#include "zconf.h"
#include "utils.h"
#include "ucos_ii.h"
#include "nwk_routing.h"
*****/

/*                      Constant declaration
*/

/*****
/*                      Type Definition
*/

*****/
#endif ZIGBEE_DEVICE_FFD == 1

```

```

/* Leave transaction information */
typedef struct
{

}NWK_LEAVE_TRANS_INFO;

#endif //!ZIGBEE_DEVICE_FFD == 1

/*****
*                               Variable declaration
*/

/*****
#if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR
extern WORD confPandId;
#endif //!ZIGBEE_DEVICE_FFD == 1

/*****
*                               Static Function
*/

/*****
*                               Function
*/

/*****
//      Function Name:      nlmeGetRequest
//      Description:        This primitive allows the next higher layer of a device can read
//                          and write attributes in the NIB
//      Parameters:         NIBAttribute      -      NWK_IB_ID
//      Return:             nothing
/*****

NWK_STATUS nlmeGetRequest(BYTE NIBAttribute,void *pNIBAttribValue,UINT8* NIBAttribLength){
    switch (NIBAttribute){
        case NWKSEQUENCENUMBER:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkSequenceNumber;
            *NIBAttribLength = sizeof(nwk_ib.nwkSequenceNumber);
            break;
        case NWKPASSIVEACKTIMEOUT:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkPassiveAckTimeout;
            *NIBAttribLength = sizeof(nwk_ib.nwkPassiveAckTimeout);
            break;
        case NWKMAXBROADCASTRETRIES:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxBroadcastRetries;
            *NIBAttribLength = sizeof(nwk_ib.nwkMaxBroadcastRetries);
            break;
        case NWKMAXCHILDREN:
            *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxChildren;
            *NIBAttribLength = sizeof(nwk_ib.nwkMaxChildren);
            break;
    }
}

```



```

case NWKMAXDEPTH:
    *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxDepth;
    *NIBAttribLength = sizeof(nwk_ib.nwkMaxDepth);
    break;
case NWKMAXROUTERS:
    *((UINT8*)pNIBAttribValue) = nwk_ib.nwkMaxRouters;
    *NIBAttribLength = sizeof(nwk_ib.nwkMaxRouters);
    break;
case NWKNEIGHBORTABLE:
    *((QWORD**)pNIBAttribValue) = nwk_ib.nwkNeighborTable;
    *NIBAttribLength = sizeof(nwk_ib.nwkNeighborTable);    /*TBD: update the
length*/

    break;
case NWKNETWORKBROADCASTDELIVERYTIME:
    *((UINT8*)pNIBAttribValue) = nwk_ib.nwkBroadcastDeliveryTime;
    *NIBAttribLength = sizeof(nwk_ib.nwkBroadcastDeliveryTime);
    break;
case NWKREPORTCOSTANTCOST:
    *((UINT8*)pNIBAttribValue) = nwk_ib.nwkReportConstantCost;
    *NIBAttribLength = sizeof(nwk_ib.nwkReportConstantCost);
    break;
case NWKROUTEDISCOVERYRETRIESPERMITTED:
    *((UINT8*)pNIBAttribValue) = nwk_ib.nwkRouteDiscoveryRetriesPermitted;
    *NIBAttribLength = sizeof(nwk_ib.nwkRouteDiscoveryRetriesPermitted);
    break;
case NWKROUTETABLE:
    *((QWORD**)pNIBAttribValue) = nwk_ib.nwkRouteTable;    /*TBD:
update the length*/

    *NIBAttribLength = sizeof(nwk_ib.nwkRouteTable);
    break;
case NWKSYMLINK:
    *((BOOL*)pNIBAttribValue) = nwk_ib.nwkSymLink;
    *NIBAttribLength = sizeof(nwk_ib.nwkSymLink);
    break;

case NWKCAPABILITYINFORMATION:
    *((BYTE*)pNIBAttribValue) = nwk_ib.nwkCapabilityInformation;
    *NIBAttribLength = sizeof(nwk_ib.nwkCapabilityInformation);
    break;
case NWKUSETREEADDRALLOC:
    *((BOOL*)pNIBAttribValue) = nwk_ib.nwkUseTreeAddrAlloc;
    *NIBAttribLength = sizeof(nwk_ib.nwkUseTreeAddrAlloc);
    break;
case NWKUSETREEROUTING:
    *((BOOL*)pNIBAttribValue) = nwk_ib.nwkUseTreeRouting;
    *NIBAttribLength = sizeof(nwk_ib.nwkUseTreeRouting);
    break;
case NWKNEXTADDRESS:
    *((UINT16*)pNIBAttribValue) = nwk_ib.nwkNextAddress;
    *NIBAttribLength = sizeof(nwk_ib.nwkNextAddress);
    break;
case NWKAVAILABLEADDRESSES:
    *((UINT16*)pNIBAttribValue) = nwk_ib.nwkAvailableAddresses;
    *NIBAttribLength = sizeof(nwk_ib.nwkAvailableAddresses);
    break;
case NWKADDRESSINCREMENT:
    *((UINT16*)pNIBAttribValue) = nwk_ib.nwkAddressIncrement;
  
```



```

        *NIBAttribLength = sizeof(nwk_ib.nwkAddressIncrement);
        break;
    case NWKTRANSACTIONPERSISTENCETIME:
        *((UINT16*)pNIBAttribValue) = nwk_ib.nwkTransactionPersistenceTime;
        *NIBAttribLength = sizeof(nwk_ib.nwkTransactionPersistenceTime);
        break;
    default:
        *NIBAttribLength = 0; /*mean UNSUPPORTED_ATTRIBUTE */
        return nwk_UNUNSUPPORTED_ATTRIBUTE;
        break;
    }
    return nwk_SUCCESS;
}

/*****
//      Function Name:      nlmeSetRequest
//      Description:      This primitive allows the next higher layer to write the value of
//                          an attribute into the NIB*/
//      Parameters:      nlmeSetRequest_s - NLME_SETREQUEST
//      Return:      nothing
*****/
NWK_STATUS nlmeSetRequest(BYTE NIBAttribute, UINT8      NIBAttributeLength, void *NIBAttributeValue){

    /***** Function body *****/
    NWK_STATUS      SetStatus;
    /***** Function body *****/
    SetStatus = nwk_SUCCESS;
    if(NIBAttribute == NWKSEQUENCENUMBER){
        nwk_ib.nwkSequenceNumber = *(WORD*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKPASSIVEACKTIMEOUT) {
        nwk_ib.nwkPassiveAckTimeout = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXBROADCASTRETRIES) {
        nwk_ib.nwkMaxBroadcastRetries = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXCHILDREN) {
        nwk_ib.nwkMaxChildren = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXDEPTH) {
        nwk_ib.nwkMaxDepth = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKMAXROUTERS) {
        nwk_ib.nwkMaxRouters = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKNEIGHBORTABLE) {
        nwk_ib.nwkNeighborTable = (void*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKNETWORKBROADCASTDELIVERYTIME) {
        nwk_ib.nwkBroadcastDeliveryTime = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKREPORTCOSTANTCOST) {
        nwk_ib.nwkReportConstantCost = *(UINT8*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKROUTEDISCOVERYRETRIESPERMITTED) {
        nwk_ib.nwkRouteDiscoveryRetriesPermitted = *(UINT8*)NIBAttributeValue;
    }

```

```

    }
    else if(NIBAttribute == NWKSYMLINK) {
        nwk_ib.nwkRouteTable = (void*)NIBAttributeValue;
    }
    else if(NIBAttribute == NWKROUTEDISCOVERYRETRIESPERMITTED) {
        nwk_ib.nwkSymLink = *(UINT8*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKCAPABILITYINFORMATION) {
        nwk_ib.nwkCapabilityInformation = *(UINT8*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKUSETREEADDRALLOC) {
        nwk_ib.nwkUseTreeAddrAlloc = *(UINT8*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKUSETREEROUTING) {
        nwk_ib.nwkUseTreeRouting = *(UINT8*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKNEXTADDRESS) {
        nwk_ib.nwkNextAddress = *(UINT16*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKAVAILABLEADDRESSES) {
        nwk_ib.nwkAvailableAddresses = *(UINT16*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKADDRESSINCREMENT) {
        nwk_ib.nwkAddressIncrement = *(UINT16*)NIBAttributeValue;
    }

    else if(NIBAttribute == NWKTRANSACTIONPERSISTENCETIME) {
        nwk_ib.nwkTransactionPersistenceTime = *(UINT8*)NIBAttributeValue;
    }
    else{
        SetStatus = nwk_UNSUPPORTED_ATTRIBUTE;
    }

    return SetStatus;
}

/*****
//      Function Name:      nlmeResetRequest
//      Description:        This primitive allows the next higher layer to request that the
//                          NWk layer performs a reset operation
//      Parameters:         nothing
//      Return:             nothing
*****/

void nlmeResetRequest(){
    // Initiate MAC RESET
    mlmeResetRequest(TRUE);
}

/*****/

```

```

//@FunctionName:
*/

//@Description:
*/
//
*/
//@Param
*/
//
*/
/*****/
void nlme_resetNwk(NWK_STATUS Status)
{
    /***** Variable declaration *****/
    /***** Function body *****/
    if (Status == nwk_SUCCESS)
    {
        // Set the current state
        NwkInfo.NwkState = NWK_STATE_INIT;

        // Set NIB to default values
        nwk_ib.nwkSequenceNumber =
        (BYTE)random_getRandomByte(); // set to random number
        nwk_ib.nwkPassiveAckTimeout = 0x03;
        nwk_ib.nwkMaxBroadcastRetries = 0x03;
        nwk_ib.nwkMaxChildren =
        NWK_MAX_CHILD;
        nwk_ib.nwkMaxDepth =
        0x05;
        nwk_ib.nwkMaxRouters =
        NWK_MAX_ROUTER;
        nwk_ib.nwkNeighborTable = NULL;
        nwk_ib.nwkBroadcastDeliveryTime =
        nwk_ib.nwkPassiveAckTimeout*nwk_ib.nwkMaxBroadcastRetries; //need to initialize
        nwk_ib.nwkReportConstantCost = 0x00;
        nwk_ib.nwkRouteDiscoveryRetriesPermitted = nwkDiscoveryRetryLimit;
        nwk_ib.nwkRouteTable =
        NULL;
        nwk_ib.nwkSymLink =
        FALSE;
        nwk_ib.nwkCapabilityInformation = 0x00,
        nwk_ib.nwkUseTreeAddrAlloc =
        USE_TREE_ADDR_ALLOC;
        nwk_ib.nwkUseTreeRouting =
        USE_TREE_ROUTING;
        nwk_ib.nwkNextAddress =
        0x0000;
        nwk_ib.nwkAvailableAddresses = 0x0000;
        nwk_ib.nwkAddressIncrement =
        0x0001;
        nwk_ib.nwkTransactionPersistenceTime = 0x01f4;

#ifdef ZIGBEE_STUB
        //inform upper layer
        nlmeResetConfirm(nwk_SUCCESS);
#endif
    }
    else
        nlmeResetConfirm(nwk_DISABLE_TRX_FAILURE);
} // End of functions

```

```

/*****
//      Function Name:      nlmeSyncRequest
//      Description:
//      Parameters:          Track - BOOL
//      Return:              nothing
*****/
Z_STATUS nlmeSyncRequest(BOOL Track)
{
    /***** Variable declaration *****/
    UINT8      nSetValue;
    /***** Function body *****/
    /*Set network state*/
    if (!nwk_setNwkState(NWK_STATE_SYNC))
        return nwk_NOT_PERMITTED;

    if ((Track == FALSE) && (NwkInfo.NwkBeaconOrder == NON_BEACON_NWK))
    {
        if (NwkInfo.pParent->NetworkAddress == INVALID_SHORT_ADDR)
            return mlmePollRequest(AM_EXTENDED_64,&NwkInfo.pParent->ExtendedAddress,
                                   NwkInfo.NwkPANId,FALSE);

        else
            return mlmePollRequest(AM_SHORT_16,&NwkInfo.pParent->NetworkAddress,
                                   NwkInfo.NwkPANId,FALSE);
    }

    if ((Track == FALSE) && (NwkInfo.NwkBeaconOrder != NON_BEACON_NWK))
    {
        nSetValue = TRUE;
        mlmeSetRequest(MAC_AUTO_REQUEST, &nSetValue);
        return mlmeSyncRequest(NwkInfo.NwkChannel, FALSE);
    }

    if ((Track == TRUE) && (NwkInfo.NwkBeaconOrder == NON_BEACON_NWK))
        return INVALID_PARAMETER;

    if ((Track == TRUE) && (NwkInfo.NwkBeaconOrder != NON_BEACON_NWK))
    {
        nSetValue = TRUE;
        mlmeSetRequest(MAC_AUTO_REQUEST, &nSetValue);
        return mlmeSyncRequest(NwkInfo.NwkChannel, TRUE);
    }
    return INVALID_PARAMETER;
} // End of functions

/*****
//      Function Name:      nlmeNetworkDiscoveryRequest
//      Description:          This primitive allows the next higher layer to request that
//                           the NWK layer discover networks currently operating within the POS
//      Parameters:          ScanChannels - DWORD
//                           ScanDuration - UINT8
//      Return:              nothing
*****/
void nlmeNetworkDiscoveryRequest(DWORD ScanChannels, UINT8 ScanDuration)
{

```

```

/* Set current request */
NwkInfo.CurRequest = NWK_STATE_DISCOVERY;

/* perform the request */
if (nwkcCoordinatorCapable == 1)
{
    mlmeScanRequest(ACTIVE_SCAN, ScanChannels, ScanDuration); //active
}
else
    if (MAC_RFD_ACTIVE_SCAN_ENABLE == 0)
    {
        mlmeScanRequest(PASSIVE_SCAN, ScanChannels, ScanDuration); //passive
    }
    else
        mlmeScanRequest(ACTIVE_SCAN, ScanChannels, ScanDuration); //active
}

/*****
//      Function Name:      nlmeJoinRequest
//      Description:        This primitive allows the next higher layer to request that the NWK layer to JOIN or
//                           associate to a parent in NEIGHBOR table
//      Parameters:          nlmeJoinRequest_s - NLME_JOINREQUEST
//      Return:              nothing
*****/
/*TBD: past to here only the capacity infor*/
void nlmeJoinRequest(UINT16 PanID,          BOOL      JoinAsRouter,  BOOL      RejoinNetwork,
                    DWORD      ScanChannels,  UINT8      ScanDuration,  UINT8
                    PowerSource,  BOOL      RxOnWhenIdle,  UINT8      MACSecurity)
{
    /***** Variable declaration *****/
    NWK_NEIGHBOR_NODE*      pRouter;
    CAPACITY_INFO*          pCapInfo;
    UINT8                   nTemp;

    /***** Function body *****/
    /* Check if currently joined nwk */
    if ((NwkInfo.NwkState == NWK_STATE_NORMAL_MODE)
        || (NwkInfo.NwkState == NWK_STATE_JOIN_SUCESS))
        nlmeJoinConfirm(PanID, nwk_INVALID_REQUEST);
    else
    {
        /* Rejoin the network or not */
        if (!RejoinNetwork) /* Join new network */
        {
            pCapInfo = (CAPACITY_INFO*)&nwk_ib.nwkCapabilityInformation;

            if (JoinAsRouter)
            {
                NwkInfo.NwkJoinedDeviveType = ZIGBEE_ROUTER;
                pCapInfo->DeviceType = TRUE;
                pCapInfo->AllocAddr = TRUE;
            }
            else
            {
                NwkInfo.NwkJoinedDeviveType = ZIGBEE_ENDPOINT;
            }
        }
    }
}

```

```

        pCapInfo->DeviceType      = FALSE;
        pCapInfo->AllocAddr       = FALSE;
    }

    pRouter = nwk_findRouterInNT(PanID,ScanChannels);
    if (pRouter)
    {
        NwkInfo.pParent          = pRouter;
        NwkInfo.NwkPANId         = PanID;
        NwkInfo.NwkChannel       = pRouter->LogicalChannel;

        pCapInfo->PwrSrc          = PowerSource;
        pCapInfo->RxOnWhenIdle    = RxOnWhenIdle;
        pCapInfo->SecurityCap     = 0;
        NwkInfo.CurRequest        = NWK_STATE_JOIN;

        pCapInfo      = &nwk_ib.nwkCapabilityInformation;
        nTemp          = pCapInfo->RxOnWhenIdle;
        mlmeSetRequest(MAC_RX_ON_WHEN_IDLE, &nTemp);

        if (pRouter->NetworkAddress >= INVALID_SHORT_ADDR)
            mlmeAssociateRequest(pRouter->LogicalChannel,AM_EXTENDED_64,PanID,
                                (ADDRESS*)&pRouter->ExtendedAddress,
                                nwk_ib.nwkCapabilityInformation,FALSE);
        else
            mlmeAssociateRequest(pRouter->LogicalChannel,AM_SHORT_16,PanID,
                                (ADDRESS*)&pRouter->NetworkAddress,
                                nwk_ib.nwkCapabilityInformation,FALSE);
    }
    else /*No device exists*/
    {
        nlmeJoinConfirm(PanID,nwk_NOT_PERMITTED);
        return;
    }
}

if (RejoinNetwork
    && NwkInfo.NwkState != NWK_STATE_NORMAL_MODE) /* Rejoin nwk*/
{
    NwkInfo.CurRequest = NWK_STATE_REJOIN;
    mlmeScanRequest(ORPHAN_SCAN, ScanChannels,ScanDuration);
}
}
} // End of functions

#if ZIGBEE_DEVICE_FFD == 1

#if ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER

/*****/
//@FunctionName:
*/
//@Description:
//
//@Param
//

```

```

/*****
void nlmeJoinDirectRequest(ADDRESS* pDeviceAddr, CAPACITY_INFO CapInfor)
{
    /***** Variable declaration *****/
    NWK_STATUS status;
    /***** Function body *****/
    status = nwk_addNewChild(pDeviceAddr,0x00,CapInfor,NwkInfo.SecurityEnable,0x00);
    nlmeJoinDirectConfirm(pDeviceAddr, status);
} // End of functions

#endif

#endif

/*****
// Function Name: nlmeLeaveRequest
// Description:
// Parameters:
// Return: nothing
*****/

void nlmeLeaveRequest(ADDRESS* pDeviceAddress, BOOL RemoveChildren, BOOL MACSecurityEnable)
{
    /***** Variable declaration *****/
    NWK_NEIGHBOR_NODE* pNode;
    //NWK_FRM_TYPE FrameType = NWK_FRM_CMD;
    //NWK_CMD_TYPE CmdType = NWK_CMD_LEAVE;
    //P_NPDU npdu;
    /***** Function body *****/
    if (NwkInfo.NwkState != NWK_STATE_NORMAL_MODE){
        nlmeLeaveConfirm(pDeviceAddress, nwk_INVALID_REQUEST);
    }
    else
    {
        /* Remove devices from the network */
        if (pDeviceAddress->Extended != 0 )
        {
            /* Remove this child from the network 2.7.1.7.2*/
            pNode = nwk_findNodeInNT(pDeviceAddress);
            if (!pNode) /* Node does not exist in the NT*/
            {
                nlmeLeaveConfirm(pDeviceAddress,nwk_UNKNOWN_DEVICE);
                return;
            }
            else /* Device found, remove it */
            {
                // Check if the device initiating the Leave is a coord or a router
                if ((ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR) ||
(ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER)){
                    /* Begin the transaction */

                    //npdu = nwk_makeNpdu(NWK_FRM_DATA,NsduLength,Nsdu);

                    /* Send th LEAVE.request */
                    /* mcpsDataRequest(Config_NWK_Leave.SrcAddrMode,
Config_NWK_Leave.SrcPANId, *Config_NWK_Leave.pSrcAddr, pNode DstAddrMode,

```



```

pNode.PanID,
pNode.NetworkAddress,
UINT8    msduLength,
BYTE     *msdu,
UINT8    msduHandle,

Config_NWK_Leave.TxOptions); */

/* Wait for a timeout */

/* Send DISASSOCIATION.request */

/* End of transaction: SUCCESS or LEAVE_UNCONFIRMED */
    }
else{
    nlmeLeaveConfirm(pDeviceAddress,
nwk_INVALID_REQUEST);
    }
    }
    /*TBD*/
}

/* Remove itself from the network following 2.7.1.7.1*/
if (pDeviceAddress->Extended == 0 && !RemoveChildren)
{
    /* Initiate self removal procedure */

    /* Clear routing table */
    /* TBD */
    /* Reset the MAC */
    nlmeResetRequest(TRUE);
}

}

} // End of functions

#if ZIGBEE_DEVICE_FFD == 1                /* Code for FFD only */

#if ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR /* Code for PAN Coordinator only */

/*****
//      Function Name:      nlmeNetworkFormationRequest

//      Description:

//      Parameters:

//      Return:

*****/
void nlmeNetworkFormationRequest(DWORD ScanChannels, UINT8 ScanDuration, UINT8 BeaconOrder,
                                UINT8 SuperframeOrder, UINT16 PanId, BOOL BatteryLifeExtension)
{
    if ((ZIGBEE_DEVICE_FFD == FALSE) ||
        ((NwkInfo.NwkState == NWK_STATE_NORMAL_MODE) && (ZIGBEE_DEVICE_TYPE
==ZIGBEE_PAN_COOR))){
        nlmeNetworkFormationConfirm(nwk_INVALID_REQUEST);
    }
}

```

```

    }
    else{
        /*Set the current state*/ /*TBD: omit ED_SCAN */
        NwkInfo.NwkState          = NWK_STATE_FORMATION;
        NwkInfo.CurRequest         = NWK_STATE_FORMATION;
        NwkInfo.NwkBeaconOrder    = BeaconOrder;
        NwkInfo.NwkSuperFrmOrder  = SuperframeOrder;
        NwkInfo.NwkPANId          = PandId;          /* Pre-assigned PanID*/
        NwkInfo.NwkChannelList    = ScanChannels;
        NwkInfo.BatLifeExt        = BatteryLifeExtension;

        NwkInfo.BcnPayload.DeviceDepth          = 0;
        NwkInfo.BcnPayload.EndDeviceCapacity    = ENDDEVICE_CAP;
        NwkInfo.BcnPayload.nwkExtendedPANID     = aExtendedAddress;
        NwkInfo.BcnPayload.RouterCapacity       = ROUTER_CAP;
        NwkInfo.BcnPayload.nwkProtocolVersion   = confProtocolVers; /*Zigbee protocol*/
        NwkInfo.BcnPayload.ProtocolId          = 0;
        NwkInfo.BcnPayload.StackProfile         = confStackProfile;

        mlmeScanRequest(ACTIVE_SCAN, ScanChannels, ScanDuration);
    }
}

/*****
//@FunctionName:
//
//@Description:
//
//@Param
//
*****/

void nlme_formNwk()
{
    /***** Variable declaration *****/
    UINT8      setAttributeValue;
    /***** Function body *****/
    if (NwkInfo.NwkPANId == 0)
    {
        /*TBD*/
    }

    if(!nlwk_checkNwkExisted(NwkInfo.NwkPANId))
    {
        /* Not exist such PanId, then, get a channel */
        NwkInfo.NwkChannel = nlwk_choseChannel(NwkInfo.NwkChannelList);
        if (NwkInfo.NwkChannel >= ZIGBEE_MIN_CHANNEL && NwkInfo.NwkChannel <=
        ZIGBEE_MAX_CHANNEL)
        {
            routing_init();

            NwkInfo.NwkAddr = 0x0000;
            mlmeSetRequest(MAC_SHORT_ADDRESS, &NwkInfo.NwkAddr);

            /* Modify necessary PIB attributes */
            setAttributeValue = TRUE;
            mlmeSetRequest(MAC_ASSOCIATION_PERMIT, &setAttributeValue);
            mlmeSetRequest(MAC_RX_ON_WHEN_IDLE, &setAttributeValue);

```

```

        setAttributeValue = sizeof(BEACON_PAYLOAD);
        mlmeSetRequest(MAC_BEACON_PAYLOAD_LENGTH, &setAttributeValue);
        mlmeSetRequest(MAC_BEACON_PAYLOAD, &NwkInfo.BcnPayload);
        setAttributeValue = TRUE;
        mlmeSetRequest(MAC_RX_ON_WHEN_IDLE, &setAttributeValue);
        mlmeStartRequest(NwkInfo.NwkPANId, NwkInfo.NwkChannel, NwkInfo.NwkBeaconOrder,
                        NwkInfo.NwkSuperFrmOrder, TRUE, NwkInfo.BatLifeExt, FALSE, FALSE, 0);

        return;
    }
}

    nlmeNetworkFormationConfirm(nwk_STARTUP_FAILURE);
} // End of functions

#endif // ZIGBEE_DEVICE_TYPE == ZIGBEE_PAN_COOR

#if ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER
/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nlmeStartRouterRequest(UINT8 BeaconOrder, UINT8 SuperframeOrder, BOOL BatLifeExt)
{
    /***** Variable declaration *****/
    UINT8 nTemp;
    CAPACITY_INFO* pCapInfor;
    /***** Function body *****/
    /* Check if it joined as router */
    if (NwkInfo.NwkJoinedDeviveType != ZIGBEE_ROUTER)
    {
        nlmeStartRouterConfirm(nwk_INVALID_REQUEST);
    }
    else
    {
        /* If already created and now change the config or initializing new router */
        if (((NwkInfo.NwkState == NWK_STATE_NORMAL_MODE) && ((NwkInfo.NwkBeaconOrder !=
BeaconOrder) ||
(NwkInfo.NwkSuperFrmOrder != SuperframeOrder) || (NwkInfo.BatLifeExt != BatLifeExt)))
|| (NwkInfo.NwkState == NWK_STATE_JOIN_SUCESS) )
        {

            NwkInfo.NwkBeaconOrder = BeaconOrder;
            NwkInfo.NwkSuperFrmOrder = SuperframeOrder;
            NwkInfo.BatLifeExt = BatLifeExt;
            NwkInfo.CurRequest = NWK_STATE_START_ROUTER;

            NwkInfo.BcnPayload.EndDeviceCapacity = ENDDEVICE_CAP;
            NwkInfo.BcnPayload.nwkExtendedPANID = 0x00; /*TBD*/
            NwkInfo.BcnPayload.RouterCapacity = ROUTER_CAP;

        }

    }

}

#endif ZIGBEE_DEVICE_FFD == 1
    NwkInfo.BcnPayload.DeviceDepth = NwkInfo.pParent->Depth+1;

```

```

    /*Init routing*/
    routing_init();

#endif

    /* Modify necessary PIB attributes */
    nTemp = TRUE;
    mlmeSetRequest(MAC_ASSOCIATION_PERMIT, &nTemp);

    nTemp = sizeof(BEACON_PAYLOAD);
    mlmeSetRequest(MAC_BEACON_PAYLOAD_LENGTH, &nTemp);
    mlmeSetRequest(MAC_BEACON_PAYLOAD, &NwkInfo.BcnPayload);

    if (NwkInfo.NwkBeaconOrder != NON_BEACON_NWK)
        nwk_schedulBeacon(); /*Beacon scheduling*/
    else
        NwkInfo.BeaconOffset = 0; /*Without beacon scheduling*/

    mlmeStartRequest(NwkInfo.NwkPANId, NwkInfo.NwkChannel,
                    BeaconOrder, SuperframeOrder, FALSE, BatLifeExt,
                    NwkInfo.NwkState == NWK_STATE_NORMAL_MODE, FALSE,
    NwkInfo.BeaconOffset);
    }
}

} // End of functions

#endif //ZIGBEE_DEVICE_TYPE == ZIGBEE_ROUTER

/*****
//@FunctionName: This is call back function to disable the PermitJoin */
//@Description: */
// */
//@Param */
// */
*****/
void nlme_timeoutPermitJoin(void* pTimer, void* pArg)
{
    /****** Variable declaration *****/
    UINT32 nErr;
    /****** Function body *****/
    OSTmrDel(pTimer, &nErr); /* Delete the timer */
    nErr = FALSE;
    mlmeSetRequest(MAC_ASSOCIATION_PERMIT, &nErr);
} // End of functions

/*****
//@FunctionName: This function will enable permitjoint att. in MAC in
// a PermitDuration */
//@Description: */
// */
//@Param */
// */
*****/
void nlmePermitJoiningRequest(UINT8 PermitDuration)

```

```

{
    /***** Variable declaration *****/
    OS_TMR*   PermitJoinTimer;
    UINT8     nErr;
    UINT32    nDur;
    /***** Function body *****/
    /* Check device */
    if (nwkCoordinatorCapable || (Cskip == 0))
        nlmePermitJoiningConfirm(nwk_INVALID_REQUEST);
    else
    {
        /* Create a timer for reset the JoinPermission */
        nDur = (UINT32)PermitDuration*1000000/(MAC_SYMBOL_DURATION*aUnitBackoffPeriod);
        PermitJoinTimer
    }
    OSTmrCreate(0,nDur,OS_TMR_OPT_ONE_SHOT,nlme_timeoutPermitJoin,NULL,NULL,&nErr);
    nErr = TRUE;
    mlmeSetRequest(MAC_ASSOCIATION_PERMIT,&nErr);
    OSTmrStart(PermitJoinTimer,&nErr);
} // End of functions

#endif // !ZIGBEE_DEVICE_FFD == 1

/*****
CONFIRM AND INDICATION FUNCTIONS
*****/

//@FunctionName:
*/

//@Description:
//
//@Param
//
/*****
void nlmeJoinConfirm(UINT16 PanID,    NWK_STATUS Status)
{
    /***** Variable declaration *****/
    ZAPP_MSG Msg;
    /***** Function body *****/
    Msg.MsgType      = MSG_ZDO_CONFIRM_JOIN;
    Msg.EndPointId   = EP_ID_ZDO;
    Msg.Data         = Status;
    /*Do not care the PanId, access through NwkInfo*/
    apf_dispatchEvent(&Msg);
} // End of functions

/*****
//@FunctionName:
*/

//@Description:
//
//@Param
//
/*****
void nlmeJoinIndication(    WORD    ShortAddr,

```

```

        QWORD* pExtAddr,
        BYTE    CapInfor,
        BOOL     Sercurity)
{
    /***** Variable declaration *****/
    ZAPP_MSG     Msg;
    MSG_JOIN_INDI* pInidcation;
    /***** Function body *****/
    pInidcation = (MSG_JOIN_INDI*)mem_getMem(sizeof(MSG_JOIN_INDI));
    if(pInidcation)
    {
        pInidcation->ExtAddr    = *pExtAddr;
        pInidcation->ShortAddr  = ShortAddr;
        pInidcation->Sercurity  = Sercurity;
        pInidcation->CapInfor   = CapInfor;

        Msg.MsgType            = MSG_ZDO_INDI_JOIN;
        Msg.EndPointId         = EP_ID_ZDO;
        Msg.Data                = (WORD)pInidcation;
        /*Do not care the PanId, access through NwkInfo*/
        apf_dispatchEvent(&Msg);
    }
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nlmeLeaveConfirm(ADDRESS* NWK_DeviceAddress, NWK_STATUS Status)
{
    /***** Variable declaration *****/

    /***** Function body *****/

} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nlmeNetworkDiscoveryConfirm(BYTE NetworkCount, NWK_DESC* NetworkDescriptor,
                                NWK_STATUS Status)
{
    /***** Variable declaration *****/
    ZAPP_MSG     Msg;
    /***** Function body *****/
    /* Initialize ZDO */
    Msg.MsgType    = MSG_ZDO_CONFIRM_DIS;
    Msg.EndPointId = EP_ID_ZDO;

```

```

    Msg.Data          = Status;
    /*we will use function to access nwk descriptor, omit it here*/
    apf_dispatchEvent(&Msg);
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nlmeNetworkFormationConfirm(NWK_STATUS status)
{
    /****** Variable declaration *****/
    ZAPP_MSG Msg;
    /****** Function body *****/
    Msg.MsgType      = MSG_ZDO_CONFIRM_FORMATION;
    Msg.EndPointId   = EP_ID_ZDO;
    apf_dispatchEvent(&Msg);
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nlmeStartRouterConfirm(NWK_STATUS Status)
{
    /****** Variable declaration *****/
    ZAPP_MSG Msg;
    /****** Function body *****/
    Msg.MsgType      = MSG_ZDO_CONFIRM_START_ROUTER;
    Msg.EndPointId   = EP_ID_ZDO;
    Msg.Data         = Status;
    apf_dispatchEvent(&Msg);
} // End of functions

/*****
//@FunctionName:
*/
//@Description:
//
//@Param
//
*****/
void nlmeResetConfirm(NWK_STATUS status)
{
    ZAPP_MSG Msg;
    /****** Function body *****/
    Msg.MsgType      = MSG_ZDO_CONFIRM_RESET;
    Msg.EndPointId   = EP_ID_ZDO;
    apf_dispatchEvent(&Msg);
}

```

} // End of functions

/******
 // @FunctionName:

*/

// @Description:

*/

//

*/

// @Param

*/

//

*/

/******

void nlmeSyncConfirm(BYTE Status)

{

/****** Variable declaration *****/

/****** Function body *****/

/*No need to implement*/

} // End of functions

/******

// @FunctionName:

*/

// @Description:

*/

//

*/

// @Param

*/

//

*/

/******

void nlmeSyncIndication()

{

/****** Variable declaration *****/

ZAPP_MSG Msg;

/****** Function body *****/

Msg.MsgType = MSG_ZDO_INDI_SYNC;

Msg.EndPointId = EP_ID_ZDO;

apf_dispatchEvent(&Msg);

} // End of functions

/******

// @FunctionName:

*/

// @Description:

*/

//

*/

// @Param

*/

//

*/

/******

void nlmePermitJoiningConfirm(NWK_STATUS Status)

{

/****** Variable declaration *****/

/****** Function body *****/

/*No need to implement*/

} // End of functions

/******

// @FunctionName:

*/

// @Description:

*/

//

*/

// @Param

*/

//

*/



```
/******  
void nlmeJoinDirectConfirm(ADDRESS* pDeviceAddress, NWK_STATUS Status)  
{  
    /****** Variable declaration *****/  
    /****** Function body *****/  
    /*No need, return directly from the call to the Request-function*/  
} // End of functions
```